CENTRO UNIVERSITÁRIO DOS GUARARAPES – UNIFG
CIÊNCIA DA COMPUTAÇÃO

ALLAN GABRIEL ALMEIDA BARROS
FRANCISCO MIGUEL DE LEMOS SENA FERREIRA
GABRIEL SOUSA VASCONCELOS
MATHEUS HENRIQUE DIAS SIQUEIRA

SBI — SUPER BUSINESS INTELLIGENCE:

análise, desenvolvimento, modelagem de dados e garantia da qualidade de um sistema integrado de gestão comercial

RECIFE
2026

ALLAN GABRIEL ALMEIDA BARROS
FRANCISCO MIGUEL DE LEMOS SENA FERREIRA
GABRIEL SOUSA VASCONCELOS
MATHEUS HENRIQUE DIAS SIQUEIRA

SBI — SUPER BUSINESS INTELLIGENCE:

análise, desenvolvimento, modelagem de dados e garantia da qualidade de um sistema integrado de gestão comercial

Relatório técnico-científico apresentado como Projeto Acadêmico A3 às unidades curriculares de Garantia da Qualidade de Software e de Banco de Dados do curso de Ciência da Computação do Centro Universitário dos Guararapes – UNIFG, como requisito parcial para aprovação.

Garantia da Qualidade de Software: Prof. Paulo Thiago Gomes da Silva e Prof. Christian Felix Gbedande Azonyetin.
Banco de Dados: Prof. Paulo Thiago Gomes da Silva e Prof. Bruno Marciano de Amorim Josino.

RECIFE
2026

RESUMO

Este relatório documenta o SBI — Super Business Intelligence, sistema web de gestão comercial desenvolvido como Projeto Acadêmico A3 das unidades curriculares de Garantia da Qualidade de Software e de Banco de Dados. O sistema atende uma loja de varejo de eletrônicos com dois perfis, vendedor e gerente, e reúne ponto de venda, clientes, estoque com reserva, preços, promoções, alçadas de desconto, metas, usuários e painéis analíticos. A arquitetura combina interface Next.js, API REST Spring Boot e banco PostgreSQL, publicados na Vercel, no Render e no Neon. O objetivo foi documentar, com evidências verificáveis, a concepção, a modelagem de dados, a garantia da qualidade e a implantação do sistema e avaliar sua conformidade com os requisitos e com o contrato OpenAPI. A metodologia combinou auditoria documental do repositório, da integração contínua, da aplicação publicada, do banco de produção, da pasta oficial e do quadro de tarefas com a reexecução não destrutiva das verificações viáveis, separando resultados executados de resultados apenas documentados. Os 26 requisitos funcionais estão implementados; o banco possui 19 tabelas, 162 colunas, 52 restrições de verificação, 32 chaves estrangeiras, 8 gatilhos e 18 *views*, com paridade estrutural entre o esquema migrado localmente e o publicado. Das 163 verificações reexecutadas, 162 foram aprovadas, incluindo 61 testes SQL de regras, permissões e concorrência, e a integração contínua da versão de referência confirmou a aprovação das suítes Java, de ponta a ponta e de contrato ao vivo. O OpenAPI publicado contém as 58 operações do contrato com os mesmos códigos de sucesso, mas documenta apenas 45,8% das respostas de erro previstas. Foram registrados 17 achados, com destaque para a dependência do framework de interface com avisos críticos, a documentação incompleta de erros e a cadeia de evidências desatualizada, e proposto um roteiro de evolução priorizado, com correções de cinco achados preparadas e validadas localmente. Conclui-se que o SBI cumpre em grau elevado os requisitos e a estrutura do contrato, e que as principais lacunas estão na documentação e na manutenção das evidências.

Palavras-chave: Garantia da qualidade de software. Banco de dados. API REST. Conformidade de contrato. Sistema de gestão comercial.

ABSTRACT

This report documents SBI — Super Business Intelligence, a web-based retail management system developed as the A3 Academic Project for the Software Quality Assurance and Database courses. The system serves an electronics retail store with two roles, salesperson and manager, and combines a point of sale, customers, stock with reservation, prices, promotions, discount limits, sales goals, user administration and analytical dashboards. Its architecture consists of a Next.js interface, a Spring Boot REST API and a PostgreSQL database, deployed on Vercel, Render and Neon. The goal was to document, with verifiable evidence, the design, data modeling, quality assurance and deployment of the system, and to assess its conformance with its requirements and with its OpenAPI contract. The method combined a documentary audit of the repository, the continuous integration pipeline, the deployed application, the production database, the official folder and the task board with the non-destructive re-execution of feasible checks, separating executed results from results that were only documented. All 26 functional requirements are implemented; the database has 19 tables, 162 columns, 52 check constraints, 32 foreign keys, 8 triggers and 18 views, with structural parity between the locally migrated schema and the deployed one. Of the 163 re-executed checks, 162 passed, including 61 SQL tests of business rules, permissions and concurrency, and the continuous integration run of the reference version confirmed that the Java, end-to-end and live contract suites passed. The deployed OpenAPI document contains all 58 contract operations with the same success codes, but documents only 45.8% of the expected error responses. Seventeen findings were recorded, notably a user interface framework dependency with critical advisories, incomplete error documentation and an outdated evidence chain, and a prioritized evolution roadmap was proposed, with fixes for five findings prepared and validated locally. SBI largely meets its requirements and the structure of its contract, and its main gaps lie in documentation and in keeping evidence up to date.

Keywords: Software quality assurance. Database. REST API. Contract conformance. Retail management system.

LISTA DE FIGURAS

Figura 1 **–** Diagrama de casos de uso do SBI 43
Figura 2 **–** Diagrama de contexto do SBI 47
Figura 3 **–** Diagrama de componentes do SBI 50
Figura 4 **–** Fluxo de autenticação, bloqueio e revogação de sessão 54
Figura 5 **–** Diagrama de sequência da venda no PDV 56
Figura 6 **–** Modelo conceitual (diagrama entidade-relacionamento) 61
Figura 7 **–** Modelo lógico relacional (sem chaves de autoria) 63
Figura 8 **–** Modelo físico do subdomínio de vendas 65
Figura 9 **–** Fluxo de atualização do estoque 81
Figura 10 **–** Estados do pedido e do pagamento 82
Figura 11 **–** Estratégia de testes e fluxo de verificação contínua 89
Figura 12 **–** Tela de login no ambiente publicado, em largura de computador 111
Figura 13 **–** Visão geral da gestão (perfil gerente) 113
Figura 14 **–** Ponto de venda, etapa de itens (perfil vendedor) 115
Figura 15 **–** Meu desempenho sem meta definida (perfil vendedor) 116
Figura 16 **–** Tela de login no ambiente publicado, em largura de celular (375 px) . 117
Figura 17 **–** Administração de usuários (perfil gerente) 119
Figura 18 **–** Lista de clientes com documentos mascarados (perfil vendedor) 120
Figura 19 **–** Posição de estoque (perfil gerente) 121
Figura 20 **–** Avatar associado a cada integrante e docente 123
Figura 21 **–** Expressões pré-renderizadas de cada avatar 124
Figura 22 **–** Linha do tempo do projeto baseada em evidências 131
Figura 23 **–** Arquitetura de implantação do SBI 136
Figura 24 **–** Modelo físico do subdomínio de catálogo e estoque 171
Figura 25 **–** Modelo físico do subdomínio de usuários, metas e auditoria 172

LISTA DE QUADROS

Quadro 1 **–** Equipe e docentes do projeto 31
Quadro 2 **–** Ferramentas identificadas no projeto 33
Quadro 3 **–** Classificação das informações quanto à verificação 36
Quadro 4 **–** Códigos do nível de verificação dos requisitos 37
Quadro 5 **–** Requisitos funcionais do SBI 38
Quadro 6 **–** Requisitos não funcionais 40
Quadro 7 **–** Regras de negócio verificadas 41
Quadro 8 **–** Resumo da rastreabilidade requisito → componente → dados → testes 46
Quadro 9 **–** Organização do repositório na versão de referência 48
Quadro 10 **–** Operações do contrato v5 por recurso e papel 51
Quadro 11 **–** Códigos de erro estáveis da API 57
Quadro 12 **–** Principais decisões arquiteturais registradas 58
Quadro 13 **–** Entidades do banco por subdomínio 61
Quadro 14 **–** Resumo do dicionário de dados por tabela 66
Quadro 15 **–** Cardinalidades impostas pelo esquema 68
Quadro 16 **–** Restrições de verificação agrupadas por finalidade 69
Quadro 17 **–** Gatilho de entrada de estoque com custo médio ponderado (trecho da V5) 69
Quadro 18 **–** View de lançamentos que compõem o realizado (V6) 71
Quadro 19 **–** Trecho do roteiro de CRUD executado como gerente 72
Quadro 20 **–** Migrações Flyway aplicadas no banco publicado 75
Quadro 21 **–** Políticas de segurança em nível de linha para pedidos 76
Quadro 22 **–** Indicadores calculados pelo SBI 85
Quadro 23 **–** Características de qualidade avaliadas e indicadores 88
Quadro 24 **–** Matriz de risco por funcionalidade 90
Quadro 25 **–** Achados desta auditoria 97
Quadro 26 **–** Conclusão das suítes na integração contínua do *commit* de referência 99
Quadro 27 **–** Riscos do OWASP API Security Top 10 (2023) e controles do SBI 104
Quadro 28 **–** Resultado da auditoria de dependências npm 106
Quadro 29 **–** Riscos de segurança identificados 108

Quadro 30 **–** Reações do avatar da própria conta 125
Quadro 31 **–** Responsabilidades planejadas por integrante 127
Quadro 32 **–** Fases do desenvolvimento segundo o histórico de versões 130
Quadro 33 **–** Entregáveis exigidos e artefatos correspondentes 132
Quadro 34 **–** Indicadores de participação observados nas fontes 133
Quadro 35 **–** Pré-requisitos técnicos 134
Quadro 36 **–** Atendimento aos objetivos específicos 140
Quadro 37 **–** Roteiro técnico de evolução 150
Quadro 38 **–** Correções preparadas e validadas localmente 152
Quadro 39 **–** Critérios de aceitação por requisito funcional 160
Quadro 40 **–** Matriz de rastreabilidade requisito → critério → caso de teste → evidência 166
Quadro 41 **–** Evidências documentadas para requisitos não funcionais 169
Quadro 42 **–** Diagramas produzidos e arquivos-fonte 170
Quadro 43 **–** Dicionário de dados da tabela `address` 173
Quadro 44 **–** Dicionário de dados da tabela `app_users` 174
Quadro 45 **–** Dicionário de dados da tabela `audit_event` 175
Quadro 46 **–** Dicionário de dados da tabela `category` 175
Quadro 47 **–** Dicionário de dados da tabela `customer` 175
Quadro 48 **–** Dicionário de dados da tabela `discount_policy` 176
Quadro 49 **–** Dicionário de dados da tabela `monthly_goal` 177
Quadro 50 **–** Dicionário de dados da tabela `order_item` 177
Quadro 51 **–** Dicionário de dados da tabela `orders` 177
Quadro 52 **–** Dicionário de dados da tabela `payment` 178
Quadro 53 **–** Dicionário de dados da tabela `product` 179
Quadro 54 **–** Dicionário de dados da tabela `product_price_history` 180
Quadro 55 **–** Dicionário de dados da tabela `promotion` 180
Quadro 56 **–** Dicionário de dados da tabela `refund` 181
Quadro 57 **–** Dicionário de dados da tabela `role` 181
Quadro 58 **–** Dicionário de dados da tabela `seller_goal` 181
Quadro 59 **–** Dicionário de dados da tabela `stock_balance` 182
Quadro 60 **–** Dicionário de dados da tabela `stock_entry` 182
Quadro 61 **–** Dicionário de dados da tabela `stock_movement` 183

Quadro 62 **–** Chaves estrangeiras do esquema 183
Quadro 63 **–** Views do esquema 185
Quadro 64 **–** Funções do esquema 185
Quadro 65 **–** Gatilhos do esquema 186
Quadro 66 **–** Especificação resumida dos casos de uso 188
Quadro 67 **–** Objetivos de qualidade do plano da equipe 191
Quadro 68 **–** Níveis de teste e ferramentas 192
Quadro 69 **–** Plano de verificação desta auditoria 193
Quadro 70 **–** Casos de teste do catálogo da equipe 205
Quadro 71 **–** Defeitos registrados pela equipe 210
Quadro 72 **–** Achados desta auditoria 213
Quadro 73 **–** Requisitos de instalação por procedimento 216
Quadro 74 **–** Perfis da API e variáveis obrigatórias 217
Quadro 75 **–** Comandos de teste 217
Quadro 76 **–** Problemas conhecidos e soluções 219
Quadro 77 **–** Tarefas do vendedor 220
Quadro 78 **–** Tarefas exclusivas do gerente 221
Quadro 79 **–** Mensagens frequentes e ação recomendada 222
Quadro 80 **–** Matriz de evidências 224
Quadro 81 **–** Glossário técnico 229
Quadro 82 **–** Situação dos *pull requests* e dos achados em 25/10/2026 231

LISTA DE TABELAS

Tabela 1 **–** Resultado do pacote de testes de banco reexecutado 94
Tabela 2 **–** Defeitos registrados pela equipe por severidade 97
Tabela 3 **–** Execuções realizadas nesta auditoria.....98
Tabela 4 **–** Métricas de código antes e depois do ciclo de melhoria 99
Tabela 5 **–** Métricas de defeitos e de execução.....100
Tabela 6 **–** Estrutura e uso do quadro do Trello.....128
Tabela 7 **–** Resultados de testes por origem.....141
Tabela 8 **–** Casos executados na camada de banco 196
Tabela 9 **–** Casos executados nos validadores de contrato.....200
Tabela 10 **–** Casos executados na lógica do frontend 202
Tabela 11 **–** Casos executados no ambiente publicado.....203

LISTA DE ABREVIATURAS E SIGLAS

A3	Avaliação 3 (projeto acadêmico semestral)
ABNT	Associação Brasileira de Normas Técnicas
ACID	Atomicidade, consistência, isolamento e durabilidade
API	Application Programming Interface
ASVS	Application Security Verification Standard
BD	Banco de dados
BI	Business Intelligence
BOLA	Broken Object Level Authorization
CA	Critério de aceitação
CI	Integração contínua (Continuous Integration)
CNPJ	Cadastro Nacional da Pessoa Jurídica
CPD	Copy/Paste Detector
CPF	Cadastro de Pessoas Físicas
CRUD	Create, Read, Update, Delete
CSP	Content Security Policy
CSRF	Cross-Site Request Forgery
CT	Caso de teste
DER	Diagrama entidade-relacionamento
DOI	Digital Object Identifier
DTO	Data Transfer Object
E2E	End-to-end (teste de ponta a ponta)
ERP	Enterprise Resource Planning
FK	Foreign key (chave estrangeira)
GQS	Garantia da Qualidade de Software
HSTS	HTTP Strict Transport Security
HTTP	Hypertext Transfer Protocol
IEC	International Electrotechnical Commission
IEEE	Institute of Electrical and Electronics Engineers
ISO	International Organization for Standardization
ISTQB	International Software Testing Qualifications Board
JDBC	Java Database Connectivity

JPA	Jakarta Persistence API
JSON	JavaScript Object Notation
JVM	Java Virtual Machine
KLOC	Mil linhas de código
LGPD	Lei Geral de Proteção de Dados Pessoais
NIST	National Institute of Standards and Technology
OWASP	Open Worldwide Application Security Project
PDV	Ponto de venda
PF	Pessoa física
PJ	Pessoa jurídica
PK	Primary key (chave primária)
PR	Pull request
RBAC	Role-Based Access Control
REST	Representational State Transfer
RF	Requisito funcional
RFC	Request for Comments
RLS	Row-Level Security
RN	Regra de negócio
RNF	Requisito não funcional
SGBD	Sistema gerenciador de banco de dados
SKU	Stock Keeping Unit
SQL	Structured Query Language
TLS	Transport Layer Security
UC	Caso de uso (use case)
UNIFG	Centro Universitário dos Guararapes
URL	Uniform Resource Locator
UTC	Tempo universal coordenado
WCAG	Web Content Accessibility Guidelines

SUMÁRIO

1 INTRODUÇÃO 19
1.1 CONTEXTUALIZAÇÃO 19
1.2 PROBLEMA DE PESQUISA 20
1.3 JUSTIFICATIVA 20
1.4 OBJETIVO GERAL 21
1.5 OBJETIVOS ESPECÍFICOS 21
1.6 DELIMITAÇÃO DO PROJETO 21
1.7 ORGANIZAÇÃO DO TRABALHO 22
2 FUNDAMENTAÇÃO TEÓRICA 23
2.1 SISTEMAS DE INFORMAÇÃO 23
2.2 BUSINESS INTELLIGENCE 23
2.3 SISTEMAS DE GESTÃO COMERCIAL 24
2.4 ENGENHARIA DE SOFTWARE 24
2.5 ARQUITETURA DE APLICAÇÕES WEB 24
2.6 ENGENHARIA DE REQUISITOS 25
2.7 BANCO DE DADOS E SISTEMAS GERENCIADORES 25
2.8 MODELAGEM RELACIONAL 26
2.9 NORMALIZAÇÃO DE DADOS 26
2.10 INTEGRIDADE E TRANSAÇÕES 27
2.11 GARANTIA DA QUALIDADE DE SOFTWARE 27
2.12 VERIFICAÇÃO E VALIDAÇÃO 28
2.13 TESTES DE SOFTWARE 28
2.14 SEGURANÇA DE APLICAÇÕES 29
2.15 USABILIDADE E ACESSIBILIDADE 29
2.16 VERSIONAMENTO E DESENVOLVIMENTO COLABORATIVO 30
2.17 METODOLOGIAS DE DESENVOLVIMENTO 30
3 METODOLOGIA 31
3.1 CARACTERIZAÇÃO DA PESQUISA E DO PROJETO 31
3.2 CONTEXTO ACADÊMICO 31
3.3 LEVANTAMENTO DE REQUISITOS 32
3.4 PLANEJAMENTO 33
3.5 ORGANIZAÇÃO DA EQUIPE 33

3.6 FERRAMENTAS UTILIZADAS....33
3.7 ESTRATÉGIA DE DESENVOLVIMENTO....34
3.8 MODELAGEM DE DADOS....34
3.9 IMPLEMENTAÇÃO....34
3.10 TESTES E VALIDAÇÃO....35
3.11 PROCEDIMENTOS DE ANÁLISE DOS RESULTADOS....35
4 ESPECIFICAÇÃO DE REQUISITOS....37
4.1 REQUISITOS FUNCIONAIS....37
4.2 REQUISITOS NÃO FUNCIONAIS....40
4.3 REGRAS DE NEGÓCIO....41
4.4 ATORES E CASOS DE USO....42
4.4.1 UC03 — Registrar venda no PDV....44
4.4.2 UC04 — Confirmar pagamento....44
4.4.3 UC12 — Registrar entrada e ajuste de estoque....45
4.4.4 UC17 — Administrar usuários....45
4.5 MATRIZ DE RASTREABILIDADE....46
5 ARQUITETURA E TECNOLOGIAS....47
5.1 VISÃO ARQUITETURAL....47
5.2 TECNOLOGIAS UTILIZADAS....48
5.3 ORGANIZAÇÃO DO PROJETO....48
5.4 ARQUITETURA FRONTEND....49
5.5 ARQUITETURA BACKEND....49
5.6 COMUNICAÇÃO ENTRE COMPONENTES....51
5.7 APIS E ENDPOINTS....51
5.8 SERVIÇOS E INTEGRAÇÕES....52
5.9 AUTENTICAÇÃO....52
5.10 CONTROLE DE PERMISSÕES....55
5.11 FLUXO DE DADOS....55
5.12 TRATAMENTO DE EXCEÇÕES....57
5.13 GERENCIAMENTO DE ESTADO....57
5.14 CONFIGURAÇÕES E DEPENDÊNCIAS....58
5.15 DECISÕES ARQUITETURAIS....58
5.16 LIMITAÇÕES IDENTIFICADAS....59
6 MODELAGEM E IMPLEMENTAÇÃO DO BANCO DE DADOS....60

6.1 BANCO DE DADOS UTILIZADO 60
6.2 JUSTIFICATIVA TÉCNICA DA TECNOLOGIA 60
6.3 LEVANTAMENTO DAS ENTIDADES 61
6.4 MODELO CONCEITUAL 61
6.5 MODELO LÓGICO 62
6.6 MODELO FÍSICO 64
6.7 DIAGRAMA ENTIDADE-RELACIONAMENTO 66
6.8 DICIONÁRIO DE DADOS 66
6.9 TIPOS DE DADOS 67
6.10 CHAVES PRIMÁRIAS 67
6.11 CHAVES ESTRANGEIRAS 67
6.12 CARDINALIDADE E RELACIONAMENTOS 68
6.13 RESTRIÇÕES DE INTEGRIDADE 68
6.14 NORMALIZAÇÃO 70
6.15 CONSULTAS SQL 71
6.16 OPERAÇÕES DE INSERÇÃO, CONSULTA, ATUALIZAÇÃO E EXCLUSÃO 72
6.17 TRANSAÇÕES 73
6.18 CONCORRÊNCIA E CONSISTÊNCIA 73
6.19 ÍNDICES E DESEMPENHO 74
6.20 MIGRATIONS 75
6.21 CONTROLE DE ACESSO 75
6.22 DADOS FICTÍCIOS E MECANISMOS DE DEMONSTRAÇÃO 76
6.23 LIMITAÇÕES E PROPOSTAS DE EVOLUÇÃO 77
7 IMPLEMENTAÇÃO DAS FUNCIONALIDADES 78
7.1 AUTENTICAÇÃO 78
7.2 USUÁRIOS E PERMISSÕES 79
7.3 CADASTRO DE PRODUTOS 80
7.4 CONTROLE DE ESTOQUE 80
7.5 GESTÃO DE VENDAS 82
7.6 METAS COMERCIAIS 83
7.7 DASHBOARDS 84
7.8 INDICADORES 84
7.9 RELATÓRIOS 85
7.10 FUNCIONALIDADES ADMINISTRATIVAS 85

7.11 DEMAIS FUNCIONALIDADES IDENTIFICADAS 86
8 GARANTIA DA QUALIDADE DE SOFTWARE 88
8.1 FUNDAMENTOS E ATRIBUTOS DE QUALIDADE 88
8.2 ESTRATÉGIA DE GARANTIA DA QUALIDADE 89
8.3 PLANEJAMENTO DOS TESTES 90
8.4 CRITÉRIOS DE ACEITAÇÃO 90
8.5 TESTES UNITÁRIOS 91
8.6 TESTES DE INTEGRAÇÃO 92
8.7 TESTES FUNCIONAIS 92
8.8 TESTES DE INTERFACE 93
8.9 TESTES DE AUTENTICAÇÃO 94
8.10 TESTES DE AUTORIZAÇÃO 94
8.11 TESTES DE BANCO DE DADOS 94
8.12 TESTES DE REGRESSÃO 95
8.13 TESTES DE RESPONSIVIDADE 95
8.14 TESTES DE ACESSIBILIDADE 96
8.15 TESTES DE DESEMPENHO 96
8.16 TESTES DE SEGURANÇA AUTORIZADOS 96
8.17 REGISTRO DE DEFEITOS 97
8.18 ANÁLISE DOS RESULTADOS 98
8.19 MÉTRICAS DE QUALIDADE 99
8.20 RECOMENDAÇÕES DE MELHORIA 101
9 SEGURANÇA DA INFORMAÇÃO 102
9.1 AUTENTICAÇÃO 102
9.2 AUTORIZAÇÃO 102
9.3 GERENCIAMENTO DE SESSÕES 103
9.4 VALIDAÇÃO DE ENTRADAS 103
9.5 PROTEÇÃO DE DADOS 103
9.6 SEGURANÇA DE APIS 104
9.7 ARMAZENAMENTO DE CREDENCIAIS 105
9.8 VARIÁVEIS DE AMBIENTE 105
9.9 CONFIGURAÇÃO DE ACESSO AO BANCO 105
9.10 TRATAMENTO DE EXCEÇÕES 106
9.11 GERENCIAMENTO DE DEPENDÊNCIAS 106

9.12 CONSIDERAÇÕES PERTINENTES SOBRE LGPD 107
9.13 RISCOS IDENTIFICADOS 108
9.14 RECOMENDAÇÕES 108
10 INTERFACE E EXPERIÊNCIA DO USUÁRIO 110
10.1 ORGANIZAÇÃO VISUAL 110
10.2 NAVEGAÇÃO 111
10.3 HIERARQUIA DE INFORMAÇÕES 112
10.4 COMPONENTES REUTILIZÁVEIS 113
10.5 FORMULÁRIOS 114
10.6 FEEDBACK DAS OPERAÇÕES 115
10.7 TRATAMENTO DE ESTADOS 115
10.8 RESPONSIVIDADE 116
10.9 ACESSIBILIDADE 118
10.10 CONSISTÊNCIA VISUAL 118
10.11 FLUXOS DE UTILIZAÇÃO 119
10.12 AVALIAÇÃO CRÍTICA 121
10.13 AVATARES DOS USUÁRIOS E REAÇÕES ANIMADAS 122
11 GERENCIAMENTO DO PROJETO 127
11.1 PLANEJAMENTO INICIAL 127
11.2 DIVISÃO DE ATIVIDADES 127
11.3 FERRAMENTAS COLABORATIVAS 128
11.4 USO DO TRELLO 128
11.5 USO DO GITHUB 129
11.6 CONTROLE DE VERSÕES 129
11.7 HISTÓRICO DE ALTERAÇÕES 130
11.8 MARCOS DE DESENVOLVIMENTO 130
11.9 CORREÇÕES E MELHORIAS 131
11.10 ENTREGAS REALIZADAS 132
11.11 PENDÊNCIAS IDENTIFICADAS 132
11.12 PARTICIPAÇÃO DOS INTEGRANTES 133
12 IMPLANTAÇÃO E DISPONIBILIZAÇÃO 134
12.1 AMBIENTE DE DESENVOLVIMENTO 134
12.2 PRÉ-REQUISITOS TÉCNICOS 134
12.3 INSTALAÇÃO DAS DEPENDÊNCIAS 134

12.4 CONFIGURAÇÕES 135
12.5 PROCESSO DE BUILD 135
12.6 IMPLANTAÇÃO NA VERCEL 135
12.7 INTEGRAÇÃO COM O GITHUB 135
12.8 VARIÁVEIS DE AMBIENTE 137
12.9 CONFIGURAÇÕES DE PRODUÇÃO 137
12.10 PROCEDIMENTOS DE EXECUÇÃO LOCAL 138
12.11 LOGS E MONITORAMENTO 139
12.12 LIMITAÇÕES DA INFRAESTRUTURA 139
13 RESULTADOS OBTIDOS 140
13.1 OBJETIVOS ALCANÇADOS 140
13.2 FUNCIONALIDADES CONCLUÍDAS 140
13.3 REQUISITOS ATENDIDOS 141
13.4 ESTRUTURAS DE DADOS IMPLEMENTADAS 141
13.5 RESULTADOS DOS TESTES 141
13.6 DEFEITOS IDENTIFICADOS 142
13.7 PROBLEMAS CORRIGIDOS 142
13.8 INTEGRAÇÕES COMPROVADAS 143
13.9 DISPONIBILIDADE DA APLICAÇÃO 143
13.10 LIMITAÇÕES 143
14 DISCUSSÃO 144
14.1 CONFORMIDADE ESTRUTURAL E CONFORMIDADE DOCUMENTAL DO CONTRATO 144
14.2 COMPARAÇÃO COM OS OBJETIVOS 144
14.3 CONTRIBUIÇÕES DA DISCIPLINA DE BANCO DE DADOS 145
14.4 CONTRIBUIÇÕES DA DISCIPLINA DE GARANTIA DA QUALIDADE DE SOFTWARE 145
14.5 ESCOPO PLANEJADO E ESCOPO ENTREGUE 146
14.6 AMEAÇAS À VALIDADE 147
14.7 LIÇÕES APRENDIDAS 148
15 PROPOSTAS DE EVOLUÇÃO 149
15.1 ROTEIRO TÉCNICO PRIORIZADO 149
15.2 FUNCIONALIDADES FORA DO ESCOPO ATUAL 152
15.3 SEQUÊNCIA RECOMENDADA 152

15.4 CORREÇÕES PREPARADAS NESTA AUDITORIA 152
16 CONCLUSÃO 154
REFERÊNCIAS 156
APÊNDICE A – CRITÉRIOS DE ACEITAÇÃO DOS REQUISITOS FUNCIONAIS 160
APÊNDICE B – MATRIZ DE RASTREABILIDADE 165
APÊNDICE C – DIAGRAMAS TÉCNICOS 170
APÊNDICE D – DICIONÁRIO DE DADOS 173
D.1 TABELA ADDRESS 173
D.2 TABELA APP_USERS 173
D.3 TABELA AUDIT_EVENT 174
D.4 TABELA CATEGORY 175
D.5 TABELA CUSTOMER 175
D.6 TABELA DISCOUNT_POLICY 176
D.7 TABELA MONTHLY_GOAL 176
D.8 TABELA ORDER_ITEM 177
D.9 TABELA ORDERS 177
D.10 TABELA PAYMENT 178
D.11 TABELA PRODUCT 179
D.12 TABELA PRODUCT_PRICE_HISTORY 180
D.13 TABELA PROMOTION 180
D.14 TABELA REFUND 181
D.15 TABELA ROLE 181
D.16 TABELA SELLER_GOAL 181
D.17 TABELA STOCK_BALANCE 182
D.18 TABELA STOCK_ENTRY 182
D.19 TABELA STOCK_MOVEMENT 183
D.20 CHAVES ESTRANGEIRAS 183
D.21 VIEWS 185
D.22 FUNÇÕES E GATILHOS 185
APÊNDICE E – CASOS DE USO 187
APÊNDICE F – PLANO DE TESTES 191
F.1 OBJETIVOS E ESCOPO 191
F.2 OBJETIVOS DE QUALIDADE E METAS 191
F.3 NÍVEIS, TÉCNICAS E FERRAMENTAS 192

F.4 DADOS E AMBIENTES DE TESTE 192
F.5 CRITÉRIOS DE ENTRADA, SAÍDA E APROVAÇÃO 193
F.6 PLANO DE VERIFICAÇÃO DESTA AUDITORIA 193
APÊNDICE G – CASOS DE TESTE E RESULTADOS 195
G.1 PACOTE SQL E MASSA OLIST 195
G.2 VALIDADORES DE CONTRATO 200
G.3 LÓGICA DO FRONTEND 201
G.4 VERIFICAÇÕES NO AMBIENTE PUBLICADO 203
G.5 CATÁLOGO DE CASOS DOCUMENTADO PELA EQUIPE 204
APÊNDICE H – REGISTRO DE DEFEITOS 209
H.1 DEFEITOS REGISTRADOS PELA EQUIPE 209
H.2 ACHADOS DESTA AUDITORIA 212
APÊNDICE I – MANUAL TÉCNICO DE INSTALAÇÃO 216
I.1 REQUISITOS 216
I.2 OBTENÇÃO DO CÓDIGO 216
I.3 PILHA DE DEMONSTRAÇÃO EM UM COMANDO 216
I.4 AMBIENTE DE DESENVOLVIMENTO 217
I.5 PERFIS E VARIÁVEIS DE AMBIENTE 217
I.6 EXECUÇÃO DOS TESTES 217
I.7 REPRODUÇÃO DO PACOTE SQL SEM DOCKER 218
I.8 PUBLICAÇÃO 219
I.9 SOLUÇÃO DE PROBLEMAS 219
APÊNDICE J – MANUAL DO USUÁRIO 220
J.1 ACESSO AO SISTEMA 220
J.2 PERFIL VENDEDOR 220
J.3 PERFIL GERENTE 221
J.4 MENSAGENS FREQUENTES 222
APÊNDICE K – MATRIZ DE EVIDÊNCIAS 223
APÊNDICE L – GLOSSÁRIO TÉCNICO 229
APÊNDICE M – SITUAÇÃO EM 25 DE OUTUBRO DE 2026 231

1 INTRODUÇÃO

1.1 CONTEXTUALIZAÇÃO

Pequenos e médios varejistas costumam registrar vendas, estoque e metas comerciais em instrumentos separados, como planilhas, cadernos e sistemas de caixa sem integração. Essa fragmentação produz três efeitos recorrentes: o valor cobrado do cliente pode divergir do valor contabilizado, a mesma unidade de estoque pode ser prometida a dois clientes e o acompanhamento de metas passa a depender de consolidações manuais tardias. Sistemas de informação integrados reduzem esses problemas ao concentrar a transação e a informação gerencial em uma mesma base de dados (LAUDON; LAUDON, 2020), e o uso analítico desses dados para apoiar decisões caracteriza o campo de *Business Intelligence* (SHARDA; DELEN; TURBAN, 2018).

O SBI — Super Business Intelligence é o sistema desenvolvido pela equipe como objeto do Projeto Acadêmico A3 das unidades curriculares de Garantia da Qualidade de Software e Banco de Dados do curso de Ciência da Computação. O sistema implementado é uma aplicação web de gestão comercial para uma loja de varejo de eletrônicos, com dois perfis de acesso: o vendedor, que opera o ponto de venda (PDV) no balcão, e o gerente, que acumula as funções do vendedor e administra catálogo, preços, estoque, promoções, metas, usuários e indicadores. A base do sistema é uma API REST de gestão de pedidos preexistente (Order Management API), publicada por um integrante da equipe em agosto de 2026 e evoluída ao longo do semestre [E-01].

O eixo temático do trabalho é a conformidade de contrato da API REST: verificar, com evidências reproduzíveis, se a API cumpre o que o documento OpenAPI promete quanto a operações, parâmetros, validação de entrada (Bean Validation), formato padronizado de erro e consistência dos objetos de transferência de dados (DTO). Esse eixo atende ao enunciado da disciplina de Garantia da Qualidade de Software, que exige demonstrar por evidências que o software atende aos requisitos e apresenta nível aceitável de qualidade (AZONYETIN, 2026), e se articula com a disciplina de Banco de Dados por meio do esquema relacional, das restrições de integridade, das transações e das consultas SQL que sustentam as regras de negócio.

1.2 PROBLEMA DE PESQUISA

O problema que orienta este relatório é o seguinte: em que medida o SBI, na versão efetivamente publicada, cumpre os requisitos definidos pela equipe e o contrato OpenAPI que declara, e como os conhecimentos de Banco de Dados e de Garantia da Qualidade de Software foram aplicados para produzir e demonstrar essa conformidade?

A pergunta tem duas dimensões. A primeira é de produto: verificar se as funcionalidades, regras de negócio e estruturas de dados existem e se comportam como especificado. A segunda é de processo: verificar se os artefatos de qualidade exigidos (requisitos, critérios de aceitação, plano de testes, casos de teste, registro de defeitos, métricas e ciclo de melhoria) foram produzidos e se os resultados declarados são reproduzíveis.

1.3 JUSTIFICATIVA

A relevância do tema decorre de três fatores. Primeiro, falhas de cálculo monetário, de controle de estoque e de autorização têm impacto financeiro direto e figuram entre os riscos mais frequentes em APIs, como a autorização inadequada em nível de objeto e de propriedade (OWASP, 2023). Segundo, o enunciado de Garantia da Qualidade de Software determina que apenas resultados obtidos por execução real e reproduzíveis durante a apresentação serão considerados válidos (AZONYETIN, 2026), o que exige uma documentação que separe com rigor o que foi verificado do que foi apenas declarado. Terceiro, a disciplina de Banco de Dados exige um banco funcional com comandos SQL e operações de inserção, consulta, atualização e exclusão [E-35], e o SBI concentra parte relevante das regras de integridade no próprio banco, o que permite avaliar a aplicação prática de modelagem relacional, normalização, restrições e controle de concorrência.

Do ponto de vista acadêmico, o projeto permite relacionar conceitos de engenharia de requisitos, arquitetura de aplicações web, modelagem de dados, verificação e validação, testes automatizados e segurança de aplicações a um sistema executável e publicado, com histórico de versões e integração contínua.

1.4 OBJETIVO GERAL

Documentar, de forma técnico-científica e baseada em evidências verificáveis, a concepção, o desenvolvimento, a modelagem de dados, a garantia da qualidade e a implantação do SBI — Super Business Intelligence, avaliando o grau de conformidade do sistema publicado com seus requisitos e com seu contrato OpenAPI.

1.5 OBJETIVOS ESPECÍFICOS

Para atingir o objetivo geral, foram definidos os seguintes objetivos específicos:

a) inventariar e auditar as fontes oficiais do projeto (repositório, aplicação publicada, pasta do Google Drive e quadro do Trello), registrando versão analisada, data de consulta e limitações de acesso;
b) especificar os requisitos funcionais e não funcionais efetivamente implementados, as regras de negócio, os atores e os casos de uso, com rastreabilidade até componentes, entidades e casos de teste;
c) descrever a arquitetura, as tecnologias, a API e o controle de acesso a partir do código-fonte;
d) documentar o modelo conceitual, lógico e físico do banco de dados, com dicionário de dados derivado do esquema real;
e) reexecutar, de forma não destrutiva, as verificações automatizadas viáveis no ambiente de análise e confrontar os resultados com os declarados pela equipe;
f) avaliar segurança, interface, gerenciamento do projeto e implantação, identificando defeitos, riscos e limitações;
g) propor um roteiro técnico de evolução priorizado.

1.6 DELIMITAÇÃO DO PROJETO

O objeto analisado é o código da branch `main` do repositório oficial no *commit* `3b0242c477bf7b5401a6eff55ae2b88c74927b1b`, de 21 de outubro de 2026, que é também a versão em produção na Vercel na data da análise, 24 de outubro de 2026

[E-01] [E-02]. Alterações locais não publicadas encontradas na pasta de trabalho de um integrante (branch `feat/ceo-executive-dashboard`, com arquivos modificados e não integrados) não fazem parte do escopo e não são apresentadas como entregas. As mudanças integradas e publicadas depois da análise, entre elas as dessa *branch*, estão registradas no Apêndice M.

O sistema opera com dados de demonstração: massa sintética e um recorte do conjunto público de pedidos da Olist (OLIST, 2018), com nomes e documentos sintéticos. Ficam fora do escopo gateway real de pagamento, emissão fiscal, múltiplas lojas e dados reais de clientes, conforme decisão registrada pela equipe [E-29].

A análise não incluiu testes ofensivos nem operações de escrita no ambiente publicado. As telas autenticadas não foram inspecionadas ao vivo, porque nenhuma credencial foi fornecida por mecanismo seguro e as contas de demonstração dos professores não existiam no banco publicado na data da análise [E-32]; para essas telas foram usadas as capturas versionadas pela equipe.

1.7 ORGANIZAÇÃO DO TRABALHO

O capítulo 2 apresenta a fundamentação teórica. O capítulo 3 descreve a metodologia, separando a forma de trabalho da equipe da metodologia aplicada para reconstruir esta documentação. Os capítulos 4 a 7 tratam de requisitos, arquitetura, banco de dados e funcionalidades. O capítulo 8 reúne a garantia da qualidade, o capítulo 9 a segurança da informação e o capítulo 10 a interface. Os capítulos 11 e 12 abordam o gerenciamento do projeto e a implantação. Os capítulos 13 a 16 apresentam resultados, discussão, propostas de evolução e conclusão. Os apêndices trazem a especificação detalhada, a rastreabilidade, o dicionário de dados, os casos de teste, o registro de defeitos, os manuais e a matriz de evidências, cujos identificadores no formato [E-nn] são citados ao longo do texto.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo reúne os conceitos que sustentam as decisões observadas no SBI. Cada seção apresenta o conceito e indica como ele se manifesta na implementação analisada, sem repetir os detalhes técnicos tratados nos capítulos seguintes.

2.1 SISTEMAS DE INFORMAÇÃO

Sistemas de informação são conjuntos inter-relacionados de componentes que coletam, processam, armazenam e distribuem informações para apoiar a tomada de decisão, a coordenação e o controle de uma organização (LAUDON; LAUDON, 2020). Os autores distinguem sistemas de processamento de transações, que registram as operações rotineiras, de sistemas de apoio à gestão, que agregam essas transações em informações para decisão.

No SBI, as duas categorias convivem sobre a mesma base: o PDV registra pedidos, reservas e pagamentos (processamento de transações), enquanto os painéis do gerente consomem visões agregadas desses registros (apoio à gestão). A consequência arquitetural é que a qualidade da informação gerencial depende diretamente da integridade transacional, tema retomado nos capítulos 6 e 7.

2.2 BUSINESS INTELLIGENCE

Business Intelligence (BI) designa o conjunto de arquiteturas, ferramentas, bases de dados, aplicações e metodologias que transformam dados em informação para apoiar decisões de negócio (SHARDA; DELEN; TURBAN, 2018). Em ambientes analíticos de maior porte, o BI costuma apoiar-se em armazéns de dados com modelagem dimensional, separados da base transacional (KIMBALL; ROSS, 2013).

O SBI adota uma abordagem mais simples e adequada ao porte do problema: os indicadores são calculados por visões (*views*) SQL diretamente sobre o esquema transacional normalizado, como faturamento diário, desempenho por vendedor, margem por produto e posição de estoque [E-16]. Essa escolha elimina a latência de cargas periódicas, ao custo de concentrar a carga analítica no banco transacional,

limitação discutida no capítulo 14. Apesar do nome do produto, o sistema não implementa armazém de dados, cubo OLAP nem processos de ETL; o termo descreve a finalidade gerencial dos painéis, não uma arquitetura de BI corporativo.

2.3 SISTEMAS DE GESTÃO COMERCIAL

Sistemas de gestão comercial para varejo integram, tipicamente, cadastro de produtos e clientes, frente de caixa, controle de estoque, precificação, promoções e acompanhamento de metas. O valor desses sistemas está na consistência entre os módulos: uma venda confirmada deve, na mesma transação, baixar o estoque, compor o faturamento e o realizado do vendedor (LAUDON; LAUDON, 2020).

O SBI implementa esse encadeamento: o pedido reserva estoque na criação, converte a reserva em saída na confirmação do pagamento e passa a compor o faturamento e as metas apenas quando o pagamento é aprovado, descontados os estornos [E-16] [E-19].

2.4 ENGENHARIA DE SOFTWARE

A engenharia de software trata da produção de software com custo, prazo e qualidade controlados, abrangendo especificação, projeto, implementação, validação e evolução (SOMMERVILLE, 2016). Pressman e Maxim (2020) enfatizam que a qualidade resulta de práticas aplicadas ao longo de todo o processo, e não apenas de testes no final.

O projeto analisado evidencia práticas desse tipo: requisitos versionados, decisões técnicas registradas em `docs/decisoes/`, revisão por *pull request*, integração contínua com testes automatizados e um ciclo de medição antes e depois das correções [E-20] [E-22].

2.5 ARQUITETURA DE APLICAÇÕES WEB

A arquitetura de software descreve os elementos de um sistema, suas relações e as propriedades que emergem dessas relações (BASS; CLEMENTS; KAZMAN, 2021). Em aplicações corporativas, a organização em camadas (apresentação, domínio e acesso a dados) é um padrão recorrente para separar

responsabilidades (FOWLER, 2002). O estilo arquitetural REST, formulado por Fielding (2000), organiza a interação entre cliente e servidor em torno de recursos identificados por URI, operações uniformes e ausência de estado no protocolo, sendo hoje especificado em termos de semântica HTTP (FIELDING; NOTTINGHAM; RESCHKE, 2022).

O SBI combina uma interface Next.js com uma API Spring Boot organizada em controladores, serviços e repositórios. A interface e a API compartilham a mesma origem por meio de um *rewrite* HTTP, o que permite manter o identificador de sessão em *cookie* `HttpOnly` sem expor *tokens* ao JavaScript (BARTH, 2011). A API não é estritamente sem estado, porque a autenticação é por sessão no servidor; a escolha e suas consequências são analisadas no capítulo 5.

2.6 ENGENHARIA DE REQUISITOS

A engenharia de requisitos compreende a elicitação, a análise, a especificação e a validação das necessidades que o sistema deve atender (WIEGERS; BEATTY, 2013). A norma ISO/IEC/IEEE 29148 estabelece características desejáveis para requisitos individuais, como necessidade, ausência de ambiguidade, completude, consistência e verificabilidade (ISO; IEC; IEEE, 2018). Requisitos não verificáveis, como “o sistema deve ser rápido”, precisam ser reescritos em termos mensuráveis (AZONYETIN, 2026).

A equipe registrou 26 requisitos funcionais e 7 não funcionais, com critérios de aceitação no formato Dado/Quando/Então e uma rodada de validação que resolveu ambiguidades (por exemplo, se pedido pendente conta para a meta) [E-36]. O capítulo 4 apresenta a especificação consolidada e o grau de verificação de cada requisito.

2.7 BANCO DE DADOS E SISTEMAS GERENCIADORES

Um sistema gerenciador de banco de dados (SGBD) é o software que permite definir, construir, manipular e compartilhar bancos de dados entre usuários e aplicações, garantindo independência de dados, controle de concorrência e recuperação (ELMASRI; NAVATHE, 2016). Silberschatz, Korth e Sudarshan (2020) destacam como funções centrais do SGBD a manutenção de restrições de

integridade, a atomicidade das transações e o isolamento entre transações concorrentes.

O SBI utiliza o PostgreSQL 18 como SGBD em produção (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2025) [E-03], com o esquema versionado por migrações Flyway, que aplicam em ordem scripts identificados por versão e registram cada aplicação em uma tabela de histórico (REDGATE, 2026).

2.8 MODELAGEM RELACIONAL

O modelo relacional, proposto por Codd (1970), representa os dados como relações (tabelas) compostas por tuplas e atributos, manipuladas por operações de álgebra relacional e independentes da organização física. A modelagem costuma seguir três níveis de abstração: conceitual, que descreve entidades e relacionamentos do domínio; lógico, que mapeia o modelo para relações com chaves; e físico, que define tipos, índices e restrições no SGBD específico (ELMASRI; NAVATHE, 2016).

O esquema do SBI é relacional e combina dois padrões: herança em tabela única (*single table inheritance*), herdada do legado JPA para clientes, produtos e pagamentos, e tabelas especializadas para estoque, metas e auditoria [E-15]. A herança em tabela única simplifica consultas polimórficas, mas obriga colunas específicas de subtipo a aceitarem nulos, o que transfere parte da integridade para restrições `CHECK` (FOWLER, 2002), como se observa em `ck_customer_document_type`.

2.9 NORMALIZAÇÃO DE DADOS

A normalização é o processo de decompor relações para eliminar redundâncias e anomalias de inserção, atualização e exclusão, com base em dependências funcionais (CODD, 1970; ELMASRI; NAVATHE, 2016). A primeira forma normal (1FN) exige atributos atômicos; a segunda (2FN) elimina dependências parciais de chaves compostas; a terceira (3FN) elimina dependências transitivas de atributos não chave; a forma normal de Boyce-Codd (FNBC) exige que todo determinante seja chave candidata.

A desnormalização controlada é aceitável quando justificada por requisitos de desempenho ou de preservação histórica. No SBI, o preço unitário copiado para `order_item.price` e o custo copiado para `order_item.unit_cost` são redundâncias intencionais: representam o valor no momento da venda, e não o valor vigente, e por isso não constituem anomalia (seção 6.14).

2.10 INTEGRIDADE E TRANSAÇÕES

Restrições de integridade declarativas (chave primária, chave estrangeira, unicidade, não nulidade e verificação) garantem que o banco rejeite estados inválidos independentemente da aplicação que o acessa (SILBERSCHATZ; KORTH; SUDARSHAN, 2020). Uma transação é uma unidade lógica de trabalho com as propriedades de atomicidade, consistência, isolamento e durabilidade (ACID), sistematizadas por Haerder e Reuter (1983). Os níveis de isolamento padronizados pelo SQL permitem diferentes anomalias de concorrência, e Berenson *et al.* (1995) demonstram que a definição por fenômenos é insuficiente para caracterizar alguns níveis, como o isolamento por *snapshot*.

No PostgreSQL, o nível padrão é *Read Committed*; nele, uma instrução `UPDATE` que encontra uma linha alterada por transação concorrente reavalia a condição `WHERE` sobre a versão confirmada mais recente (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2025). O SBI explora essa propriedade na reserva de estoque com `UPDATE ... WHERE on_hand - reserved >= q`, que impede a venda da mesma última unidade a dois clientes sem bloqueio explícito de tabela (seção 6.18).

2.11 GARANTIA DA QUALIDADE DE SOFTWARE

Garantia da qualidade de software (SQA) é o conjunto de atividades planejadas e sistemáticas que fornecem confiança de que o processo e o produto atendem aos requisitos estabelecidos (PRESSMAN; MAXIM, 2020). A norma ISO/IEC 25010 define o modelo de qualidade de produto; a edição de 2023 organiza a qualidade em nove características — adequação funcional, eficiência de desempenho, compatibilidade, capacidade de interação, confiabilidade, segurança, manutenibilidade, flexibilidade e proteção (*safety*) — e substitui a usabilidade e a

portabilidade da edição de 2011 por capacidade de interação e flexibilidade (ISO; IEC, 2023).

A documentação da equipe utiliza a terminologia da edição de 2011 (por exemplo, usabilidade e portabilidade) [E-37]. Este relatório adota a edição de 2023 na análise do capítulo 8 e indica a correspondência com os termos da equipe.

2.12 VERIFICAÇÃO E VALIDAÇÃO

Verificação é a confirmação de que um produto de trabalho atende à sua especificação; validação é a confirmação de que atende ao uso pretendido pelas partes interessadas (IEEE, 2016). Sommerville (2016) resume a distinção pelas perguntas “estamos construindo o produto corretamente?” e “estamos construindo o produto certo?”.

No SBI, a verificação é dominante: testes automatizados comparam o comportamento com critérios de aceitação e com o contrato OpenAPI. A validação com usuários finais é limitada a um relatório de avaliação heurística de usabilidade e a ensaios de demonstração conduzidos pela própria equipe [E-38], o que é uma limitação discutida no capítulo 14.

2.13 TESTES DE SOFTWARE

Testar é executar um programa com a intenção de encontrar defeitos (MYERS; SANDLER; BADGETT, 2011). As técnicas de projeto de casos dividem-se em caixa-preta, baseadas na especificação, como particionamento de equivalência, análise de valor-limite, tabela de decisão e transição de estados, e caixa-branca, baseadas na estrutura interna, como cobertura de instruções, de decisões e de caminhos independentes (DELAMARO; MALDONADO; JINO, 2016; ISO; IEC; IEEE, 2021). A complexidade ciclomática de McCabe (1976) fornece o número de caminhos linearmente independentes de um método e orienta o esforço de teste estrutural.

Os níveis de teste — unitário, integração, sistema e aceitação — e o teste de regressão compõem a estratégia descrita no programa de certificação ISTQB (ISTQB, 2023) e na série ISO/IEC/IEEE 29119 (ISO; IEC; IEEE, 2022). A proporção

entre testes rápidos de unidade e testes mais lentos de ponta a ponta é frequentemente representada pela pirâmide de testes (COHN, 2010).

Na conformidade de contrato, o documento OpenAPI 3.1 funciona como oráculo de teste: descreve operações, parâmetros e esquemas de resposta em JSON Schema 2020-12 (OPENAPI INITIATIVE, 2021; JSON SCHEMA, 2020), e as respostas reais da API são validadas contra esses esquemas. A padronização do corpo de erro pode seguir o formato *Problem Details* (NOTTINGHAM; WILDE; DALAL, 2023) ou um formato próprio documentado no contrato, como ocorre no SBI.

2.14 SEGURANÇA DE APLICAÇÕES

A segurança de aplicações web e de APIs é orientada por referências como o OWASP API Security Top 10, cuja edição de 2023 lista como primeiro risco a autorização quebrada em nível de objeto (BOLA) e como terceiro a autorização quebrada em nível de propriedade (OWASP, 2023), e pelo padrão de verificação ASVS, cuja versão 5.0 organiza requisitos verificáveis por domínio (OWASP, 2025). Diretrizes de autenticação recomendam senhas longas, verificação contra listas de senhas comprometidas e limitação de tentativas, em vez de regras de composição arbitrárias (NIST, 2025).

No contexto brasileiro, a Lei Geral de Proteção de Dados Pessoais disciplina o tratamento de dados pessoais, incluindo os princípios de necessidade, segurança e prevenção (BRASIL, 2018). Para dados de cartão de pagamento, o padrão PCI DSS restringe o armazenamento do número completo do cartão (PCI SECURITY STANDARDS COUNCIL, 2024). O SBI armazena apenas os quatro últimos dígitos do cartão, mascara documentos para o vendedor e trabalha somente com dados sintéticos (capítulo 9).

2.15 USABILIDADE E ACESSIBILIDADE

Usabilidade é a medida em que um sistema pode ser usado por usuários específicos para atingir objetivos com eficácia, eficiência e satisfação em um contexto de uso; Nielsen (1993) propõe heurísticas para sua avaliação, como visibilidade do estado do sistema, prevenção de erros e consistência. A acessibilidade web é normatizada pelas Diretrizes de Acessibilidade para Conteúdo

Web (WCAG), cuja versão 2.2 acrescentou critérios como tamanho mínimo de alvo e foco não obscurecido (W3C, 2024).

A equipe verifica automaticamente a conformidade WCAG 2.2 nível AA com a biblioteca axe-core nos testes de ponta a ponta e na fumaça (*smoke test*) do ambiente publicado [E-21]. Ferramentas automáticas detectam apenas parte dos problemas de acessibilidade; avaliação manual com tecnologias assistivas permanece necessária (capítulo 10).

2.16 VERSIONAMENTO E DESENVOLVIMENTO COLABORATIVO

Sistemas de controle de versão distribuídos, como o Git, registram o histórico de alterações e permitem o trabalho paralelo em ramificações integradas por mesclagem (CHACON; STRAUB, 2014). A integração contínua executa automaticamente a construção e os testes a cada alteração integrada, e a entrega contínua estende essa automação até a publicação (HUMBLE; FARLEY, 2010).

O SBI utiliza GitHub com *pull requests*, integração contínua no GitHub Actions e implantação automática na Vercel e no Render a cada alteração na branch `main`, além de bancos efêmeros no Neon por *pull request* [E-20] [E-02].

2.17 METODOLOGIAS DE DESENVOLVIMENTO

Metodologias ágeis privilegiam entregas incrementais, colaboração e resposta a mudanças (BECK *et al.*, 2001). O Scrum organiza o trabalho em *sprints* com papéis e eventos definidos (SCHWABER; SUTHERLAND, 2020), e o Kanban enfatiza a visualização do fluxo e a limitação do trabalho em andamento (ANDERSON, 2010).

A atribuição de uma metodologia a um projeto exige evidência de seus elementos característicos. O quadro do projeto no Trello apresenta listas de estado (Backlog, A fazer, Em andamento, Em revisão, Concluído), mas não registra *sprints*, limites de trabalho em andamento, cerimônias nem métricas de fluxo, e nenhum documento cita Scrum ou Kanban como método adotado [E-26]. Por isso, este relatório descreve o processo observado — desenvolvimento incremental orientado a etapas, com integração por *pull request* — sem enquadrá-lo formalmente em uma metodologia específica (capítulo 11).

3 METODOLOGIA

Este capítulo distingue duas metodologias: a forma de trabalho efetivamente adotada pela equipe durante o desenvolvimento, reconstruída a partir dos registros do projeto, e a metodologia aplicada posteriormente para auditar as fontes e produzir esta documentação.

3.1 CARACTERIZAÇÃO DA PESQUISA E DO PROJETO

Quanto à natureza, o trabalho é uma pesquisa aplicada, pois produz conhecimento para um problema concreto: demonstrar a conformidade de um sistema com seus requisitos e com seu contrato de API. Quanto aos objetivos, é descritivo e avaliativo, porque descreve a solução implementada e avalia sua qualidade com critérios explícitos. Quanto aos procedimentos, combina pesquisa documental (análise de repositório, documentos, quadro de tarefas e registros de implantação), estudo de caso (o SBI como unidade de análise) e verificação experimental reproduzível (reexecução de testes automatizados e consultas ao sistema publicado).

O projeto em si é classificado como desenvolvimento de software com finalidade acadêmica, no qual a aplicação é o sistema sob teste (*System Under Test*, SUT) e o processo de garantia da qualidade é o objeto principal de avaliação da disciplina de Garantia da Qualidade de Software (AZONYETIN, 2026).

3.2 CONTEXTO ACADÊMICO

O projeto atende a duas unidades curriculares do curso de Ciência da Computação do Centro Universitário dos Guararapes (UNIFG), no segundo semestre de 2026, cujos docentes estão identificados no **Quadro** 1.

Quadro 1 **–** Equipe e docentes do projeto

Papel	Nome	Registro acadêmico
Integrante	Allan Gabriel Almeida Barros	1352524052
Integrante	Gabriel Sousa Vasconcelos	13526110176
Integrante	Francisco Miguel de Lemos Sena Ferreira	1352523939
Integrante	Matheus Henrique Dias Siqueira	1352623492
Docente — Garantia da Qualidade de Software	Paulo Thiago Gomes da Silva; Christian Felix Gbedande Azonyetin	—

Papel	Nome	Registro acadêmico
Docente — Banco de Dados	Paulo Thiago Gomes da Silva; Bruno Marciano de Amorim Josino	—

Fonte: dados informados pela equipe; registros acadêmicos conferidos com o Manual do projeto v1.0 [E-29].

Três condições do contexto acadêmico influenciam a análise. Primeiro, o enunciado de Garantia da Qualidade estabelece grupos de no máximo três alunos, e a equipe tem quatro integrantes; nem o Drive, nem o Trello, nem as mensagens de correio eletrônico consultadas registram autorização docente para essa composição [E-27] [E-26]. Segundo, não há enunciado nem rubrica escrita da disciplina de Banco de Dados nas fontes oficiais nem nas mensagens consultadas; a exigência conhecida, registrada em documento do projeto e no cartão BD-04 do Trello, é implementar um banco de dados funcional com comandos SQL e operações CRUD [E-35]. Terceiro, a entrega final está prevista até 30 de novembro de 2026, e a janela de avaliação A3 do calendário institucional vai de 30 de novembro a 5 de dezembro de 2026 [E-27]. Esses pontos são tratados como pendências de validação no checklist final.

3.3 LEVANTAMENTO DE REQUISITOS

Pelos registros, o levantamento ocorreu em duas fases. Na primeira, entre 26 de setembro e 3 de outubro de 2026, o Plano de Execução e o Manual do projeto definiram como sistema sob teste a Order Management API, com doze requisitos funcionais sobre clientes, produtos, endereços, pedidos e pagamentos, e excluíram expressamente do escopo principal a interface de loja e a autenticação completa [E-29]. Na segunda, a partir de 1º de outubro, dezesseis cartões do Trello introduziram o escopo “Loja • Gestão”, com perfis de gerente e vendedor, metas, estoque com reserva, custo histórico e quatro telas [E-26]; em 3 de outubro, o documento `docs/requisitos.md` consolidou os requisitos RF-01 a RF-16 com status Observado, Alvo, Parcial ou Pendente, e as evoluções de 14 de outubro acrescentaram RF-17 a RF-26 [E-36].

A validação dos requisitos registrou ambiguidades resolvidas (por exemplo, AMB-04: pagamento pendente não entra no realizado) e decisões pendentes, como o limite numérico de desconto, depois resolvido pela decisão DEC-01 com alçadas de 5% para vendedor e 20% para gerente [E-36].

3.4 PLANEJAMENTO

O planejamento registrado compreende um cronograma de nove semanas (28 de setembro a 30 de novembro), marcos internos, estimativa de 226 horas de esforço e distribuição de papéis por integrante [E-29]. O quadro do Trello foi criado em 26 de setembro com 34 cartões e chegou a 94 cartões distribuídos em seis listas [E-26]. A partir de 13 de outubro, o planejamento técnico passou a ser registrado em documentos de etapa (2A banco, 2B segurança, 2C endpoints, 3 frontend, 4 testes e CI, 5 documentação) e em pacotes de evolução WP-EVO e WP-PUB, versionados no repositório ou mantidos na pasta local de planejamento [E-34].

3.5 ORGANIZAÇÃO DA EQUIPE

A distribuição planejada de responsabilidades aparece em três versões que não coincidem integralmente: o Plano de Execução de 26 de setembro, o Plano de Qualidade de 9 de outubro e o cartão “COMO COMEÇAR” de 12 de outubro [E-29] [E-37] [E-26]. A análise da participação efetivamente comprovada, que difere da planejada, é apresentada no capítulo 11.

3.6 FERRAMENTAS UTILIZADAS

O **Quadro** 2 relaciona as ferramentas identificadas nos manifestos de dependência, nas configurações e nos fluxos de integração contínua do repositório.

Quadro 2 **–** Ferramentas identificadas no projeto

Finalidade	Ferramenta e versão	Evidência
Linguagem e plataforma da API	Java 25, Spring Boot 4.1.0, Spring Security, Spring Data JPA, `JdbcClient`	`pom.xml`
Banco de dados e migrações	PostgreSQL 18 (CI e produção), Flyway (`spring-boot-starter-flyway`)	`docker-compose*.yml`, `pom.xml`, Neon
Contrato da API	OpenAPI 3.1, Springdoc 2.8.5, Ajv 8.20.0 com ajv-formats 3.0.1	`pom.xml`, `package.json`
Interface web	Next.js 16.3.8, React 19.3.0, TypeScript 6.0.3, Zod 4.6.7, TanStack Query 5.104.2, React Hook Form 7.90.0, orval 8.42.0	`frontend/package.json`
Testes	JUnit 5, Spring Boot Test, MockMvc, `node:test`, Playwright 1.63.0, @axe-core/playwright 4.13.0	`pom.xml`, `frontend/package.json`
Cobertura e análise estática	JaCoCo 0.8.15, PMD (plugin 3.28.0) com CPD, SpotBugs (plugin 4.10.4.1)	`pom.xml`
Integração e implantação	GitHub Actions, Docker, Vercel, Render, Neon	`.github/workflows/`, `Dockerfile`, `docs/deploy.md`

Finalidade	Ferramenta e versão	Evidência
Gestão	Trello, Google Drive, GitHub (PRs)	quadro `ombHh0vo`, pasta oficial

Fonte: elaborado a partir do repositório no *commit* `3b0242c` [E-01].

3.7 ESTRATÉGIA DE DESENVOLVIMENTO

O histórico de versões mostra uma estratégia incremental em três momentos [E-01]. De 31 de agosto a 1º de setembro, a base Order Management API foi publicada no repositório de origem. De 3 a 10 de outubro, a equipe estabeleceu a linha de base de qualidade, corrigiu defeitos do legado, introduziu autenticação por sessão, Flyway e métricas iniciais. Em 14 de outubro, concentraram-se 58 dos 113 *commits* da branch `main` (51,3%), integrando as etapas de banco, segurança, endpoints, interface, testes e documentação e, em seguida, a evolução com administração de usuários, resumo do cliente, CNPJ alfanumérico, desconto manual, massa Olist e marca. De 15 a 21 de outubro, o sistema foi publicado e recebeu correções e melhorias.

Duas regras de processo aparecem de forma consistente nos registros: toda operação nova do contrato entra como `planned` e só passa a `implemented` com teste verde, e todo defeito ganha um teste que falha antes da correção e passa depois [E-37] [E-06].

3.8 MODELAGEM DE DADOS

A modelagem partiu do esquema legado gerado pelo JPA (endereço, cliente, produto, pedido, item e pagamento) e evoluiu por migrações versionadas. O modelo-alvo do ERP/PDV foi documentado e ensaiado em um pacote SQL da disciplina de Banco de Dados em 12 de outubro, com 51 verificações aprovadas, e depois incorporado às migrações V5 e V6 com adaptações registradas (P1 a P9 e D1 a D10) [E-39]. A partir de então, o Flyway passou a ser a única fonte do esquema, com o Hibernate apenas em modo de validação [E-16].

3.9 IMPLEMENTAÇÃO

A implementação seguiu a separação entre módulos novos, em `api/<módulo>`, com controladores finos, serviços com autorização por método e transações

declarativas, e repositórios com SQL explícito por `JdbcClient`, e o código legado em JPA, mantido para as rotas `/orders`, `/payment` e `/address` e para a leitura de contas e a gravação da auditoria [E-01]. A interface web consome a API por esquemas Zod gerados automaticamente a partir do contrato, o que transforma divergências entre contrato e cliente em falha de compilação ou de integração contínua.

3.10 TESTES E VALIDAÇÃO

A estratégia de testes declarada pela equipe é baseada em risco e orientada pelo contrato (*contract-first*): a intensidade dos testes segue a matriz probabilidade × impacto, e o contrato OpenAPI aprovado é o oráculo dos testes HTTP [E-37]. Os testes são distribuídos em unitários, integração com banco PostgreSQL real descartável, testes de banco por `psql`, testes de contrato e de sessão contra a API em execução, e testes de ponta a ponta com verificação de acessibilidade. O detalhamento e os resultados estão no capítulo 8.

3.11 PROCEDIMENTOS DE ANÁLISE DOS RESULTADOS

A metodologia aplicada para produzir esta documentação seguiu oito etapas, executadas em 24 de outubro de 2026:

a) **levantamento documental:** inventário do repositório (1.245 arquivos versionados, 113 *commits* na `main`), da pasta do Drive (4 arquivos), do quadro do Trello (94 cartões, 448 ações) e dos documentos locais de planejamento [E-01] [E-27] [E-26] [E-34];
b) **auditoria técnica:** leitura do código, das migrações, do contrato e das configurações; extração do catálogo do banco a partir de um esquema migrado localmente; consulta somente leitura ao banco publicado [E-15] [E-03];
c) **reconstrução histórica:** cronologia a partir dos *commits*, das ações do Trello, das datas do Drive, do histórico de implantações na Vercel e no Render e das execuções da integração contínua [E-01] [E-26] [E-02] [E-47] [E-46];
d) **análise de requisitos:** confronto entre requisitos planejados, requisitos documentados e funcionalidades presentes no código;

e) **verificação e validação:** reexecução não destrutiva das suítes viáveis e verificações caixa-preta somente leitura no sistema publicado (seção 8.2);
f) **fundamentação científica:** associação dos conceitos às decisões observadas, com referências verificadas;
g) **redação acadêmica:** produção dos capítulos com indicação de evidências;
h) **revisão de consistência:** conferência das afirmações contra as fontes e registro das pendências no checklist final.

Cada afirmação técnica relevante recebeu uma das classificações do **Quadro** 3, aplicadas também à matriz de evidências do Apêndice K.

Quadro 3 **–** Classificação das informações quanto à verificação

Classificação	Critério aplicado
Verificada	Confirmada nesta auditoria por execução, consulta ao sistema publicado ou leitura direta do código na versão de referência.
Documentada	Registrada pela equipe em artefato versionado, mas não reexecutada nesta auditoria.
Parcialmente verificada	Parte da afirmação foi confirmada; outra parte depende de evidência não acessível.
Não verificada	Afirmada em alguma fonte, sem evidência acessível que a sustente.
Planejada	Prevista em plano ou cartão, sem implementação na versão de referência.
Não implementada	Verificada a ausência na versão de referência.

Fonte: elaborado pelos autores.

O ambiente de reexecução impôs restrições que delimitam os resultados. A rede do ambiente de análise não alcançava o repositório Maven Central, a distribuição do JDK 25 nem o serviço de fontes do Google; por isso, as suítes Java e a construção do frontend não puderam ser executadas localmente [E-10]. A conclusão dessas suítes, das de ponta a ponta e do contrato ao vivo foi consultada na execução da integração contínua do *commit* de referência, que terminou com sucesso em todas as etapas; como os relatórios e os *logs* dessa execução não puderam ser baixados, a aprovação é tratada como verificada e os totais por suíte, como documentados [E-46]. O pacote de testes de banco foi reexecutado em PostgreSQL 16.15, e não 18, com as migrações aplicadas por `psql` com substituição do marcador `${flyway:defaultSchema}` e com a migração Java de contas substituída por inserção SQL equivalente com senha gerada na execução [E-11]. Essas adaptações são ameaças à validade e são discutidas no capítulo 14.

4 ESPECIFICAÇÃO DE REQUISITOS

Os requisitos deste capítulo correspondem aos registrados pela equipe em `docs/requisitos.md` e `docs/criterios-aceitacao.md`, conferidos contra o código da versão de referência [E-36] [E-01]. A prioridade foi atribuída nesta documentação a partir da matriz de risco da equipe (probabilidade × impacto) [E-40]: requisitos ligados a riscos críticos ou altos (pagamento, cálculo do total, estoque, autenticação e autorização) receberam prioridade Alta. A coluna de verificação usa os códigos do **Quadro** 4.

Quadro 4 **–** Códigos do nível de verificação dos requisitos

Código	Significado
COD	Implementação localizada e lida no código-fonte da versão de referência.
BD	Regra verificada por teste de banco reexecutado nesta auditoria (pacote SQL, 61 verificações aprovadas).
PUB	Comportamento observado no ambiente publicado em 24/10/2026, somente por requisições de leitura.
DOC	Coberto por teste automatizado da equipe (Java, E2E ou contrato) com resultado documentado, não reexecutado.

Fonte: elaborado pelos autores.

4.1 REQUISITOS FUNCIONAIS

O sistema possui 26 requisitos funcionais, apresentados no **Quadro** 5. Todos estão implementados na versão de referência; nenhum requisito permanece com status Alvo ou Pendente no código, embora a matriz final do documento de critérios de aceitação ainda os marque como pendentes, inconsistência registrada como achado A-01 (seção 8.17).

Quadro 5 **–** Requisitos funcionais do SBI

ID	Nome	Descrição	Prior.	Regra de negócio	Status	Evidência
RF-01	Autenticação	Autenticar usuários da equipe com perfil Gerente ou Vendedor, por sessão e CSRF.	Alta	RN-15, RN-16	Implementado	COD, PUB, DOC
RF-02	Autorização no servidor	Autorizar cada operação por perfil e identidade da sessão, com 401/403/404 padronizados.	Alta	RN-01	Implementado	COD, BD, PUB, DOC
RF-03	Clientes PF e PJ	Manter clientes pessoa física e jurídica com o documento coerente com o tipo.	Média	RN-14	Implementado	COD, BD, DOC
RF-04	Catálogo físico e digital	Manter produtos físicos (peso) e digitais (link de download).	Média	RN-22	Implementado	COD, DOC
RF-05	Pedido (rota legada)	Criar pedido para cliente existente ou novo, com itens, endereço e pagamento.	Média	RN-02, RN-24	Implementado	COD, DOC
RF-06	Vendedor pela sessão	Atribuir o vendedor do pedido pela sessão autenticada, nunca pelo corpo da requisição.	Alta	RN-01	Implementado	COD, BD, DOC
RF-07	Cálculo monetário no servidor	Calcular valores no servidor, com preço unitário histórico e quantidade multiplicada uma única vez.	Alta	RN-02, RN-03, RN-21	Implementado	COD, BD, DOC
RF-08	Pagamento e estorno	Processar pagamento simulado, confirmar de forma idempotente e registrar estornos.	Alta	RN-07, RN-13, RN-19	Implementado	COD, BD, DOC
RF-09	Metas mensais	Registrar meta mensal da loja e distribuição por vendedor com soma igual à meta.	Média	RN-10	Implementado	COD, BD, DOC
RF-10	Realizado	Calcular o realizado como pagamentos confirmados menos estornos; meta zero exibe “Sem meta”.	Média	RN-09, RN-10	Implementado	COD, BD, DOC
RF-11	Escopo do vendedor	Restringir o vendedor à própria meta, ao próprio realizado e às próprias vendas.	Alta	RN-01	Implementado	COD, BD, DOC
RF-12	Visão consolidada	Oferecer ao gerente a visão da loja, da equipe, das metas e dos alertas de estoque.	Média	RN-09	Implementado	COD, DOC
RF-13	Reserva de estoque	Reservar na criação, baixar na confirmação e liberar no cancelamento do pedido.	Alta	RN-04, RN-05	Implementado	COD, BD, DOC
RF-14	Entradas e custo histórico	Registrar entradas com custo, mantendo custo médio e custo histórico da venda, ocultos ao vendedor.	Média	RN-08	Implementado	COD, BD, DOC
RF-15	Área Vender	Permitir atendimento de cliente novo ou existente, seleção de produtos e criação do pedido.	Alta	RN-02, RN-06	Implementado	COD, DOC
RF-16	Interface por perfil	Oferecer as áreas de Gestão, Meu desempenho, Vender e Estoque conforme o perfil.	Média	RN-01	Implementado	COD, DOC

ID	Nome	Descrição	Prior.	Regra de negócio	Status	Evidência
RF-17	Hierarquia de perfis	Garantir que o Gerente herde tudo do Vendedor e que nada exclusivo do Gerente alcance o Vendedor.	Alta	RN-01	Implementado	COD, BD, DOC
RF-18	Login resistente a força bruta	Bloquear a conta por 15 minutos após 5 falhas, com resposta idêntica e tempo equivalente.	Alta	RN-15	Implementado	COD, DOC
RF-19	Venda no balcão (PDV)	Buscar por SKU ou nome, montar carrinho, cotar no servidor, finalizar de forma idempotente e confirmar pagamento.	Alta	RN-02 a RN-07	Implementado	COD, BD, DOC
RF-20	Cadastro de clientes com validação	Validar CPF/CNPJ (numérico ou alfanumérico) com dígito verificador e unicidade; exclusão só pelo gerente e sem pedidos.	Média	RN-14	Implementado	COD, BD, DOC
RF-21	Histórico do cliente	Exibir histórico paginado de pedidos, com valores de outros vendedores ocultos ao vendedor.	Baixa	RN-01, RN-14	Implementado	COD, DOC
RF-22	Backoffice do gerente	Manter produtos e insumos, preço com motivo, entradas, ajustes com motivo, extrato e estoque mínimo.	Média	RN-08, RN-20, RN-21, RN-22	Implementado	COD, BD, DOC
RF-23	Painel de faturamento	Exibir faturamento, lucro e margem a partir das *views*, atualizado a cada 15 s com a aba visível.	Média	RN-08, RN-09	Implementado	COD, BD, DOC
RF-24	Administração de usuários	Cadastrar, editar, alterar papel e situação, desbloquear e redefinir senha, com revogação de sessão e ao menos um gerente ativo.	Alta	RN-16, RN-17, RN-18	Implementado	COD, BD, DOC
RF-25	Indicadores do cliente	Exibir total comprado, ticket médio, pedidos por situação, datas, produtos e formas de pagamento.	Baixa	RN-09, RN-14	Implementado	COD, BD, DOC
RF-26	Desconto manual com alçada	Aplicar desconto manual dentro da alçada do papel, com motivo, sem venda abaixo do custo e com aprovação do gerente acima da alçada.	Alta	RN-12	Implementado	COD, BD, DOC

Fonte: elaborado pelos autores a partir de `docs/requisitos.md` e do código no *commit* `3b0242c` [E-36] [E-01].

Os critérios de aceitação somam 85 itens no formato Dado/Quando/Então, listados no Apêndice A. Dois aspectos merecem registro. Primeiro, RF-03 a RF-05 descrevem comportamentos herdados da API original e continuam ativos nas rotas legadas `/orders`, `/payment` e `/address`, restritas ao gerente e fora do contrato v5 (seção 5.7). Segundo, RF-23 especifica um intervalo de atualização de 15 segundos, verificado pela equipe em teste de ponta a ponta e não reexecutado nesta auditoria.

4.2 REQUISITOS NÃO FUNCIONAIS

O **Quadro** 6 consolida os requisitos não funcionais. A coluna de critério reproduz a meta verificável definida pela equipe; a coluna de situação informa o que foi possível confirmar.

Quadro 6 **–** Requisitos não funcionais

ID	Característica (ISO/IEC 25010:2023)	Requisito e critério verificável	Situação
RNF-01	Segurança	Acesso mínimo por perfil: rota restrita responde 401 sem sessão e 403 a perfil indevido.	Verificado parcialmente: 401 confirmado no ambiente publicado; 403 documentado (`AccessMatrixTest`).
RNF-02	Confiabilidade	Integridade transacional: reserva, saída, cancelamento e pagamento não deixam estado parcial.	Verificado na camada de banco (BD-15, BD-19, BD-21, BD-28, CC-01).
RNF-03	Adequação funcional	Precisão monetária: NUMERIC(12,2), arredondamento HALF_UP e preservação histórica.	Verificado no esquema (V4) e em BD-24 a BD-26; regras Java documentadas.
RNF-04	Manutenibilidade	Reprodutibilidade: ambiente, massa, versões, comandos e relatórios versionados.	Verificado parcialmente: pacote SQL reproduzido; suítes Java dependem de JDK 25 e Maven Central.
RNF-05	Capacidade de interação	Usabilidade e acessibilidade: estados de carregamento, vazio, erro e sucesso; WCAG 2.2 AA.	Documentado (axe sem violação nos E2E e nas capturas); login publicado inspecionado.
RNF-06	Eficiência de desempenho	Desempenho sob protocolo definido (p95 por endpoint crítico).	Não verificado: protocolo previsto, medição não realizada pela equipe.
RNF-07	Manutenibilidade	Rastreabilidade: cada RF ligado a critério, caso de teste, execução e defeito.	Verificado: matriz em `docs/rastreabilidade.md` (Apêndice B), com divergências documentais apontadas no achado A-01.

Fonte: elaborado pelos autores a partir de `docs/requisitos.md` [E-36].

O RNF-06 é o único requisito não funcional sem evidência de medição. As respostas observadas no ambiente publicado em 24/10/2026, entre 237 ms e 731 ms para requisições GET sem sessão com a API aquecida [E-04], são uma amostra isolada e não substituem um teste de desempenho com carga, aquecimento e percentis definidos.

4.3 REGRAS DE NEGÓCIO

As regras do **Quadro** 7 foram extraídas do código, das migrações e das decisões registradas. A coluna de implementação indica onde a regra é imposta; regras com dupla barreira (aplicação e banco) resistem a acessos que não passem pela API.

Quadro 7 **–** Regras de negócio verificadas

ID	Regra	Implementação
RN-01	O vendedor do pedido é sempre o usuário da sessão; vendedor só acessa os próprios pedidos (objeto de outro dono responde 404).	`SalesService`, `CurrentUser`; RLS demonstrativa em `03_rbac.sql`
RN-02	Preço, desconto e total são calculados no servidor; propriedade desconhecida no corpo responde 400.	`Pricing`, `fail-on-unknown-properties=true`
RN-03	Dinheiro em NUMERIC(12,2) com HALF_UP; menor valor aceito 0,005 (arredonda para 0,01).	V4, `Money`, `@DecimalMin`
RN-04	Disponível = saldo − reservado; reserva na criação, saída na confirmação, liberação no cancelamento.	`SalesService`; `stock_balance` e CHECKs
RN-05	Itens processados em ordem crescente de `product_id` para evitar *deadlock*.	`SalesService`, `fn_criar_pedido`
RN-06	Venda idempotente por `Idempotency-Key` + SHA-256 do corpo; corpo diferente com a mesma chave responde 409.	`RequestHash`, `uq_orders_seller_request`
RN-07	Só pedido pendente é cancelado; confirmação repetida não tem efeito.	`SalesService`, `fn_confirmar_pagamento`
RN-08	Custo médio ponderado recalculado na entrada; custo desconhecido propaga nulo e a margem fica “Não informado”.	gatilho `stock_entry_apply`, *views* V6
RN-09	Realizado = pagamentos APPROVED − estornos; pendente não conta; datas em America/Fortaleza.	`v_lancamento_realizado`
RN-10	Soma das metas dos vendedores = meta da loja, conferida no COMMIT; meta individual só para VENDEDOR; meta zero = “Sem meta”.	gatilhos `seller_goal_sum`, `seller_goal_is_seller`
RN-11	Promoção só liberada por GERENTE, com janela válida, percentual ≤ 100 e escopo por produto ou categoria.	gatilho `promotion_only_manager`, CHECKs
RN-12	Desconto manual na alçada do papel (VENDEDOR 5%, GERENTE 20%), com motivo ≥ 5 caracteres, sem venda abaixo do custo médio; acima da alçada, aprovação do gerente no pedido pendente.	`discount_policy`, `SalesService`, V11
RN-13	Estorno só pelo gerente, só de pagamento aprovado e até o saldo não estornado.	`RefundService`, `fn_estornar`
RN-14	CPF/CNPJ com dígito verificador e único; CNPJ alfanumérico aceito; documento imutável e mascarado ao vendedor; exclusão só pelo gerente e sem pedidos.	`@CPF`/`@CNPJ` nos DTOs, `DocumentFormat`, V5, V10
RN-15	Cinco falhas de login bloqueiam a conta por 15 minutos; respostas idênticas e BCrypt executado em todos os casos.	`LockoutAuthenticationProvider`, `JdbcLoginCounter`
RN-16	Senha com 12 a 72 caracteres e até 72 bytes, fora da lista comum e diferente do e-mail; senha provisória obriga troca.	`StrongPasswordValidator`, `common-passwords.txt`
RN-17	Mudança de papel, situação, senha ou desbloqueio revoga as sessões abertas na requisição seguinte.	`session_version`, `SessionVersionFilter`

ID	Regra	Implementação
RN-18	Sempre existe pelo menos um GERENTE ativo.	gatilho `keep_one_active_manager` (V8)
RN-19	Do cartão, somente os quatro últimos dígitos são armazenados.	`card_last4`, `ck_payment_card_number_absent` (V7)
RN-20	Ajuste de estoque exige motivo; ajuste de entrada só por inventário.	CHECKs de `stock_movement`
RN-21	Alteração de preço gera histórico com autor e motivo; item vendido mantém o preço histórico.	gatilho `product_price_history`
RN-22	Produto inativo ou insumo (`sellable = false`) não é vendido.	`v_catalogo_vendedor`, `SalesService`
RN-23	Eventos de auditoria nunca contêm senha, hash nem documento completo.	`AuditService`, `audit_event`
RN-24	Entrega (DELIVERY) exige endereço.	`ck_orders_delivery_address`

Fonte: elaborado pelos autores a partir do código e das migrações [E-16] [E-17] [E-19].

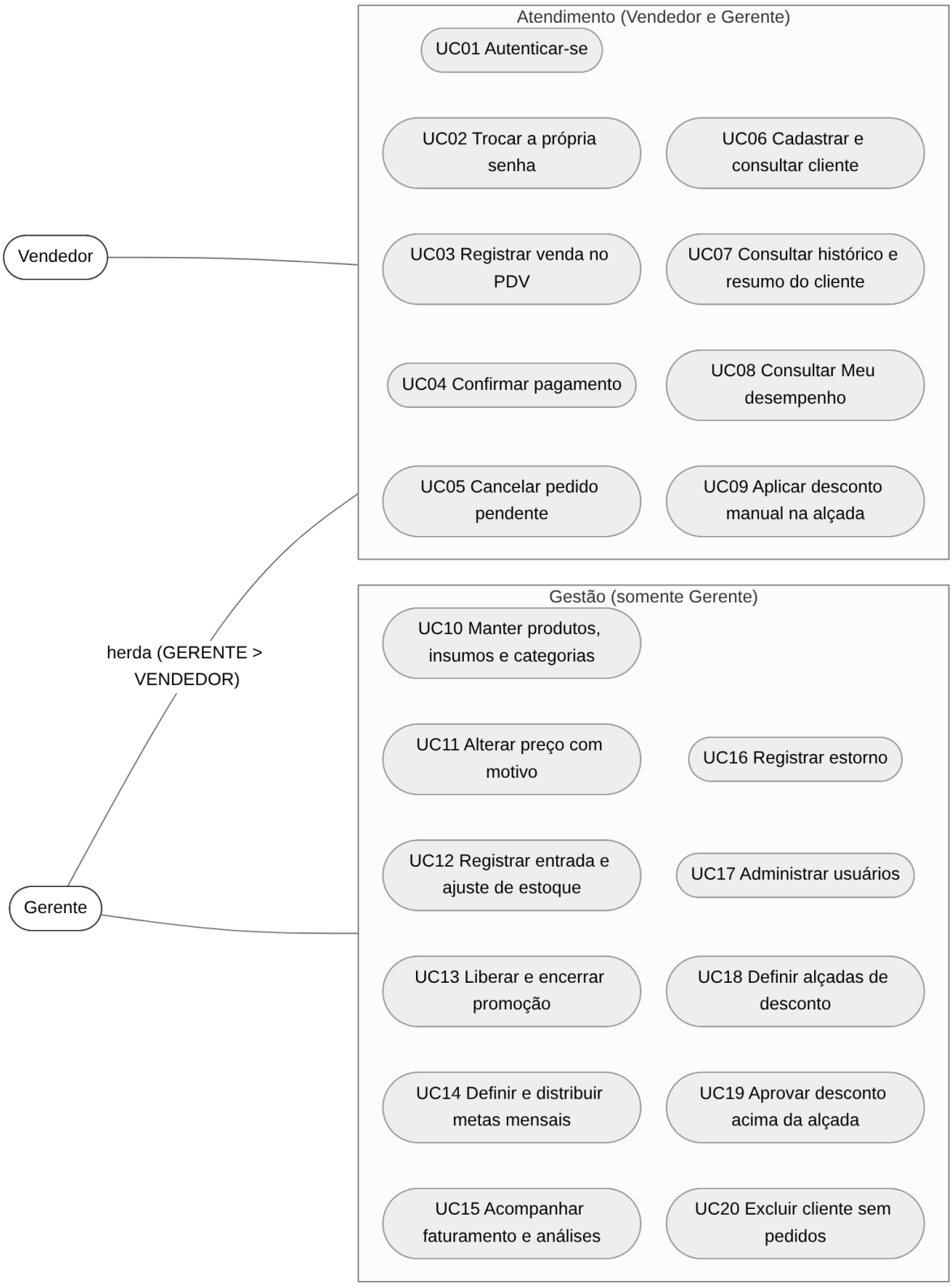
4.4 ATORES E CASOS DE USO

Foram identificados dois atores humanos e um ator técnico:

- **Vendedor**: atende o cliente no balcão, registra vendas, consulta clientes e acompanha o próprio desempenho;
- **Gerente**: herda as capacidades do vendedor pela hierarquia GERENTE > VENDEDOR e administra catálogo, estoque, promoções, metas, usuários, alçadas e indicadores;
- **Avaliador acadêmico**: acessa o ambiente de demonstração com contas autorizadas; na data da análise, não havia contas próprias dos docentes no banco publicado [E-32].

A **Figura** 1 apresenta os vinte casos de uso identificados no código e nas rotas da interface.

Figura 1 **–** Diagrama de casos de uso do SBI



Fonte: elaborado pelos autores. Notação adaptada de UML; o Gerente herda os casos de uso do Vendedor.

Os quatro casos de uso de maior risco são detalhados a seguir; os demais estão no Apêndice E.

4.4.1 UC03 — Registrar venda no PDV

Ator principal: Vendedor (ou Gerente). **Pré-condições:** sessão ativa sem troca de senha pendente; produto ativo e vendável com disponível maior que zero.

Fluxo principal: (1) o vendedor busca produtos por SKU ou prefixo do nome; (2) adiciona itens ao carrinho, limitado ao disponível; (3) seleciona ou cadastra o cliente, a forma de pagamento (Pix, cartão ou boleto) e a modalidade (retirada ou entrega); (4) o sistema cota a venda no servidor e exibe subtotal, descontos e total; (5) o vendedor finaliza; o sistema cria o pedido PENDING_PAYMENT com chave de idempotência, reserva o estoque item a item e cria o pagamento PENDING.

Fluxos alternativos: (A1) desconto manual dentro da alçada, com motivo; (A2) promoção vigente aplicada ao item; (A3) cliente com documento já cadastrado: o sistema oferece usar o cliente existente.

Exceções: (E1) disponível insuficiente: 409 `STOCK_INSUFFICIENT`, nenhuma reserva permanece; (E2) repetição com a mesma chave e o mesmo corpo: 200 com o mesmo pedido; com corpo diferente: 409 `IDEMPOTENCY_CONFLICT`; (E3) desconto acima da alçada: 403 `DISCOUNT_ABOVE_LIMIT`; (E4) desconto abaixo do custo: 409 `BELOW_COST`, sem revelar o custo; (E5) campo de preço, total ou vendedor no corpo: 400 `MALFORMED_REQUEST`.

Pós-condições: pedido pendente com vendedor igual ao usuário da sessão, reserva registrada como movimento RESERVA e pagamento pendente com valor igual ao total.

4.4.2 UC04 — Confirmar pagamento

Ator principal: Vendedor dono do pedido ou Gerente. **Pré-condição:** pedido PENDING_PAYMENT.

Fluxo principal: (1) o usuário confirma o pagamento; (2) o sistema bloqueia o pedido (`SELECT ... FOR UPDATE`); (3) baixa saldo e reserva de cada item, grava o custo médio vigente no item e registra a SAIDA; (4) aprova o pagamento e marca o pedido como PAID.

Exceções: (E1) pedido já pago: 200 sem efeito; (E2) pedido cancelado: 409 `STATE_CONFLICT`; (E3) pedido de outro vendedor: 404.

Pós-condições: a venda passa a compor o faturamento, o realizado do vendedor e o resumo do cliente.

4.4.3 UC12 — Registrar entrada e ajuste de estoque

Ator principal: Gerente. **Pré-condição:** produto cadastrado.

Fluxo principal: (1) o gerente informa quantidade e custo unitário da entrada; (2) o gatilho soma ao saldo, recalcula o custo médio ponderado e registra o movimento ENTRADA.

Fluxo alternativo: (A1) ajuste de saída com motivo (perda, avaria, uso interno, inventário ou devolução ao fornecedor) ou ajuste de entrada por inventário, auditado como `STOCK_ADJUSTED`.

Exceções: (E1) ajuste de saída acima do disponível: 409, sem saldo negativo; (E2) ajuste sem motivo: 400.

Pós-condições: saldo, custo médio e extrato atualizados; alerta de estoque mínimo reavaliado.

4.4.4 UC17 — Administrar usuários

Ator principal: Gerente. **Pré-condição:** sessão de gerente.

Fluxo principal: (1) o gerente cadastra usuário com nome, e-mail, papel e senha provisória que atenda à política; (2) o usuário entra e é obrigado a trocar a senha antes de acessar outras rotas.

Fluxos alternativos: alterar papel, desativar, desbloquear ou redefinir senha; cada mudança incrementa a versão de sessão e encerra as sessões abertas do usuário na requisição seguinte (401 `SESSION_REVOKED`).

Exceções: (E1) e-mail duplicado: 409; (E2) rebaixar ou desativar o último gerente ativo: 409 `LAST_MANAGER`; (E3) alterar o próprio papel, situação ou senha pela administração: 409 `SELF_CHANGE_FORBIDDEN`.

Pós-condições: alteração auditada sem senha nem hash no detalhe.

4.5 MATRIZ DE RASTREABILIDADE

O **Quadro** 8 resume a rastreabilidade por grupo de requisitos; a matriz completa por requisito, com critérios, casos de teste, componentes e entidades, está no Apêndice B.

Quadro 8 **–** Resumo da rastreabilidade requisito → componente → dados → testes

Requisitos	Componentes principais	Entidades	Casos de teste (exemplos)
RF-01, RF-02, RF-17, RF-18, RF-24	`SecurityConfig`, `security/*`, `api/users`	`app_users`, `role`, `audit_event`	CT-AUTH-001/002/101/102, CT-SEC-101 a 122, CT-USR-101 a 107, CT-BD-30
RF-06, RF-07, RF-13, RF-15, RF-19, RF-26	`api/sales`, `Pricing`, `RequestHash`	`orders`, `order_item`, `payment`, `stock_balance`, `stock_movement`, `discount_policy`	CT-SALE-101 a 104, CT-STOCK-101 a 107, CT-DISC-101 a 107, BD-19 a BD-29, CC-01
RF-03, RF-20, RF-21, RF-25	`api/customers`, `DocumentFormat`	`customer`, `address`, `v_cliente_resumo`	CT-CRM-101 a 106, CT-CUS-121 a 123, BD-31, CT-DEMO-05/06
RF-04, RF-14, RF-22	`api/backoffice`	`product`, `category`, `stock_entry`, `product_price_history`	CT-PRODUCT-001/002, CT-STOCK-108/109, BD-18, BD-24, RG-03, RG-05
RF-08	`api/sales`, `api/management/RefundService`, legado `PaymentService`	`payment`, `refund`	CT-PAY-001 a 011, BD-22, BD-23
RF-09 a RF-12, RF-23	`api/management`	`monthly_goal`, `seller_goal`, *views* V6	CT-GOAL-001 a 005, CT-MGMT-001/002, BD-01 a BD-05, BD-16, BD-17
RF-16	`frontend/app`, `lib/areas.ts`	—	CT-UI-001 a 003, CT-WEB-103, CT-WEB-120

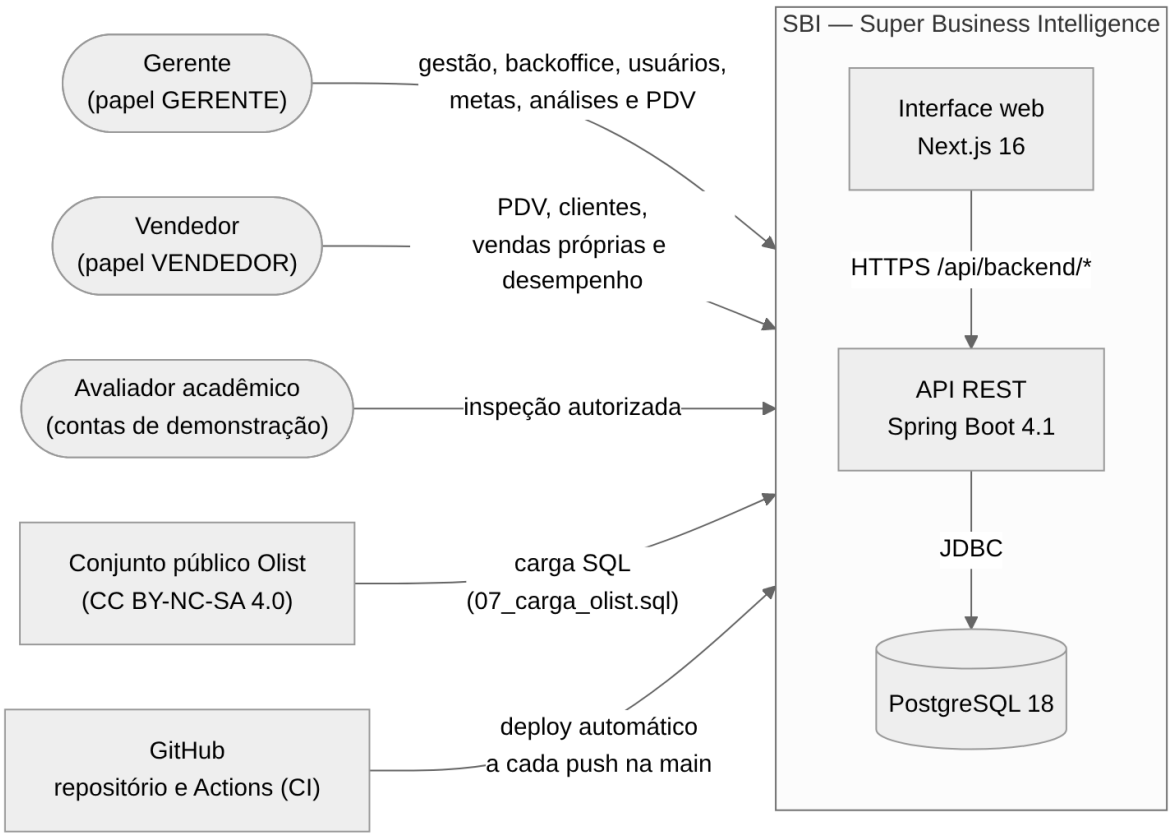
Fonte: elaborado pelos autores a partir de `docs/rastreabilidade.md` e do código [E-36] [E-01].

5 ARQUITETURA E TECNOLOGIAS

5.1 VISÃO ARQUITETURAL

O SBI é uma aplicação web em três camadas físicas: uma interface Next.js executada no navegador e no servidor da Vercel, uma API REST Spring Boot executada em contêiner Docker no Render e um banco PostgreSQL gerenciado no Neon. A **Figura** 2 apresenta o contexto do sistema, com atores e sistemas externos.

Figura 2 **–** Diagrama de contexto do SBI



Fonte: elaborado pelos autores a partir do repositório e da configuração de implantação [E-01] [E-02].

Logicamente, a API segue o padrão em camadas descrito por Fowler (2002): controladores recebem e validam a requisição, serviços aplicam regras e autorização em transação, e repositórios executam SQL. Duas gerações de código convivem: os módulos novos em `api/<módulo>`, que usam `JdbcClient` com SQL explícito e objetos `record`, e o código legado em JPA/Hibernate (`controllers`, `services`, `repositories`, `entities`), mantido para as rotas `/orders`, `/payment`, `/address` e `POST /customers/{id}/address` e ainda usado pelos módulos novos na

leitura de contas para o login (`AppUserRepository`) e na gravação da trilha de auditoria (`AuditEventRepository`) [E-01].

5.2 TECNOLOGIAS UTILIZADAS

As tecnologias foram identificadas nos manifestos de dependências e não presumidas. O **Quadro** 2 (capítulo 3) lista as versões. Três escolhas merecem destaque: Java 25 com Spring Boot 4.1, que exige JDK 25 para compilar e executar; PostgreSQL 18 nos ambientes de integração contínua e produção, confirmado pela versão 18.6 do banco publicado [E-03]; e geração automática de esquemas Zod a partir do contrato OpenAPI pelo orval, que liga a interface ao contrato em tempo de compilação.

5.3 ORGANIZAÇÃO DO PROJETO

O repositório contém 1.245 arquivos versionados, organizados conforme o **Quadro** 9.

Quadro 9 **–** Organização do repositório na versão de referência

Diretório	Conteúdo	Volume
`src/main/java`	API: `api/*` (módulos novos), `config`, `security`, `exceptions`, legado JPA	134 arquivos; 5.952 linhas lógicas
`src/main/resources/db`	Migrações Flyway V1–V11 (`migration`) e massa demo (`demo`)	14 arquivos SQL
`src/test/java`	Testes unitários e de integração	49 arquivos; 44 classes `*Test` e a classe `OrderManagementSpringApplicationTests`
`frontend/`	Next.js: `app` (páginas), `components`, `lib` (API, regras de tela), `tests`, `e2e`	146 arquivos (92 TS/TSX)
`config/openapi`	Contratos versionados (linha de base, v2 a v5) e esquema de validação	9 contratos e 1 esquema
`docs/`	Requisitos, decisões, planos, defeitos, banco, gestão e referências	136 arquivos (110 em Markdown)
`scripts/`	Execução de testes, contrato ao vivo, métricas e carga de dados	22 arquivos (14 `.mjs`, 4 `.ps1`)
`reports/`	Evidências versionadas: contrato, execuções, gestão, métricas e capturas	706 arquivos, incluindo as 20 capturas de tela de `reports/marca/`
`.github/workflows`	Integração contínua e automações do Neon	4 *workflows*

Fonte: elaborado pelos autores; contagens obtidas por `git ls-files` no *commit* `3b0242c` [E-01].

5.4 ARQUITETURA FRONTEND

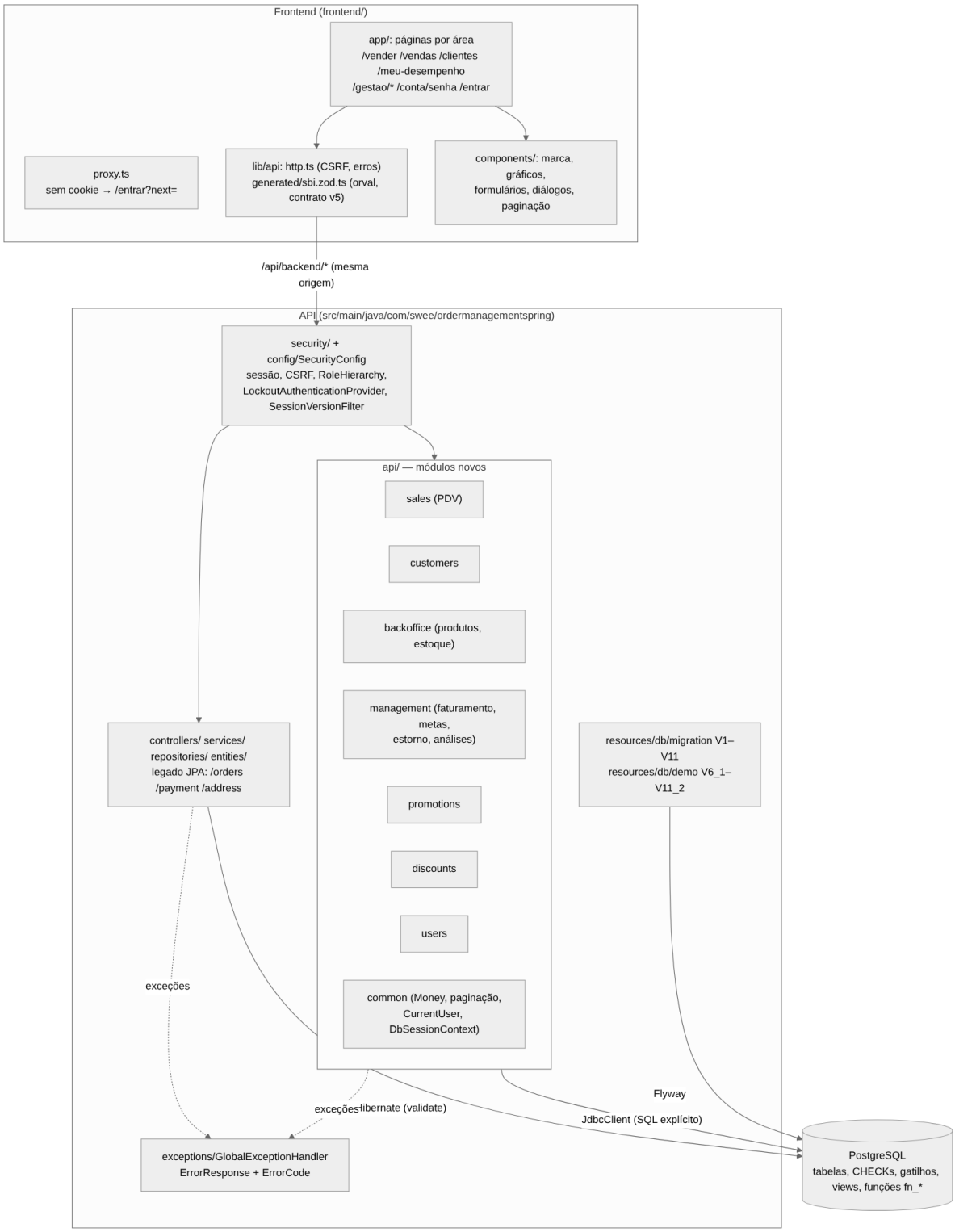
A interface usa o App Router do Next.js 16 com React 19 e TypeScript em modo estrito. As rotas autenticadas ficam no grupo `(app)`: `/vender`, `/vendas`, `/vendas/[id]`, `/clientes`, `/clientes/[id]`, `/meu-desempenho`, `/gestao`, `/gestao/produtos`, `/gestao/promocoes`, `/gestao/estoque`, `/gestao/analises`, `/gestao/usuarios` e `/creditos`, além de `/entrar` e `/conta/senha` [E-01]. O arquivo `proxy.ts` redireciona qualquer rota interna para `/entrar?next=...` quando não há *cookie* de sessão, comportamento confirmado no ambiente publicado [E-04]; a autorização efetiva é sempre da API.

A camada `lib/api` concentra o cliente HTTP, que obtém o *token* CSRF, envia-o no cabeçalho `X-CSRF-TOKEN` em toda mutação e traduz cada `ErrorCode` para mensagem em português; os esquemas de validação de respostas e formulários são gerados do contrato v5 em `lib/api/generated/sbi.zod.ts`.

5.5 ARQUITETURA BACKEND

A **Figura** 3 mostra os componentes da API e suas dependências.

Figura 3 **–** Diagrama de componentes do SBI



Fonte: elaborado pelos autores a partir de `src/main/java` [E-01].

Os módulos de `api/` são: `sales` (PDV), `customers`, `backoffice` (produtos, categorias e estoque), `management` (faturamento, metas, estorno e análises), `promotions`, `discounts` (alçadas), `users` e `common` (dinheiro, paginação, usuário da

sessão e contexto de banco). O componente `DbSessionContext` grava o usuário e o motivo da operação em variáveis de sessão do PostgreSQL (`set_config('app.user_id', ..., true)`), que os gatilhos usam para preencher autoria e histórico de preço [E-16].

5.6 COMUNICAÇÃO ENTRE COMPONENTES

O navegador conversa apenas com a Vercel. As chamadas à API usam o prefixo `/api/backend/*`, que o `next.config.mjs` reescreve para a origem configurada em `BACKEND_ORIGIN`; a resposta observada no ambiente publicado traz o cabeçalho `x-render-origin-server: Render`, o que confirma a cadeia Vercel → Render [E-04]. Como navegador, interface e API compartilham a mesma origem do ponto de vista do navegador, não há CORS, e o *cookie* de sessão permanece `HttpOnly`, `Secure` e `SameSite=Lax`. A API acessa o banco por JDBC com TLS, em conexão direta, sem o *pooler* do Neon, porque o Flyway e as variáveis de sessão usadas pelos gatilhos exigem sessão de banco estável [E-41].

5.7 APIS E ENDPOINTS

O contrato vigente é o `target-loja-gestao-openapi-2026-10-14-v5.json`, em OpenAPI 3.1.0, com 46 caminhos, 58 operações e 113 esquemas de componentes, todas as operações marcadas como `implemented` [E-06]. O **Quadro** 10 agrupa as operações por recurso.

Quadro 10 **–** Operações do contrato v5 por recurso e papel

Recurso	Operações	Papel mínimo
`/auth`	`GET /auth/csrf`, `POST /auth/login`, `POST /auth/logout`, `GET /auth/me`, `PUT /auth/password`	público (csrf, login, logout); autenticado (me, password)
`/sales`	catálogo, cotação, criar, listar e obter pedido, confirmar pagamento; aprovação de desconto	VENDEDOR; aprovação: GERENTE
`/orders/{id}/cancel`	cancelar pedido pendente	VENDEDOR (dono)
`/customers`	listar, criar, obter, atualizar, excluir, histórico e resumo	VENDEDOR; exclusão: GERENTE
`/performance/me`, `/promotions/active`	desempenho próprio, promoções vigentes	VENDEDOR
`/products`, `/categories`	produtos: cadastro, consulta, edição, exclusão de não vendidos, preço com motivo, situação e histórico de preço; categorias: listar, criar e renomear	GERENTE
`/stock`	posição, entradas, ajustes, mínimo e movimentos	GERENTE

Recurso	Operações	Papel mínimo
`/promotions`, `/goals`, `/management`, `/analytics`	promoções, metas, resumo, faturamento, análises de vendas e estoque	GERENTE
`/payment/{id}/refund`	estorno	GERENTE
`/users`, `/discount-policy`	administração de usuários e alçadas	GERENTE

Fonte: elaborado pelos autores a partir do contrato v5 [E-06].

O OpenAPI gerado em tempo de execução pelo Springdoc e publicado em `/v3/api-docs` expõe 76 operações: as 58 do contrato e 18 rotas legadas de endereço, pedido e pagamento [E-05]. A comparação realizada nesta auditoria encontrou as 58 operações do contrato no documento publicado, com os mesmos códigos de sucesso, mas 109 dos 201 códigos de erro previstos no contrato (54,2%) não aparecem no documento gerado (39 respostas 400, 31 respostas 404, 22 respostas 409 e 17 respostas 403) [E-05]. A implementação pode responder corretamente a esses erros, como indicam os testes da equipe, mas o Swagger publicado não os promete; o achado é registrado como A-02 e discutido no capítulo 14.

5.8 SERVIÇOS E INTEGRAÇÕES

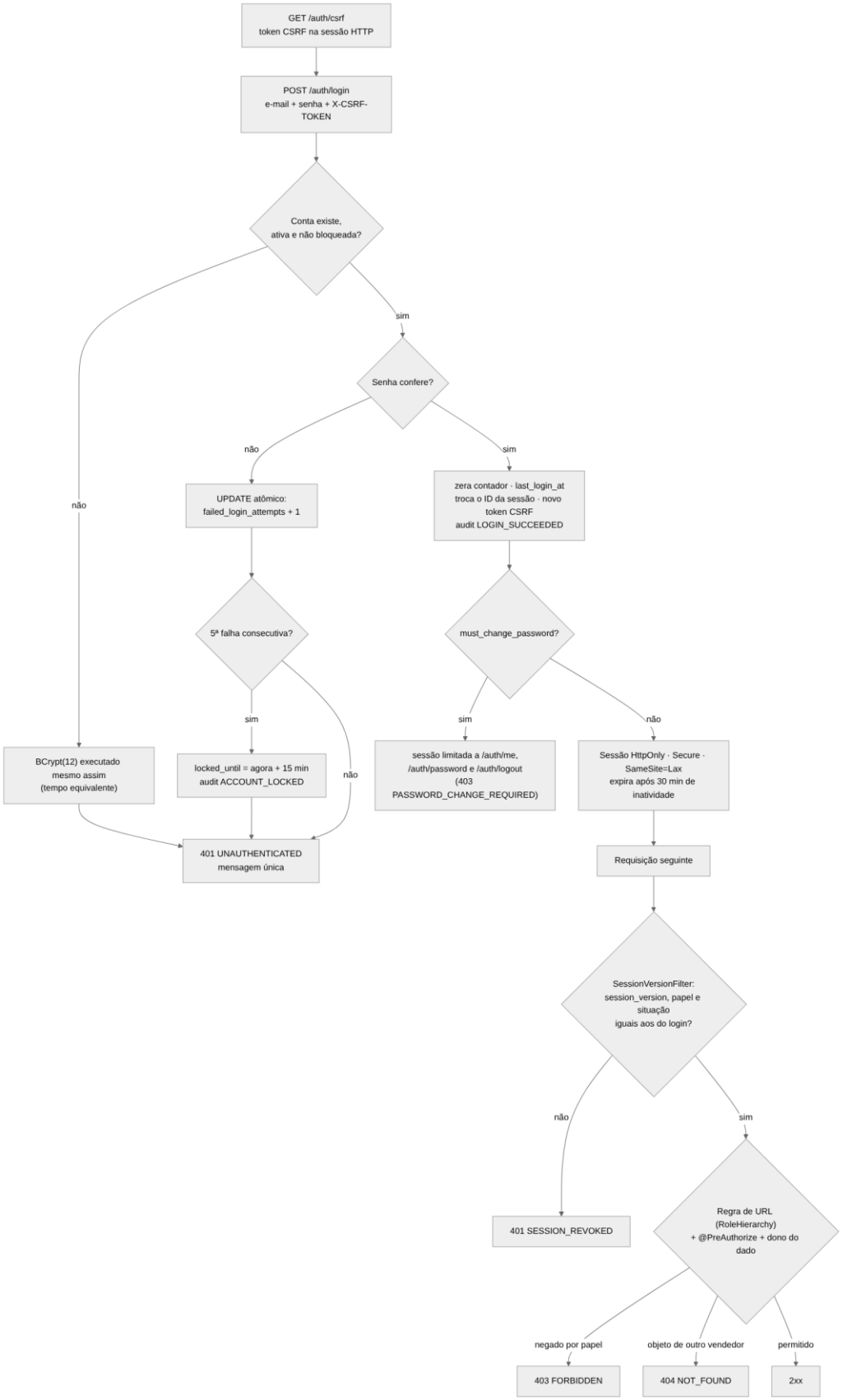
As integrações externas são de infraestrutura: GitHub (código, *pull requests* e Actions), Vercel (interface), Render (API), Neon (banco e *branches* por *pull request*) e o conjunto público Olist, carregado por SQL. Não há integração com gateway de pagamento, emissor fiscal ou serviço de e-mail; os pagamentos são simulados e confirmados pelo próprio usuário no PDV.

5.9 AUTENTICAÇÃO

A autenticação é por sessão no servidor, com proteção CSRF por *token* sincronizador armazenado na sessão (`HttpSessionCsrfTokenRepository`) [E-17]. No login, a estratégia composta troca o identificador da sessão (proteção contra fixação de sessão) e renova o *token* CSRF. As senhas são armazenadas com BCrypt de custo 12. O provedor `LockoutAuthenticationProvider` executa o BCrypt mesmo para conta inexistente, inativa ou bloqueada, para que o tempo de resposta não revele o estado da conta, e o contador de falhas é incrementado por `UPDATE` atômico,

o que evita condição de corrida entre tentativas paralelas [E-17]. A sessão expira após 30 minutos de inatividade. O fluxo completo está na **Figura** 4.

Figura 4 **–** Fluxo de autenticação, bloqueio e revogação de sessão



Fonte: elaborado pelos autores a partir de `SecurityConfig`, `LockoutAuthenticationProvider` e `SessionVersionFilter` [E-17].

5.10 CONTROLE DE PERMISSÕES

O controle de acesso aplica três barreiras independentes:

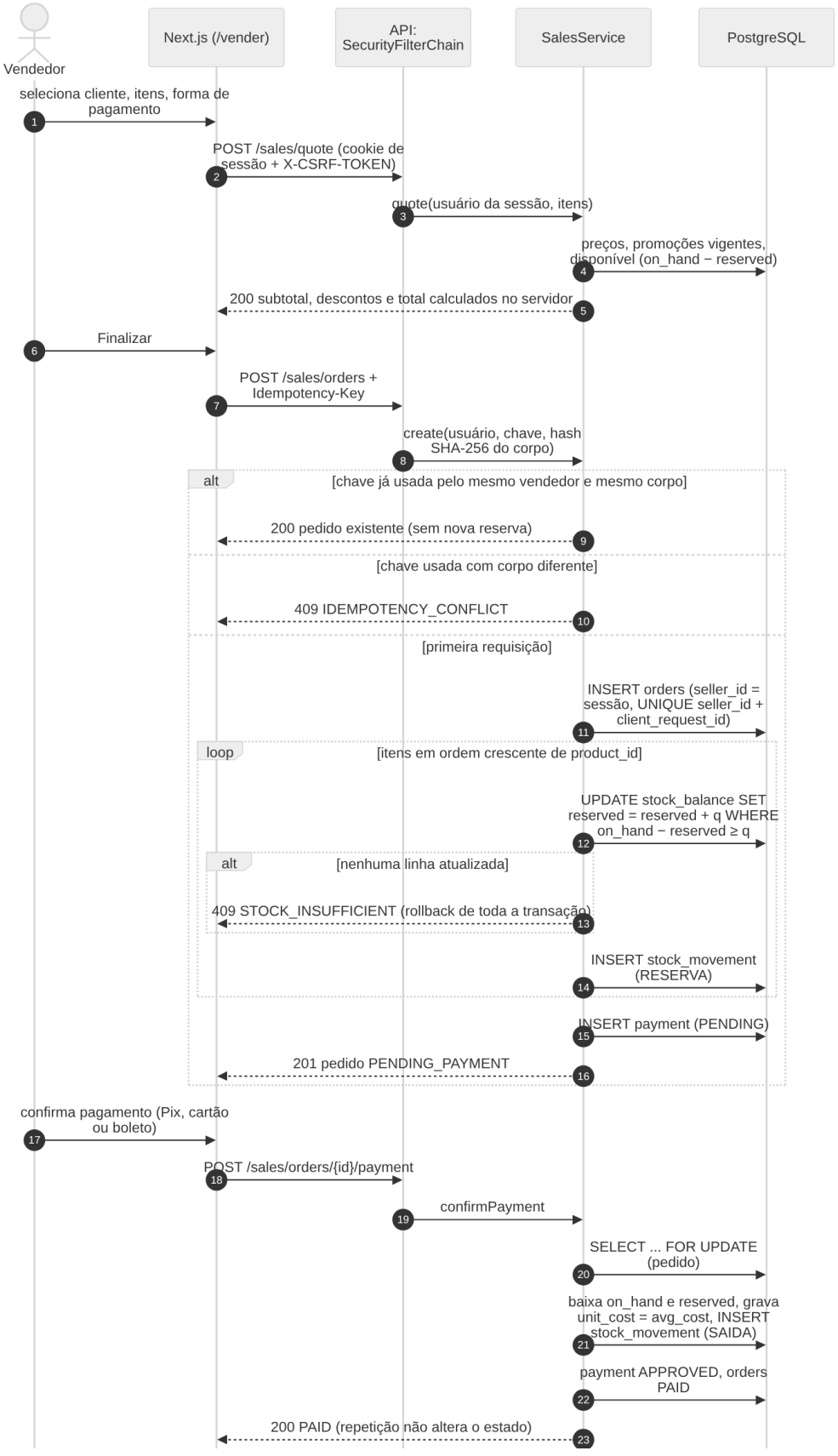
a) **URL**: regras do `SecurityFilterChain` com hierarquia `ROLE_GERENTE > ROLE_VENDEDOR` e negação por padrão (`anyRequest().denyAll()`);
b) **método**: `@PreAuthorize` nos serviços dos módulos novos e nos controladores legados, com o mesmo `RoleHierarchy`;
c) **dono do dado**: o serviço compara o vendedor do pedido com o usuário da sessão e responde 404 quando o objeto pertence a outro vendedor, sem revelar sua existência.

A matriz de acesso é executável: o arquivo `src/test/resources/security/access-matrix.csv` alimenta o `AccessMatrixTest`, que falha se surgir rota sem linha na matriz; as 82 linhas do arquivo, uma delas com curinga para cinco métodos, geram 258 casos parametrizados (86 combinações de rota e método × 3 perfis), que, somados a dois testes adicionais, correspondem aos 260 casos documentados pela equipe [E-22]. No banco, o script `03_rbac.sql` demonstra defesa em profundidade com papéis de banco, concessões por coluna e políticas de segurança em nível de linha (RLS) para pedidos, itens e pagamentos; esse mecanismo é usado na demonstração por `psql` e no job `bd`, não pela API [E-11].

5.11 FLUXO DE DADOS

O fluxo de uma venda, desde a cotação até a confirmação, está na **Figura** 5. Os valores monetários trafegam como objeto `Money` (`amount` e `currency`), calculados no servidor; o vendedor nunca envia preço, total ou identificação do vendedor.

Figura 5 **–** Diagrama de sequência da venda no PDV



Fonte: elaborado pelos autores a partir de `SalesService` e `SalesRepository` [E-19].

5.12 TRATAMENTO DE EXCEÇÕES

Todo erro da API segue o esquema `ErrorResponse`, com os campos `timestamp` (RFC 3339 com deslocamento), `status`, `error`, `code`, `message` e `details` [E-18]. O `GlobalExceptionHandler` converte exceções de validação, de leitura do corpo, de tipo de parâmetro, de método e de mídia em 400, 405 ou 415; violações de integridade do banco em 409 com código específico (`DUPLICATE_DOCUMENT`, `IDEMPOTENCY_CONFLICT`, `RESOURCE_IN_USE`, `STOCK_INSUFFICIENT`); e qualquer exceção inesperada em 500 `INTERNAL_ERROR` sem *stack trace*, SQL ou nome de classe. Os erros gerados antes da camada MVC, como 401 e 403 dos filtros de segurança, são escritos pela classe `SecurityJson` no mesmo formato. O **Quadro** 11 relaciona os vinte códigos.

Quadro 11 **–** Códigos de erro estáveis da API

HTTP	Códigos
400	`VALIDATION_FAILED`, `MALFORMED_REQUEST`
401	`UNAUTHENTICATED`, `SESSION_REVOKED`
403	`FORBIDDEN`, `PASSWORD_CHANGE_REQUIRED`, `DISCOUNT_ABOVE_LIMIT`
404	`NOT_FOUND`
405	`METHOD_NOT_ALLOWED`
409	`STOCK_INSUFFICIENT`, `IDEMPOTENCY_CONFLICT`, `STATE_CONFLICT`, `DUPLICATE_DOCUMENT`, `RESOURCE_IN_USE`, `CONCURRENT_UPDATE`, `LAST_MANAGER`, `SELF_CHANGE_FORBIDDEN`, `BELOW_COST`
415	`UNSUPPORTED_MEDIA_TYPE`
500	`INTERNAL_ERROR`

Fonte: elaborado pelos autores a partir de `ErrorCode`, `GlobalExceptionHandler` e `SecurityJson` [E-18].

As respostas 401 observadas no ambiente publicado seguem exatamente esse formato, inclusive para uma rota inexistente, que responde 401 a um usuário anônimo em vez de 404 e, assim, não revela quais rotas existem [E-04].

5.13 GERENCIAMENTO DE ESTADO

No servidor, o estado de autenticação vive na sessão HTTP, e o estado de negócio vive no banco. Na interface, o TanStack Query mantém o cache das consultas; o painel do gerente recarrega os dados a cada 15 segundos apenas com a aba visível (`refetchInterval: 15_000`, `refetchIntervalInBackground: false`) [E-01]. O carrinho do PDV é persistido em `sessionStorage` por usuário, o que preserva

itens e desconto em uma recarga da página sem compartilhar dados entre usuários do mesmo navegador.

5.14 CONFIGURAÇÕES E DEPENDÊNCIAS

As configurações estão em `application.properties` e nos perfis `demo`, `migration`, `contract` e `integration`. Destacam-se: `server.port=${PORT:8080}`; senha do banco obrigatória por variável de ambiente, com o `DatasourcePasswordGuard` interrompendo a inicialização quando ausente; Flyway como única fonte do esquema e Hibernate em `validate`; `fail-on-unknown-properties=true` e `accept-float-as-int=false` no Jackson, que transformam propriedades desconhecidas e decimais em campos inteiros em erro 400 [E-01]. O perfil padrão mantém `spring.jpa.show-sql=true`, que registra cada instrução SQL em log; no Render, a variável `SPRING_JPA_SHOW_SQL=false` desativa esse comportamento [E-41].

5.15 DECISÕES ARQUITETURAIS

As decisões estão registradas em `docs/decisoes/`. O **Quadro** 12 resume as que mais influenciam a arquitetura.

Quadro 12 **–** Principais decisões arquiteturais registradas

Decisão	Escolha	Justificativa registrada
Autenticação	Sessão no servidor + CSRF, sem *token* no JavaScript	Reduz exposição de credenciais a XSS e simplifica revogação
Integração interface–API	Mesma origem por *rewrite*	Evita CORS e mantém o *cookie* `HttpOnly`
Fonte do esquema	Flyway único; Hibernate em `validate`	Migrações revisáveis e reprodutíveis
Acesso a dados nos módulos novos	`JdbcClient` com SQL explícito; *views* não viram entidade	Controle do SQL e das projeções sem custo para o vendedor
Dinheiro	NUMERIC(12,2), HALF_UP, objeto `Money`	Precisão decimal e arredondamento comercial
Contrato	*Contract-first*: operação `planned` só vira `implemented` com teste	Conformidade verificável
Banco de testes	PostgreSQL 18 descartável por Docker Compose, sem Testcontainers (D-EVO-38)	Mesmo motor da produção com menor dependência de ferramenta

Fonte: elaborado pelos autores a partir de `docs/decisoes/` [E-42].

5.16 LIMITAÇÕES IDENTIFICADAS

As limitações arquiteturais observadas são: convivência de dois estilos de persistência (JPA no legado e `JdbcClient` nos módulos novos), que aumenta o custo de manutenção; rotas legadas fora do contrato v5, ainda expostas ao gerente; sessão em memória da instância da API, que impede escalonamento horizontal sem sessão compartilhada; plano gratuito do Render, que suspende a API após inatividade e exige de 1 a 3 minutos na primeira requisição, segundo a equipe [E-41]; e o OpenAPI gerado que não documenta parte das respostas de erro do contrato (A-02).

6 MODELAGEM E IMPLEMENTAÇÃO DO BANCO DE DADOS

Este capítulo documenta o banco de dados com a profundidade exigida pela disciplina de Banco de Dados. As estruturas descritas foram extraídas do catálogo de um esquema criado nesta auditoria pela aplicação, em ordem, das migrações da versão de referência, e conferidas contra o banco publicado no Neon [E-15] [E-03]. Nenhuma tabela, campo ou relacionamento foi inferido sem correspondência no esquema.

6.1 BANCO DE DADOS UTILIZADO

O SGBD é o PostgreSQL. A versão em produção é a 18.6, no serviço gerenciado Neon (projeto `wispy-glade-88962628`, região AWS us-east-2, banco `neondb`) [E-03]. A integração contínua e os testes de integração usam a imagem oficial `postgres:18` em contêiner descartável [E-20]. O esquema `public` contém 19 tabelas de negócio, 162 colunas, 18 *views*, 13 funções, 8 gatilhos e 48 índices, além da tabela de controle `flyway_schema_history`; o esquema `stg_olist` contém seis tabelas de preparação usadas apenas na carga da massa de demonstração [E-15] [E-03].

6.2 JUSTIFICATIVA TÉCNICA DA TECNOLOGIA

A escolha do PostgreSQL é coerente com os requisitos observados por cinco razões verificáveis no código. Primeiro, o tipo `NUMERIC(12,2)` oferece aritmética decimal exata para valores monetários (RN-03). Segundo, restrições `CHECK`, gatilhos em PL/pgSQL e gatilhos de restrição adiáveis (`DEFERRABLE INITIALLY DEFERRED`) permitem impor no banco regras que envolvem várias linhas, como a soma das metas (RN-10) e a existência de pelo menos um gerente ativo (RN-18). Terceiro, a semântica de `UPDATE` sob *Read Committed* permite a reserva condicional de estoque sem bloqueio de tabela (seção 6.18). Quarto, a segurança em nível de linha (RLS) e as concessões por coluna permitem demonstrar defesa em profundidade (seção 6.21). Quinto, o tipo `jsonb` armazena o detalhe dos eventos de auditoria (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2025).

6.3 LEVANTAMENTO DAS ENTIDADES

As entidades foram levantadas a partir das migrações e agrupadas em quatro subdomínios, conforme o **Quadro** 13.

Quadro 13 **–** Entidades do banco por subdomínio

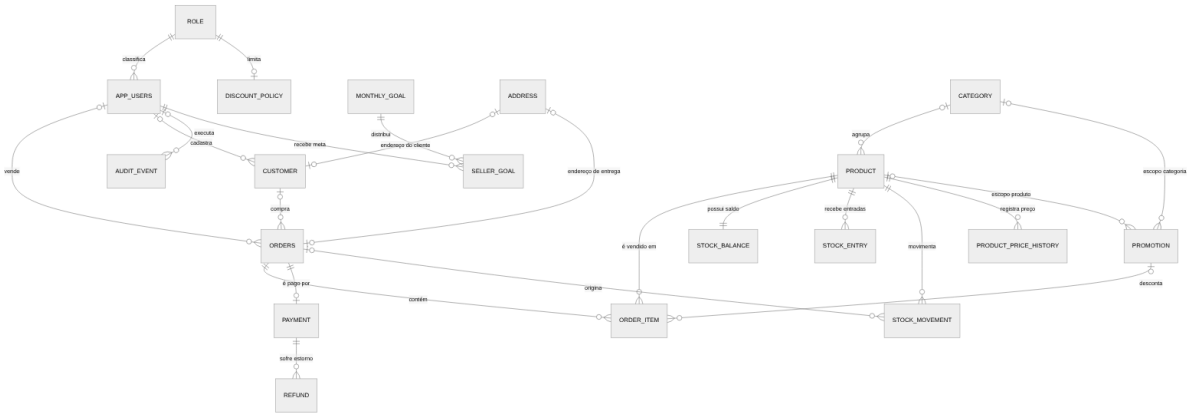
Subdomínio	Tabelas	Origem
Vendas	`customer`, `address`, `orders`, `order_item`, `payment`, `refund`	V1 a V5, V7, V11
Catálogo e estoque	`category`, `product`, `product_price_history`, `promotion`, `stock_balance`, `stock_entry`, `stock_movement`	V1, V5
Usuários, metas e políticas	`role`, `app_users`, `monthly_goal`, `seller_goal`, `discount_policy`	V2, V5, V8, V11
Auditoria	`audit_event`	V5, V8, V11

Fonte: elaborado pelos autores a partir das migrações [E-16].

6.4 MODELO CONCEITUAL

A **Figura** 6 apresenta o modelo conceitual em notação de pés de galinha (*crow's foot*), com as entidades e os relacionamentos do domínio, sem atributos.

Figura 6 **–** Modelo conceitual (diagrama entidade-relacionamento)



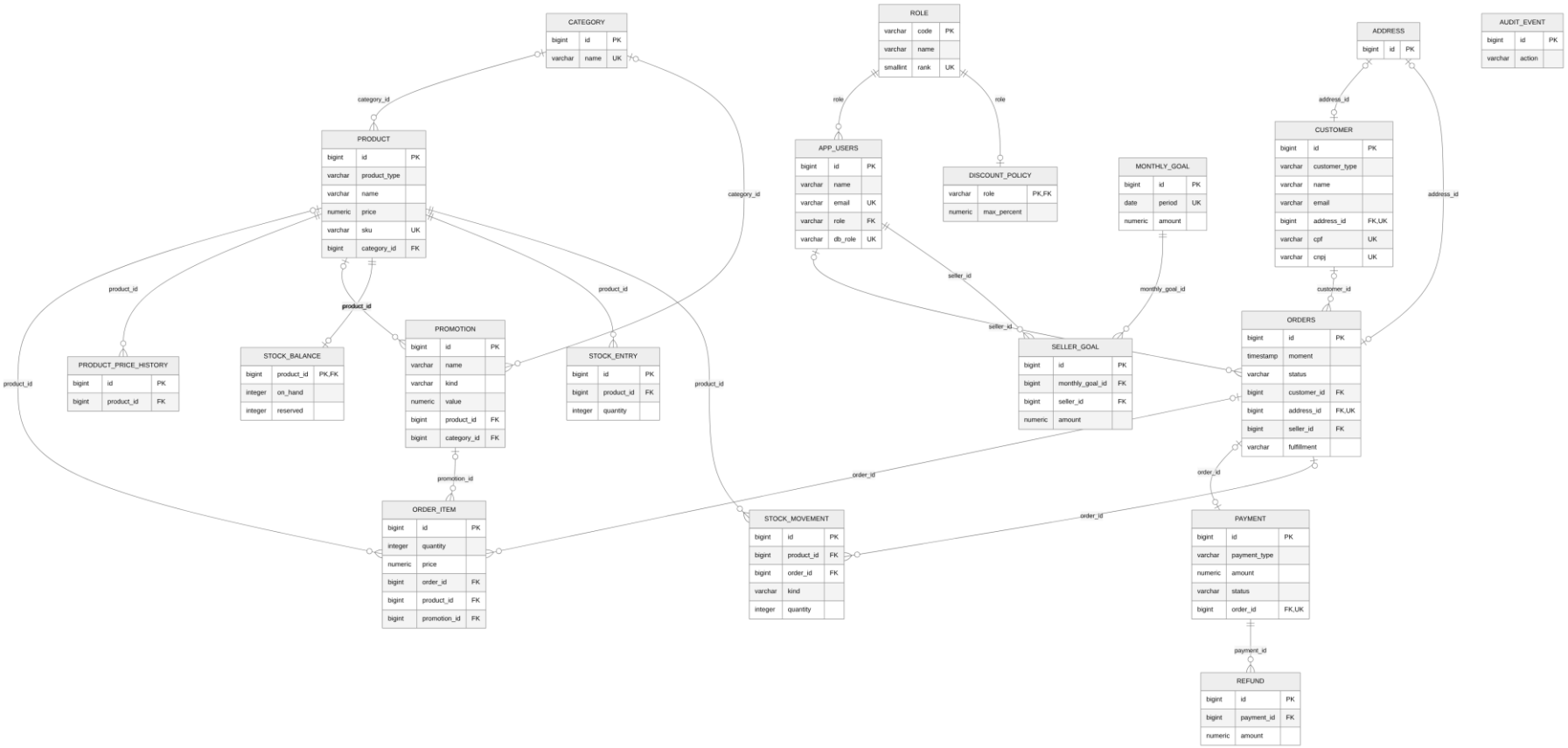
Fonte: elaborado pelos autores a partir do esquema migrado [E-15].

O centro do modelo é o pedido (`ORDERS`), que relaciona cliente, vendedor, itens, pagamento, endereço de entrega e movimentos de estoque. O produto concentra o subdomínio de catálogo e estoque, com saldo único (`STOCK_BALANCE`, relação 1:1), entradas, movimentos e histórico de preço. O usuário participa como vendedor, cadastrador, autor de eventos de auditoria e destinatário de metas.

6.5 MODELO LÓGICO

O modelo lógico da **Figura** 7 mostra as relações com chaves primárias, estrangeiras e únicas e os atributos principais. Para legibilidade, as chaves estrangeiras de autoria (`created_by`, `changed_by`, `updated_by`, `actor_id` e `discount_approved_by`), todas referenciando `app_users`, foram omitidas; elas aparecem no modelo físico.

Figura 7 – Modelo lógico relacional (sem chaves de autoria)

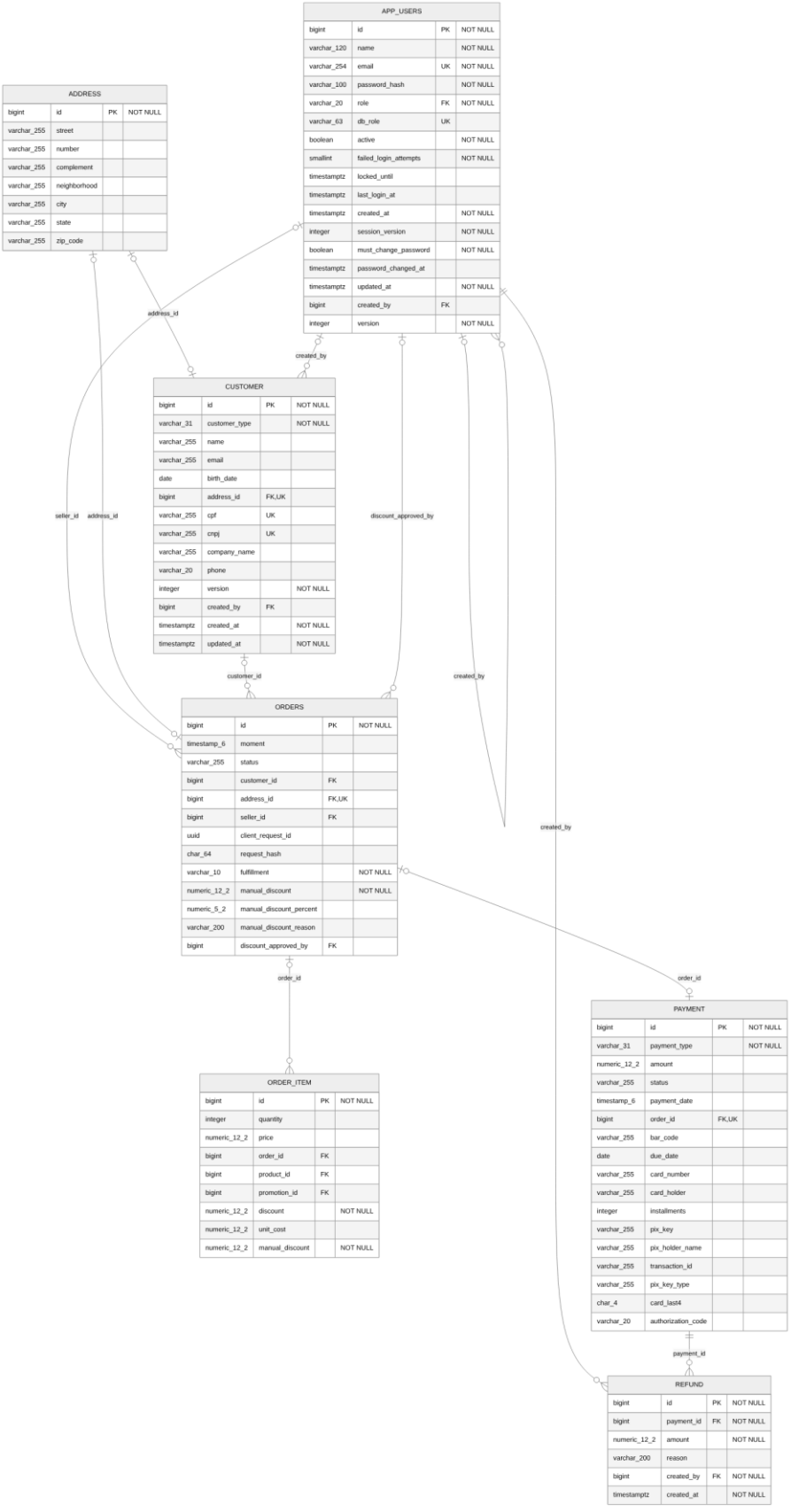


Fonte: elaborado pelos autores a partir do catálogo do esquema [E-15].

6.6 MODELO FÍSICO

O modelo físico, com todos os atributos, tipos do PostgreSQL e indicação de nulidade, foi dividido em três diagramas por subdomínio, apresentados no Apêndice C. A **Figura** 8 mostra o subdomínio de vendas.

Figura 8 **–** Modelo físico do subdomínio de vendas



Fonte: elaborado pelos autores a partir do catálogo do esquema [E-15].

6.7 DIAGRAMA ENTIDADE-RELACIONAMENTO

O diagrama entidade-relacionamento completo corresponde à combinação das figuras **Figura** 6 (entidades e relacionamentos) e **Figura** 7 (chaves). As 32 chaves estrangeiras do esquema estão enumeradas no Apêndice D, com tabela de origem, coluna e tabela referenciada. Os arquivos-fonte editáveis dos diagramas, em Mermaid, acompanham a entrega no diretório `diagramas/fonte`.

6.8 DICIONÁRIO DE DADOS

O **Quadro** 14 resume o dicionário por tabela. O dicionário completo, com as 162 colunas, tipo, nulidade, valor padrão, chave e descrição, está no Apêndice D e na planilha `SBI_DICIONARIO_DE_DADOS.xlsx`.

Quadro 14 **–** Resumo do dicionário de dados por tabela

Tabela	Descrição	Colunas	Chave primária	Chaves estrangeiras
address	Endereço de cliente ou de entrega	8	id	—
app_users	Conta de gerente ou vendedor	17	id	role, created_by
audit_event	Trilha de auditoria	7	id	actor_id
category	Categoria de produto	2	id	—
customer	Cliente PF ou PJ (tabela única)	14	id	address_id, created_by
discount_policy	Alçada de desconto por papel	5	role	role, updated_by
monthly_goal	Meta mensal da loja	5	id	created_by
order_item	Item do pedido	9	id	order_id, product_id, promotion_id
orders	Pedido	13	id	customer_id, address_id, seller_id, discount_approved_by
payment	Pagamento (tabela única)	17	id	order_id
product	Produto físico ou digital	12	id	category_id
product_price_history	Histórico de preço	7	id	product_id, changed_by
promotion	Promoção	11	id	product_id, category_id, created_by
refund	Estorno	6	id	payment_id, created_by
role	Papel de acesso	3	code	—
seller_goal	Meta individual	4	id	monthly_goal_id, seller_id
stock_balance	Saldo de estoque	6	product_id	product_id
stock_entry	Entrada de estoque	7	id	product_id, created_by

Tabela	Descrição	Colunas	Chave primária	Chaves estrangeiras
stock_movement	Movimento de estoque	9	id	product_id, order_id, created_by

Fonte: elaborado pelos autores a partir do catálogo do esquema [E-15].

6.9 TIPOS DE DADOS

Os tipos foram escolhidos de acordo com a natureza do dado. Identificadores usam `bigint` com `GENERATED BY DEFAULT AS IDENTITY`, padrão SQL que substitui sequências implícitas. Valores monetários usam `numeric(12,2)` desde a V4, que converteu os antigos `double precision` com arredondamento HALF_UP e falha explícita para valores acima do limite (correção do defeito D020). Os percentuais do desconto manual e das alçadas usam `numeric(5,2)`; o valor da promoção, que pode ser percentual ou monetário, usa `numeric(12,2)`. Instantes de auditoria e controle usam `timestamptz`; o momento do pedido (`orders.moment`) e a data do pagamento usam `timestamp` sem fuso, gravado no horário da loja (America/Fortaleza). A chave de idempotência usa `uuid`, o hash do corpo usa `char(64)` e o detalhe da auditoria usa `jsonb`. Textos legados mantêm `varchar(255)`, herdados do mapeamento JPA original.

6.10 CHAVES PRIMÁRIAS

As 19 tabelas têm chave primária. Dezesseis usam chave substituta `id bigint`; duas usam chave natural: `role.code` (código do papel) e `discount_policy.role`, que é ao mesmo tempo chave primária e estrangeira para `role`, caracterizando relação 1:1 opcional. A décima nona, `stock_balance`, usa `product_id` como chave primária e estrangeira, garantindo exatamente uma linha de saldo por produto, criada automaticamente pelo gatilho `product_stock_row`.

6.11 CHAVES ESTRANGEIRAS

O esquema possui 32 chaves estrangeiras [E-15]. Nenhuma usa exclusão em cascata, exceto `seller_goal.monthly_goal_id`, com `ON DELETE CASCADE`, porque a distribuição só existe em função da meta mensal. A ausência de cascata nas demais é uma decisão de integridade: um cliente com pedidos ou um produto já vendido não

pode ser excluído, e a API responde 409 (`RESOURCE_IN_USE` no caso do cliente). Categorias não têm operação de exclusão no contrato v5. Essa recusa foi observada na execução do roteiro de CRUD desta auditoria, em que a exclusão de um cliente com pedidos foi rejeitada pela restrição `orders_client_id_fkey` [E-12].

6.12 CARDINALIDADE E RELACIONAMENTOS

O **Quadro** 15 apresenta os relacionamentos principais com a cardinalidade imposta pelo esquema.

Quadro 15 **–** Cardinalidades impostas pelo esquema

Relacionamento	Cardinalidade	Mecanismo
Cliente — pedido	1:N (pedido pode não ter cliente nas rotas legadas)	FK anulável `orders.customer_id`
Pedido — item	1:N	FK `order_item.order_id`
Pedido — pagamento	1:1 opcional	FK única `payment.order_id`
Pagamento — estorno	1:N	FK `refund.payment_id`
Produto — saldo	1:1 obrigatório	PK/FK `stock_balance.product_id` + gatilho
Produto — entrada, movimento, histórico de preço	1:N	FKs `product_id`
Categoria — produto	1:N opcional	FK anulável `product.category_id`
Usuário (vendedor) — pedido	1:N opcional	FK anulável `orders.seller_id`
Meta mensal — meta do vendedor	1:N, soma igual à meta	FK + gatilho de restrição adiado
Papel — usuário	1:N	FK `app_users.role`
Endereço — cliente; endereço — pedido	1:1 opcional	FKs únicas `address_id`

Fonte: elaborado pelos autores a partir do catálogo do esquema [E-15].

A anulabilidade de `orders.seller_id` e de `orders.customer_id` é herança do modelo legado: pedidos anteriores à autenticação não têm vendedor, e a política registrada é não atribuí-los retroativamente sem evidência [E-39]. Nos pedidos criados pelo PDV, o vendedor é sempre preenchido pela sessão.

6.13 RESTRIÇÕES DE INTEGRIDADE

O esquema declara 52 restrições `CHECK`, 13 restrições `UNIQUE`, 19 chaves primárias, 32 chaves estrangeiras e 3 gatilhos de restrição [E-15]. O **Quadro** 16 agrupa as restrições de verificação por finalidade.

Quadro 16 **–** Restrições de verificação agrupadas por finalidade

Finalidade	Exemplos	Regra
Domínios fechados	`ck_orders_status`, `ck_payment_status`, `ck_payment_type`, `ck_stock_movement_kind`, `ck_audit_event_action`	Valores enumerados
Valores não negativos ou positivos	`ck_product_price`, `ck_order_item_qty`, `ck_payment_amount`, `on_hand >= 0`	RN-03, RN-04
Coerência entre colunas	`ck_customer_document_type`, `ck_orders_delivery_address`, `ck_orders_request_pair`, `ck_payment_card_last4_type`	RN-14, RN-24, RN-06
Formato	`ck_customer_cpf`, `ck_customer_cnpj`, `ck_orders_request_hash`, `ck_product_sku_upper`	RN-14, RN-06
Estoque	`ck_stock_reserved_le_on_hand`, `ck_stock_movement_reason_required`, `ck_stock_movement_in_reason`	RN-04, RN-20
Desconto	`ck_order_item_discount`, `ck_order_item_manual_discount`, `ck_orders_manual_discount_reason`, `ck_promotion_percent`	RN-11, RN-12
Dados sensíveis	`ck_payment_card_number_absent`	RN-19

Fonte: elaborado pelos autores a partir das migrações [E-16].

Regras que dependem de outras linhas ou tabelas são impostas por gatilhos: autor gerente da promoção (`promotion_only_manager`), meta individual apenas para vendedor (`seller_goal_is_seller`), soma das metas no COMMIT (`seller_goal_sum` e `monthly_goal_sum`, adiados), pelo menos um gerente ativo (`keep_one_active_manager`, adiado e serializado por *advisory lock*), histórico de preço (`product_price_history`), linha de saldo por produto (`product_stock_row`) e aplicação da entrada com custo médio (`stock_entry_apply`). O **Quadro** 17 reproduz o cálculo do custo médio ponderado.

Quadro 17 **–** Gatilho de entrada de estoque com custo médio ponderado (trecho da V5)

```
UPDATE stock_balance
   SET avg_cost = CASE
                    WHEN NEW.unit_cost IS NULL THEN NULL
                    WHEN b.on_hand = 0 THEN NEW.unit_cost
                    WHEN b.avg_cost IS NULL THEN NULL
                    ELSE round((b.on_hand * b.avg_cost + NEW.quantity * NEW.unit_cost)
                               / (b.on hand + NEW.quantity), 2)
                  END,
       on_hand = on_hand + NEW.quantity,
       updated_at = now()
 WHERE product_id = NEW.product_id;
```

Fonte: `src/main/resources/db/migration/V5__catalogo_estoque_metas_promocoes.sql` [E-16].

A propagação de nulo é uma escolha deliberada: se uma entrada não informa custo, o custo médio passa a ser desconhecido, e a margem correspondente é exibida como “Não informado”, nunca como zero (RN-08). Nesta auditoria, BD-18 e RG-05 confirmaram o cálculo do custo médio ponderado com custos conhecidos, e

BD-07 confirmou a margem nula para item vendido sem custo [E-11]; a propagação de nulo a partir de uma entrada sem custo não é exercitada pelos testes reexecutados.

6.14 NORMALIZAÇÃO

A análise de normalização foi feita tabela a tabela sobre o esquema final.

Primeira forma normal (1FN). Todas as colunas armazenam valores atômicos. A única coluna estruturada, `audit_event.detail` (`jsonb`), contém metadados de auditoria que não são consultados por partes nem participam de regras de negócio; por isso, não compromete a 1FN no uso efetivo.

Segunda forma normal (2FN). As tabelas com chave substituta simples não podem ter dependência parcial. As únicas chaves candidatas compostas são `uq_seller_goal (monthly_goal_id, seller_id)` e `uq_orders_seller_request (seller_id, client_request_id)`; em `seller_goal`, o atributo `amount` depende do par (meta, vendedor), e em `orders` os atributos dependem do pedido, não de parte da chave candidata. O esquema está em 2FN.

Terceira forma normal (3FN). Não foram encontradas dependências transitivas não justificadas. Os atributos descritivos de categoria, papel e cliente estão em tabelas próprias e são referenciados por chave.

Redundâncias controladas. Quatro redundâncias são intencionais e documentadas:

- `order_item.price` copia o preço vigente no momento da venda, porque o preço do produto muda e a venda deve preservar o valor histórico (RN-21; teste BD-25);
- `order_item.unit_cost` copia o custo médio no momento da confirmação, para que a margem histórica não mude com entradas futuras;
- `stock_balance.on_hand` e `reserved` são saldos derivados que poderiam ser calculados a partir de `stock_movement`; são mantidos por desempenho e protegidos por CHECKs e por atualização transacional;
- `payment.amount` repete o total do pedido; a igualdade é conferida pela aplicação (correção do defeito D012).

Herança em tabela única. As tabelas `customer`, `product` e `payment` usam uma tabela por hierarquia, com discriminador (`customer_type`, `product_type`,

`payment_type`) e colunas específicas anuláveis. Essa forma não viola a 3FN, mas gera colunas que só fazem sentido para um subtipo; o esquema compensa com CHECKs de coerência, como `ck_customer_document_type`. Das 162 colunas, 77 aceitam nulo, grande parte por essa herança e por compatibilidade com o legado [E-15].

Inconsistência de nomenclatura. A V3 renomeou a tabela `client` para `customer`, mas preservou os nomes das restrições originais (`client_pkey`, `client_cpf_key`, `orders_client_id_fkey`). O efeito é apenas de legibilidade das mensagens de erro e foi mantido para não editar migração publicada.

6.15 CONSULTAS SQL

As consultas analíticas da aplicação estão nas *views* da V6, V9 e V11, que concentram as regras de cálculo e 13 delas são lidas diretamente pela API por `JdbcClient`; as demais (`v_catalogo_vendedor`, `v_evolucao_semanal`, `v_minha_evolucao_semanal`, `v_minhas_vendas` e `v_resumo_loja`) servem ao pacote SQL, à demonstração de permissões por `psql` ou a outras *views*. O **Quadro** 18 mostra a *view* que define o realizado, base de faturamento, metas e resumo do cliente.

Quadro 18 **–** View de lançamentos que compõem o realizado (V6)

```
CREATE OR REPLACE VIEW v_lancamento_realizado AS
SELECT o.seller_id, p.payment_date::date AS data, o.id AS order_id, p.payment_type,
       (SELECT sum(oi.price * oi.quantity - oi.discount)
          FROM order_item oi WHERE oi.order_id = o.id) AS valor,
       'VENDA'::varchar AS tipo
  FROM orders o JOIN payment p ON p.order_id = o.id
 WHERE p.status = 'APPROVED'
UNION ALL
SELECT o.seller_id, (r.created_at AT TIME ZONE 'America/Fortaleza')::date, o.id,
       p.payment_type, -r.amount, 'ESTORNO'
  FROM refund r JOIN payment p ON p.id = r.payment_id JOIN orders o ON o.id = p.order_id;
```

Fonte: `src/main/resources/db/migration/V6__views_dashboards.sql` [E-16].

O esquema tem 18 *views*: lançamentos realizados, desempenho por vendedor, resumo da loja, evolução semanal (por vendedor e da loja), posição e alerta de estoque, catálogo do vendedor (sem custo), margem por produto, vendas por forma de pagamento, promoções ativas e uso de promoções, faturamento diário, resumo do cliente, descontos manuais e três *views* filtradas pela identidade da sessão (`v_meu_desempenho`, `v_minha_evolucao_semanal` com `security_barrier` e `v_minhas_vendas` com `security_invoker`). A correção numérica das *views* foi

verificada pelos testes BD-01 a BD-09, BD-17, BD-20, BD-22 e BD-31, todos aprovados nesta auditoria; por exemplo, BD-03 confirma o resumo da loja de setembro com R$ 186.400,00 realizados sobre meta de R$ 240.000,00 (77,7%), e BD-31 confirma que o resumo dos clientes soma R$ 186.400,00 em 65 pedidos pagos, igual ao faturamento líquido da *view* diária [E-11].

6.16 OPERAÇÕES DE INSERÇÃO, CONSULTA, ATUALIZAÇÃO E EXCLUSÃO

A disciplina de Banco de Dados exige demonstrar operações CRUD por SQL [E-35]. O repositório traz dois roteiros executáveis, ambos encerrados em `ROLLBACK` para preservar a massa: `05_crud_exemplos.sql`, sobre catálogo, estoque e painéis, executado com o login de banco do gerente, e `08_crud_clientes_olist.sql`, sobre clientes e histórico de compras [E-39]. O **Quadro** 19 reproduz um trecho do primeiro.

Quadro 19 **–** Trecho do roteiro de CRUD executado como gerente

```
BEGIN;
INSERT INTO product(product_type, name, price, weight, sku, category_id)
VALUES ('PHYSICAL', 'iPad 10a geracao', 3600.00, 0.48, 'IPD-10',
        (SELECT id FROM category WHERE name = 'Tablets'))
RETURNING id, sku, name, price;
INSERT INTO stock_entry(product_id, quantity, unit_cost, note)
VALUES ((SELECT id FROM product WHERE sku = 'IPD-10'), 8, 2700.00, 'Entrada de demonstracao');
SET LOCAL app.reason = 'Reajuste de tabela do fornecedor';
UPDATE product SET price = 3800.00 WHERE sku = 'IPD-10';
UPDATE product SET active = false WHERE sku = 'IPD-10';
DELETE FROM category WHERE name = 'Tablets'
   AND NOT EXISTS (SELECT 1 FROM product WHERE category_id = category.id);
ROLLBACK;
```

Fonte: `docs/bd/pacote-sql/05_crud_exemplos.sql` (trecho) [E-39].

Nesta auditoria, os dois roteiros foram executados sobre o banco local migrado com a massa Olist. O primeiro concluiu sem erro, criando o produto, registrando a entrada com custo médio de R$ 2.700,00 e o histórico de preço com autor gerente, e desfazendo tudo no `ROLLBACK`. O segundo produziu exatamente as três recusas de integridade esperadas — CPF com máscara (`ck_customer_cpf`), pessoa física com CNPJ (`ck_customer_document_type`) e exclusão de cliente com pedidos (`orders_client_id_fkey`) —, o que corresponde ao caso CT-DEMO-06 aprovado [E-12]. A exclusão física é usada apenas quando não há vínculos; para produtos com histórico, a operação recomendada é a inativação (`active = false`).

6.17 TRANSAÇÕES

As transações aparecem em três níveis. No nível das migrações, o Flyway executa cada script em uma transação, e a falha de uma pré-checagem aborta a migração inteira, como nas validações de dados legados da V4, V5, V8 e V10. No nível da aplicação, os serviços anotados com `@Transactional` executam cada caso de uso de escrita em uma única transação; na criação do pedido, a falha da reserva do segundo item desfaz a inserção do pedido e a reserva do primeiro (CT-STOCK-103, documentado) [E-19]. No nível do banco, as funções `fn_criar_pedido`, `fn_confirmar_pagamento`, `fn_cancelar_pedido` e `fn_estornar` encapsulam as mesmas operações para uso por `psql` e pelo job `bd`; a API não as chama, mas elas permitem demonstrar transações completas diretamente no SGBD [E-16].

6.18 CONCORRÊNCIA E CONSISTÊNCIA

Quatro mecanismos tratam a concorrência:

a) **reserva condicional**: `UPDATE stock_balance SET reserved = reserved + :q WHERE product_id = :id AND on_hand - reserved >= :q`. Sob *Read Committed*, uma segunda transação que tenta reservar a mesma linha espera o término da primeira e reavalia a condição sobre a versão confirmada; se o disponível não basta, nenhuma linha é atualizada e a venda é rejeitada (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2025);
b) **bloqueio pessimista**: confirmação e cancelamento leem o pedido com `SELECT ... FOR UPDATE`, o que serializa operações sobre o mesmo pedido (correção do defeito D019);
c) **ordem fixa de bloqueio**: os itens são processados em ordem crescente de `product_id`, o que elimina *deadlocks* entre vendas com os mesmos produtos em ordens diferentes (RN-05);
d) **controle otimista**: `customer.version` e `app_users.version` detectam edição concorrente e produzem 409 `CONCURRENT_UPDATE`.

O teste CC-01, reexecutado nesta auditoria, dispara duas transações concorrentes que tentam reservar a última unidade de um produto; o resultado esperado e obtido foi exatamente uma reserva aceita, com saldo 1, reservado 1 e

um item, e a segunda transação rejeitada com “estoque insuficiente” [E-11]. O gatilho `keep_one_active_manager` usa `pg_advisory_xact_lock` para serializar a verificação entre transações concorrentes, de modo que dois gerentes rebaixando um ao outro simultaneamente não deixem a loja sem gerente.

6.19 ÍNDICES E DESEMPENHO

Além das 19 chaves primárias, o esquema possui 29 índices: 13 sustentam restrições de unicidade, 1 é índice único por expressão (`ux_app_users_email_lower`, que impede e-mails iguais com diferença apenas de maiúsculas) e 15 são índices secundários criados para consultas específicas, como `ix_customer_name_prefix` e `ix_product_name_prefix` (`lower(name) text_pattern_ops`, para busca por prefixo), `ix_orders_customer_moment (customer_id, moment DESC)` para o histórico do cliente e `ix_stock_movement_product (product_id, created_at)` para o extrato [E-15].

Os planos de execução obtidos nesta auditoria com `EXPLAIN ANALYZE` sobre a massa de demonstração (66 clientes, 287 pedidos, 333 itens e 49 produtos) mostram que, nesse volume, o otimizador escolhe varredura sequencial, que é mais barata que o acesso por índice em tabelas pequenas. Com `enable_seqscan` desativado, os índices previstos são usados: `ix_customer_name_prefix` na busca por prefixo, `ix_orders_customer_moment` no histórico do cliente e `product_sku_key` com `stock_balance_pkey` no catálogo por SKU [E-13]. Na consulta de pedidos por vendedor, o otimizador preferiu `uq_orders_seller_request`, cujo primeiro campo é `seller_id`, o que torna `idx_orders_seller_id` funcionalmente redundante (achado A-11).

No banco publicado, as estatísticas de uso mostraram 16 índices sem nenhuma varredura desde o último reinício de estatísticas (15 do esquema de negócio e 1 da tabela de histórico do Flyway), entre eles `ix_customer_name_prefix` e `ix_product_name_prefix` [E-14]. Como o Neon suspende o *compute* ocioso e as estatísticas recomeçam, o dado é uma amostra parcial e não autoriza remover índices; indica, porém, que a eficácia dos índices de prefixo depende de a consulta da aplicação usar exatamente a expressão indexada, o que deve ser verificado com volume maior.

6.20 MIGRATIONS

O **Quadro** 20 relaciona as migrações aplicadas no banco publicado, conforme a tabela `flyway_schema_history` consultada em 24/10/2026 [E-03].

Quadro 20 **–** Migrações Flyway aplicadas no banco publicado

Versão	Descrição	Tipo	Aplicada em (UTC)
1	Esquema legado	SQL	15/10/2026 11:45
2	Usuários e autoria do pedido	SQL	15/10/2026 11:45
3	Renomeação de `client` para `customer`	SQL	15/10/2026 11:45
4	Dinheiro em NUMERIC	SQL	15/10/2026 11:45
5	Catálogo, estoque, metas, promoções, auditoria e funções	SQL	15/10/2026 11:45
6	*Views* dos painéis	SQL	15/10/2026 11:45
6.1	Contas de demonstração (BCrypt)	JDBC (Java)	15/10/2026 11:45
6.2	Massa sintética de demonstração	SQL	15/10/2026 11:46
7	Mascaramento de cartões	SQL	15/10/2026 11:46
8	Administração de usuários e versão de sessão	SQL	15/10/2026 11:46
9	*View* de resumo do cliente	SQL	15/10/2026 11:46
10	CNPJ alfanumérico	SQL	15/10/2026 11:46
11	Desconto manual com alçada	SQL	15/10/2026 11:46
11.1	Promoção de demonstração até dezembro	SQL	15/10/2026 12:57
11.2	Custos da carga inicial de demonstração	SQL	15/10/2026 17:00

Fonte: tabela `flyway_schema_history` do banco publicado [E-03].

Todas as 15 migrações foram registradas com sucesso. As versões 6.1, 6.2, 11.1 e 11.2 pertencem à localização `db/demo` (a 6.1 é uma migração Java, as demais são SQL) e só são aplicadas no perfil `demo`; com `out-of-order` desativado, esse perfil precisa estar ativo desde a primeira migração. A V10 atualiza a validação de CNPJ para o formato alfanumérico instituído pela Receita Federal a partir de julho de 2026 (BRASIL, 2024). Migrações publicadas não são editadas; correções entram como novas versões.

6.21 CONTROLE DE ACESSO

O controle de acesso principal está na API (seção 5.10). No banco, o script `03_rbac.sql` implementa defesa em profundidade para a demonstração por SQL: cria os papéis `loja_gerente` e `loja_vendedor`, revoga privilégios públicos, concede ao vendedor apenas *views* sem custo e colunas selecionadas de `order_item` e

`payment`, e ativa RLS em pedidos, itens e pagamentos com políticas que restringem o vendedor às próprias vendas, conforme o **Quadro** 21.

Quadro 21 **–** Políticas de segurança em nível de linha para pedidos

```
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
CREATE POLICY orders_gerente  ON orders FOR SELECT TO loja_gerente  USING (true);
CREATE POLICY orders_vendedor ON orders FOR SELECT TO loja_vendedor
       USING (seller_id = app_current_user_id());
```

Fonte: `docs/bd/pacote-sql/03_rbac.sql` (trecho) [E-39].

Os testes RB-01 a RB-14 e RG-01 a RG-07, executados com os logins de vendedor e de gerente, confirmaram nesta auditoria que o vendedor não lê custo, margem, resultado da loja nem pedidos de colegas, não altera estoque nem cria promoções, e que o gerente lê o conjunto completo [E-11]. As funções de negócio são `SECURITY DEFINER` com `search_path` fixado no esquema da migração, o que evita sequestro de objetos por esquema temporário. Os logins de banco sem senha criados pelo script são próprios de ambiente local descartável e não existem no banco publicado.

6.22 DADOS FICTÍCIOS E MECANISMOS DE DEMONSTRAÇÃO

A massa de demonstração tem duas origens. A migração V6_2 cria um cenário sintético de setembro de 2026: quatro categorias, seis produtos, seis clientes com CPF de dígito verificador válido, meta da loja de R$ 240.000,00 e vendas que reproduzem o guia visual (Allan R$ 68.400,00, Francisco R$ 62.000,00, Gabriel R$ 56.000,00). O script `07_carga_olist.sql` carrega, por `\copy` e `INSERT ... SELECT` em uma transação idempotente, um recorte do conjunto público da Olist (OLIST, 2018), licenciado em CC BY-NC-SA 4.0, com nomes e documentos substituídos por dados sintéticos e o catálogo trocado por 42 produtos Apple: 60 clientes (8 pessoas jurídicas), 219 pedidos e 237 itens, totalizando R$ 449.690,00 [E-12]. Os testes CT-DEMO-01 a 04 e 07 confirmaram essas contagens, a idempotência da carga repetida e a coerência do saldo de estoque nesta auditoria.

O banco publicado contém, em 24/10/2026, 4 usuários, 67 clientes, 48 produtos, 7 categorias, 291 pedidos, 337 itens, 388 movimentos de estoque, 4 metas mensais e 55 eventos de auditoria, valores que incluem as operações feitas durante ensaios e demonstrações após a carga [E-03].

6.23 LIMITAÇÕES E PROPOSTAS DE EVOLUÇÃO

As limitações do banco são: (a) colunas anuláveis herdadas do legado em relacionamentos que o domínio novo trata como obrigatórios, como `order_item.order_id` e `orders.seller_id`; (b) nomes de restrições anteriores à renomeação de `client`; (c) carga analítica executada sobre o banco transacional, sem *views* materializadas; (d) índices de prefixo sem uso comprovado em produção; (e) RLS usada apenas na demonstração por SQL, não na conexão da API. As propostas correspondentes estão no capítulo 15, entre elas tornar `NOT NULL` as chaves obrigatórias por nova migração após saneamento dos dados legados, remover `idx_orders_seller_id` após medir a carga real e avaliar *views* materializadas para os painéis em volumes maiores.

7 IMPLEMENTAÇÃO DAS FUNCIONALIDADES

Este capítulo descreve cada módulo encontrado no código da versão de referência. Para cada um, são indicados objetivo, requisitos, estrutura técnica, regras, interações com o banco, validações, tratamento de erros, comportamento observado, evidências e limitações. As capturas de tela das funcionalidades estão concentradas no capítulo 10, para evitar repetição.

7.1 AUTENTICAÇÃO

Objetivo e requisitos. Identificar o usuário e manter a sessão com proteção contra força bruta e CSRF (RF-01, RF-18).

Estrutura técnica. `AuthController` expõe `GET /auth/csrf`, `POST /auth/login`, `POST /auth/logout`, `GET /auth/me` e `PUT /auth/password`. A autenticação usa `ProviderManager` com `LockoutAuthenticationProvider`, que lê as contas pelo `AppUserRepository`, `LoginAttemptService` e `JdbcLoginCounter` (a classe `DatabaseUserDetailsService` existe no código, mas não é usada); a revogação de sessão usa `SessionVersionFilter` e `JdbcSessionStateReader` [E-17].

Regras e fluxo. O fluxo da **Figura** 4 aplica RN-15 a RN-17: cinco falhas consecutivas bloqueiam a conta por 15 minutos, todas as falhas recebem a mesma resposta 401 e o BCrypt é executado mesmo quando a conta não existe. O login bem-sucedido zera o contador, troca o identificador de sessão e grava `LOGIN_SUCCEEDED` na auditoria. Senha provisória restringe a sessão às rotas da própria conta até a troca.

Banco. Lê e atualiza `app_users` (`failed_login_attempts`, `locked_until`, `last_login_at`, `session_version`) e grava `audit_event`; em falha de e-mail inexistente, o detalhe guarda apenas o SHA-256 do e-mail tentado.

Validações e erros. Corpo malformado: 400 `MALFORMED_REQUEST`; sem *token* CSRF: 403; credencial inválida, conta inexistente, bloqueada ou inativa: 401 `UNAUTHENTICATED`; conta alterada após o login: 401 `SESSION_REVOKED`; senha provisória: 403 `PASSWORD_CHANGE_REQUIRED`.

Comportamento observado e evidências. No ambiente publicado, `GET /auth/csrf` respondeu 200 com o nome do cabeçalho e o *token*, e `GET /auth/me` sem sessão respondeu 401 no formato padronizado [E-04]. O formulário de login

usa campos rotulados, `autocomplete` adequado (`username`, `current-password`), opção “Mostrar senha” e aviso do bloqueio após cinco tentativas [E-04]. O bloqueio por concorrência é coberto pelos testes `LoginLockoutIntegrationTest` e `LoginAttemptConcurrencyTest` (dez falhas paralelas bloqueiam a conta uma única vez), com resultado documentado [E-22].

Limitações. Não há limitação de taxa por endereço IP, duração absoluta da sessão nem *cookie* com prefixo `__Host-`; esses itens foram registrados pela equipe como trabalho futuro (D-EVO-27) [E-42]. Como o bloqueio é por conta, um atacante pode bloquear contas conhecidas enviando senhas erradas, risco discutido no capítulo 9.

7.2 USUÁRIOS E PERMISSÕES

Objetivo e requisitos. Permitir ao gerente administrar contas e garantir a hierarquia de perfis (RF-17, RF-24).

Estrutura técnica. Módulo `api/users` (`UsersController`, `UsersService`, `UsersRepository`, `PasswordService`) e validador `StrongPasswordValidator`, com lista de senhas comuns em `security/common-passwords.txt`. A interface é a tela `/gestao/usuarios` e a troca de senha própria é `/conta/senha`.

Regras. RN-16 a RN-18: senha de 12 a 72 caracteres e até 72 bytes, fora da lista comum e diferente do e-mail; senha provisória obrigatória no cadastro e na redefinição; qualquer mudança de papel, situação, senha ou desbloqueio incrementa `session_version` e derruba as sessões abertas; o próprio gerente não altera seu papel, sua situação ou sua senha pela administração; sempre existe ao menos um gerente ativo.

Banco. `app_users` com `version` para controle otimista e o gatilho de restrição `keep_one_active_manager`, adiado para o COMMIT e serializado por *advisory lock*. A violação produz `P0001` com prefixo `LAST_MANAGER`, convertido em 409.

Comportamento observado e evidências. O teste CT-BD-30, reexecutado nesta auditoria, confirmou que o último gerente ativo não pode ser rebaixado nem desativado e que o rebaixamento é aceito quando há outro gerente ativo [E-11]. O banco publicado contém quatro contas, duas de gerente e duas de vendedor, todas ativas, sem bloqueio e sem troca de senha pendente na consulta de 24/10/2026 [E-32].

Limitações. O papel é binário (GERENTE, VENDEDOR); não há perfis intermediários, como caixa ou estoquista, nem permissões configuráveis por recurso.

7.3 CADASTRO DE PRODUTOS

Objetivo e requisitos. Manter o catálogo de produtos e insumos, categorias e preços (RF-04, RF-22).

Estrutura técnica. Módulo `api/backoffice` (`ProductsController`, `CatalogAdminService`, `BackofficeRepository`), com cadastro, consulta, edição e exclusão de produtos nunca vendidos, cadastro e renomeação de categorias (sem exclusão de categoria), `PATCH /products/{id}/price` (com motivo), `PATCH /products/{id}/status` e `GET /products/{id}/price-history`. Interface: `/gestao/produtos`.

Regras. Produto físico exige peso e produto digital exige link (tabela de decisão por tipo); SKU único em maiúsculas; insumo é produto com `sellable = false` e não aparece no PDV; alteração de preço registra histórico com autor e motivo (RN-21, RN-22); exclusão física só sem vínculos, com inativação recomendada.

Banco. `product`, `category`, `product_price_history` e o gatilho `product_price_history`, que lê o autor e o motivo das variáveis de sessão gravadas por `DbSessionContext`.

Comportamento observado e evidências. O roteiro de CRUD reexecutado registrou o histórico de preço com o gerente como autor e o motivo informado (RG-03), e BD-24 e BD-25 confirmaram o histórico e a preservação do preço de itens já vendidos [E-11].

Limitações. Não há importação em lote nem variações de produto (cor, capacidade) como entidade própria; cada variação é um produto.

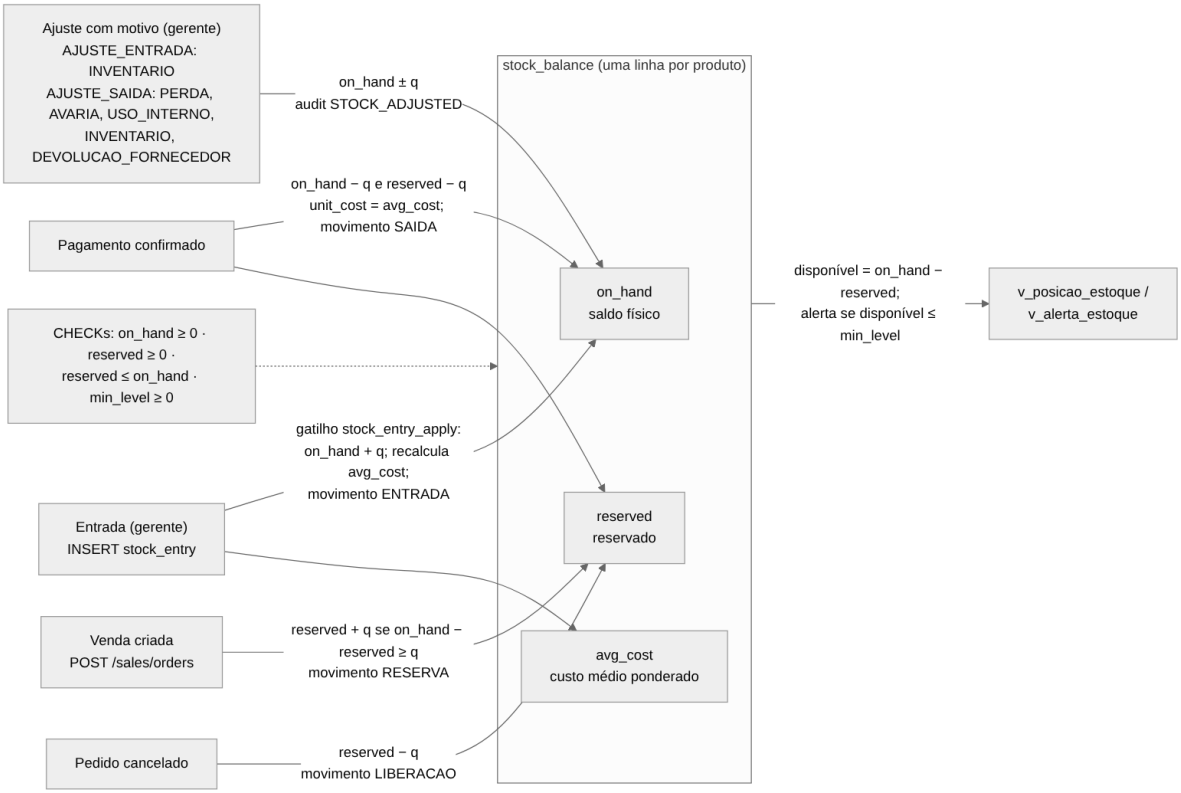
7.4 CONTROLE DE ESTOQUE

Objetivo e requisitos. Controlar saldo, reservas, entradas, ajustes e estoque mínimo (RF-13, RF-14, RF-22).

Estrutura técnica. `StockController` e `StockService` (`/stock/products`, entradas, ajustes, mínimo e `/stock/movements`); reservas, saídas e liberações são

feitas pelo módulo de vendas. Interface: `/gestao/estoque`. A **Figura** 9 mostra como cada operação afeta o saldo.

Figura 9 **–** Fluxo de atualização do estoque



Fonte: elaborado pelos autores a partir da V5, de `SalesService` e de `StockService` [E-16] [E-19].

Regras. RN-04, RN-08 e RN-20: disponível igual a saldo menos reservado; custo médio ponderado recalculado na entrada; ajuste com motivo; nenhuma operação deixa saldo ou reserva negativos.

Banco. `stock_balance`, `stock_entry`, `stock_movement`, gatilhos `product_stock_row` e `stock_entry_apply` e as *views* `v_posicao_estoque` e `v_alerta_estoque`.

Validações e erros. Entradas e ajustes aceitam quantidades inteiras de 1 a 100.000 e estoque mínimo de 0 a 100.000; valores fora desses limites, ou decimais, respondem 400. Ajuste acima do disponível responde 409 `STOCK_INSUFFICIENT`, e a violação de CHECK de estoque também é convertida em 409, nunca em 500.

Comportamento observado e evidências. BD-06, BD-09, BD-15, BD-18 e RG-05/RG-06 confirmaram nesta auditoria o cálculo de disponível, o alerta de mínimo, a barreira `reserved <= on_hand` e o custo médio de entradas sucessivas (2

× R$ 1.900,00 + 5 × R$ 1.950,00) [E-11]. A captura da tela de estoque mostra saldo, reservado, disponível, mínimo e custo médio por produto (capítulo 10).

Limitações. Não há múltiplos depósitos, lotes, validade nem inventário cíclico; a devolução de cliente (`DEVOLUCAO`) existe no domínio do movimento, mas não há fluxo de interface correspondente.

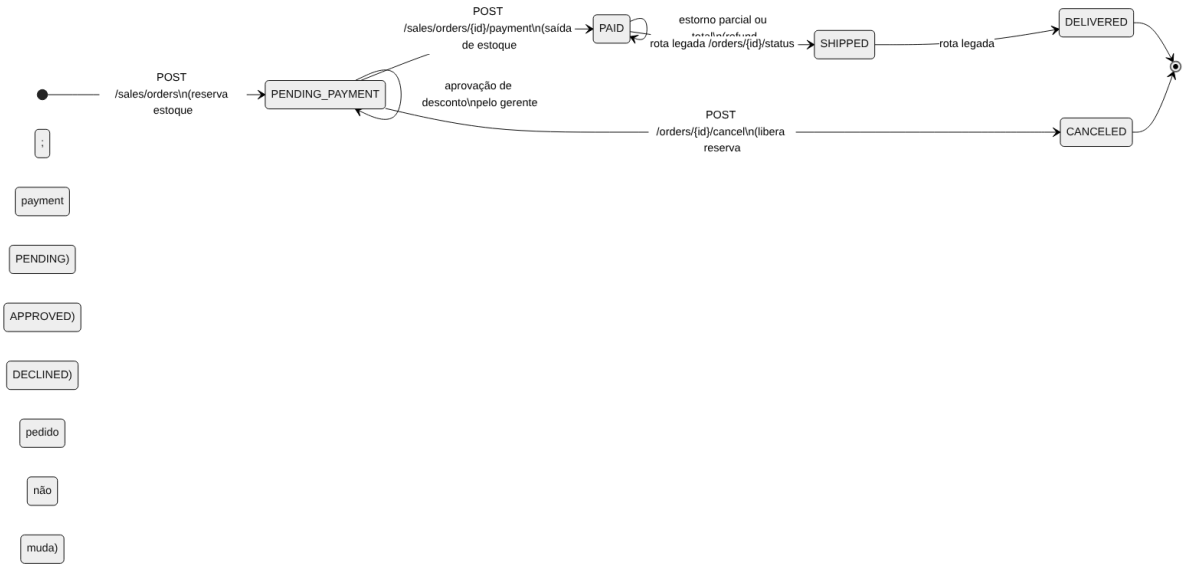
7.5 GESTÃO DE VENDAS

Objetivo e requisitos. Registrar vendas no balcão com cálculo no servidor, idempotência e reflexo imediato no estoque (RF-06, RF-07, RF-15, RF-19, RF-26).

Estrutura técnica. Módulo `api/sales`: `SalesController` (catálogo, cotação, pedidos, confirmação e aprovação de desconto), `OrderCancelController`, `SalesService`, `SalesRepository`, `Pricing` (cálculo com HALF_UP) e `RequestHash` (SHA-256 canônico do corpo). Interface: `/vender` (etapas Itens, Cliente e pagamento, Confirmação), `/vendas` e `/vendas/[id]`.

Regras e fluxo. O fluxo está na **Figura** 5 e o ciclo de vida do pedido na **Figura** 10. Aplicam-se RN-01 a RN-07, RN-12, RN-19, RN-22 e RN-24.

Figura 10 **–** Estados do pedido e do pagamento



Fonte: elaborado pelos autores a partir de `SalesService`, `OrderStatus` e da V5 [E-19].

Banco. Inserções em `orders`, `order_item`, `payment` e `stock_movement`; atualização condicional de `stock_balance`; leitura de promoções vigentes e de `discount_policy`.

Validações e erros. O corpo da venda aceita apenas cliente, itens (produto, quantidade, promoção), pagamento, modalidade, endereço e desconto manual; qualquer outra propriedade, como preço, total ou vendedor, responde 400. Os conflitos de estoque, idempotência e estado respondem 409 com código específico.

Comportamento observado e evidências. Na camada de banco, os testes BD-10 a BD-15, BD-19 a BD-21, BD-26 a BD-29, RB-13, RB-14 e CC-01 foram aprovados nesta auditoria, incluindo a venda simultânea da última unidade [E-11]. A camada HTTP do PDV é coberta por `SalesApiTest` (23 métodos), `SalesConcurrencyTest` e `ManualDiscountApiTest`, e pelos testes de ponta a ponta `sale.spec.ts` e `discount.spec.ts`, com resultados documentados [E-21] [E-22].

Limitações. O pagamento é simulado e confirmado pelo próprio usuário, sem conciliação com adquirente; não há emissão de cupom fiscal nem troca ou devolução de mercadoria após o pagamento.

7.6 METAS COMERCIAIS

Objetivo e requisitos. Definir a meta mensal da loja, distribuí-la entre vendedores e acompanhar o realizado (RF-09, RF-10, RF-11).

Estrutura técnica. `GoalsController` e `GoalsService`: `GET` e `PUT /goals/monthly` (gerente) e `GET /performance/me` (vendedor). Interface: cartão de meta em `/gestao` e tela `/meu-desempenho`.

Regras. RN-09 e RN-10: realizado igual a pagamentos aprovados menos estornos, no fuso da loja; soma das metas individuais igual à meta da loja, conferida no COMMIT; meta individual apenas para vendedor; meta zero exibida como “Sem meta”, sem percentual.

Banco. `monthly_goal`, `seller_goal`, gatilhos de soma e de papel, *views* `v_lancamento_realizado`, `v_desempenho_vendedor` e `v_meu_desempenho`, lidas pelo `GoalsService`; `v_resumo_loja` e `v_minha_evolucao_semanal` são usadas pelo pacote SQL.

Comportamento observado e evidências. BD-01 a BD-05, BD-16, BD-17 e BD-20, além de RB-11 e RB-12, foram aprovados nesta auditoria: Allan com R$ 68.400,00 de R$ 120.000,00 (57,0%), Francisco com 51,7%, loja com 77,7% e falta de R$ 53.600,00, pedidos pendentes fora do realizado, distribuição divergente rejeitada e meta zero sem percentual [E-11].

Limitações. A interface exibe as metas, mas não oferece tela para cadastrá-las ou distribuí-las: a definição é feita pela API (`PUT /goals/monthly`, por exemplo pelo Swagger) ou por SQL, a meta de setembro vem da massa de demonstração, e as de outubro a dezembro de 2026 do banco publicado foram registradas fora da interface [E-01] [E-41] [E-03]. O gerente vende, mas não pode receber meta individual (o gatilho exige papel VENDEDOR); não há comissão nem histórico de revisão de metas.

7.7 DASHBOARDS

Objetivo e requisitos. Oferecer ao gerente a visão consolidada da loja (RF-12, RF-23).

Estrutura técnica. `ManagementController` (`/management/summary` por mês e `/management/revenue` com períodos TODAY, D7, D30 e D90) e a página `/gestao`, que exibe faturamento líquido, lucro bruto, margem bruta, pedidos pagos, gráfico diário de faturamento e lucro, formas de pagamento, ranking de vendedores, produtos mais vendidos, cartão de meta e alerta de estoque.

Regras. Os indicadores leem `v_faturamento_diario` e as demais *views*; lucro e margem ficam nulos quando algum item vendido não tem custo, e a cobertura de custo é informada; o painel recarrega a cada 15 segundos com a aba visível e nunca exibe erro de carga como zero [E-01].

Comportamento observado e evidências. A captura versionada do painel mostra os quatro indicadores, o gráfico com o atalho “Ver dados em tabela” e o cartão “Meta do mês não definida” (capítulo 10) [E-30]. A correção do faturamento diário frente ao seed foi confirmada por BD-31 nesta auditoria [E-11].

Limitações. Os períodos são fixos (hoje, 7, 30 e 90 dias); não há filtro por intervalo arbitrário nem exportação de dados.

7.8 INDICADORES

Os indicadores calculados pelo sistema estão no **Quadro** 22. Todos derivam de pagamentos confirmados, e não de pedidos criados, o que impede que vendas pendentes ou canceladas inflem os resultados.

Quadro 22 **–** Indicadores calculados pelo SBI

Indicador	Definição implementada	Origem
Faturamento bruto	Soma de (preço × quantidade − desconto) dos itens de pedidos com pagamento aprovado	`v_faturamento_diario`
Faturamento líquido	Faturamento bruto − estornos do período	`v_faturamento_diario`
Lucro bruto	Faturamento líquido − custo conhecido; nulo se houver item sem custo	`v_faturamento_diario`
Margem bruta	Lucro bruto ÷ faturamento líquido × 100 (1 casa, HALF_UP)	`ManagementService`
Cobertura de custo	Receita com custo conhecido ÷ faturamento bruto × 100	`ManagementService`
Ticket médio	Faturamento bruto ÷ pedidos pagos	`ManagementService`, sobre `v_faturamento_diario`
Atingimento da meta	Realizado ÷ meta × 100; nulo com meta zero	`GoalsService`, sobre `v_desempenho_vendedor` e `v_meu_desempenho`
Margem por produto	(Receita líquida − custo) ÷ receita líquida × 100 por mês	`v_margem_produto`
Valor em estoque	Saldo × custo médio e saldo × preço de venda	`v_posicao_estoque`

Fonte: elaborado pelos autores a partir das *views* V6 e do código do módulo `management` [E-16] [E-01].

7.9 RELATÓRIOS

O sistema não gera arquivos de relatório (PDF ou planilha). As funções de relatório são consultas na interface: análises de vendas do mês (`/analytics/sales`, com evolução semanal, formas de pagamento, margem por produto, uso de promoções e descontos manuais por vendedor), análise de estoque (`/analytics/stock`, com quantidade de produtos e unidades, valor a custo e a preço, produtos sem custo e abaixo do mínimo), histórico de preço, extrato de movimentos e histórico e resumo do cliente [E-06]. A tela `/gestao/analises` apresenta essas consultas, e os gráficos oferecem alternativa em tabela para acessibilidade [E-30].

7.10 FUNCIONALIDADES ADMINISTRATIVAS

As funções exclusivas do gerente são: administração de usuários (seção 7.2), alçadas de desconto (`GET /discount-policy` e `PUT /discount-policy/{role}`, com motivo e auditoria `DISCOUNT_POLICY_CHANGED`), aprovação de desconto acima da alçada em pedido pendente (`POST /sales/orders/{id}/discount-approval`), promoções (`/promotions`, criar, listar e encerrar), estorno (`POST /payment/{id}/refund`, até o saldo não estornado), exclusão de cliente sem pedidos e acesso às rotas legadas `/orders`, `/payment` e `/address` [E-06] [E-17]. Quatro

funções existem apenas na API, sem tela na interface: o estorno, a definição de metas e a exclusão de cliente e de produto. A trilha de auditoria registra 19 tipos de ação, mas também não possui tela de consulta; a leitura é feita por SQL.

7.11 DEMAIS FUNCIONALIDADES IDENTIFICADAS

Clientes (RF-03, RF-20, RF-21, RF-25). O módulo `api/customers` permite cadastrar e editar clientes pessoa física e jurídica, com validação de dígito verificador de CPF e de CNPJ numérico ou alfanumérico pelas anotações `@CPF` e `@CNPJ` do Hibernate Validator nos DTOs (`AlphanumericCnpjTest`), busca e máscara por `DocumentFormat`, documento imutável após o cadastro, máscara do documento para o vendedor, histórico paginado de pedidos com valores de outros vendedores ocultos e resumo com indicadores de compra lidos de `v_cliente_resumo`. Ao tentar cadastrar um documento existente pelo PDV, a interface oferece usar o cliente já cadastrado (CT-WEB-128) [E-01]. O teste CT-DEMO-05, que verifica a máscara de documento pela API, não foi reexecutado nesta auditoria por depender da API em execução.

Promoções (RF-22). O módulo `api/promotions` permite ao gerente liberar promoções percentuais ou em valor, por produto ou categoria e com vigência; o vendedor consulta as promoções ativas, que aparecem pré-preenchidas no carrinho. BD-14, BD-27 e RG-04 confirmaram nesta auditoria que apenas o gerente libera promoção e que a promoção fora do escopo do item é rejeitada [E-11].

Desconto manual (RF-26). Calculado sobre o subtotal já promocionado, limitado à alçada do papel, com motivo obrigatório e rateado entre os itens em `order_item.manual_discount`, o que mantém o sentido de faturamento e metas. BD-32 confirmou as alçadas iniciais (vendedor 5%, gerente 20%) e as restrições de motivo e rateio [E-11].

Créditos de imagens. A rota `/creditos` lista a autoria e a licença das fotos de produtos obtidas do Wikimedia Commons, exigência das licenças CC BY e CC BY-SA [E-01].

Rotas legadas. As rotas `/orders`, `/payment` e `/address` da API original permanecem ativas, restritas ao gerente, com as correções de defeitos D003 a D024 aplicadas. Elas não fazem parte do contrato v5, não são usadas pela interface e permitem, por exemplo, alterar o estado de pedidos pela tabela de transições legada

(SHIPPED, DELIVERED) [E-06].

8 GARANTIA DA QUALIDADE DE SOFTWARE

Este capítulo apresenta o processo de garantia da qualidade aplicado ao SBI e os resultados obtidos. Para cada item, distinguem-se os resultados **documentados** pela equipe, registrados em artefatos versionados, dos resultados **verificados** nesta auditoria por reexecução em 24/10/2026. Nenhum resultado foi inferido: testes não reexecutados aparecem como não executados nesta auditoria, com a referência do resultado declarado.

8.1 FUNDAMENTOS E ATRIBUTOS DE QUALIDADE

O modelo de referência é a ISO/IEC 25010:2023 (ISO; IEC, 2023). O **Quadro** 23 relaciona as características avaliadas, a nomenclatura usada no plano de qualidade da equipe (baseado na edição de 2011) e os indicadores com evidência disponível.

Quadro 23 **–** Características de qualidade avaliadas e indicadores

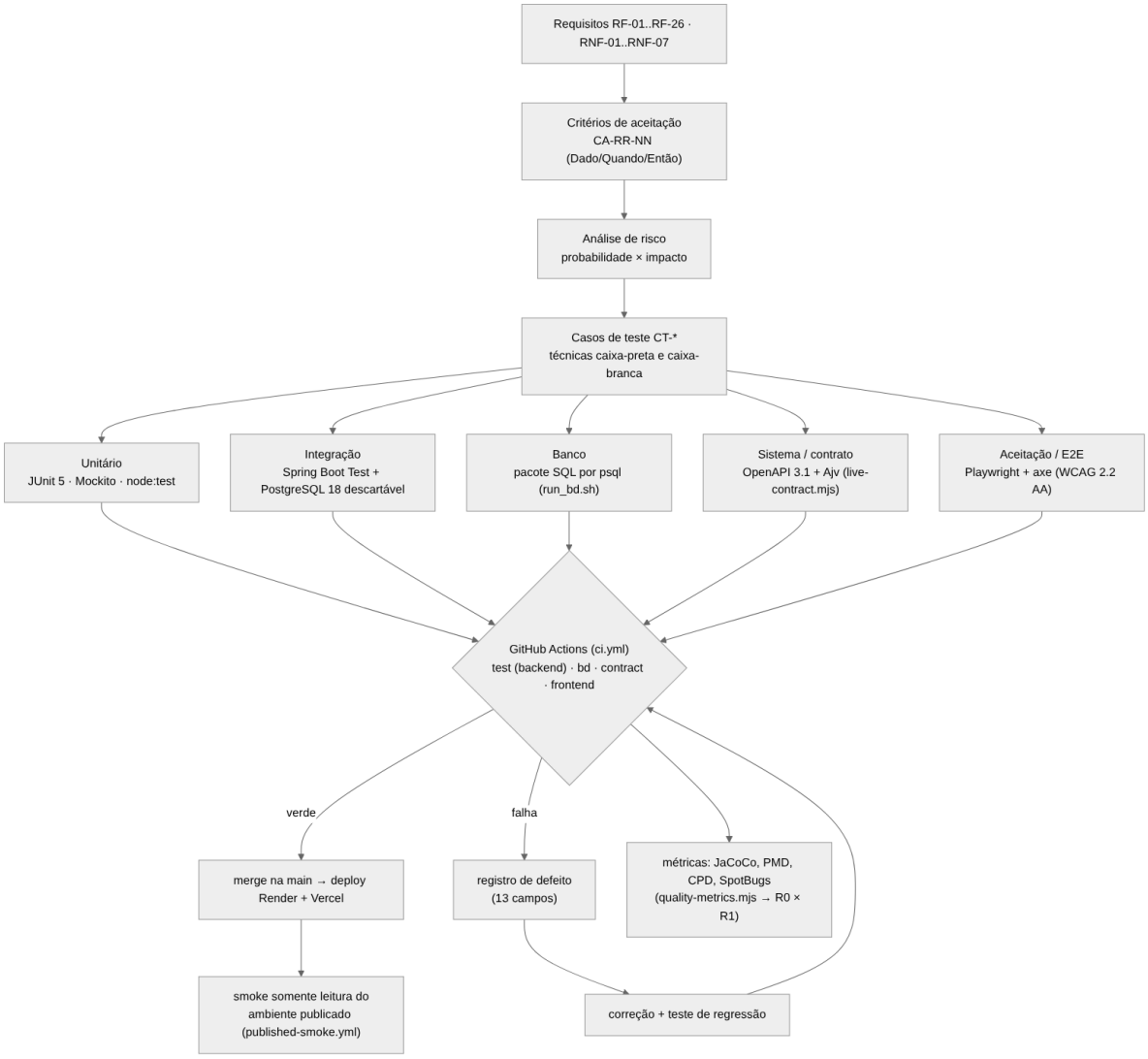
Característica (ISO/IEC 25010:2023)	**Termo no plano da equipe**	**Indicador utilizado**	**Evidência**
Adequação funcional	Adequação funcional	Requisitos com critérios e testes aprovados; correção numérica das *views*	Apêndices A e B; pacote SQL 61/61 [E-11]
Confiabilidade	Confiabilidade	Taxa de falha das suítes; defeitos por severidade; concorrência	Seção 8.18; CC-01 [E-11]
Segurança	Segurança	Respostas 401/403/404 por perfil; dados sensíveis ocultos; vulnerabilidades de dependências	RB-01 a RB-14 [E-11]; `npm audit` [E-24]
Manutenibilidade	Manutenibilidade	Complexidade ciclomática, duplicação, achados de análise estática, cobertura	Métricas R0 × R1 [E-22]
Eficiência de desempenho	Eficiência	p95 por operação crítica	Não medido (RNF-06)
Capacidade de interação	Usabilidade	WCAG 2.2 AA (axe), estados de interface, avaliação heurística	Capturas com axe sem violação [E-30]; relatório de usabilidade [E-38]
Flexibilidade	Portabilidade	Execução em ambientes distintos (Windows local, Linux CI, nuvem)	Execuções desta auditoria em Linux, PostgreSQL 16 e Node 22 (seção 8.18)
Compatibilidade	Compatibilidade	Respostas conformes ao contrato OpenAPI 3.1	Auditoria estática do v5 [E-07]; OpenAPI publicado [E-05]

Fonte: elaborado pelos autores a partir de `docs/plano-qualidade.md` e das execuções desta auditoria [E-37].

8.2 ESTRATÉGIA DE GARANTIA DA QUALIDADE

A estratégia documentada pela equipe combina quatro princípios [E-37]: intensidade de teste proporcional ao risco; contrato OpenAPI aprovado como oráculo dos testes HTTP (*contract-first*); teste que reproduz o defeito antes da correção, mantido como regressão; e medição antes e depois das correções com as mesmas ferramentas. A integração contínua aplica a estratégia em cada *push* e *pull request* por meio de quatro jobs independentes, cada um com seu próprio PostgreSQL 18 descartável, conforme a **Figura** 11.

Figura 11 **–** Estratégia de testes e fluxo de verificação contínua



Fonte: elaborado pelos autores a partir de `.github/workflows/ci.yml` e de `docs/plano-qualidade.md` [E-20] [E-37].

A estratégia desta auditoria foi complementar: reexecutar tudo o que fosse viável sem escrita no ambiente publicado, comparar o contrato com o OpenAPI efetivamente publicado e inspecionar dependências e configuração de segurança.

8.3 PLANEJAMENTO DOS TESTES

O plano de testes da equipe define objetivos, escopo, níveis, técnicas, dados de teste, ambiente e critérios de entrada e saída [E-43]. A priorização segue a matriz de risco do **Quadro** 24, em que risco é o produto de probabilidade e impacto em escala de 1 a 3.

Quadro 24 **–** Matriz de risco por funcionalidade

ID	Funcionalidade	P	I	Risco
R01	Registrar e processar pagamento	3	3	9 Crítico
R02	Criar pedido e calcular o total	3	3	9 Crítico
R03	Reserva e saída de estoque	3	3	9 Crítico
R04	Transição de status do pedido	3	2	6 Alto
R05	Autenticação e autorização por perfil	2	3	6 Alto
R06	Precisão monetária	2	3	6 Alto
R07	Migração do banco	2	3	6 Alto
R08	Metas e realizado	2	2	4 Médio
R09	Cadastro de clientes PF/PJ	2	2	4 Médio
R10	Configuração e credenciais do ambiente	2	2	4 Médio
R11	Catálogo de produtos	1	2	2 Baixo
R12	Endereço de entrega	1	2	2 Baixo
R13	Tela de acesso	1	2	2 Baixo

Fonte: `docs/analise-risco.md`, versão de 09/10/2026 [E-40].

A justificativa registrada para a distribuição do esforço é verificável: em 09/10/2026, pagamento (risco crítico) tinha 55 testes Java, enquanto endereço (risco baixo) tinha 8 [E-40]. A matriz não foi reavaliada após a implementação do PDV e do estoque em 14/10, embora o próprio documento preveja revisão a cada nova medição; os riscos R03 e R05, que se referiam a funcionalidades então inexistentes ou recentes, deveriam ser reclassificados com base nos testes atuais.

8.4 CRITÉRIOS DE ACEITAÇÃO

Os 85 critérios de aceitação estão no formato Dado/Quando/Então e cobrem os 26 requisitos funcionais (Apêndice A). O plano de qualidade estabelece que um

requisito só é considerado atendido quando todos os seus critérios têm caso de teste aprovado com evidência reproduzível, e que um caso só é aprovado quando o resultado obtido é igual ao esperado [E-37]. Os critérios de saída de uma rodada exigem execução completa dos casos selecionados, preservação dos relatórios brutos, classificação de cada falha (produto, teste, contrato ou ambiente) e registro de defeito com os 13 campos do enunciado.

8.5 TESTES UNITÁRIOS

A suíte Java possui 24 classes de teste sem a etiqueta `integration`, com 128 métodos anotados (`@Test` ou `@ParameterizedTest`) [E-21]. As maiores cobrem pagamento (`PaymentServiceTest`, 18 métodos), regras de catálogo e cliente (`CatalogServicesTest`, 16), regras de pedido (`OrderServiceRulesTest`, 13) e cálculo de pedido (`ProductAndOrderCalculationTest`, 10); outras classes cobrem formato de documento, CNPJ alfanumérico, política de bloqueio, cálculo de desconto, *hash* de idempotência, política de senha, hierarquia de método e formato de erro. A equipe documenta 233 execuções unitárias aprovadas, número maior que o de métodos porque os testes parametrizados geram uma execução por conjunto de dados [E-22]. **Situação nesta auditoria:** não executado localmente, por indisponibilidade do JDK 25 e do Maven Central no ambiente de análise. Na integração contínua do *commit* de referência (execução 39668791724, de 21/10/2026), a etapa que executa os testes unitários e de integração com JaCoCo, PMD, CPD e SpotBugs terminou com sucesso, o que confirma a aprovação da suíte, mas não o total de 233, porque os relatórios não puderam ser baixados [E-46].

No frontend, os testes de lógica usam o executor nativo do Node (`node:test`) e estão em oito arquivos, com 42 casos. **Resultado nesta auditoria:** 41 aprovados e 1 reprovado em Node 22.23.4 [E-09]. O caso reprovado, CT-WEB-124, espera que `compactMoney(47000)` produza “R$ 47 mil” e obteve “R$ 47,0 mil”. A causa é a dependência da implementação de `Intl.NumberFormat` do ambiente de execução: a integração contínua usa Node 24, enquanto o `package.json` declara compatibilidade com Node 20.9 ou superior. O defeito é de portabilidade do teste e da formatação, registrado como A-03. Com as correções propostas nesta auditoria (seção 15.4), que calculam a escala e o sufixo sem a notação compacta do `Intl`, a suíte passou a 43 casos, todos aprovados no mesmo ambiente [E-48].

Na raiz do repositório, os testes unitários dos validadores de contrato e da verificação de mensagens de *commit* somam 47 casos. **Resultado nesta auditoria:** 47 aprovados e 0 reprovados [E-08].

8.6 TESTES DE INTEGRAÇÃO

A suíte de integração possui 20 classes com a etiqueta `integration` e 119 métodos, executados contra PostgreSQL 18 real em contêiner descartável (porta 15432), com as tabelas de negócio esvaziadas entre testes [E-21]. As maiores classes são `SalesApiTest` (23 métodos), `UsersApiTest` (13), `FlywayMigrationTest` (12), `CustomersApiTest` (11) e `BackofficeApiTest` (10). Testes de concorrência (`SalesConcurrencyTest`, `ConcurrencyTest`, `LoginAttemptConcurrencyTest`) repetem cenários com várias *threads* para expor condições de corrida. A equipe documenta 649 testes aprovados, sem falhas nem testes ignorados, no perfil de integração após a evolução de 14/10 [E-22]. **Situação nesta auditoria:** não executado localmente, pelo mesmo motivo dos testes unitários Java; aprovado na mesma etapa da integração contínua do *commit* de referência, seguida da validação do esquema do Flyway pelo Hibernate, também concluída com sucesso [E-46].

8.7 TESTES FUNCIONAIS

Os testes funcionais em nível de sistema verificam a API completa em execução contra o contrato. O `scripts/live-contract.mjs` compara o OpenAPI gerado pela API com o contrato v5 (caminho, método, parâmetros, presença de corpo e códigos de sucesso) e valida com Ajv as respostas reais de sucesso e de erro; `http-contract.mjs` e `http-session.mjs` cobrem erros HTTP e sessão [E-20]. A equipe declara 58 de 58 operações exercitadas sem divergência [E-22]; o relatório versionado em `reports/contrato/live-v5/live-contract.json`, porém, registra 54 operações exercitadas em 14/10/2026, antes da inclusão das quatro últimas operações no contrato (achado A-09) [E-33].

Verificações desta auditoria. A auditoria estática do contrato v5 (`audit-contract.mjs`) encontrou 46 caminhos, 58 operações e 0 erros de política, e a validação dos esquemas (`validate-contract.mjs`) conferiu 236 esquemas em JSON Schema 2020-12 sem achados [E-07]. A comparação entre o contrato v5 e o

OpenAPI publicado em `https://sbi-app-alpha.vercel.app/api/backend/v3/api-docs` mostrou:

- 58 de 58 operações do contrato presentes no documento publicado (100%);
- 58 de 58 operações com os mesmos códigos de sucesso (100%);
- 92 de 201 códigos de erro previstos no contrato documentados no OpenAPI publicado (45,8%) [E-05].

O resultado indica conformidade estrutural nas operações e nos caminhos de sucesso e uma lacuna de documentação dos caminhos de erro, discutida no capítulo 14. Na integração contínua do *commit* de referência, a etapa que executa o OpenAPI ao vivo, os erros HTTP e a sessão terminou com sucesso [E-46]; como o `live-contract.mjs` não compara os códigos de erro do OpenAPI gerado, esse sucesso é compatível com a lacuna medida.

8.8 TESTES DE INTERFACE

Os testes de ponta a ponta usam Playwright com Chromium e estão em oito arquivos de especificação, com 26 testes, cobrindo login e sessão, venda completa, gestão, fluxos de estoque, preço e promoção, clientes, usuários, desconto e marca [E-21]. A equipe documenta 24 de 24 aprovados após a evolução de 14/10 [E-22]; a diferença para os 26 testes atuais é compatível com casos acrescentados depois (CT-WEB-126 a 128), mas a correspondência exata não foi verificada. Há ainda uma suíte separada de fumaça do ambiente publicado (`frontend/e2e/published/smoke.spec.ts`, 5 testes, CT-PUB-01 a 05), somente leitura, declarada como aprovada em 15/10/2026 [E-22]. **Situação nesta auditoria:** não executados localmente, porque dependem da API em execução local ou de credenciais do ambiente publicado. Na integração contínua do *commit* de referência, a etapa de ponta a ponta com axe terminou com sucesso [E-46]. O *workflow* da fumaça publicada (`published-smoke.yml`), por sua vez, não tem nenhuma execução registrada no GitHub Actions, de modo que o resultado de 15/10/2026 consta apenas do documento da equipe, sem execução rastreável [E-46].

8.9 TESTES DE AUTENTICAÇÃO

A autenticação é coberta por `LoginLockoutPolicyTest` (unitário, valores-limite da quinta falha e do fim do bloqueio), `LoginLockoutIntegrationTest`, `LoginAttemptConcurrencyTest`, `SessionSecurityTest`, `SessionRevocationTest` e `StrongPasswordValidatorTest`, e pelos testes de ponta a ponta `session.spec.ts` e `users.spec.ts` [E-21]. **Verificado nesta auditoria:** no ambiente publicado, requisições sem sessão a `/auth/me`, `/sales/catalog` e `/management/summary` responderam 401 `UNAUTHENTICATED` no formato padronizado, e a navegação para `/gestao` sem *cookie* redirecionou para `/entrar?next=%2Fgestao` [E-04]. O fluxo de login com credenciais não foi exercitado, por ausência de credenciais fornecidas por mecanismo seguro.

8.10 TESTES DE AUTORIZAÇÃO

O `AccessMatrixTest` executa a matriz de acesso versionada em CSV para anônimo, vendedor e gerente; a equipe documenta 260 combinações aprovadas [E-22]. `MethodSecurityHierarchyTest` verifica a segunda barreira (`@PreAuthorize`) e os testes de API verificam a terceira (dono do dado), por exemplo `SalesApiTest.ownershipOnPaymentAndCancel` e `CustomersApiTest.historyHidesOtherSellersTotals`. **Verificado nesta auditoria, na camada de banco:** os 14 testes RB executados com o login de vendedor e os 7 testes RG executados com o login de gerente foram aprovados, incluindo leitura apenas dos próprios pedidos (RB-10), negação de custo, margem e resultado da loja (RB-01 a RB-04) e negação de cancelamento de pedido alheio (RB-13) [E-11].

8.11 TESTES DE BANCO DE DADOS

O pacote SQL da disciplina foi reexecutado integralmente nesta auditoria pelo mesmo script usado no job `bd` da integração contínua, adaptado apenas para executar o `psql` localmente em vez de dentro de um contêiner [E-11]. O **Tabela** 1 resume o resultado.

Tabela 1 **–** Resultado do pacote de testes de banco reexecutado

Grupo	Escopo	Executados	Aprovados	Reprovados

Grupo	Escopo	Executados	Aprovados	Reprovados
BD-01 a BD-29 (com BD-07b)	Regras de cálculo, integridade e transações (superusuário)	30	30	0
RB-01 a RB-14	Restrições do vendedor (RLS e concessões)	14	14	0
RG-01 a RG-07	Operações do gerente	7	7	0
BD-30 a BD-32 (a, b, c)	Evolução: último gerente, resumo do cliente, desconto manual	9	9	0
CC-01	Duas reservas simultâneas da última unidade	1	1	0
Subtotal job `bd`		**61**	**61**	**0**
CT-DEMO-01 a 04, 06 e 07	Massa Olist: contagens, idempotência, estoque, CRUD	7	7	0
Total		**68**	**68**	**0**

Fonte: execução desta auditoria em 24/10/2026, PostgreSQL 16.15, *commit* `3b0242c` [E-11] [E-12].

O resultado reproduz o número declarado pela equipe para o job `bd` (61 PASS). Da etapa Olist, foram executados 7 dos 11 testes declarados: os quatro restantes (CT-DEMO-05) dependem da API em execução para verificar a máscara de documentos e não foram executados.

8.12 TESTES DE REGRESSÃO

A regressão é sustentada por três mecanismos: cada defeito corrigido tem teste nomeado com o identificador do defeito (por exemplo, `[D012]`, `CT-REG-D017`), testes de defeitos conhecidos são marcados com `known-defect` e excluídos apenas enquanto o defeito está aberto, e a integração contínua executa todas as suítes a cada *push* e *pull request* [E-23] [E-20]. O catálogo da equipe registra 16 casos de regressão de defeitos [E-44]. **Verificado nesta auditoria:** os testes de banco que protegem correções de integridade (BD-12, preço mínimo; BD-15, reservado ≤ saldo; RG-07, exclusão física de pedido negada) foram aprovados [E-11].

8.13 TESTES DE RESPONSIVIDADE

Os testes de ponta a ponta verificam ausência de rolagem horizontal em 320 px de largura nas telas dos dois perfis (CT-WEB-109, CT-WEB-121), e as capturas versionadas incluem telas em 390 × 844 px [E-21] [E-30]. **Verificado nesta auditoria:** a tela de login publicada, emulada em 375 × 812 px, reorganiza o painel de identidade e o formulário em uma coluna, sem rolagem horizontal visível (capítulo 10) [E-04].

8.14 TESTES DE ACESSIBILIDADE

A acessibilidade é verificada automaticamente com axe-core contra WCAG 2.2 AA nos testes de ponta a ponta e nas capturas versionadas: as 17 verificações registradas em `reports/marca/index.json`, que correspondem a 20 imagens, indicam zero violação [E-30]. **Verificado nesta auditoria:** na tela de login publicada, os campos possuem rótulos associados, o idioma do documento é `pt-BR` e há um único `h1` [E-04]. Testes com leitores de tela e navegação exclusivamente por teclado não foram realizados nesta auditoria; ferramentas automáticas não detectam todos os problemas de acessibilidade (W3C, 2024).

8.15 TESTES DE DESEMPENHO

Não há teste de desempenho com carga, aquecimento e percentis executado pela equipe, e o RNF-06 permanece não verificado. As medidas disponíveis são operacionais: a equipe registra que a partida a frio da API no plano gratuito do Render caiu de cerca de 280 s para 67 s após o uso de cache AOT da JVM (PR #43) [E-41], e esta auditoria observou tempos de 237 ms a 731 ms em cinco requisições GET sem sessão, com a API aquecida [E-04]. Esses números não caracterizam desempenho sob carga.

8.16 TESTES DE SEGURANÇA AUTORIZADOS

Esta auditoria não executou testes ofensivos, varredura ativa nem exploração de vulnerabilidades. As verificações de segurança realizadas foram passivas e não destrutivas: inspeção de cabeçalhos HTTP das páginas e da API publicadas, verificação do formato das respostas 401, auditoria de dependências com `npm audit` e busca por credenciais no histórico do repositório sem reprodução de valores. Os resultados estão no capítulo 9.

8.17 REGISTRO DE DEFEITOS

O registro da equipe segue o modelo de 13 campos exigido pelo enunciado e contém 20 defeitos: 2 históricos sem identificador do quadro (H-01 e H-02) e 18 com identificador de D003 a D026 [E-23]. A distribuição por severidade está na **Tabela** 2.

Tabela 2 **–** Defeitos registrados pela equipe por severidade

Severidade	Quantidade	Defeitos
Crítica	2	H-01 (quantidade multiplicada duas vezes), D012 (pagamento diferente do total)
Alta	7	D003, D004, D006, D010, D017, D018, D022
Média	7	H-02, D011, D016, D019, D023, D024, D026
Baixa	4	D008, D020, D021, D025
Total	**20**	

Fonte: `docs/defeitos.md` [E-23].

Os 20 defeitos constam como corrigidos, cada um com teste de regressão; nenhum consta como “Encerrado”, porque o critério da própria equipe exige correção na `main`, regressão verde e reteste por quem não implementou, e o registro, em versão 0.1 de 09/10/2026, não foi atualizado com esse reteste [E-23].

Esta auditoria registrou 17 achados adicionais, identificados por A-01 a A-17 e detalhados no Apêndice H. O **Quadro** 25 apresenta o resumo.

Quadro 25 **–** Achados desta auditoria

ID	Artefato	Descrição resumida	Severidade	Prioridade
A-01	Documentação	Documentos com estado desatualizado ou contraditório em relação ao código e ao banco publicado	Baixa	Alta
A-02	Contrato	OpenAPI publicado omite 109 de 201 respostas de erro do contrato v5	Média	Alta
A-03	Teste	CT-WEB-124 depende da versão do Node (falha em 22.23.4)	Baixa	Média
A-04	Dependências	`next` 16.3.8 com avisos de segurança críticos publicados após o último *commit*	Alta	Alta
A-05	Configuração	Páginas sem cabeçalhos CSP, anti-enquadramento e afins	Média	Média
A-06	Repositório	Senha de banco de desenvolvimento em texto no histórico do Git	Média	Média
A-07	Gestão	Pasta oficial do Drive editável por qualquer pessoa com o link	Média	Alta
A-08	Ambiente	Contas de demonstração dos docentes inexistentes no banco publicado	Média	Alta
A-09	Evidência	Relatório versionado do contrato ao vivo cobre 54, e não 58, operações	Baixa	Média
A-10	Evidência	Métricas R1 anteriores à evolução de 14/10/2026 e medição R2 não realizada	Baixa	Média

ID	Artefato	Descrição resumida	Severidade	Prioridade
A-11	Banco	Índice `idx_orders_seller_id` redundante; índices sem uso na amostra de produção	Baixa	Baixa
A-12	Configuração	`spring.jpa.show-sql=true` no perfil padrão	Baixa	Baixa
A-13	Gestão de defeitos	Registro sem encerramento formal e com estados internos divergentes	Baixa	Média
A-14	Gestão	Quadro do Trello sem atualização desde 12/10/2026	Baixa	Média
A-15	Evidência	Capturas de 20/10 com miniaturas de produto quebradas	Baixa	Baixa
A-16	Interface	Barra de progresso quase cheia em “Meu desempenho” quando não há meta	Baixa	Média
A-17	Segurança	Bloqueio por conta permite negação de serviço direcionada a contas conhecidas	Média	Média

Fonte: elaborado pelos autores (Apêndice H).

8.18 ANÁLISE DOS RESULTADOS

A **Tabela** 3 consolida as execuções desta auditoria.

Tabela 3 **–** Execuções realizadas nesta auditoria

Suíte	Ambiente	Executados	Aprovados	Reprovados	Taxa de aprovação
Pacote SQL (job `bd`)	PostgreSQL 16.15	61	61	0	100,0%
Massa Olist (sem API)	PostgreSQL 16.15	7	7	0	100,0%
Validadores de contrato (`npm test` raiz)	Node 22.23.4	47	47	0	100,0%
Lógica do frontend (`npm test`)	Node 22.23.4	42	41	1	97,6%
Verificação de tipos do frontend	TypeScript 6.0.3	1	1	0	100,0%
Verificações GET no ambiente publicado	Navegador integrado	5	5	0	100,0%
Total		**163**	**162**	**1**	**99,4%**

Fonte: execuções desta auditoria em 24/10/2026 [E-04] [E-08] [E-09] [E-11] [E-12].

Não foram executados localmente: testes Java unitários e de integração, testes de contrato ao vivo, testes de ponta a ponta, fumaça do ambiente publicado e construção do frontend. A construção falhou antes da compilação porque o Next.js precisou baixar fontes do Google, bloqueadas no ambiente de análise; a construção da mesma versão está comprovada pela etapa de construção do *job* `frontend` na integração contínua e pela implantação de produção na Vercel em estado READY para o *commit* `3b0242c` [E-10] [E-46] [E-02]. O **Quadro** 26 apresenta a conclusão, na integração contínua, das etapas que executam essas suítes.

Quadro 26 **–** Conclusão das suítes na integração contínua do *commit* de referência

Job	Etapa	Suítes cobertas	Duração do *job*	Conclusão
`test` (*backend*)	*Unit, integration, JaCoCo, PMD, CPD and SpotBugs*; *Verify Flyway schema with Hibernate validation*	Java unitários e de integração, cobertura, análise estática, validação do esquema	2 min 20 s	Sucesso
`bd`	*Run RBAC, CRUD, package tests and CC-01 by psql*; *Olist demo dataset by psql*	Pacote SQL, massa Olist e mascaramento pela API (CT-DEMO-05)	47 s	Sucesso
`contract`	*Static contract audit*; *Live OpenAPI, HTTP errors, session and v4 responses*	Auditoria estática, contrato ao vivo, erros HTTP e sessão	57 s	Sucesso
`frontend`	*Typecheck, generated schemas, unit tests and build*; *End-to-end and accessibility (axe)*	Tipos, esquemas gerados, lógica, construção, ponta a ponta e acessibilidade	4 min 10 s	Sucesso

Fonte: API do GitHub Actions, execução 39668791724 do *workflow* CI, disparada pelo *push* do *commit* `3b0242c` em 21/10/2026 [E-46].

A interpretação dos resultados é a seguinte. A taxa de 99,4% indica que as partes reexecutáveis se comportam como documentado, e a única falha é ambiental. A cobertura da reexecução, contudo, é desigual: a camada de banco foi verificada em profundidade, enquanto a camada HTTP da API, onde residem as barreiras de autorização de URL e de método e o formato de erro da maior parte das respostas, depende de resultados documentados. A aprovação dessas suítes na versão de referência está confirmada pela integração contínua [E-46]; os totais por suíte (233 unitários, 649 de integração, 58 operações e 24 casos de ponta a ponta), porém, permanecem documentados, porque os relatórios e os *logs* da execução não puderam ser baixados nesta auditoria.

8.19 MÉTRICAS DE QUALIDADE

As métricas de código foram medidas pela equipe em duas rodadas, R0 (06/10/2026, antes dos novos testes e correções) e R1 (14/10/2026, após as etapas 2A a 4), a partir dos relatórios de Surefire, JaCoCo, PMD, CPD e SpotBugs gerados por `mvnw -Pintegration verify` [E-22]. A **Tabela** 4 reproduz os valores.

Tabela 4 **–** Métricas de código antes e depois do ciclo de melhoria

Indicador	R0 (06/10)	R1 (14/10)
Testes executados / aprovados	17 / 17	544 / 544
Cobertura de instruções	33,3% (1.385/4.160)	93,1% (13.320/14.310)
Cobertura de *branches*	10,0% (22/221)	80,5% (598/743)
Cobertura de linhas	32,6% (349/1.070)	90,4% (2.168/2.398)
Complexidade ciclomática total / média / máxima	566 / 1,26 / 18	1.300 / 1,42 / 18

Indicador	R0 (06/10)	R1 (14/10)
Métodos com complexidade > 10	1	2
Violações PMD	44	105
Duplicações CPD (blocos / linhas)	9 / 241	18 / 253
Achados SpotBugs	46	83

Fonte: `reports/metricas/comparativo-r0-r1.md`, valores recalculados nesta auditoria a partir dos relatórios brutos de `reports/metricas/r0-baseline/` e `reports/metricas/r1/` [E-22].

A interpretação exige cautela por três motivos. Primeiro, o código cresceu 3,4 vezes entre as rodadas, de modo que o aumento absoluto de achados estáticos não indica piora proporcional: PMD passou de 44 para 105 violações, com 44 delas do tipo `ControlStatementBraces` (estilo), e 72 dos 83 achados SpotBugs são `EI_EXPOSE_REP`/`EI_EXPOSE_REP2` em *records* com listas, padrão aceito pela equipe; nenhum achado SpotBugs tem prioridade alta. Segundo, a cobertura de *branches* de 80,5% deixa cerca de 145 decisões sem exercício, concentradas, segundo a equipe, em caminhos alternativos de regras de negócio; essa é a métrica de maior risco residual. Terceiro, a R1 é anterior à evolução de 14/10 (usuários, desconto, CNPJ alfanumérico, Olist): suas 544 execuções não incluem as classes de teste dessa evolução, como `UsersApiTest` e `ManualDiscountApiTest`, e a medição não foi refeita (A-10). Os relatórios brutos de R0 e R1 (Surefire, JaCoCo, PMD, CPD e SpotBugs) estão versionados em `reports/metricas/`, e esta auditoria recalculou a partir deles todos os valores da **Tabela** 4, com resultado idêntico ao declarado [E-22]. As metas de qualidade propostas (cobertura de instruções ≥ 93%, *branches* ≥ 80%, nenhuma violação nova de prioridade alta) aguardam aprovação da equipe e não estão ativadas na integração contínua [E-45].

As métricas de defeitos e de execução estão na **Tabela** 5.

Tabela 5 **–** Métricas de defeitos e de execução

Métrica	Valor	Base de cálculo
Defeitos registrados pela equipe	20	`docs/defeitos.md`
Defeitos críticos ou altos	9 (45%)	2 críticos + 7 altos
Defeitos corrigidos com teste de regressão	20 (100%)	registro da equipe
Defeitos formalmente encerrados	0	critério de reteste independente não registrado
Densidade de defeitos	3,36 por KLOC	20 ÷ 5,952 KLOC de `src/main/java` (sem linhas em branco, comentários, `import` e `package`)
Achados desta auditoria	17	Apêndice H
Taxa de aprovação nesta auditoria	99,4%	162 ÷ 163
Conformidade estrutural do OpenAPI publicado	100%	58 ÷ 58 operações e códigos de sucesso

Métrica	Valor	Base de cálculo
Documentação de erros no OpenAPI publicado	45,8%	92 ÷ 201 códigos de erro do contrato

Fonte: elaborado pelos autores [E-23] [E-05].

A densidade de 3,36 defeitos por KLOC difere dos 6,2 por KLOC calculados pela equipe em 09/10/2026 porque o denominador cresceu de 2,43 para 5,952 KLOC com o código novo, sem novos defeitos registrados no período; o valor atual não deve ser lido como melhoria de qualidade do código legado, mas como efeito do crescimento do código.

8.20 RECOMENDAÇÕES DE MELHORIA

As recomendações de qualidade, priorizadas no capítulo 15, são: completar as anotações de resposta de erro para que o OpenAPI gerado coincida com o contrato v5 e incluir os códigos de erro na comparação do `live-contract.mjs`; refazer a medição R2 após a evolução, com os relatórios brutos versionados como em R0 e R1; ativar na integração contínua as metas MQ-01 a MQ-07 depois do voto da equipe; atualizar a matriz de risco para o PDV e o estoque; executar e registrar o reteste independente que encerra formalmente os defeitos; definir e executar um protocolo de desempenho para o RNF-06; e fixar a versão do Node usada nos testes de frontend, ou tornar a formatação independente do ambiente, alternativa já preparada nas correções propostas da seção 15.4; e executar a fumaça publicada pelo *workflow* existente, para que cada execução fique registrada como evidência.

9 SEGURANÇA DA INFORMAÇÃO

Este capítulo separa os controles comprovadamente implementados, verificados no código ou no sistema publicado, das boas práticas recomendadas. Nenhum teste ofensivo, exploração de vulnerabilidade ou alteração destrutiva foi realizado, e nenhum dado sigiloso é reproduzido.

9.1 AUTENTICAÇÃO

Implementado. Autenticação por sessão no servidor, senhas com BCrypt de custo 12, execução do BCrypt em todos os casos de falha para evitar enumeração por tempo, mensagem única para falhas, bloqueio de 15 minutos após 5 falhas consecutivas com contador atômico, troca do identificador de sessão no login e política de senha com 12 a 72 caracteres, limite de 72 bytes (limite do BCrypt), verificação contra lista de senhas comuns e proibição de senha igual ao e-mail [E-17]. A política está alinhada às diretrizes atuais, que privilegiam comprimento mínimo e verificação contra senhas comprometidas em vez de regras de composição (NIST, 2025).

Risco identificado. O bloqueio por conta, sem limitação por endereço de origem, permite que alguém que conheça o e-mail de uma conta a bloqueie repetidamente enviando cinco senhas erradas a cada 15 minutos. Os e-mails das quatro contas da equipe estão documentados no README (achado A-17). O efeito é de indisponibilidade da conta, não de acesso indevido.

9.2 AUTORIZAÇÃO

Implementado. Três barreiras independentes (URL, método e dono do dado), negação por padrão e hierarquia GERENTE > VENDEDOR (seção 5.10). A resposta 404 para objetos de outro vendedor atende ao risco API1:2023, autorização quebrada em nível de objeto, e as projeções sem custo, margem ou documento completo para o vendedor atendem ao risco API3:2023, autorização quebrada em nível de propriedade (OWASP, 2023). A rejeição de propriedades desconhecidas no corpo impede atribuição em massa, como o envio do vendedor, do preço ou do total pelo navegador [E-17] [E-19]. **Verificado nesta auditoria:** respostas 401 para rotas

protegidas e para rota inexistente sem sessão no ambiente publicado [E-04]; restrições equivalentes na camada de banco pelos testes RB [E-11].

9.3 GERENCIAMENTO DE SESSÕES

Implementado. *Cookie* `JSESSIONID` com `HttpOnly`, `Secure` (configurável por `SESSION_COOKIE_SECURE`, verdadeiro por padrão) e `SameSite=Lax`; expiração por inatividade de 30 minutos; *token* CSRF sincronizador exigido em toda mutação; revogação de sessões na requisição seguinte à mudança de papel, situação ou senha, por meio de `session_version` [E-17].

Recomendado. Duração absoluta máxima de sessão, prefixo `__Host-` no *cookie* e limitação de tentativas por origem, itens registrados pela equipe como trabalho futuro (D-EVO-27) [E-42].

9.4 VALIDAÇÃO DE ENTRADAS

Implementado. Bean Validation nos DTOs (`@NotNull`, `@Min`, `@Max`, `@Size`, `@Pattern`, `@DecimalMin`, validadores de CPF e CNPJ), rejeição de propriedades desconhecidas e de decimais em campos inteiros, consultas SQL parametrizadas no `JdbcClient` e no JPA, e restrições `CHECK` no banco como última barreira. Requisições barradas pelo *firewall* HTTP do Spring Security (por exemplo, com `;` ou `%2e%2e` no caminho) respondem 400 no formato padronizado, e não 500 [E-17] [E-18].

9.5 PROTEÇÃO DE DADOS

Implementado. O sistema trabalha apenas com dados sintéticos e com um recorte público anonimizado da Olist, com nomes e documentos substituídos (OLIST, 2018); do cartão, apenas os quatro últimos dígitos são armazenados, e a coluna legada `card_number` é obrigatoriamente nula por restrição (`ck_payment_card_number_absent`), o que atende à restrição de armazenamento do número completo do cartão (PCI SECURITY STANDARDS COUNCIL, 2024); documentos de clientes são mascarados para o vendedor; a trilha de auditoria não registra senha, *hash* nem documento completo, e falhas de login com e-mail inexistente registram apenas o *hash* SHA-256 do e-mail tentado [E-16] [E-17].

Não implementado. Criptografia de colunas específicas no banco e política de retenção e descarte de dados de auditoria.

9.6 SEGURANÇA DE APIS

O **Quadro** 27 relaciona os riscos do OWASP API Security Top 10 de 2023 aos controles observados.

Quadro 27 **–** Riscos do OWASP API Security Top 10 (2023) e controles do SBI

Risco	Controle observado	Situação
API1 — Autorização em nível de objeto	Dono do dado conferido no serviço; 404 para objeto alheio	Implementado
API2 — Autenticação quebrada	Sessão, BCrypt, bloqueio, CSRF, revogação	Implementado, com risco A-17
API3 — Autorização em nível de propriedade	Projeções sem custo e com documento mascarado; propriedades desconhecidas rejeitadas	Implementado
API4 — Consumo irrestrito de recursos	Paginação e limites de quantidade nos DTOs	Parcial: sem limitação de taxa
API5 — Autorização em nível de função	Regras de URL e `@PreAuthorize` com hierarquia; matriz executável	Implementado
API6 — Acesso irrestrito a fluxos sensíveis	Idempotência da venda; desconto limitado por alçada	Parcial: sem limitação de taxa no login
API7 — Falsificação de requisição no servidor	API não busca URLs fornecidas pelo usuário	Não aplicável
API8 — Configuração de segurança inadequada	Erros sem detalhes internos; cabeçalhos de segurança na API	Parcial: páginas sem CSP (A-05)
API9 — Gestão inadequada de inventário	Contrato versionado; rotas legadas fora do contrato ainda ativas	Parcial
API10 — Consumo inseguro de APIs	Sem consumo de APIs de terceiros em tempo de execução	Não aplicável

Fonte: elaborado pelos autores com base em OWASP (2023) e no código [E-17].

Verificado nesta auditoria. As respostas da API publicada incluem `Cache-Control: no-store`, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY` e `Strict-Transport-Security` [E-04]. As páginas HTML servidas pela Vercel incluem HSTS, mas não incluem `Content-Security-Policy`, `X-Frame-Options` ou diretiva `frame-ancestors`, `X-Content-Type-Options`, `Referrer-Policy` nem `Permissions-Policy` (achado A-05). Sem proteção contra enquadramento, as páginas podem ser exibidas em `iframe` de outro site, o que viabiliza ataques de *clickjacking*; o risco é mitigado, mas não eliminado, pelo `SameSite=Lax` do *cookie*. O documento OpenAPI e a interface Swagger são públicos (`permitAll`), o que é coerente com o objetivo acadêmico de conformidade de contrato, mas expõe o inventário de rotas, inclusive as legadas.

9.7 ARMAZENAMENTO DE CREDENCIAIS

Implementado. Nenhuma senha de banco ou de conta está no código da versão de referência: a senha do banco é lida de `SPRING_DATASOURCE_PASSWORD`, e o `DatasourcePasswordGuard` interrompe a inicialização se ela estiver ausente (correção do defeito D021); as senhas das contas de demonstração vêm de variáveis de ambiente e só são gravadas como BCrypt; os arquivos `.env` e `.env.local` estão no `.gitignore` [E-01].

Risco identificado. O histórico do Git contém, em texto, uma senha de banco de desenvolvimento em `application.properties` e `docker-compose.yml`, introduzida em `application.properties` no *commit* `182a5a2`, de 31/08/2026, e em `docker-compose.yml` no *commit* `153093a`, de 03/10/2026, e removida no *commit* `3753711`, de 14/10/2026 (achado A-06) [E-25]. O valor não é reproduzido neste relatório. O repositório exigiu autorização para clonagem nesta auditoria, o que indica acesso restrito, e não há indício de que a mesma senha seja usada no Neon; ainda assim, a credencial deve ser considerada exposta e rotacionada onde quer que tenha sido usada. O cartão SEC-02 do Trello registrou o problema em 12/10, e o próprio cartão chegou a conter o valor da senha na descrição [E-26].

9.8 VARIÁVEIS DE AMBIENTE

As variáveis estão documentadas por nome, sem valores, em `docs/deploy.md`: no Render, perfil, URL, usuário e senha do banco, senha das contas de demonstração, segurança do *cookie*, registro de SQL, porta e opções da JVM; na Vercel, `BACKEND_ORIGIN` e a variável opcional de aquecimento; no GitHub, o segredo `NEON_API_KEY`, a variável `NEON_PROJECT_ID` e os segredos da fumaça publicada [E-41]. A configuração do `next.config.mjs` rejeita `BACKEND_ORIGIN` com credenciais ou caminho embutidos [E-01].

9.9 CONFIGURAÇÃO DE ACESSO AO BANCO

A API acessa o Neon por conexão direta com TLS obrigatório (`sslmode=require`), com usuário e senha definidos apenas no painel do Render [E-41]. No projeto Neon, a lista de IPs permitidos está vazia e as conexões públicas

não estão bloqueadas, de modo que o banco aceita conexões autenticadas de qualquer origem [E-03]. A medida é comum em planos gratuitos, mas a restrição por IP, quando disponível, reduziria a superfície de ataque. O usuário da aplicação é o dono do esquema; não há usuário de banco com privilégios mínimos para a API, o que faz com que a RLS demonstrada no pacote SQL não proteja os acessos feitos pela aplicação (seção 6.21).

9.10 TRATAMENTO DE EXCEÇÕES

O tratamento centralizado garante que nenhuma entrada inválida resulte em 500 e que o erro 500 não exponha *stack trace*, SQL ou nome de classe (seção 5.12). Os testes `ErrorContractTest`, `ErrorResponseTest` e `DataIntegrityMappingTest` cobrem esse comportamento, com resultado documentado [E-22]. O registro em log do erro inesperado inclui a exceção completa no servidor, o que é adequado para diagnóstico desde que os logs tenham acesso restrito.

9.11 GERENCIAMENTO DE DEPENDÊNCIAS

As versões das dependências do frontend e dos validadores são fixadas de forma exata nos `package.json` e travadas nos `package-lock.json`; as do backend são gerenciadas pelo *parent* do Spring Boot 4.1.0, com versões explícitas para plugins de qualidade e Springdoc. A imagem Docker da API é construída em dois estágios e executada por usuário sem privilégios (UID 10001). Nos *workflows*, as ações oficiais do GitHub são referenciadas por versão principal (`@v4`) e as ações de terceiros do Neon por SHA completo; o *workflow* principal não declara `permissions` mínimas [E-20]. Não há configuração de atualização automática de dependências (Dependabot) no repositório.

Verificado nesta auditoria. O `npm audit` executado em 24/10/2026 apontou seis pacotes vulneráveis no frontend e um na raiz [E-24], conforme o **Quadro** 28.

Quadro 28 **–** Resultado da auditoria de dependências npm

Pacote	Escopo	Severidade	Avisos	Correção indicada

Pacote	Escopo	Severidade	Avisos	Correção indicada
`next` 16.3.8	produção	Crítica	GHSA-2464-fhr4-w9xw (RCE na otimização de imagens), GHSA-673q-mjm6-r5xr (RCE em funções `use cache`), GHSA-p3m5-59mp-jv5r (negação de serviço com *Server Actions*)	`next` ≥ 16.4.1 (o `npm audit` indica 16.4.3)
`undici`	desenvolvimento (via `orval`)	Alta	GHSA-xjj6-f33q-929c, GHSA-4qv8-2p9j-2685, GHSA-f72v-6w45-q472	o `npm audit` propõe `orval` 8.35.0, regressão da versão instalada (8.42.0); requer avaliação manual
`fast-uri`	desenvolvimento (via `orval` e `ajv`)	Moderada	GHSA-w74x-v6f9-j3xg e três avisos de severidade baixa a moderada	`fast-uri` ≥ 3.1.9
`@scalar/json-magic`, `@scalar/openapi-parser`, `orval`	desenvolvimento	Moderada	dependências transitivas dos avisos acima	idem (faixa afetada do `orval`: ≥ 8.36.0)

Fonte: `npm audit` sobre os *lockfiles* do *commit* `3b0242c` [E-24].

O aviso GHSA-2464-fhr4-w9xw foi publicado no banco de avisos do GitHub em 22/10/2026, um dia após o último *commit* analisado, e atribui à biblioteca de imagens `sharp` a possibilidade de execução remota de código quando o Next.js executa em Linux com glibc (GITHUB, 2026). O SBI não usa `next/image`, a diretiva `use cache` nem *Server Actions*, e a otimização de imagens na Vercel é executada pela plataforma, o que reduz a probabilidade de exploração; como o endpoint de otimização existe por padrão, a exposição não foi descartada nesta auditoria e a atualização é recomendada com prioridade alta (achado A-04). Os avisos de `undici`, `fast-uri` e `@scalar/*` afetam ferramentas de desenvolvimento (geração de esquemas e validação de contrato) e não são carregados pela aplicação em produção.

9.12 CONSIDERAÇÕES PERTINENTES SOBRE LGPD

A Lei nº 13.709/2018 aplica-se ao tratamento de dados pessoais, inclusive de clientes de uma loja (BRASIL, 2018). No ambiente acadêmico, o SBI não trata dados pessoais reais de clientes, o que elimina o risco principal. Os dados pessoais efetivamente tratados são os das contas da equipe (nome e e-mail institucional

fictício `@sbi.tech`) e os metadados de acesso na auditoria. Em um uso real, o sistema já incorpora práticas compatíveis com os princípios da necessidade e da segurança: minimização de dados de cartão, mascaramento de documentos por perfil e auditoria de acessos. Faltariam, entre outros, base legal e finalidade documentadas, política de retenção, mecanismo de atendimento aos direitos do titular (acesso, correção, eliminação) e registro das operações de tratamento.

9.13 RISCOS IDENTIFICADOS

O **Quadro** 29 consolida os riscos, diferenciando problemas observados de riscos potenciais.

Quadro 29 **–** Riscos de segurança identificados

ID	Risco	Natureza	Probabilidade	Impacto
A-04	Dependência `next` com avisos críticos	Potencial (exposição não confirmada)	Baixa a média	Alto
A-05	Ausência de CSP e de proteção contra enquadramento nas páginas	Observado	Média	Médio
A-06	Senha de desenvolvimento no histórico do Git	Observado	Baixa (repositório restrito)	Médio
A-07	Pasta oficial do Drive editável por qualquer pessoa com o link	Observado	Média	Médio (integridade das evidências)
A-17	Bloqueio de contas conhecidas por terceiros	Potencial	Média	Baixo a médio (indisponibilidade)
—	Banco publicado aceita conexões de qualquer IP	Observado	Baixa (exige credencial)	Alto
—	Rotas legadas fora do contrato, ativas para o gerente	Observado	Baixa	Médio

Fonte: elaborado pelos autores [E-04] [E-24] [E-25] [E-27].

9.14 RECOMENDAÇÕES

As recomendações de segurança, em ordem de prioridade, são: (1) atualizar `next` para versão corrigida e as ferramentas de desenvolvimento afetadas; (2) configurar cabeçalhos de segurança nas páginas pelo `next.config.mjs` (`Content-Security-Policy` com `frame-ancestors 'none'`, `X-Content-Type-Options`, `Referrer-Policy` e `Permissions-Policy`); (3) restringir a pasta do Drive a leitores nomeados; (4) rotacionar a senha exposta no histórico e, se a política da equipe permitir, reescrever o histórico para removê-la; (5) adicionar limitação de taxa por origem no

login, para reduzir o bloqueio deliberado de contas; (6) criar usuário de banco com privilégios mínimos para a API; (7) declarar `permissions: contents: read` nos *workflows* e habilitar atualização automática de dependências; (8) restringir o Swagger público ao contrato v5 ou desativar as rotas legadas. As recomendações (1), quanto ao `next`, e (2), além da desativação do registro de SQL no perfil padrão, foram implementadas em correções propostas e validadas localmente (seção 15.4); com elas, o `npm audit` deixou de apontar vulnerabilidades nas dependências de produção, e os avisos restantes ficaram restritos às ferramentas de desenvolvimento trazidas pelo `orval` [E-48].

10 INTERFACE E EXPERIÊNCIA DO USUÁRIO

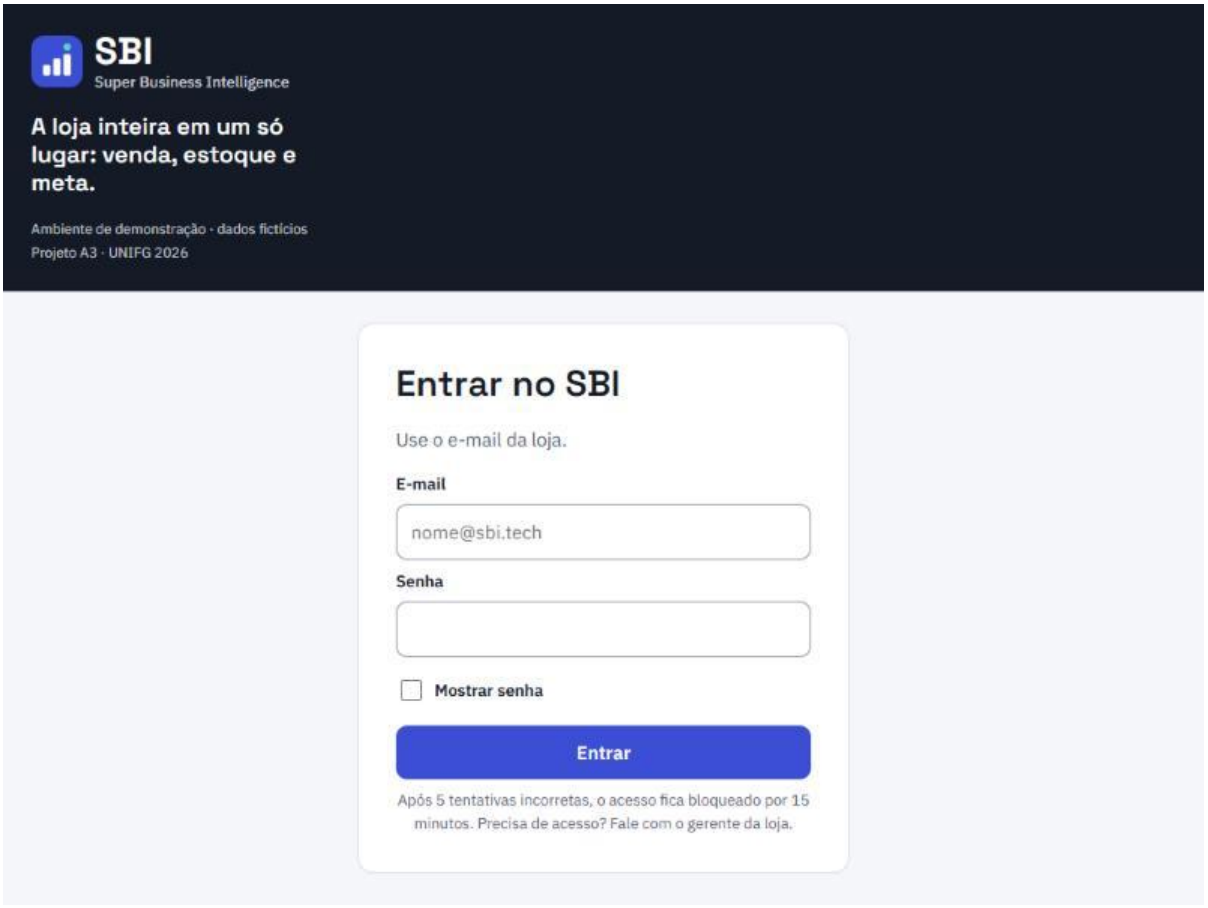
As capturas deste capítulo têm duas origens, indicadas na fonte de cada figura: capturas feitas nesta auditoria no ambiente publicado, restritas à tela de login, que não exige credencial [E-04], e capturas versionadas pela equipe em `reports/marca/`, geradas em 20/10/2026 na pilha de demonstração local com dados fictícios e verificadas com axe [E-30]. As telas autenticadas não foram capturadas ao vivo nesta auditoria pelos motivos expostos na seção 1.6.

10.1 ORGANIZAÇÃO VISUAL

A interface aplica o manual de marca SBI v1.0, com *tokens* de cor, tipografia e espaçamento em `frontend/app/brand-tokens.css`, símbolo, logotipo e ícones em `frontend/components/brand/` e três famílias tipográficas (Space Grotesk para títulos, IBM Plex Sans para texto e IBM Plex Mono para valores) [E-01]. O *layout* autenticado usa barra lateral fixa com o logotipo, o menu por seções e, no rodapé, o avatar, o nome e o papel do usuário e o botão Sair; a área principal tem cabeçalho com título que expressa a conclusão da tela, ação principal à direita e cartões de conteúdo.

A tela de login, verificada no ambiente publicado, divide-se em painel de identidade (logotipo, proposta de valor, aviso de dados fictícios e identificação do projeto A3) e formulário, conforme a **Figura** 12.

Figura 12 **–** Tela de login no ambiente publicado, em largura de computador



Fonte: captura desta auditoria em https://sbi-app-alpha.vercel.app/entrar, 24/10/2026 [E-04].

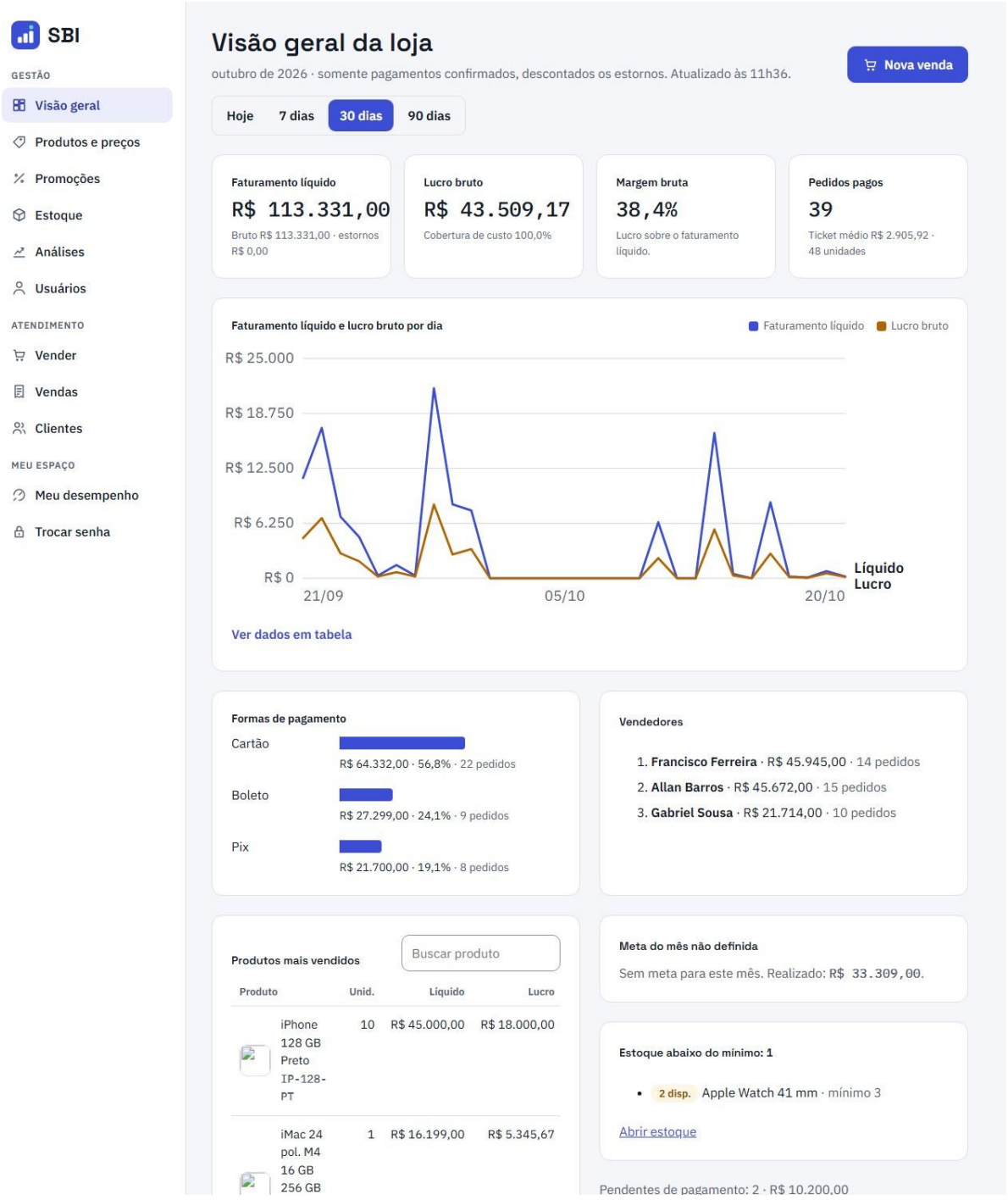
10.2 NAVEGAÇÃO

O menu é agrupado em três seções: GESTÃO (Visão geral, Produtos e preços, Promoções, Estoque, Análises e Usuários), visível apenas ao gerente; ATENDIMENTO (Vender, Vendas e Clientes); e MEU ESPAÇO (Meu desempenho e Trocar senha). O item ativo é marcado visualmente e por `aria-current`, verificado pelo teste CT-WEB-120 em todas as rotas dos dois perfis [E-21]. Rotas internas acessadas sem sessão redirecionam para o login com o parâmetro `next`, e o login ignora destinos externos nesse parâmetro, o que evita redirecionamento aberto (CT-WEB-110) [E-04] [E-22]. Quando o vendedor acessa uma URL de gestão, a interface exibe “Sem acesso a esta área” e a API responde 403.

10.3 HIERARQUIA DE INFORMAÇÕES

As telas seguem a ordem conclusão, indicadores, detalhe. Na visão geral do gerente, os quatro indicadores (faturamento líquido, lucro bruto, margem bruta e pedidos pagos) precedem o gráfico diário, as formas de pagamento, o ranking de vendedores, os produtos mais vendidos, a meta e o alerta de estoque, conforme a **Figura** 13. O subtítulo informa o período e a regra de cálculo (“somente pagamentos confirmados, descontados os estornos”) e o horário da última atualização.

Figura 13 **–** Visão geral da gestão (perfil gerente)



Fonte: captura versionada `reports/marca/gerente-gestao.png`, 20/10/2026, recortada na parte superior [E-30].

10.4 COMPONENTES REUTILIZÁVEIS

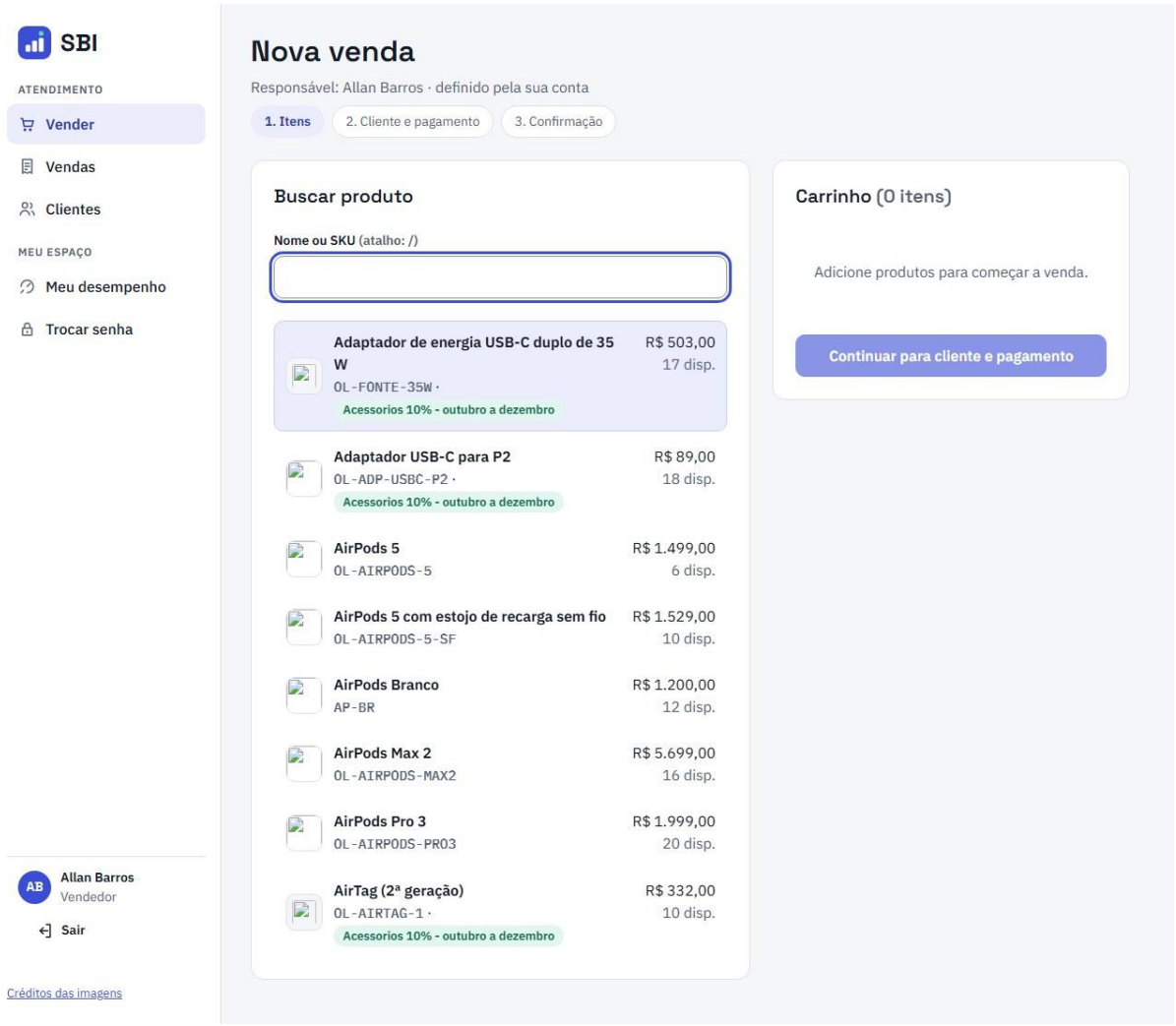
Os componentes compartilhados em `frontend/components/` incluem barra lateral e *links* de navegação, gráficos (linha, barras e barra de meta) com alternativa em tabela, diálogo de confirmação para ações irreversíveis (`ConfirmDialog`), aviso

de sucesso (`StatusMessage`), mensagens de estado, gaveta lateral para formulários, paginação, abas, miniatura de produto, formulário de cliente e opção “Mostrar senha” [E-01]. Os diálogos de confirmação e os avisos de sucesso foram introduzidos após a avaliação heurística de 20/10/2026, que apontou cancelamento de pedido, encerramento de promoção e inativação de produto executados no primeiro clique [E-38].

10.5 FORMULÁRIOS

Os formulários validam os dados com os esquemas gerados do contrato, o que mantém as regras da interface iguais às da API; o formulário de cliente usa, além disso, React Hook Form. Os campos têm rótulo visível, mensagens de erro em português derivadas do `ErrorCode` e foco movido para o primeiro campo com erro (CT-WEB-112) [E-22]. A tela de login publicada usa `type="email"`, `autocomplete="username"` e `autocomplete="current-password"`, o que favorece gerenciadores de senha [E-04]. O PDV, na **Figura** 14, organiza a venda em três etapas (Itens, Cliente e pagamento, Confirmação), com busca por nome ou SKU acionável pelo atalho de teclado “/”, disponível por produto, promoção vigente identificada no item e carrinho lateral.

Figura 14 **–** Ponto de venda, etapa de itens (perfil vendedor)



Fonte: captura versionada `reports/marca/vendedor-vender.png`, 20/10/2026 [E-30].

10.6 FEEDBACK DAS OPERAÇÕES

O sistema informa o resultado das operações por avisos de sucesso, mensagens de erro específicas e estados de carregamento. No ambiente publicado, a tela de login avisa após 8 segundos que o servidor pode estar acordando, consequência da suspensão da API no plano gratuito [E-41]. A política de nunca exibir erro de carga como zero nos indicadores está registrada e coberta por teste [E-22].

10.7 TRATAMENTO DE ESTADOS

As telas tratam os estados de carregamento, vazio, erro e sucesso. A **Figura** 15 mostra o estado “Sem meta” na tela Meu desempenho: os campos Meta,

Atingimento e Falta exibem “Sem meta”, conforme RN-10. A mesma figura evidencia o achado A-16: sem meta, a barra “Realizado em relação à meta” aparece preenchida em cerca de 87% da largura. A análise do componente `GoalBar` confirma a causa: a escala é `max(meta, realizado, 1) × 1,15`, que, com meta zero, torna a barra proporcional apenas ao próprio realizado. O rótulo acessível informa corretamente “sem meta”, mas a representação visual sugere um atingimento inexistente.

Figura 15 **–** Meu desempenho sem meta definida (perfil vendedor)



Fonte: captura versionada `reports/marca/vendedor-meu-desempenho.png`, 20/10/2026 [E-30].

10.8 RESPONSIVIDADE

A interface adapta-se a telas estreitas com menu recolhível de 44 px de altura e reorganização dos cartões em uma coluna; os testes de ponta a ponta verificam a ausência de rolagem horizontal em 320 px [E-21]. A **Figura** 16 mostra a tela de login publicada emulada em 375 × 812 px nesta auditoria.

Figura 16 **–** Tela de login no ambiente publicado, em largura de celular (375 px)



Fonte: captura desta auditoria, emulação de 375 × 812 px, 24/10/2026 [E-04].

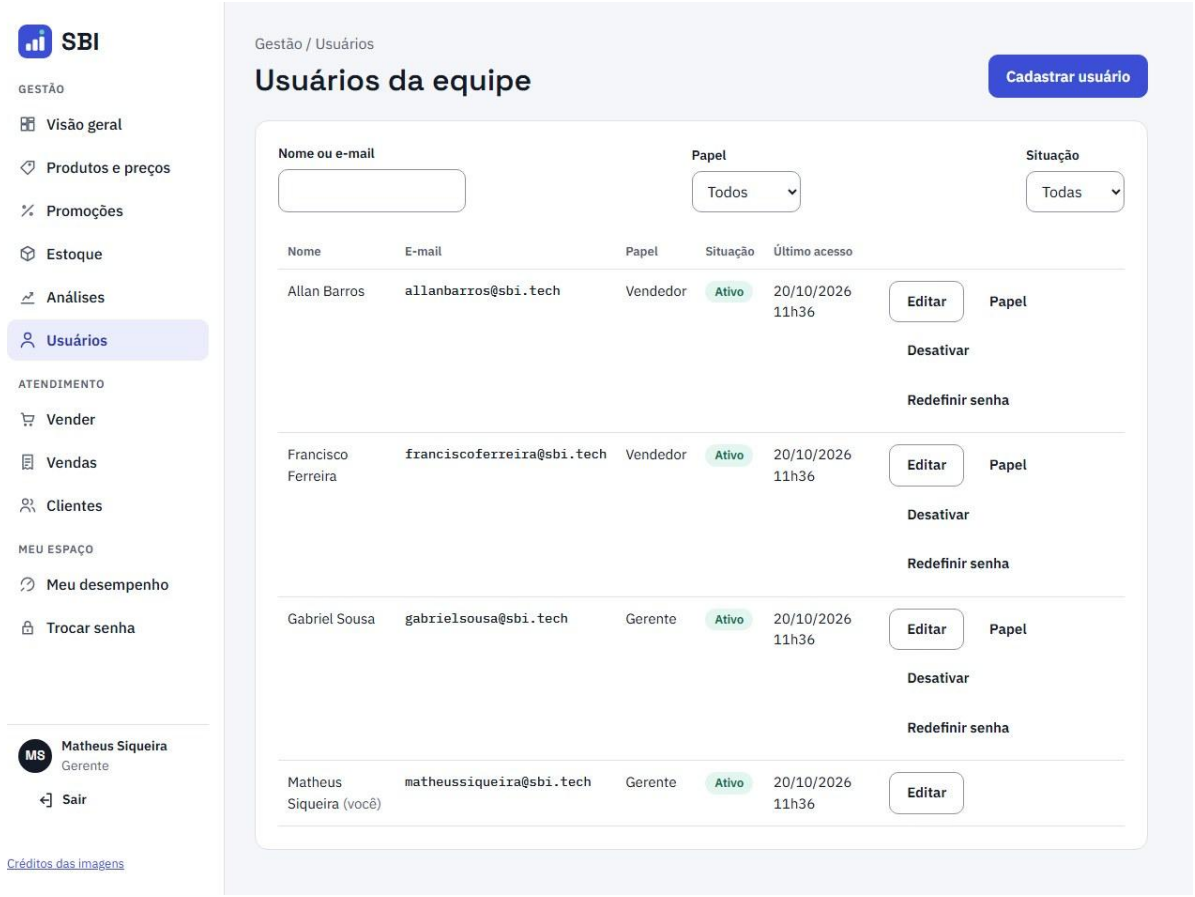
10.9 ACESSIBILIDADE

Os mecanismos implementados incluem idioma `pt-BR`, rótulos em todos os campos, nome acessível único para o logotipo, `aria-current` no item de menu ativo, ícones decorativos com `aria-hidden`, gráficos com rótulo e alternativa em tabela, foco visível e operação do PDV por teclado [E-01]. As 17 verificações registradas no índice das capturas versionadas, que correspondem a 20 imagens, indicam zero violação WCAG 2.2 AA no axe [E-30]. A avaliação automática cobre parte dos critérios; contraste em estados de foco, ordem de leitura com leitor de tela e compreensão das mensagens por usuários com deficiência não foram avaliados com usuários reais [E-38].

10.10 CONSISTÊNCIA VISUAL

A consistência é sustentada pelos *tokens* de marca e por testes que verificam a geometria do símbolo, o conjunto de ícones, os formatos de data, hora e moeda e a ausência de formas proibidas do nome do produto (CT-WEB-122 a CT-WEB-125) [E-21]. Duas inconsistências foram observadas nas capturas. Na tela de usuários, na **Figura** 17, apenas a ação “Editar” tem aparência de botão; “Papel”, “Desativar” e “Redefinir senha” aparecem como texto, o que reduz a percepção de que são ações. Nas capturas de 20/10/2026, as miniaturas de produto aparecem como imagens quebradas (A-15); o relatório de usabilidade atribui o problema à imagem Docker da demonstração local, que não incluía a pasta `public/`, corrigida no PR #45 de 21/10, e os arquivos de imagem respondem 200 no ambiente publicado [E-38] [E-04], mas as capturas não foram refeitas.

Figura 17 **–** Administração de usuários (perfil gerente)



Fonte: captura versionada `reports/marca/gerente-gestao-usuarios.png`, 20/10/2026 [E-30].

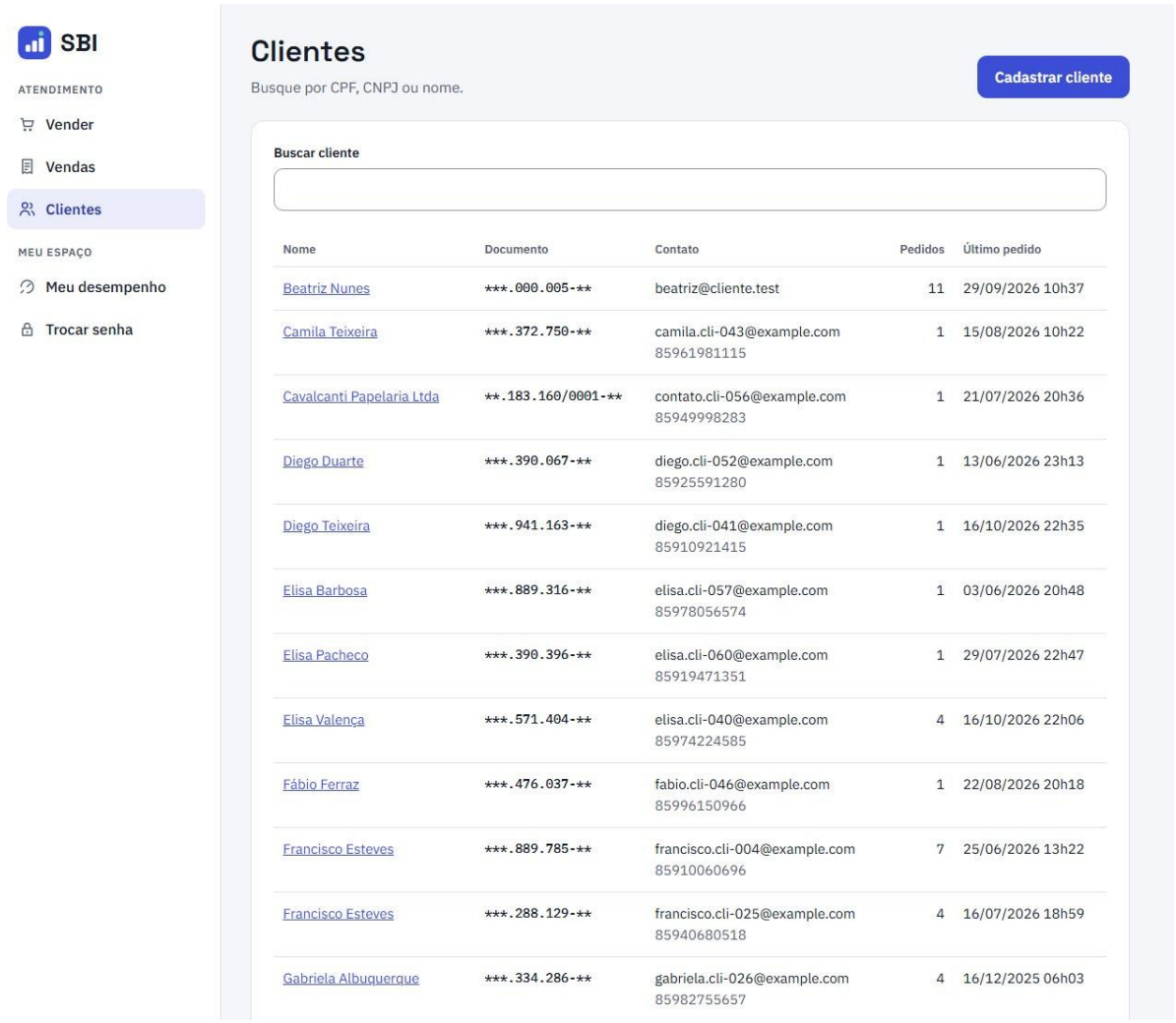
10.11 FLUXOS DE UTILIZAÇÃO

Os fluxos principais, por perfil, são:

- **vendedor**: entrar → Vender (buscar, adicionar itens, escolher cliente e pagamento, finalizar) → confirmar pagamento → consultar Vendas e Meu desempenho;
- **gerente**: entrar → Visão geral → agir sobre exceções (estoque abaixo do mínimo, pedidos pendentes, descontos acima da alçada) → Produtos e preços, Estoque, Promoções, Análises e Usuários;
- **ambos**: Clientes (buscar por CPF, CNPJ ou nome, cadastrar, consultar histórico e indicadores) e Trocar senha.

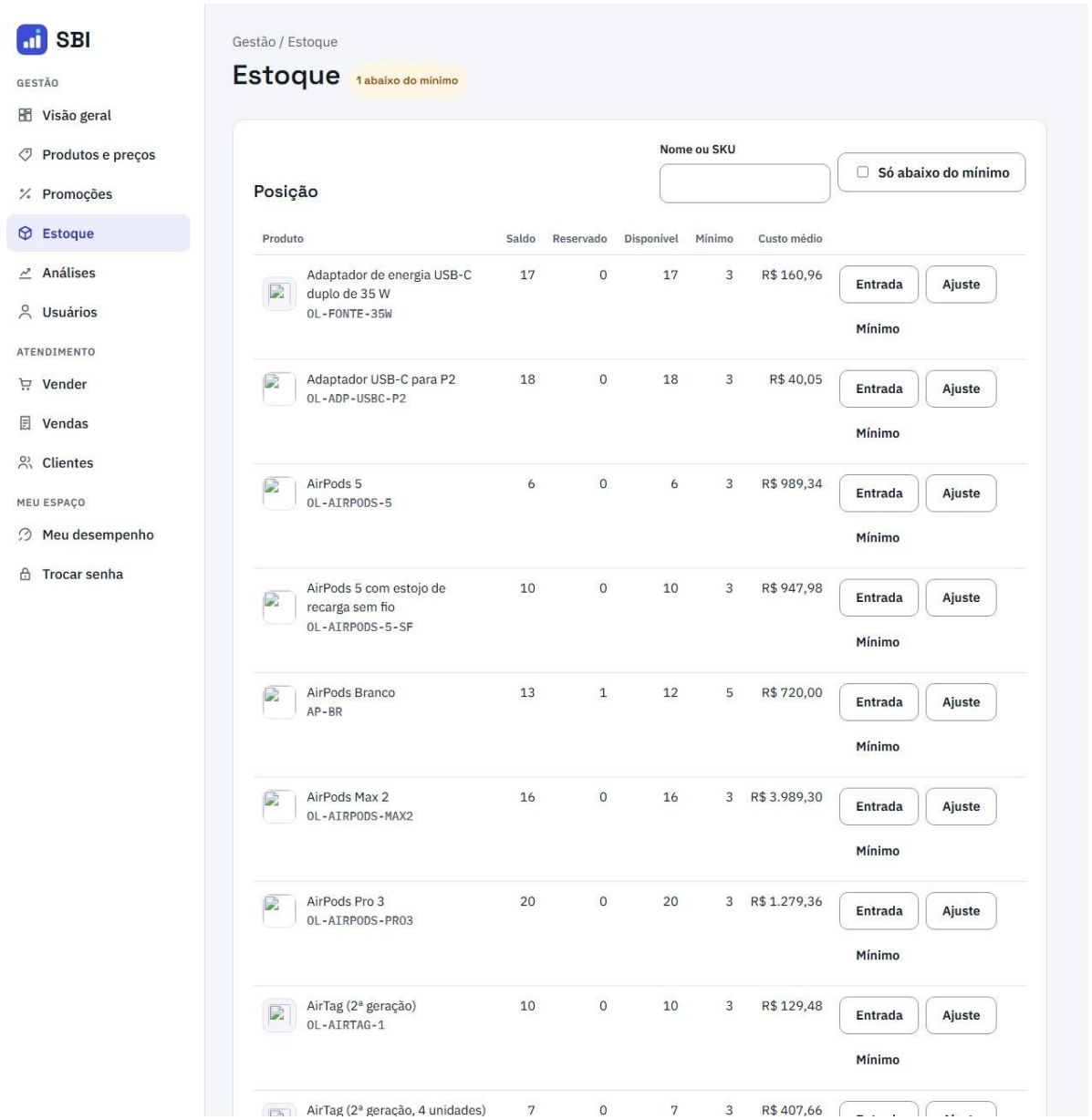
A **Figura** 18 mostra a lista de clientes vista pelo vendedor, com documentos mascarados, conforme RN-14, e a **Figura** 19 mostra a posição de estoque do gerente.

Figura 18 **–** Lista de clientes com documentos mascarados (perfil vendedor)



Fonte: captura versionada `reports/marca/vendedor-clientes.png`, 20/10/2026, recortada [E-30].

Figura 19 **–** Posição de estoque (perfil gerente)



Fonte: captura versionada `reports/marca/gerente-gestao-estoque.png`, 20/10/2026, recortada na parte superior [E-30].

10.12 AVALIAÇÃO CRÍTICA

A interface atende ao objetivo de separar o trabalho do vendedor e do gerente sem expor dados indevidos, e a ligação dos esquemas ao contrato reduz divergências entre tela e API. Os pontos fortes são a hierarquia de informação orientada à conclusão, o tratamento explícito de estados e a verificação automatizada de acessibilidade. As fragilidades são: ausência de validação com usuários reais, reconhecida pela própria equipe [E-38]; representação visual enganosa da meta inexistente (A-16); affordance inconsistente das ações na tela de

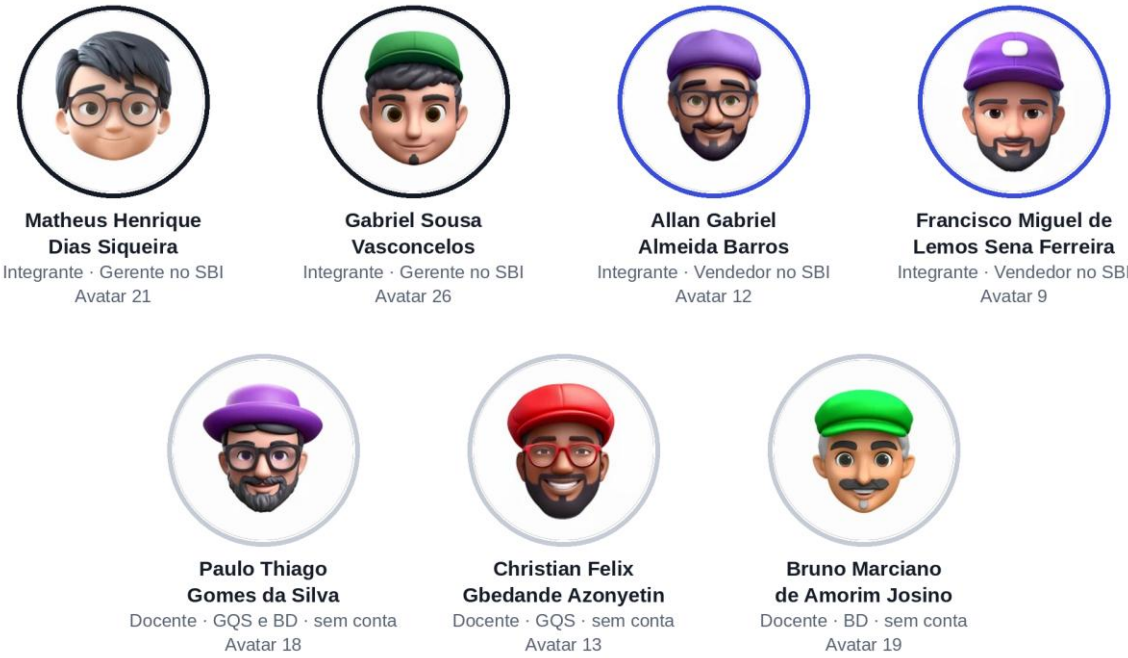
usuários; dependência de uma espera de até 3 minutos na primeira entrada do ambiente gratuito; e evidências visuais desatualizadas (A-15). A recomendação é executar o roteiro de validação com pessoas previsto no relatório de usabilidade, com tarefas cronometradas para o vendedor e o gerente, e refazer as capturas após as correções.

10.13 AVATARES DOS USUÁRIOS E REAÇÕES ANIMADAS

Depois da versão de referência, a equipe abriu, em 24/10/2026 às 22h13 (horário de Brasília), o *pull request* nº 47 (*branch* `feat/user-profile-avatars`, *commit* `a54635d`), que acrescenta avatares ilustrados aos perfis e reações animadas às operações, em 33 arquivos e 685 linhas adicionadas [E-49]. Na data desta análise, o *pull request* estava aberto e não mesclado. A integração contínua aprovou os cinco *jobs* na segunda tentativa; as implantações de pré-visualização na Vercel e uma tentativa de implantação de produção do mesmo *commit* falharam por erro de provisionamento da plataforma (“Resource provisioning failed”), e o ambiente publicado continua no *commit* `3b0242c`, sem os avatares, cujo arquivo de imagem responde 404 [E-49]. A descrição a seguir refere-se ao código do *pull request*. Em 25/10/2026, o *pull request* foi integrado e publicado; a situação posterior à análise está no Apêndice M.

Associação dos avatares. Cada integrante e cada docente recebeu um avatar, conforme a **Figura** 20. O catálogo `frontend/lib/user-avatars.ts` associa o avatar pelo e-mail das quatro contas da equipe ou pelo nome completo, sem diferenciar maiúsculas, acentos e espaços repetidos, e nunca por primeiro nome ou trecho do nome; usuário sem correspondência mantém as iniciais no círculo com a cor do papel. Como os docentes ainda não têm conta no banco publicado (achado A-08), seus avatares só aparecerão se as contas forem cadastradas com o nome completo exatamente como no catálogo. As imagens são arquivos WebP de 192 × 192 px servidos pela própria aplicação e aparecem na área da conta da barra lateral, no cabeçalho compacto do celular e na lista de usuários da gestão; são decorativas (texto alternativo vazio), e o nome continua escrito ao lado.

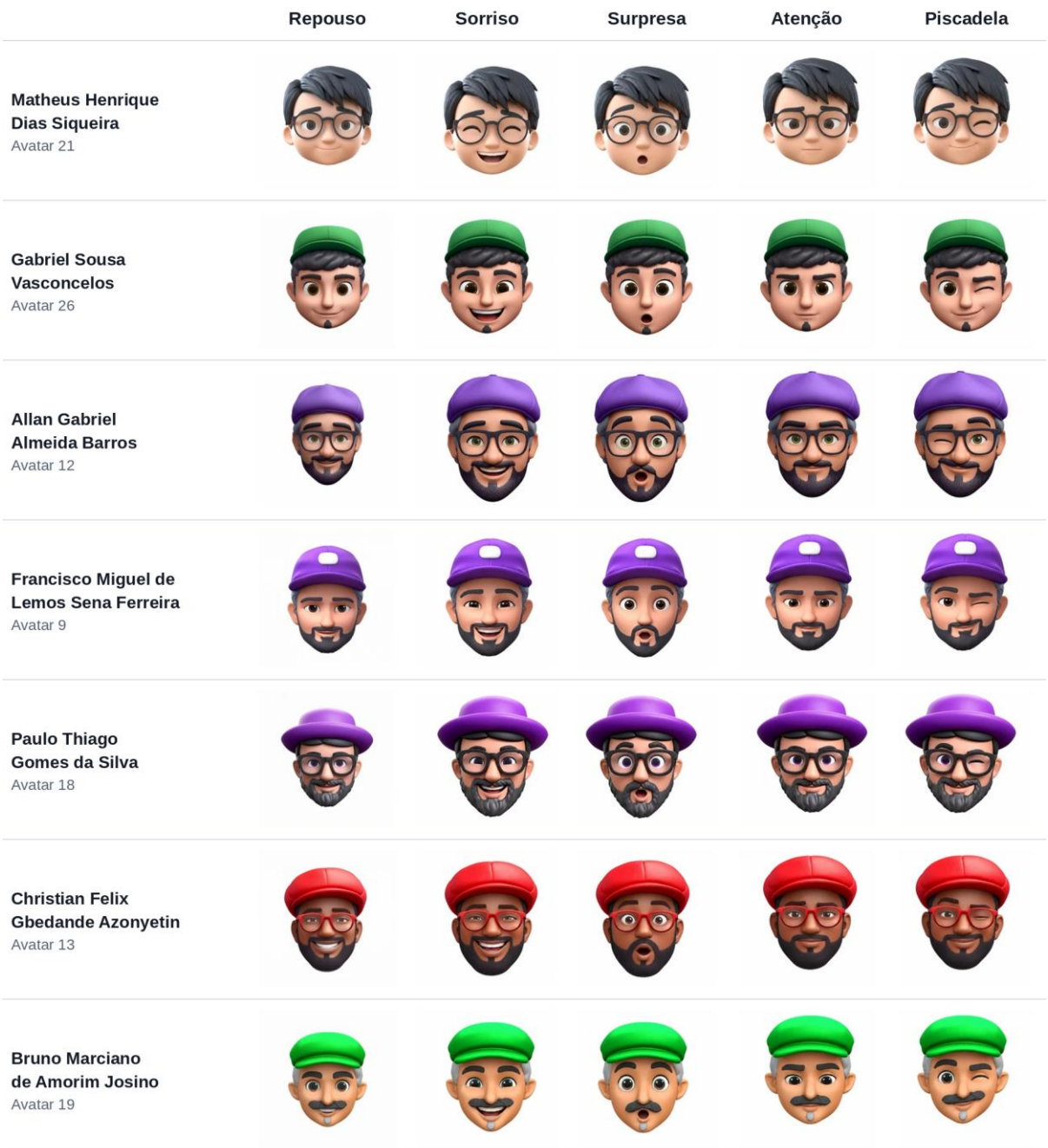
Figura 20 **–** Avatar associado a cada integrante e docente



Fonte: elaborado pelos autores a partir de `frontend/public/avatares/` e `frontend/lib/user-avatars.ts` no *commit* `a54635d` [E-49]. O contorno usa a cor do papel no círculo de iniciais da interface (tinta para gerente, índigo para vendedor) e cinza para quem não tem conta.

Movimento e expressões. Cada avatar tem, além da imagem de repouso, um conjunto de quatro expressões pré-renderizadas (sorriso, surpresa, atenção e piscadela) reunidas em uma única imagem de 384 × 384 px, da qual a interface exibe um quadrante por vez, como mostra a **Figura** 21. O movimento não depende de biblioteca de animação: combina animações e transições CSS, como a oscilação contínua de 4,8 s, o aceno de 650 ms no sorriso, a ampliação de 450 ms na surpresa e a inclinação de até 10 graus em perspectiva sob o cursor, com temporizadores que alternam as expressões em uma sequência ambiente de quadros de 380 a 1.400 ms. A sequência só roda enquanto o avatar está visível e a aba está ativa. Passar o cursor ou levar o foco ao avatar provoca uma piscadela; clique, toque, Enter ou Espaço provocam uma reação, que a cada terceira interação é de surpresa.

Figura 21 **–** Expressões pré-renderizadas de cada avatar



Fonte: elaborado pelos autores a partir das imagens de `frontend/public/avatares/` e `frontend/public/avatares/expressoes/` no *commit* `a54635d` [E-49].

Reações às operações. O avatar da própria conta acompanha as operações de escrita: o `MutationCache` do TanStack Query, configurado no *provider* da aplicação, registra o início, o sucesso e a falha de cada mutação, sem alterar requisições, respostas ou mensagens, e metadados nas mutações do PDV e do detalhe da venda distinguem criação de pedido, confirmação de pagamento e cancelamento [E-49]. Além disso, todo clique dentro das telas internas gera uma

resposta breve no avatar da conta, sem registrar o texto ou os dados do elemento clicado. O **Quadro** 30 resume as regras.

Quadro 30 **–** Reações do avatar da própria conta

Evento	Expressão	Mensagem associada	Duração
Operação de escrita em andamento	Atenção	“Processando a operação…”	Enquanto pendente
Cadastro ou alteração salva	Sorriso, com aceno	“Alteração salva com sucesso!”	3,2 s
Pedido criado no PDV	Sorriso, com aceno	“Pedido criado com sucesso!”	3,2 s
Pagamento confirmado	Sorriso, com aceno	“Pagamento confirmado!”	3,2 s
Pedido cancelado	Atenção	“Pedido cancelado.”	3,2 s
Falha `STOCK_INSUFFICIENT` ou `DISCOUNT_ABOVE_LIMIT`	Surpresa, com ampliação	“Uma condição da operação precisa de atenção.”	3,2 s
Outras falhas	Atenção	“A operação não foi concluída. Confira a mensagem na tela.”	3,2 s
Clique em qualquer ponto das telas internas	Piscadela; surpresa a cada terceiro clique	“Interação recebida.”	0,85 s

Fonte: `frontend/lib/avatar-reactions.ts`, `frontend/app/providers.tsx` e `frontend/components/user-avatar.tsx` no *commit* `a54635d` [E-49].

As regras protegem a informação principal: um sucesso concorrente não esconde um alerta ativo, cancelamentos e falhas nunca exibem sorriso, consultas periódicas não disparam reações, e a saída ou troca de conta descarta resultados anteriores. A mensagem associada aparece apenas no título do botão do avatar; a explicação acessível de cada operação continua nas mensagens de texto da tela. Os colegas listados na gestão não reagem às ações de outro usuário.

Acessibilidade e controle. O avatar é um botão de 44 × 44 px com nome acessível (“Interagir com o avatar de…”) e foco visível. O controle “Pausar avatares”, na área da conta, interrompe o movimento de todos os avatares e as reações a cliques, o que atende ao critério 2.2.2 (Pausar, parar, ocultar) das WCAG 2.2 para conteúdo em movimento que dura mais de cinco segundos (W3C, 2024). Com a preferência do sistema por movimento reduzido (`prefers-reduced-motion: reduce`), não há piscadela automática, inclinação nem transições, e as expressões de resultado mudam de forma estática. Se o conjunto de expressões não carregar, permanece a imagem original; se esta também falhar, permanecem as iniciais.

Verificação. Os testes unitários do *pull request* somam 11 casos de avatar (8 de reações e 3 de associação), todos aprovados nesta auditoria em Node 22.23.4; a suíte completa do frontend no *commit* `a54635d` teve 52 de 53 aprovados, e a única falha é o CT-WEB-124 (achado A-03). Com as correções propostas da seção 15.4 aplicadas sobre o *pull request*, sem conflitos, a suíte passou a 54 de 54 [E-49]. O

teste de ponta a ponta `avatars.spec.ts`, que confere os 14 arquivos de imagem, exercita uma conta autenticada com expressões contínuas, cliques repetidos, teclado, pausa e movimento reduzido, executa o axe com as regras WCAG 2.2 AA e verifica as larguras de 320, 390 e 760 px, foi aprovado no *job* `frontend` da integração contínua do *pull request*, sem reexecução local [E-49].

Avaliação. O recurso acrescenta retorno visual às operações sem substituir as mensagens de texto e respeita a preferência de movimento reduzido e o controle de pausa. Quatro pontos merecem atenção antes da integração: (1) as imagens vêm do arquivo comunitário “Memoji Avatars (Community)” do Figma, e a equipe registra que não presumiu licença específica, de modo que convém confirmar os termos de uso antes de mantê-las no ambiente público, já que as expressões são versões derivadas, conforme a página de créditos da aplicação; (2) a associação dos docentes depende do nome completo, e o catálogo deve receber os e-mails das contas quando forem criadas; (3) a pausa não persiste após recarregar a página, o que pode incomodar quem prefere a interface estática sem ter configurado movimento reduzido no sistema; e (4) a implantação depende da resolução das falhas de provisionamento na Vercel e da restrição de faturamento que impediu a primeira execução da integração contínua.

11 GERENCIAMENTO DO PROJETO

11.1 PLANEJAMENTO INICIAL

O planejamento inicial está em dois documentos de 26 de setembro de 2026 armazenados no Drive: o Plano de Execução e o Manual do projeto v1.0 [E-29]. Eles definem como sistema sob teste a Order Management API, organizam o trabalho em nove semanas (28/09 a 30/11), estimam 226 horas de esforço (260 horas com 15% de reserva), fixam três marcos internos (planejamento aprovado, suíte completa e rascunho revisado) e registram três pendências da primeira semana: autorização para grupo de quatro integrantes, rubrica de Banco de Dados e confirmação do prazo. O escopo de loja com gerente, vendedor, metas e estoque não aparece nesses documentos; ele surge no Trello em 1º de outubro e se torna o escopo efetivo a partir de 3 de outubro [E-26] [E-36].

11.2 DIVISÃO DE ATIVIDADES

As responsabilidades planejadas aparecem em três versões, resumidas no **Quadro** 31.

Quadro 31 **–** Responsabilidades planejadas por integrante

Integrante	Plano de Execução (26/09)	Plano de Qualidade (09/10)	Cartão “COMO COMEÇAR” (12/10)
Allan Gabriel Almeida Barros	Requisitos, critérios, catálogo, testes de sistema e aceitação	Verificação: risco, catálogo, defeitos, inspeção, análise estática	Testes e defeitos
Gabriel Sousa Vasconcelos	Liderança técnica, ambiente, CI, caixa-branca, revisão de PRs	Automação, contrato, caixa-branca e níveis	Java, integração e CI
Francisco Miguel de Lemos Sena Ferreira	Plano de Qualidade, inspeção, análise estática, defeitos, relatório	Documentação de qualidade, revisão e segurança/ética	Documentação e interface
Matheus Henrique Dias Siqueira	Banco de dados, métricas, comparação antes e depois	Métricas, regressão, banco e ambiente	Banco, métricas e telas

Fonte: Plano de Execução e Manual (Drive), `docs/plano-qualidade.md` e quadro do Trello [E-29] [E-37] [E-26].

A regra de trabalho registrada é que todo artefato entra na `main` por *pull request* revisado por quem não é o autor e que cada cartão tem responsável e revisor diferentes [E-37].

11.3 FERRAMENTAS COLABORATIVAS

As ferramentas colaborativas identificadas são o Trello, para planejamento e acompanhamento; o Google Drive, para documentos de referência; o GitHub, para código, revisão por *pull request* e integração contínua; e os serviços de implantação Vercel, Render e Neon. Não foram encontradas atas de reunião em nenhuma das fontes; o ritual semanal de 30 minutos previsto no plano não tem registro [E-27] [E-26].

11.4 USO DO TRELLO

O quadro “Conformidade de Contrato da API REST” foi criado em 26/09/2026 e teve a última ação registrada em 12/10/2026 [E-26]. Sua estrutura e seus números estão na **Tabela** 6.

Tabela 6 **–** Estrutura e uso do quadro do Trello

Indicador	Valor
Listas	6
Cartões (nenhum arquivado)	94
Cartões por lista: Referências / Backlog / A fazer / Em andamento / Em revisão / Concluído	5 / 27 / 20 / 17 / 20 / 5
Cartões por responsável no título: Matheus / Gabriel / Allan / Francisco / sem nome	40 / 24 / 15 / 13 / 2
Checklists / itens concluídos / itens totais	46 / 26 / 216 (12,0%)
Cartões com prazo / prazos vencidos em 24/10/2026 fora de “Concluído”	61 / 25
Etiquetas usadas	0
Ações registradas no histórico	448
Comentários	20

Fonte: extração somente leitura do quadro `ombHh0vo` em 24/10/2026 [E-26].

As listas representam estados de um fluxo (Backlog → A fazer → Em andamento → Em revisão → Concluído), e o cartão “COMO COMEÇAR” descreve a regra de só concluir um cartão após *merge* na `main` com integração contínua verde e aprovação de quem não é o autor. Não há registro de *sprints*, limites de trabalho em andamento nem métricas de fluxo, razão pela qual não se atribui Scrum ou Kanban ao projeto (seção 2.17). O quadro deixou de ser atualizado em 12/10/2026, antes da fase de maior volume de entregas (14 a 21/10), e por isso não reflete o estado do código: cartões de funcionalidades implementadas permanecem em Backlog, e 25 prazos estão vencidos (achado A-14).

11.5 USO DO GITHUB

O repositório oficial é `matheussiqueira-dev/SBI--Super-Business-Intelligence`; o histórico mostra que ele é o mesmo repositório antes nomeado `Conformidade-de-Contrato-da-API-REST`, ainda referenciado nos metadados da implantação da Vercel e no *remote* da cópia local de trabalho [E-01] [E-02]. A base de código foi importada do repositório original `JkbSousa/order-management-api`, mantido como referência histórica sem alterações desde 1º/09/2026 [E-26].

O fluxo observado usa *branches* por etapa ou funcionalidade (20 *branches* remotas na data da análise), *pull requests* numerados até o #46 e *merge* na `main`, com *squash* nas integrações mais recentes. Os números de *pull request* citados em mensagens de *commit* e documentos incluem #1 a #10, #13 a #23, #29, #35, #40, #43, #45 e #46 [E-01]. Duas integrações (#21 e #22) foram mescladas pela conta de Gabriel Sousa em 14/10/2026. A integração contínua executa os jobs de higiene (mensagens de *commit*), backend, banco, contrato e frontend; os *workflows* do Neon criam um banco por *pull request* e carregam a massa Olist [E-20].

11.6 CONTROLE DE VERSÕES

A branch `main` tem 113 *commits*, de 31/08/2026 a 21/10/2026, com mensagens no padrão Conventional Commits a partir de 14/10 [E-01]. A distribuição temporal é concentrada: 23 *commits* em 04/10 e 58 em 14/10 (51,3% do total), dia em que as etapas 2A a 5 e os pacotes de evolução foram integrados. A versão de produção corresponde à ponta da `main` (`3b0242c`), e a Vercel mantém as três últimas implantações de produção em estado READY, dos *commits* `b2a3b13`, `884b8d3` e `3b0242c` [E-02].

Os registros indicam duas reescritas do histórico da `main`, em 08/10 e em 20/10/2026, com alteração de nomes e e-mails de autoria e mudança dos *hashes*; os *bundles* de backup estão na pasta local do projeto [E-26] [E-34]. Por esse motivo, *hashes* citados em documentos anteriores a essas datas (por exemplo, `192f068` e `a5e3b51` no registro de defeitos) não existem na `main` atual, e os metadados de autoria devem ser lidos com cautela.

11.7 HISTÓRICO DE ALTERAÇÕES

O **Quadro** 32 resume as alterações por fase, com base no histórico de versões.

Quadro 32 **–** Fases do desenvolvimento segundo o histórico de versões

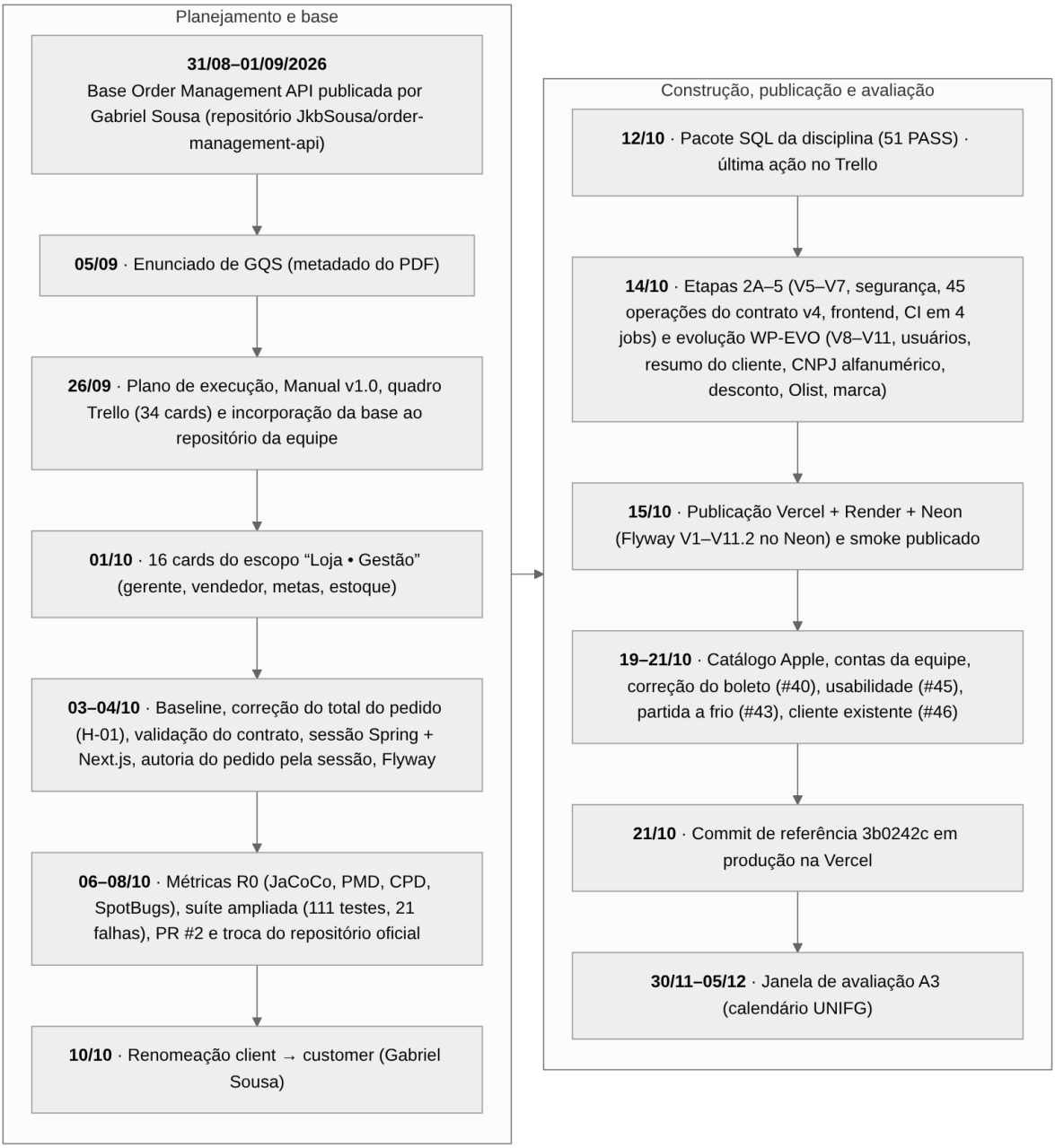
Período	Fase	Entregas principais
31/08 a 01/09	Base	Order Management API (clientes, produtos, endereços, pedidos, pagamentos)
26/09	Incorporação	Base importada para o repositório da equipe
03 a 04/10	Linha de base e acesso	Correção do total do pedido (H-01), contratos OpenAPI e validação, sessão Spring e interface Next.js, autoria do pedido pela sessão, Flyway
06 a 10/10	Medição e correções	R0 (JaCoCo, PMD, CPD, SpotBugs), suíte unitária ampliada, renomeação `client` → `customer`
14/10	ERP/PDV e evolução	V5 a V11, segurança em três barreiras, 45 operações do contrato v4, frontend, CI em quatro jobs, relatório final; usuários, resumo do cliente, CNPJ alfanumérico, desconto manual, massa Olist, marca SBI v1.0
15/10	Publicação	Implantação Vercel + Render + Neon, carga Olist no Neon, fumaça publicada
19 a 21/10	Refinamento	Catálogo Apple com preços e miniaturas, contas da equipe, correção de leitura de boleto, usabilidade, partida a frio, cliente existente no PDV

Fonte: histórico da `main` no *commit* `3b0242c` [E-01].

11.8 MARCOS DE DESENVOLVIMENTO

A linha do tempo da **Figura** 22 combina as datas do histórico de versões, do Trello, do Drive e das implantações.

Figura 22 – Linha do tempo do projeto baseada em evidências



Fonte: elaborado pelos autores a partir de [E-01], [E-02], [E-03], [E-26] e [E-27].

Os marcos previstos no planejamento (M1 em 11/10, M2 em 08/11 e M3 em 22/11) não têm registro formal de aceite. O marco técnico efetivo foi a integração de 14/10, seguida da publicação de 15/10.

11.9 CORREÇÕES E MELHORIAS

As correções seguiram o ciclo registro, teste que falha, correção e regressão. Os 20 defeitos do registro da equipe foram corrigidos, com destaque para os críticos

H-01 (quantidade multiplicada duas vezes) e D012 (pagamento com valor diferente do total) [E-23]. As melhorias posteriores à publicação incluem a correção de erro 500 na leitura de pagamentos por boleto (PR #40), 24 achados de usabilidade corrigidos a partir de avaliação heurística (PR #45), a redução da partida a frio da API (PR #43) e o aproveitamento de cliente existente no cadastro pelo PDV (PR #46) [E-01] [E-38].

11.10 ENTREGAS REALIZADAS

O **Quadro** 33 confronta os dez entregáveis exigidos pelo enunciado de Garantia da Qualidade de Software com os artefatos encontrados.

Quadro 33 **–** Entregáveis exigidos e artefatos correspondentes

Entregável exigido (AZONYETIN, 2026)	**Artefato encontrado**	**Situação**
Código-fonte com `/src`, `/tests`, `/docs`, `/reports`, `/config` e README	Repositório no *commit* `3b0242c` (testes em `src/test` e `frontend/tests`)	Entregue
Plano de qualidade	`docs/plano-qualidade.md` (v0.1, rascunho de 09/10)	Entregue, sem aprovação nominal registrada
Plano de testes	`docs/plano-testes.md`	Entregue
Catálogo de casos de teste	`docs/catalogo-casos.md` (43 casos detalhados) e casos da evolução em `plano-testes.md`	Entregue
Relatório de defeitos	`docs/defeitos.md` (20 defeitos, 13 campos)	Entregue, sem encerramento formal
Relatório de análise estática	`reports/metricas/comparativo-r0-r1.md` e relatórios brutos em `reports/metricas/`	Entregue; valores recalculados nesta auditoria
Relatórios de testes automatizados	Artefatos da CI; `reports/execucoes/`, `reports/contrato/`	Parcial no repositório
Relatório de métricas	`comparativo-r0-r1.md`, `docs/decisoes/metas-qualidade.md`	Entregue; R2 não realizado
Relatório técnico final	`docs/relatorio-final.md` e este relatório	Entregue
Apresentação e demonstração prática	`docs/roteiro-demonstracao.md`, ambiente publicado	Roteiro entregue; apresentação prevista para 30/11 a 05/12

Fonte: elaborado pelos autores a partir do enunciado e do repositório [E-28] [E-01].

11.11 PENDÊNCIAS IDENTIFICADAS

As pendências de gestão são: autorização docente para grupo de quatro integrantes; rubrica escrita da disciplina de Banco de Dados; criação das contas de demonstração dos docentes no ambiente publicado (A-08); aprovação nominal do plano de qualidade, da análise de risco e das metas de qualidade; reteste

independente e encerramento formal dos defeitos (A-13); medição R2; atualização do Trello (A-14) e dos documentos desatualizados (A-01); e restrição de acesso à pasta do Drive (A-07). A situação dessas pendências em 25/10/2026 está no Apêndice M.

11.12 PARTICIPAÇÃO DOS INTEGRANTES

A participação foi analisada com as evidências disponíveis, sem atribuir responsabilidades apenas pelo número de *commits*. O **Quadro** 34 apresenta os indicadores observados.

Quadro 34 **–** Indicadores de participação observados nas fontes

Integrante	***Commits*** **(todas as *branches*)**	**Ações no Trello**	**Outras evidências**
Allan Gabriel Almeida Barros	0	0	Responsável nominal de 15 cartões; indicado como responsável ou revisor em documentos de qualidade (risco, catálogo, defeitos)
Francisco Miguel de Lemos Sena Ferreira	0	1	Responsável nominal de 13 cartões; uma atualização de cartão em 07/10; indicado como responsável pelo plano de qualidade
Gabriel Sousa Vasconcelos	11	0	Autor da base Order Management API (31/08 a 01/09); renomeação `client` → `customer` (10/10); *merge* dos PRs #21 e #22 e tarefa de inicialização do ambiente (14/10)
Matheus Henrique Dias Siqueira	116	447	Autor de 102 dos 113 *commits* da `main`, de todos os 20 comentários e das atribuições de cartões; documentos do repositório indicam preparação por Matheus

Fonte: histórico Git de todas as *branches* e histórico do Trello [E-01] [E-26].

As evidências permitem afirmar a contribuição técnica de Gabriel Sousa (base do sistema, refatoração e integração de *pull requests*) e de Matheus Siqueira (maior parte da implementação, documentação e gestão do quadro). As fontes analisadas não registram produção de código ou de artefatos versionados por Allan Barros e Francisco Ferreira; os documentos atribuem a eles responsabilidades de revisão e reteste, mas as revisões nominais previstas não estão registradas. Essa ausência de registro não prova ausência de contribuição, porque atividades como discussões, testes manuais, revisões orais e preparação da apresentação podem ocorrer fora das ferramentas analisadas, e porque o histórico foi reescrito duas vezes. Recomenda-se que a equipe registre, antes da entrega, a contribuição de cada integrante com evidência verificável, como revisões aprovadas em *pull requests*, retestes assinados e atas.

12 IMPLANTAÇÃO E DISPONIBILIZAÇÃO

12.1 AMBIENTE DE DESENVOLVIMENTO

O ambiente de desenvolvimento documentado pela equipe é Windows com PowerShell, Docker Desktop, JDK 25 e Node 20.9 ou superior, com tarefa do VS Code (“SBI: iniciar tudo”, `Ctrl+Shift+B`) que sobe um PostgreSQL descartável em `127.0.0.1:5434`, a API no perfil `demo` em `localhost:8080` e o frontend em `localhost:3000` [E-01]. Há quatro perfis da API: padrão (banco de desenvolvimento `postgres-order`), `demo` (contas e massa de demonstração), `integration` (testes) e `contract` (API isolada para testes HTTP de contrato).

12.2 PRÉ-REQUISITOS TÉCNICOS

O **Quadro** 35 lista os pré-requisitos e indica quais foram confirmados nesta auditoria.

Quadro 35 **–** Pré-requisitos técnicos

Componente	Versão exigida	Uso	Confirmado nesta auditoria
JDK	25	Compilar e executar a API e os testes Java	Não (indisponível no ambiente de análise)
Maven	3.9.11 via Maven Wrapper	Build e testes Java	Não (Maven Central inacessível)
Node.js	≥ 20.9 (CI usa 24)	Frontend e validadores de contrato	Sim, com Node 22.23.4 (uma falha de formatação, A-03)
PostgreSQL	18 (CI e produção)	Banco	Sim, com 16.15 para o pacote SQL
Docker e Docker Compose	Docker Desktop	Banco descartável e demonstração em um comando	Não
Acesso a `fonts.googleapis.com`	—	Build do Next.js (`next/font/google`)	Não (bloqueado no ambiente de análise)

Fonte: README, `docs/decisoes/stack.md` e execuções desta auditoria [E-01] [E-10].

12.3 INSTALAÇÃO DAS DEPENDÊNCIAS

As dependências são instaladas por `npm ci` na raiz (validadores de contrato: `ajv` e `ajv-formats`) e em `frontend/` (119 pacotes instalados nesta auditoria), e pelo Maven Wrapper para a API (`./mvnw -B -q dependency:go-offline`, usado no `Dockerfile`) [E-01]. A instalação por `npm ci` foi executada com sucesso nesta auditoria nos dois diretórios.

12.4 CONFIGURAÇÕES

As configurações da API estão em `application.properties` e nos perfis; a configuração do frontend está em `next.config.mjs` (desativação do cabeçalho `X-Powered-By` e *rewrite* para a API). Nenhum segredo está no repositório; os valores ficam em variáveis de ambiente locais (`.env.local`, ignorado pelo Git) ou nos painéis das plataformas (seção 9.8).

12.5 PROCESSO DE BUILD

A API é construída por `./mvnw -B -q -DskipTests package` e empacotada em imagem Docker de dois estágios: o primeiro, sobre `eclipse-temurin:25-jdk`, compila, extrai o *jar* e treina um cache AOT da JVM; o segundo, sobre `eclipse-temurin:25-jre`, executa a aplicação com usuário sem privilégios e opções de JVM ajustadas para 512 MB de memória [E-01]. O frontend é construído por `next build` na Vercel, que grava o *rewrite* para `BACKEND_ORIGIN` no momento do build; por isso, mudar a URL da API exige novo build, e não apenas reinício [E-41].

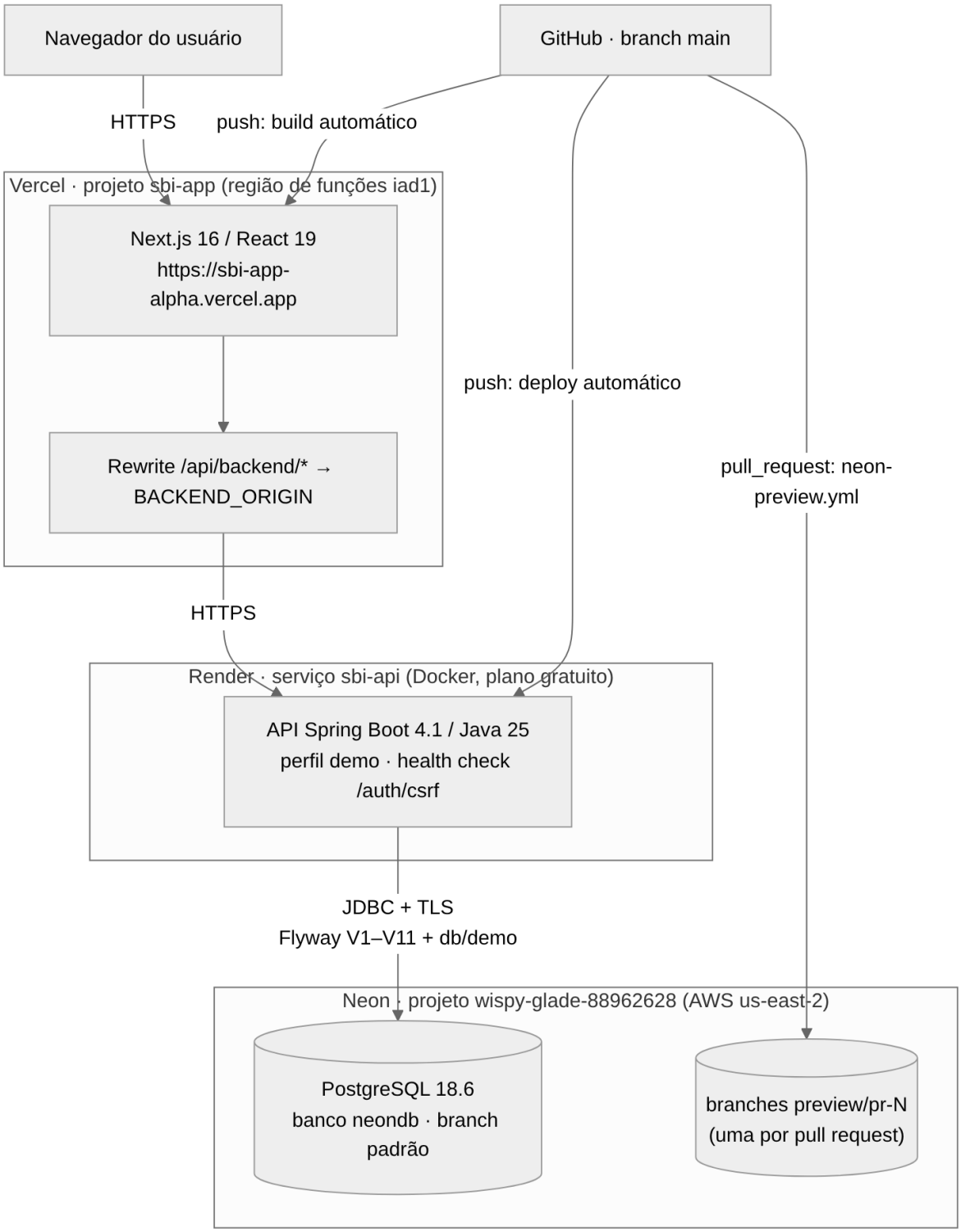
12.6 IMPLANTAÇÃO NA VERCEL

O projeto `sbi-app` da Vercel usa `frontend` como diretório raiz e publica a branch `main` em produção no endereço `https://sbi-app-alpha.vercel.app` [E-41]. Na data da análise, a implantação de produção era `dpl_ApRPtFqstvaRHnEgkRRG2h2NKe7o`, em estado READY, construída a partir do *commit* `3b0242c` da `main` [E-02]. As páginas são servidas pela borda mais próxima do usuário; as requisições desta auditoria foram atendidas pela borda de São Paulo (`gru1`) [E-04].

12.7 INTEGRAÇÃO COM O GITHUB

A **Figura** 23 apresenta a arquitetura de implantação: o navegador acessa o frontend na Vercel, que encaminha as chamadas `/api/backend/*` à API no Render pelo *rewrite*; a API acessa o banco no Neon por JDBC com TLS; e o GitHub dispara a construção e a implantação a cada *push* na `main`.

Figura 23 **–** Arquitetura de implantação do SBI



Fonte: elaborado pelos autores a partir de `docs/deploy.md`, do `next.config.mjs`, dos *workflows* e das plataformas [E-41] [E-02] [E-03] [E-20].

A integração é contínua em dois sentidos. A cada *push* na `main`, a Vercel constrói o frontend e o Render implanta a API, e o Flyway aplica as migrações pendentes no Neon na inicialização. A cada *pull request*, o *workflow* `neon-`

`preview.yml` cria uma *branch* do banco `preview/pr-<número>`, aplica as migrações do PR e a apaga quando o PR é fechado. Os *workflows* manuais `neon-olist.yml` e `published-smoke.yml` carregam a massa Olist no Neon e executam a fumaça somente leitura do site publicado, respectivamente [E-20] [E-41]. Na consulta somente leitura ao Render, o serviço `sbi-api` (serviço web Docker, plano gratuito, região Virgínia, uma instância e verificação de saúde em `/auth/csrf`) tinha implantação automática a cada *commit* na `main` e *previews* de *pull request* desativados. A implantação ativa era a do *commit* `3b0242c`, disparada pelo próprio *commit* e concluída em 21/10/2026 às 17h18 (horário de Brasília), 2 min 17 s após o início; as variáveis de ambiente do serviço não foram lidas [E-47]. O mesmo *push* disparou a integração contínua, concluída com sucesso nos quatro *jobs* (seção 8.18) [E-46]. A cadeia Vercel → Render também foi confirmada pelo cabeçalho `x-render-origin-server` das respostas [E-04].

12.8 VARIÁVEIS DE AMBIENTE

As variáveis por plataforma estão na seção 9.8 e em `docs/deploy.md`. As obrigatórias para a API são `SPRING_DATASOURCE_URL`, `SPRING_DATASOURCE_USERNAME` e `SPRING_DATASOURCE_PASSWORD`; no perfil `demo`, também `SBI_DEMO_PASSWORD` ou as senhas por conta (`SBI_DEMO_PASSWORD_MATHEUS`, `_GABRIEL`, `_ALLAN`, `_FRANCISCO`), lidas apenas na primeira migração [E-41].

12.9 CONFIGURAÇÕES DE PRODUÇÃO

Em produção, a API executa no plano gratuito do Render, em uma instância, com o perfil `demo`, *cookie* `Secure`, registro de SQL desativado e verificação de saúde em `/auth/csrf`; o banco é o PostgreSQL 18.6 do Neon, com conexão direta e TLS obrigatório, *autoscaling* entre 0,25 e 2 unidades de computação e suspensão do *compute* ocioso [E-47] [E-03] [E-41]. A janela de manutenção do projeto Neon é aos sábados, das 07h00 às 08h00 (UTC) [E-03].

12.10 PROCEDIMENTOS DE EXECUÇÃO LOCAL

O procedimento abaixo contém apenas comandos executados com sucesso nesta auditoria, em Linux, sobre o *commit* `3b0242c`. Os scripts auxiliares estão no diretório `reproducao/` entregue com este relatório.

a) Clonar o repositório e posicionar no *commit* de referência: `git clone <repositório> sbi && cd sbi && git checkout 3b0242c`.

b) Validadores de contrato: `npm ci` e `npm test` (47 testes); `node scripts/audit-contract.mjs config/openapi/target-loja-gestao-openapi-2026-10-14-v5.json` e `node scripts/validate-contract.mjs config/openapi/target-loja-gestao-openapi-2026-10-14-v5.json`.

c) Frontend: `cd frontend && npm ci && npm run typecheck && npm test` (42 testes; em Node 22, um falha por formatação, A-03).

d) Banco: iniciar um PostgreSQL local com autenticação `trust` restrita ao *socket* local e superusuário `postgres_test`; executar `reproducao/apply_migrations.sh <repositório> order_management_test`, que aplica V1 a V11 e `db/demo` na ordem do Flyway, substituindo `${flyway:defaultSchema}` por `public`, em transação por arquivo, e criando as contas de demonstração com BCrypt de uma senha aleatória.

e) Pacote SQL: copiar `reproducao/run_bd_local.sh` para `docs/bd/pacote-sql/`, executar nessa pasta `DB=order_management_test SUPERUSER=postgres_test bash run_bd_local.sh` (61 verificações esperadas) e, em seguida, `07_carga_olist.sql` duas vezes, `08_crud_clientes_olist.sql` e `09_testes_olist.sql` com o `psql`.

O procedimento oficial da equipe, que inclui a API e a interface, está documentado no README e não foi executado nesta auditoria por depender de Docker, JDK 25 e Maven Central: definir `SBI_DEMO_PASSWORD` e `SBI_DB_PASSWORD` e executar `docker compose -f docker-compose.demo.yml -p sbi-demo up --build`, com acesso em `http://localhost:3000` e banco de demonstração em `127.0.0.1:5433` [E-01]. O manual técnico completo está no Apêndice I.

12.11 LOGS E MONITORAMENTO

Não há serviço de monitoramento, métricas de aplicação ou alertas configurados no repositório. O diagnóstico depende dos *logs* das plataformas (Vercel e Render) e dos artefatos da integração contínua, que preservam relatórios de testes, cobertura, análise estática e *logs* da API [E-20]. A fumaça publicada é disparada manualmente e funciona como verificação pontual de disponibilidade e acessibilidade, não como monitoramento contínuo; até 24/10/2026, o *workflow* correspondente não tinha nenhuma execução registrada no GitHub Actions [E-46].

12.12 LIMITAÇÕES DA INFRAESTRUTURA

As limitações decorrem do uso de planos gratuitos: a API dorme após inatividade e a primeira requisição pode levar de 1 a 3 minutos, exigindo aquecimento antes de demonstrações [E-41]; o Neon suspende o *compute* ocioso; a sessão em memória de uma única instância impede escalonamento horizontal; não há ambiente de homologação separado além das *branches* por *pull request*; e não há rotina documentada de cópia de segurança do banco publicado além do histórico de seis horas do Neon (`history_retention_seconds` igual a 21.600) [E-03].

13 RESULTADOS OBTIDOS

Este capítulo apresenta apenas resultados demonstrados por evidência. Os indicadores numéricos vêm de execução ou consulta realizada nesta auditoria ou de artefato versionado identificado.

13.1 OBJETIVOS ALCANÇADOS

O **Quadro** 36 confronta os objetivos específicos definidos na seção 1.5 com o resultado obtido.

Quadro 36 **–** Atendimento aos objetivos específicos

Objetivo específico	Resultado	Situação
Inventariar e auditar as fontes oficiais	Repositório, aplicação publicada, integração contínua, Vercel, Render, Neon, Drive e Trello auditados; versão e datas registradas	Atendido
Especificar requisitos, regras, atores e casos de uso	26 RF, 7 RNF, 24 regras, 20 casos de uso e rastreabilidade	Atendido
Descrever arquitetura, tecnologias, API e controle de acesso	Capítulo 5, com diagramas derivados do código	Atendido
Documentar o banco com dicionário derivado do esquema real	19 tabelas e 162 colunas catalogadas; modelos conceitual, lógico e físico	Atendido
Reexecutar verificações viáveis e confrontar resultados	163 verificações executadas, 162 aprovadas; suítes Java, E2E e de contrato ao vivo não executadas localmente, com aprovação confirmada na integração contínua	Parcialmente atendido
Avaliar segurança, interface, gerenciamento e implantação	Capítulos 9 a 12; 17 achados registrados	Atendido
Propor roteiro de evolução	Capítulo 15, com correções de cinco achados preparadas e validadas localmente	Atendido

Fonte: elaborado pelos autores.

13.2 FUNCIONALIDADES CONCLUÍDAS

Na versão de referência, estão implementados e acessíveis pela interface os módulos de autenticação, PDV com cotação, idempotência, reserva e confirmação, vendas, clientes com histórico e indicadores, meu desempenho, visão geral da gestão, produtos e preços, promoções e alçadas de desconto, estoque, análises e usuários [E-01]. Não foram encontradas funcionalidades planejadas para o escopo da Loja apresentadas como entregues sem implementação. Os avatares dos usuários com reações animadas, do *pull request* nº 47, estão em integração e não fazem parte da versão publicada (seção 10.13) [E-49]. Os itens não implementados

são os declarados como trabalho futuro pela equipe (limitação por IP e duração absoluta da sessão) e os ausentes do escopo (pagamento real, fiscal, múltiplas lojas) [E-42]. A situação em 25/10/2026, com os avatares já publicados, está no Apêndice M.

13.3 REQUISITOS ATENDIDOS

Os 26 requisitos funcionais estão implementados (seção 4.1). Quanto ao nível de verificação, 18 deles têm regras confirmadas nesta auditoria pela camada de banco (código BD no **Quadro** 5), 2 têm comportamento confirmado no ambiente publicado (PUB) e todos têm testes automatizados documentados (DOC). Dos 7 requisitos não funcionais, 1 está verificado (RNF-07), 4 estão verificados parcialmente (RNF-01 a RNF-04), 1 está documentado (RNF-05) e 1 não foi verificado (RNF-06, desempenho).

13.4 ESTRUTURAS DE DADOS IMPLEMENTADAS

O banco implementa 19 tabelas, 162 colunas, 18 *views*, 13 funções, 8 gatilhos (3 deles gatilhos de restrição adiados), 52 restrições `CHECK`, 32 chaves estrangeiras, 13 restrições de unicidade e 48 índices, com paridade entre o esquema migrado localmente e o banco publicado nas contagens de tabelas, *views*, funções, gatilhos, restrições `CHECK`, chaves estrangeiras e índices, as únicas consultadas no banco publicado [E-15] [E-03]. As 15 migrações registradas no banco publicado foram aplicadas com sucesso em 15/10/2026.

13.5 RESULTADOS DOS TESTES

Os resultados consolidados estão na **Tabela** 7, que separa os executados nesta auditoria, os confirmados pela integração contínua do *commit* de referência e os documentados pela equipe.

Tabela 7 **–** Resultados de testes por origem

Suíte	Executada nesta auditoria	Resultado nesta auditoria	Resultado documentado pela equipe
Pacote SQL (job `bd`)	Sim	61/61	61 PASS
Massa Olist	Sim (7 de 11)	7/7	11 PASS

Suíte	Executada nesta auditoria	Resultado nesta auditoria	Resultado documentado pela equipe
Validadores de contrato (Node)	Sim	47/47	40/40 (14/10)
Lógica do frontend	Sim	41/42	36/36 (14/10)
Auditoria estática do contrato v5	Sim	0 erros; 236 esquemas sem achados	sem erros
Java unitários	Não	Aprovados na CI de 21/10 (sem totais)	233 aprovados
Java integração	Não	Aprovados na CI de 21/10 (sem totais)	649 aprovados
Contrato ao vivo	Não	Aprovado na CI de 21/10 (sem totais)	58/58 declarado; 54/54 no relatório versionado
E2E com axe	Não	Aprovados na CI de 21/10 (sem totais)	24/24
Fumaça publicada	Não	Sem execução registrada no GitHub Actions	5/5 (15/10)
Verificações GET no publicado	Sim	5/5	—
Comparação do OpenAPI publicado com o v5	Sim	58/58 operações; 92/201 códigos de erro	—

Fonte: execuções desta auditoria, execução 39668791724 da integração contínua e `docs/relatorio-final.md` [E-11] [E-12] [E-08] [E-09] [E-07] [E-05] [E-46] [E-22].

13.6 DEFEITOS IDENTIFICADOS

Foram identificados 37 registros de problema: 20 defeitos do registro da equipe, todos corrigidos, e 17 achados desta auditoria, ainda abertos (seção 8.17 e Apêndice H). Dos achados desta auditoria, 1 tem severidade alta (A-04, dependência com avisos críticos), 6 têm severidade média e 10 têm severidade baixa; 7 são de documentação, evidência ou gestão, 5 de segurança, dependências ou configuração, 2 de contrato ou teste, 1 de interface, 1 de banco e 1 de ambiente.

13.7 PROBLEMAS CORRIGIDOS

Os problemas corrigidos com teste de regressão incluem os críticos H-01 (quantidade multiplicada duas vezes no total) e D012 (pagamento com valor diferente do total), além de boleto vencido aprovado (D003), reprocessamento de pagamento (D004), regressão de status (D010), valores abaixo de meio centavo gravados como zero (D017), condição de corrida no pagamento (D019), número completo do cartão armazenado (D022) e divergências entre o OpenAPI ao vivo e o contrato v4 (D026) [E-23]. Após a publicação, foram corrigidos o erro 500 na leitura de pagamentos por boleto (PR #40) e 24 problemas de usabilidade (PR #45) [E-01]

[E-38]. Além disso, esta auditoria preparou correções para cinco de seus achados (A-03, A-04, A-05, A-12 e A-16), validadas localmente e ainda não aplicadas ao repositório (seção 15.4) [E-48].

13.8 INTEGRAÇÕES COMPROVADAS

As integrações comprovadas nesta auditoria são: GitHub → Vercel (implantação de produção do *commit* `3b0242c`) [E-02]; GitHub → Render (implantação ativa da API do mesmo *commit*) [E-47]; GitHub → GitHub Actions (integração contínua do mesmo *commit*, com os quatro *jobs* aprovados) [E-46]; Vercel → Render (respostas da API pelo *rewrite*, com cabeçalho de origem Render) [E-04]; aplicação → Neon (15 migrações Flyway aplicadas e dados de 291 pedidos) [E-03]; Olist → banco (carga reproduzida localmente com contagens conferidas) [E-12]; e contrato → interface (esquemas Zod gerados do v5 e verificação de tipos aprovada) [E-09].

13.9 DISPONIBILIDADE DA APLICAÇÃO

O site `https://sbi-app-alpha.vercel.app` respondeu às requisições desta auditoria em 24/10/2026, entre 18h58 e 19h00 (horário de Brasília), com a API aquecida e tempos de 237 ms a 731 ms nas requisições GET [E-04]. Não há medição contínua de disponibilidade; a disponibilidade do ambiente gratuito depende de aquecimento prévio, e a primeira requisição após inatividade pode levar de 1 a 3 minutos, segundo a equipe [E-41].

13.10 LIMITAÇÕES

Os resultados têm cinco limitações: a camada HTTP da API e a interface autenticada não foram exercitadas diretamente nesta auditoria, e para elas há apenas a conclusão da integração contínua, sem os totais por suíte; as métricas de cobertura e análise estática, recalculadas dos relatórios versionados, são anteriores à evolução de 14/10; o pacote SQL foi executado em PostgreSQL 16, e não 18; não há medição de desempenho; e a participação dos integrantes não pôde ser plenamente comprovada pelas fontes.

14 DISCUSSÃO

14.1 CONFORMIDADE ESTRUTURAL E CONFORMIDADE DOCUMENTAL DO CONTRATO

O resultado central para o eixo temático é a diferença entre duas medidas de conformidade. A conformidade estrutural do OpenAPI publicado com o contrato v5 é integral: as 58 operações existem, com os mesmos caminhos, métodos e códigos de sucesso [E-05]. A conformidade documental das respostas de erro, porém, é de 45,8%: 109 dos 201 códigos de erro declarados no contrato não aparecem no documento gerado pela API (A-02). Os testes da equipe indicam que esses erros são de fato retornados pela implementação com o formato `ErrorResponse` [E-22]; o defeito, portanto, não é de comportamento, mas de descrição. Ainda assim, ele é relevante para o tema: um consumidor que se oriente apenas pelo Swagger publicado não saberá que uma venda pode responder 409 `STOCK_INSUFFICIENT` ou `IDEMPOTENCY_CONFLICT`, e um verificador automatizado de contrato que compare apenas caminhos e códigos de sucesso, como o relatório versionado do `live-contract.mjs`, não detecta a lacuna.

A interpretação técnica é que o contrato v5 cumpre o papel de especificação normativa e o OpenAPI gerado cumpre o papel de documentação derivada, e as duas fontes só coincidem quando cada resposta de erro é anotada no controlador. A estratégia *contract-first* adotada pela equipe [E-42] é adequada, mas a verificação que a sustenta precisa incluir as respostas de erro, como prevê a especificação OpenAPI ao tratar o objeto de respostas como parte do contrato da operação (OPENAPI INITIATIVE, 2021). O formato de erro adotado, com `code` estável e `details` por campo, aproxima-se do modelo de detalhes de problema da RFC 9457 (NOTTINGHAM; WILDE; DALAL, 2023), embora não use o tipo de mídia `application/problem+json`.

14.2 COMPARAÇÃO COM OS OBJETIVOS

Seis dos sete objetivos específicos foram atendidos integralmente ou com limitação explícita; o objetivo de reexecutar as verificações foi atendido parcialmente (seção 13.1). A limitação decorre do ambiente de análise, sem acesso ao Maven

Central, ao Docker e à fonte remota de fontes tipográficas, e da decisão de não executar operações de escrita no ambiente publicado. Essa decisão protegeu a integridade dos dados de demonstração, mas deixou a camada HTTP autenticada sem verificação independente.

14.3 CONTRIBUIÇÕES DA DISCIPLINA DE BANCO DE DADOS

O banco do SBI não é um repositório passivo da aplicação. Ele concentra regras de integridade que normalmente ficariam apenas no código: 52 restrições `CHECK`, gatilhos que impedem metas inconsistentes e garantem pelo menos um gerente ativo, gatilhos de restrição adiados até o `COMMIT` para validar somas entre linhas e a reserva de estoque por `UPDATE ... WHERE on_hand - reserved >= q`, que resolve a disputa pela última unidade sem bloqueio explícito de leitura [E-16] [E-11]. Essa dupla barreira, na aplicação e no banco, é coerente com a recomendação de manter a integridade declarativa no SGBD sempre que possível (ELMASRI; NAVATHE, 2016) e com o modelo de transações ACID (HAERDER; REUTER, 1983). O teste de concorrência CC-01, reexecutado nesta auditoria, demonstra que duas transações simultâneas sobre a última unidade resultam em exatamente uma reserva [E-11].

Dois pontos limitam a avaliação. Primeiro, as consultas analíticas dos painéis são executadas por *views* sobre o banco transacional. A escolha é adequada ao volume atual, de 291 pedidos, e evita a duplicação de dados, mas a literatura de *data warehousing* recomenda separar a carga analítica em modelo dimensional quando o volume ou a frequência de consulta crescerem (KIMBALL; ROSS, 2013). Segundo, colunas herdadas do legado continuam anuláveis em relacionamentos que o domínio novo trata como obrigatórios, o que transfere para a aplicação uma garantia que poderia estar no esquema. Nenhum dos dois pontos gerou defeito observado, mas ambos condicionam a evolução.

14.4 CONTRIBUIÇÕES DA DISCIPLINA DE GARANTIA DA QUALIDADE DE SOFTWARE

O ciclo de melhoria documentado pela equipe é consistente com o processo de verificação e validação previsto na IEEE 1012 (IEEE, 2016): linha de base com

métricas (R0), correção dos defeitos registrados com testes de regressão, nova medição (R1) e evidência por integração contínua. Os ganhos de cobertura de instruções (33,3% para 93,1%) e de *branches* (10,0% para 80,5%) são expressivos e foram recalculados nesta auditoria a partir dos relatórios brutos versionados [E-22]. O enunciado exige pelo menos 30 casos de teste e três técnicas de caixa-preta (AZONYETIN, 2026); o catálogo da equipe tem 43 casos formalizados, com partição de equivalência, valor-limite, tabela de decisão e transição de estados [E-44], e o inventário estático do repositório registra 44 classes de teste Java e 16 arquivos de teste do frontend [E-21].

A fragilidade está na cadeia de evidência, não na existência dos testes. Quatro achados têm essa natureza: o relatório versionado do contrato ao vivo cobre 54, e não 58, operações (A-09); as métricas R1 são anteriores à evolução integrada no mesmo dia, e a R2 não foi realizada (A-10); nenhum defeito foi formalmente encerrado pelo critério de reteste independente que a própria equipe definiu (A-13); e parte da documentação descreve estados anteriores do sistema (A-01). Como o enunciado só considera válidos resultados reproduzíveis durante a apresentação (AZONYETIN, 2026), esses pontos têm peso maior do que a severidade técnica sugere.

14.5 ESCOPO PLANEJADO E ESCOPO ENTREGUE

O escopo entregue é substancialmente maior que o planejado em 26 de setembro. O Plano de Execução previa testar a Order Management API com doze requisitos funcionais e excluía a interface de loja e a autenticação completa [E-29]; o sistema publicado tem 26 requisitos funcionais, autenticação por sessão com três barreiras de autorização, PDV, estoque com reserva, metas, promoções, alçadas de desconto, administração de usuários e painel analítico. A expansão ocorreu sem atualização do plano no Drive e com o Trello abandonado a partir de 12 de outubro (A-14) [E-26] [E-27].

A expansão tem dois efeitos opostos. Ela aumenta o valor demonstrativo do projeto e sua aderência ao tema de *Business Intelligence* e de banco de dados, mas concentra o trabalho no tempo (51,3% dos *commits* da `main` em um único dia) e dilui a rastreabilidade entre planejamento, execução e evidência, como mostram os achados A-01, A-13 e A-14. A participação comprovada nas fontes é desigual

(capítulo 11), e a composição de quatro integrantes diverge do limite de três estabelecido no enunciado de Garantia da Qualidade, sem autorização registrada [E-28].

14.6 AMEAÇAS À VALIDADE

As ameaças à validade das conclusões, classificadas segundo a prática usual de estudos empíricos em engenharia de software, são as seguintes:

- **validade interna:** o pacote SQL foi executado em PostgreSQL 16.15, e não no 18, e as migrações foram aplicadas por `psql`, e não pelo Flyway; a migração Java V6_1 foi substituída por SQL equivalente. A paridade de contagens com o banco publicado (seção 13.4) reduz, mas não elimina, o risco de diferença de comportamento entre versões;
- **validade de construto:** as 163 verificações executadas não têm o mesmo peso. Sessenta e oito delas exercitam o banco, enquanto a camada HTTP autenticada não foi exercitada diretamente, apenas confirmada pela conclusão da integração contínua. A taxa de aprovação de 99,4% mede a reprodutibilidade do que foi reexecutado, e não a qualidade global do sistema;
- **validade externa:** os dados são de demonstração, com uma loja e quatro usuários; os resultados não se generalizam para volumes ou perfis de uso reais, em particular quanto a desempenho e concorrência;
- **confiabilidade:** a análise é um retrato de 24/10/2026. O ambiente publicado pode mudar a qualquer *push* na `main`, e os dados do Neon mudam a cada demonstração; por isso, cada evidência registra data, versão e origem (Apêndice K);
- **dependência de resultados declarados:** a aprovação das suítes Java, E2E e de contrato ao vivo no *commit* de referência foi confirmada pela integração contínua, mas os totais atuais (233, 649, 24 e 58) vêm de documentos da equipe, porque os relatórios da execução não puderam ser baixados; apenas as rodadas R0 e R1 têm relatórios brutos versionados, e a fumaça publicada não tem execução registrada no GitHub Actions [E-46].

14.7 LIÇÕES APRENDIDAS

Três lições emergem da análise. A primeira é que um contrato só é verificável se a verificação cobrir todas as respostas que ele declara, inclusive as de erro. A segunda é que regras de integridade no banco, quando acompanhadas de testes SQL executáveis, produzem a evidência mais fácil de reproduzir: foi a única camada que esta auditoria conseguiu verificar integralmente fora da infraestrutura da equipe. A terceira é que a qualidade da documentação acompanha a do processo de gestão: quando o quadro de tarefas deixa de ser atualizado, os documentos passam a divergir do sistema, e o custo de demonstrar o que foi feito aumenta.

15 PROPOSTAS DE EVOLUÇÃO

As propostas deste capítulo derivam dos achados e limitações registrados nos capítulos anteriores. Elas foram classificadas por prioridade (P1: antes da apresentação da A3; P2: curto prazo, após a avaliação; P3: médio prazo, para uso fora do contexto acadêmico), impacto e esforço estimado. O esforço é uma estimativa qualitativa desta documentação: baixo (até um dia de trabalho), médio (até uma semana) e alto (mais de uma semana).

15.1 ROTEIRO TÉCNICO PRIORIZADO

O **Quadro** 37 apresenta o roteiro completo.

Quadro 37 **–** Roteiro técnico de evolução

ID	Proposta	Origem	Prioridade	Impacto	Esforço	Dependências
EV-01	Atualizar `next` para versão corrigida (16.4.1 ou superior) e as ferramentas de desenvolvimento com avisos; reexecutar build, testes de frontend e E2E	A-04	P1	Alto	Baixo	Integração contínua verde
EV-02	Criar as contas de avaliação dos docentes no banco publicado, com entrega das senhas por canal seguro e troca obrigatória no primeiro acesso	A-08	P1	Alto	Baixo	Decisão da equipe sobre os e-mails das contas
EV-03	Restringir a pasta oficial do Drive a leitores nomeados e mover para o Drive a versão final dos documentos	A-07	P1	Médio	Baixo	—
EV-04	Completar as anotações de resposta de erro nos controladores para que o OpenAPI gerado coincida com o contrato v5; incluir os códigos de erro na comparação do `live-contract.mjs`	A-02	P1	Alto	Médio	—
EV-05	Reexecutar o contrato ao vivo com 58 operações e versionar o relatório; medir R2 após a evolução, versionando os relatórios brutos como em R0 e R1	A-09, A-10	P1	Médio	Baixo	EV-04 recomendada antes
EV-06	Executar o reteste independente dos 20 defeitos e registrar o encerramento formal	A-13	P1	Médio	Baixo	EV-05
EV-07	Atualizar README, rastreabilidade, critérios de aceitação, dicionário de dados e revisão da equipe; atualizar o Trello	A-01, A-14	P1	Médio	Médio	—
EV-08	Rotacionar a senha de banco exposta no histórico, caso ainda esteja em uso em algum ambiente	A-06	P1	Médio	Baixo	Acesso aos ambientes de desenvolvimento
EV-09	Configurar cabeçalhos de segurança nas páginas (`Content-Security-Policy` com `frame-ancestors 'none'`, `X-Content-Type-Options`, `Referrer-Policy`, `Permissions-Policy`)	A-05	P2	Médio	Baixo	Teste E2E para detectar bloqueio de recursos
EV-10	Corrigir a barra de meta sem meta definida e refazer as capturas de tela	A-16, A-15	P2	Baixo	Baixo	—
EV-11	Fixar a versão do Node nos testes (`engines` e `.nvmrc`) ou tornar a formatação compacta independente do ambiente	A-03	P2	Baixo	Baixo	—
EV-12	Adicionar limitação de taxa por origem no login e bloqueio progressivo, reduzindo o bloqueio deliberado de contas	A-17	P2	Médio	Médio	Decisão sobre armazenamento do contador

ID	Proposta	Origem	Prioridade	Impacto	Esforço	Dependências
EV-13	Desativar `show-sql` no perfil padrão; criar usuário de banco com privilégios mínimos para a API; restringir conexões ao Neon por IP quando o plano permitir	A-12; seção 9.9	P2	Médio	Baixo	Acesso às plataformas
EV-14	Definir e executar protocolo de desempenho com carga, aquecimento e percentil 95 por *endpoint* crítico	RNF-06	P2	Médio	Médio	Ambiente estável fora do plano gratuito
EV-15	Ativar na integração contínua as metas de qualidade MQ-01 a MQ-07 e a atualização automática de dependências	Seção 8.19	P2	Médio	Baixo	Aprovação da equipe
EV-16	Executar a validação de usabilidade com usuários reais, com tarefas cronometradas para vendedor e gerente	Seção 10.12	P2	Médio	Médio	Participantes voluntários
EV-17	Remover `idx_orders_seller_id` e revisar os índices sem uso após carga representativa	A-11	P3	Baixo	Baixo	EV-14
EV-18	Tornar `NOT NULL` as chaves obrigatórias herdadas do legado, após saneamento dos dados	Seção 6.23	P3	Médio	Médio	Migração com verificação prévia
EV-19	Substituir as rotas legadas `/orders`, `/payment` e `/address` pelos módulos do contrato v5, ou removê-las	Seção 5.16	P3	Médio	Médio	Revisão dos testes legados
EV-20	Adotar sessão compartilhada (por exemplo, sessão em banco) para permitir mais de uma instância da API	Seção 5.16	P3	Médio	Médio	EV-14
EV-21	Adotar RLS na conexão da API ou documentar formalmente a decisão de mantê-la apenas demonstrativa	Seção 6.21	P3	Médio	Alto	EV-13
EV-22	Implantar monitoramento com verificação periódica de disponibilidade, métricas e alertas	Seção 12.11	P3	Médio	Médio	—
EV-23	Definir rotina de cópia de segurança e teste de restauração do banco publicado	Seção 12.12	P3	Alto	Baixo	—
EV-24	Separar a carga analítica em *views* materializadas ou modelo dimensional quando o volume crescer	Seção 6.23	P3	Médio	Alto	EV-14
EV-25	Documentar base legal, retenção e atendimento a titulares, caso o sistema trate dados reais	Seção 9.12	P3	Alto	Médio	Uso fora do contexto acadêmico

Fonte: elaborado pelos autores.

15.2 FUNCIONALIDADES FORA DO ESCOPO ATUAL

Ficam fora do roteiro técnico, por terem sido excluídas do escopo pela equipe, a integração com gateway real de pagamento, a emissão de documentos fiscais, o suporte a várias lojas e o uso de dados reais de clientes [E-29]. Qualquer dessas extensões exigiria, além de desenvolvimento, conformidade regulatória específica: requisitos do padrão PCI DSS para pagamento com cartão (PCI SECURITY STANDARDS COUNCIL, 2024) e obrigações da LGPD para dados pessoais reais (BRASIL, 2018). Duas melhorias já declaradas pela equipe como trabalho futuro, a limitação de tentativas por IP e a duração absoluta da sessão [E-42], estão contempladas em EV-12 e EV-20.

15.3 SEQUÊNCIA RECOMENDADA

A sequência recomendada para a apresentação da A3 é: EV-01, EV-02 e EV-03 na mesma semana, por serem de baixo esforço e alto impacto; em seguida EV-04 e EV-05, que fecham a lacuna de documentação de erros e restabelecem a evidência do contrato ao vivo; e, por fim, EV-06 a EV-08. Essa ordem maximiza a reprodutibilidade exigida pelo enunciado antes de qualquer nova funcionalidade. A parte de EV-01 referente ao `next` já está preparada (seção 15.4) e pode ser aplicada junto com EV-09, EV-10, EV-11 e EV-13.

15.4 CORREÇÕES PREPARADAS NESTA AUDITORIA

Para comprovar a viabilidade das ações de menor esforço, esta auditoria preparou correções para cinco achados em uma *branch* local criada a partir do *commit* `3b0242c`. Em respeito à restrição de escopo de não alterar o repositório nem a aplicação publicada (Apêndice F, seção F.6), as alterações não foram enviadas ao GitHub: foram entregues como arquivo de *patch*, com instruções de aplicação, no diretório `correcoes-propostas/` que acompanha este relatório. O **Quadro** 38 resume as alterações e a validação realizada.

Quadro 38 **–** Correções preparadas e validadas localmente

Ação	Achado	Alteração	Validação local

Ação	Achado	Alteração	Validação local
EV-01 (parcial)	A-04	`next` 16.3.8 → 16.4.3, com o *lockfile* atualizado	`npm audit` sem vulnerabilidades nas dependências de produção; avisos das ferramentas de desenvolvimento do `orval` mantidos
EV-09	A-05	`Content-Security-Policy` com `frame-ancestors 'none'`, `base-uri`, `form-action` e `object-src`, além de `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy` e `Permissions-Policy` em todas as páginas, exceto as rotas da API	Teste unitário da configuração; cabeçalhos conferidos no servidor de desenvolvimento em quatro rotas e ausentes nas rotas `/api/backend/*`
EV-10 (parcial)	A-16	“Meu desempenho” deixa de desenhar a barra de meta no estado “Sem meta”, como já ocorre na visão geral	Verificação de tipos; capturas de tela não refeitas
EV-11	A-03	`compactMoney` calcula escala e sufixo (mil, mi, bi) sem a notação compacta do `Intl`	43/43 testes do frontend em Node 22.23.4, com seis novos valores de referência no caso CT-WEB-124
EV-13 (parcial)	A-12	`spring.jpa.show-sql` desativado por padrão, ativável pela variável `SPRING_JPA_SHOW_SQL`	Leitura da configuração; comportamento de produção inalterado

Fonte: elaborado pelos autores [E-48].

A validação local não substitui a da equipe: a construção de produção, os testes de ponta a ponta com axe e os testes Java dependem da integração contínua, que será executada quando o *pull request* for aberto. A política de conteúdo proposta não restringe *scripts* nem estilos, porque o Next.js injeta *scripts* em linha; a adoção de `script-src` com *nonce* por requisição fica como complemento de EV-09. O *patch* também se aplica sem conflitos sobre o *pull request* nº 47, dos avatares (seção 10.13), com 54 de 54 testes do frontend aprovados [E-49]. As ações EV-02, EV-03 e EV-04 não foram preparadas: as duas primeiras dependem de decisões e acessos da equipe, e a terceira exige compilar a API com o JDK 25, indisponível no ambiente de análise. Em 25/10/2026, o *patch* foi integrado pelo *pull request* nº 49, depois de aprovado nessas verificações da integração contínua (Apêndice M).

16 CONCLUSÃO

Este relatório documentou o SBI — Super Business Intelligence na versão em produção em 24 de outubro de 2026, correspondente ao *commit* `3b0242c` da branch `main`, a partir da auditoria do repositório, da integração contínua, da aplicação publicada, do serviço da API no Render, do banco de produção, da pasta oficial do Drive e do quadro do Trello. Cada afirmação relevante foi associada a uma evidência identificada, e os resultados executados nesta auditoria foram separados dos resultados declarados pela equipe.

Quanto ao problema de pesquisa, a conclusão é que o SBI cumpre em grau elevado os requisitos definidos pela equipe e a estrutura do contrato OpenAPI que declara, com uma lacuna relevante na documentação das respostas de erro. Os 26 requisitos funcionais estão implementados; as 58 operações do contrato v5 existem no OpenAPI publicado com os mesmos códigos de sucesso; e o banco de dados implementa 19 tabelas com as restrições, gatilhos e funções que sustentam as regras de negócio, com paridade estrutural entre o esquema migrado localmente e o banco de produção. Das 163 verificações reexecutadas, 162 foram aprovadas, e a única falha decorre da versão do ambiente de execução. A conformidade documental das respostas de erro, contudo, é de 45,8%, porque o OpenAPI gerado omite 109 dos 201 códigos de erro do contrato.

Quanto à aplicação das disciplinas, o projeto demonstra o uso de modelagem relacional, normalização, restrições declarativas, gatilhos, transações, controle de concorrência e operações CRUD, todas verificadas por um pacote de 61 testes SQL reexecutado nesta auditoria. Demonstra também um ciclo de garantia da qualidade com requisitos, critérios de aceitação, plano e catálogo de casos de teste com técnicas de caixa-preta, registro de defeitos, métricas antes e depois e integração contínua, cujas métricas R0 e R1 foram recalculadas nesta auditoria a partir dos relatórios versionados e cujas suítes Java, de ponta a ponta e de contrato ao vivo constam como aprovadas na integração contínua da versão de referência, embora não tenham sido reproduzidas localmente.

As principais contribuições do trabalho são três: a reconstrução documental do sistema efetivamente entregue, incluindo o dicionário de dados completo derivado do esquema real; a separação explícita entre evidência executada, evidência documentada e afirmação não comprovada; e o registro de 17 achados

com roteiro de evolução priorizado, dos quais os de maior prioridade podem ser resolvidos com baixo esforço antes da apresentação; para cinco deles, as correções já foram preparadas e validadas localmente, à espera de aplicação pela equipe.

As limitações deste trabalho são a ausência de reexecução local das suítes Java, de ponta a ponta e de contrato ao vivo, das quais se obteve apenas a conclusão na integração contínua, a inspeção apenas das telas públicas no ambiente publicado, a execução do pacote SQL em versão do PostgreSQL diferente da de produção e a impossibilidade de comprovar plenamente a participação de cada integrante pelas fontes disponíveis.

Como trabalhos futuros, recomenda-se executar as ações de prioridade P1 do capítulo 15, em especial a aplicação das correções preparadas, entre elas a atualização da dependência `next`, além da criação das contas de avaliação, da restrição da pasta do Drive e da correção da documentação de erros no OpenAPI; refazer as medições de qualidade após a evolução (R2); e, fora do contexto acadêmico, medir desempenho, implantar monitoramento e cópia de segurança e separar a carga analítica do banco transacional.

REFERÊNCIAS

ANDERSON, D. J. **Kanban**: successful evolutionary change for your technology business. Sequim: Blue Hole Press, 2010.

AZONYETIN, C. F. G. **Projeto integrado de garantia e avaliação da qualidade de software**: projeto final de disciplina – Garantia de Qualidade de Software. [*S. l.*: *s. n.*], 2026. Enunciado de projeto da disciplina.

BARTH, A. **HTTP state management mechanism**. Fremont: Internet Engineering Task Force, 2011. (RFC 6265). DOI: https://doi.org/10.17487/RFC6265.

BASS, L.; CLEMENTS, P.; KAZMAN, R. **Software architecture in practice**. 4. ed. Boston: Addison-Wesley, 2021.

BECK, K. *et al*. **Manifesto for agile software development**. [*S. l.*], 2001. Disponível em: https://agilemanifesto.org/. Acesso em: 24 out. 2026.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). **Diário Oficial da União**: seção 1, Brasília, DF, 15 ago. 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 24 out. 2026.

BRASIL. Secretaria Especial da Receita Federal do Brasil. Instrução Normativa RFB nº 2.229, de 15 de outubro de 2024. Altera a Instrução Normativa RFB nº 2.119, de 6 de dezembro de 2022, que dispõe sobre o Cadastro Nacional da Pessoa Jurídica (CNPJ). **Diário Oficial da União**: seção 1, Brasília, DF, 16 out. 2024.

CHACON, S.; STRAUB, B. **Pro Git**. 2. ed. Berkeley: Apress, 2014. Disponível em: https://git-scm.com/book. Acesso em: 24 out. 2026.

CODD, E. F. A relational model of data for large shared data banks. **Communications of the ACM**, New York, v. 13, n. 6, p. 377-387, June 1970. DOI: https://doi.org/10.1145/362384.362685.

COHN, M. **Succeeding with agile**: software development using Scrum. Upper Saddle River: Addison-Wesley, 2010.

DELAMARO, M. E.; MALDONADO, J. C.; JINO, M. (org.). **Introdução ao teste de software**. 2. ed. Rio de Janeiro: Elsevier, 2016.

ELMASRI, R.; NAVATHE, S. B. **Fundamentals of database systems**. 7. ed. Hoboken: Pearson, 2016.

FIELDING, R. T. **Architectural styles and the design of network-based software architectures**. 2000. Dissertation (Doctor of Philosophy in Information and Computer Science) – University of California, Irvine, 2000.

FIELDING, R.; NOTTINGHAM, M.; RESCHKE, J. (ed.). **HTTP semantics**. Fremont: Internet Engineering Task Force, 2022. (RFC 9110). DOI: https://doi.org/10.17487/RFC9110.

FOWLER, M. **Patterns of enterprise application architecture**. Boston: Addison-Wesley, 2002.

GITHUB. **GHSA-2464-fhr4-w9xw**: Next.js: unauthenticated remote code execution in Image Optimization API when SVG files are used. [*S. l.*], 2026. GitHub Advisory Database. Disponível em: https://github.com/advisories/GHSA-2464-fhr4-w9xw. Acesso em: 24 out. 2026.

HAERDER, T.; REUTER, A. Principles of transaction-oriented database recovery. **ACM Computing Surveys**, New York, v. 15, n. 4, p. 287-317, Dec. 1983. DOI: https://doi.org/10.1145/289.291.

HUMBLE, J.; FARLEY, D. **Continuous delivery**: reliable software releases through build, test, and deployment automation. Upper Saddle River: Addison-Wesley, 2010.

IEEE. **IEEE Std 1012-2016**: IEEE standard for system, software, and hardware verification and validation. New York: IEEE, 2016.

ISO; IEC. **ISO/IEC 25010:2023**: systems and software engineering – systems and software quality requirements and evaluation (SQuaRE) – product quality model. Geneva: ISO, 2023.

ISO; IEC; IEEE. **ISO/IEC/IEEE 29148:2018**: systems and software engineering – life cycle processes – requirements engineering. Geneva: ISO, 2018.

ISO; IEC; IEEE. **ISO/IEC/IEEE 29119-4:2021**: software and systems engineering – software testing – part 4: test techniques. Geneva: ISO, 2021.

ISO; IEC; IEEE. **ISO/IEC/IEEE 29119-1:2022**: software and systems engineering – software testing – part 1: general concepts. Geneva: ISO, 2022.

ISTQB. **Certified Tester Foundation Level syllabus**: version 4.0. [*S. l.*]: International Software Testing Qualifications Board, 2023.

JSON SCHEMA. **JSON Schema**: a media type for describing JSON documents – draft 2020-12. [*S. l.*], 2020. Disponível em: https://json-schema.org/draft/2020-12/json-schema-core. Acesso em: 24 out. 2026.

KIMBALL, R.; ROSS, M. **The data warehouse toolkit**: the definitive guide to dimensional modeling. 3. ed. Indianapolis: Wiley, 2013.

LAUDON, K. C.; LAUDON, J. P. **Management information systems**: managing the digital firm. 16. ed. Hoboken: Pearson, 2020.

McCABE, T. J. A complexity measure. **IEEE Transactions on Software Engineering**, New York, v. SE-2, n. 4, p. 308-320, Dec. 1976. DOI: https://doi.org/10.1109/TSE.1976.233837.

MYERS, G. J.; SANDLER, C.; BADGETT, T. **The art of software testing**. 3. ed. Hoboken: Wiley, 2011.

NIELSEN, J. **Usability engineering**. Boston: Academic Press, 1993.

NIST. **Digital identity guidelines**: authentication and authenticator management. Gaithersburg: National Institute of Standards and Technology, 2025. (NIST Special Publication 800-63B-4). DOI: https://doi.org/10.6028/NIST.SP.800-63B-4.

NOTTINGHAM, M.; WILDE, E.; DALAL, S. **Problem details for HTTP APIs**. Fremont: Internet Engineering Task Force, 2023. (RFC 9457). DOI: https://doi.org/10.17487/RFC9457.

OLIST. **Brazilian e-commerce public dataset by Olist**. [*S. l.*]: Kaggle, 2018. Conjunto de dados. Licença CC BY-NC-SA 4.0. Disponível em: https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce. Acesso em: 24 out. 2026.

OPENAPI INITIATIVE. **OpenAPI specification**: version 3.1.0. [*S. l.*], 2021. Disponível em: https://spec.openapis.org/oas/v3.1.0. Acesso em: 24 out. 2026.

OWASP. **OWASP API security top 10**: 2023. [*S. l.*]: OWASP Foundation, 2023. Disponível em: https://owasp.org/API-Security/editions/2023/en/0x11-t10/. Acesso em: 24 out. 2026.

OWASP. **Application security verification standard**: version 5.0.0. [*S. l.*]: OWASP Foundation, 2025. Disponível em: https://owasp.org/www-project-application-security-verification-standard/. Acesso em: 24 out. 2026.

PCI SECURITY STANDARDS COUNCIL. **Payment card industry data security standard**: requirements and testing procedures, version 4.0.1. Wakefield: PCI Security Standards Council, 2024.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 18 documentation**. [*S. l.*], 2025. Disponível em: https://www.postgresql.org/docs/18/. Acesso em: 24 out. 2026.

PRESSMAN, R. S.; MAXIM, B. R. **Software engineering**: a practitioner's approach. 9. ed. New York: McGraw-Hill, 2020.

REDGATE. **Flyway documentation**. Cambridge: Redgate Software, 2026. Disponível em: https://documentation.red-gate.com/flyway. Acesso em: 24 out. 2026.

SCHWABER, K.; SUTHERLAND, J. **The Scrum guide**: the definitive guide to Scrum: the rules of the game. [*S. l.*], 2020. Disponível em: https://scrumguides.org/scrum-guide.html. Acesso em: 24 out. 2026.

SHARDA, R.; DELEN, D.; TURBAN, E. **Business intelligence, analytics, and data science**: a managerial perspective. 4. ed. Hoboken: Pearson, 2018.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. **Database system concepts**. 7. ed. New York: McGraw-Hill, 2020.

SOMMERVILLE, I. **Software engineering**. 10. ed. Boston: Pearson, 2016.

W3C. **Web content accessibility guidelines (WCAG) 2.2**. [*S. l.*]: World Wide Web Consortium, 2024. W3C Recommendation. Disponível em: https://www.w3.org/TR/WCAG22/. Acesso em: 24 out. 2026.

WIEGERS, K.; BEATTY, J. **Software requirements**. 3. ed. Redmond: Microsoft Press, 2013.

APÊNDICE A – CRITÉRIOS DE ACEITAÇÃO DOS REQUISITOS FUNCIONAIS

Este apêndice complementa a especificação de requisitos do capítulo 4 com os 85 critérios de aceitação registrados pela equipe em `docs/criterios-aceitacao.md`, no formato Dado/Quando/Então [E-36]. O texto de cada critério foi mantido, com a acentuação restaurada; os identificadores de erro e os nomes técnicos aparecem como no código. A associação a cada requisito segue a matriz de rastreabilidade da equipe (Apêndice B).

Quadro 39 **–** Critérios de aceitação por requisito funcional

RF	Critério	Descrição (Dado/Quando/Então)
RF-01	CA-01-01	Dado um usuário sem sessão, quando acessar rota protegida, então a API retorna 401 e não revela dados de negócio.
RF-02	CA-01-02	Dado um vendedor autenticado, quando tentar acessar rota de gerente, então a API retorna 403 e não inclui custo, margem ou resultado de colegas no JSON.
RF-02	CA-01-03	Dado um gerente autenticado, quando acessar dados consolidados, então a API retorna loja, equipe, metas e estoque conforme o período solicitado.
RF-03	CA-03-01	Dado cliente individual sem CPF, quando criar cliente, então a API rejeita a requisição com erro 400 explícito.
RF-03	CA-03-02	Dado cliente corporativo sem CNPJ, quando criar cliente, então a API rejeita a requisição com erro 400 explícito.
RF-04	CA-04-01	Dado produto físico sem peso, quando criar produto, então a API rejeita a requisição.
RF-04	CA-04-02	Dado produto digital sem link de download, quando criar produto, então a API rejeita a requisição.
RF-05	CA-05-01	Dado cliente existente e itens válidos, quando criar pedido, então a API cria pedido com status `PENDING_PAYMENT`, endereço e pagamento pendente.
RF-05	CA-05-02	Dado cliente novo válido, quando criar pedido, então a API persiste o cliente junto com o pedido.
RF-05	CA-05-03	Dado pedido sem cliente existente nem dados de cliente novo, quando criar pedido, então a API retorna erro 400.
RF-06	CA-06-01	Dado vendedor autenticado, quando criar pedido, então o vendedor histórico do pedido é atribuído pela sessão.
RF-06	CA-06-02	Dado payload com campo de vendedor manipulado, quando criar pedido, então o servidor ignora ou rejeita o campo e preserva a identidade da sessão.
RF-07	CA-07-01	Dado item de R$ 4.500 e item de R$ 1.200 com quantidade 1, quando criar pedido sem desconto, então subtotal e total retornam R$ 5.700.
RF-07	CA-07-02	Dado produto com quantidade 2 e preço unitário R$ 100, quando criar pedido, então o total do item é R$ 200, não R$ 400.
RF-07	CA-07-03	Dado alteração posterior no catálogo, quando consultar venda passada, então o preço unitário histórico do item permanece inalterado.
RF-08	CA-08-01	Dado pagamento pendente, quando processar com dados válidos, então status do pagamento fica aprovado e pedido fica pago.
RF-08	CA-08-02	Dado evento repetido de confirmação de pagamento, quando reenviado, então realizado e estoque não duplicam efeitos.
RF-08	CA-08-03	Dado estorno aprovado, quando calcular realizado do período, então o valor estornado é subtraído.

RF	Critério	Descrição (Dado/Quando/Então)
RF-09	CA-09-01	Dado meta da loja de R$ 240.000 e três vendedores com R$ 80.000, quando salvar distribuição, então a soma dos vendedores corresponde à meta da loja.
RF-09	CA-09-02	Dado distribuição diferente da meta da loja, quando salvar, então a API rejeita a configuração ou registra pendência explícita.
RF-10	CA-10-01	Dado Allan com meta R$ 80.000 e realizado R$ 68.400, quando consultar desempenho, então o atingimento retorna 85,5%.
RF-10	CA-10-02	Dado vendedor com meta zero, quando consultar desempenho, então a resposta exibe estado `Sem meta` e não divide por zero.
RF-10	CA-10-03	Dado pedido com pagamento pendente, quando calcular realizado, então o pedido não entra no total.
RF-11	CA-11-01	Dado vendedor Allan autenticado, quando consultar Meu desempenho, então a resposta traz apenas meta, realizado e vendas do Allan.
RF-11	CA-11-02	Dado vendedor Allan autenticado, quando tentar consultar desempenho de Francisco por ID, então a API retorna 403 ou 404 sem vazar dados.
RF-12	CA-12-01	Dado gerente autenticado, quando consultar Gestão do período, então a resposta traz total da loja, equipe, metas e alertas de estoque.
RF-12	CA-12-02	Dado período sem dados, quando consultar Gestão, então a interface/API retorna estado vazio claro e sem números fictícios.
RF-13	CA-13-01	Dado saldo físico 1 e reservado 0, quando dois pedidos tentam reservar a última unidade ao mesmo tempo, então apenas um reserva com sucesso.
RF-13	CA-13-02	Dado pedido cancelado antes da confirmação, quando cancelar, então a reserva é liberada e o disponível aumenta.
RF-13	CA-13-03	Dado pagamento confirmado, quando aplicar efeitos, então reserva vira saída e não permanece como reservado.
RF-14	CA-14-01	Dado gerente autenticado, quando registrar entrada de lote, então quantidade, custo e preço são persistidos com auditoria.
RF-14	CA-14-02	Dado vendedor autenticado, quando listar produtos disponíveis, então custo e margem não aparecem no JSON.
RF-14	CA-14-03	Dado venda passada, quando custo ou preço de catálogo muda, então o custo/preço atribuído à venda passada permanece histórico.
RF-15	CA-15-01	Dado cliente existente, quando iniciar venda, então o fluxo permite selecionar cliente, produtos disponíveis e pagamento antes de criar pedido.
RF-15	CA-15-02	Dado falta de estoque, quando criar pedido, então o erro preserva o formulário e informa o motivo sem duplicar pedido.
RF-16	CA-16-01	Dado gerente autenticado, quando abrir a interface, então a navegação mostra Gestão, Vender e Estoque administrativo.
RF-16	CA-16-02	Dado vendedor autenticado, quando abrir a interface, então a navegação mostra Meu desempenho e Vender, sem Estoque administrativo.
RF-16	CA-16-03	Dado erro de rede ou API, quando carregar qualquer área, então a interface mostra estado de erro recuperável.
RF-17	CA-17-01	Dado gerente autenticado, quando acessar qualquer operação liberada ao vendedor, então a API não retorna 401 nem 403, tanto na regra de URL quanto na regra de método.
RF-17	CA-17-02	Dado vendedor autenticado, quando acessar operação exclusiva do gerente, então a API retorna 403 `FORBIDDEN`.
RF-17	CA-17-03	Dado usuário autenticado, quando consultar `GET /auth/me`, então `areas` lista VENDER, VENDAS, CLIENTES e MEU_DESEMPENHO para o vendedor e acrescenta GESTAO, PRODUTOS, ESTOQUE, PROMOCOES, ANALISES e USUARIOS para o gerente; com senha temporária, `areas` traz apenas TROCAR_SENHA.
RF-18	CA-18-01	Dada conta ativa, quando ocorrerem 4 falhas seguidas, então a conta não é bloqueada; na 5ª falha, fica bloqueada por 15 minutos.
RF-18	CA-18-02	Dada conta bloqueada, quando a senha correta for enviada, então a API retorna 401 com corpo igual (salvo timestamp) ao de usuário inexistente.

RF	Critério	Descrição (Dado/Quando/Então)
RF-18	CA-18-03	Dada conta bloqueada há 15 minutos, quando a senha correta for enviada, então a API retorna 200 e zera o contador de falhas.
RF-18	CA-18-04	Dada conta inativa, quando a senha correta for enviada, então a API retorna o mesmo 401.
RF-18	CA-18-05	Dado qualquer um dos quatro casos de falha, quando o login for processado, então a comparação BCrypt é executada e um `audit_event` é gravado sem senha e com o e-mail apenas como SHA-256.
RF-19	CA-19-01	Dado vendedor no PDV, quando buscar por SKU ou prefixo do nome, então o SKU exato vem primeiro e a resposta não contém custo, saldo físico nem margem.
RF-19	CA-19-02	Dado carrinho, quando cotar, então preço, desconto e total vêm do servidor sem efeito colateral; corpo de venda com vendedor, preço, desconto ou total retorna 400 sem criar pedido.
RF-19	CA-19-03	Dada a mesma `Idempotency-Key` e o mesmo corpo, quando a venda for reenviada, então a API retorna 200 com o mesmo pedido; com corpo diferente, 409 `IDEMPOTENCY_CONFLICT`.
RF-19	CA-19-04	Dado item sem estoque suficiente, quando finalizar a venda, então a API retorna 409 `STOCK_INSUFFICIENT` e nenhuma reserva do pedido permanece.
RF-19	CA-19-05	Dada venda pendente, quando o pagamento for confirmado duas vezes, então o estoque baixa uma única vez e a segunda chamada retorna 200 sem efeito.
RF-19	CA-19-06	Dada a última unidade, quando 10 vendas simultâneas tentarem reservá-la, então exatamente uma é criada e nove recebem 409; o disponível nunca fica negativo.
RF-19	CA-19-07	Dada venda pendente do próprio vendedor, quando ele cancelar, então a reserva é liberada; venda de outro vendedor retorna 404; venda paga retorna 409 `STATE_CONFLICT`.
RF-20	CA-20-01	Dado CPF ou CNPJ com dígito verificador inválido, quando cadastrar, então a API retorna 400; documento válido com máscara é gravado só com dígitos.
RF-20	CA-20-02	Dado documento já cadastrado, quando cadastrar outro cliente com ele, então a API retorna 409 `DUPLICATE_DOCUMENT`.
RF-20	CA-20-03	Dado vendedor autenticado, quando listar ou abrir cliente, então o documento vem mascarado.
RF-20	CA-20-04	Dada edição de cliente, quando o corpo trouxer `cpf` ou `cnpj`, então a API retorna 400; com `version` desatualizada, 409 `CONCURRENT_UPDATE`.
RF-20	CA-20-05	Dado vendedor, quando excluir cliente, então a API retorna 403; dado gerente e cliente com pedido, 409 `RESOURCE_IN_USE`.
RF-20	CA-20-06	Dado CNPJ alfanumérico válido (ex.: 12.ABC.345/01DE-35), quando cadastrar pessoa jurídica, então a API grava sem máscara em maiúsculas, mascara as pontas para o vendedor e acha o cliente pela busca exata; dígito verificador errado ou letra minúscula no corpo retorna 400 `VALIDATION_FAILED`; repetido retorna 409 `DUPLICATE_DOCUMENT`.
RF-21	CA-21-01	Dado cliente com pedidos, quando consultar o histórico, então os pedidos vêm paginados do mais recente para o mais antigo, com itens.
RF-21	CA-21-02	Dado vendedor, quando consultar o histórico, então total e vendedor aparecem só nos pedidos dele, e nenhum pedido traz custo ou margem.
RF-22	CA-22-01	Dado gerente, quando alterar preço com motivo de 5 a 200 caracteres, então o histórico registra autor e motivo, e itens vendidos antes mantêm o preço antigo.
RF-22	CA-22-02	Dada entrada de estoque com custo, quando registrada, então o custo médio ponderado é recalculado pelo gatilho; entrada sem custo deixa o custo médio desconhecido.
RF-22	CA-22-03	Dado ajuste de saída acima do disponível, quando registrado, então a API retorna 409 `STOCK_INSUFFICIENT`; ajuste sem motivo retorna 400; ajuste de entrada não altera o custo médio.

RF	Critério	Descrição (Dado/Quando/Então)
RF-22	CA-22-04	Dado produto já vendido, quando o gerente tentar excluí-lo, então a API retorna 409; inativado, ele some do PDV.
RF-22	CA-22-05	Dado insumo (`sellable = false`), quando buscar no PDV, então ele não aparece; venda que o inclua retorna 409 `STATE_CONFLICT`.
RF-22	CA-22-06	Dado extrato de movimentos, quando filtrar por produto, tipo e período, então cada movimento traz tipo, quantidade, motivo, pedido e autor.
RF-22	CA-22-07	Dado estoque mínimo definido, quando o disponível ficar igual ou abaixo dele, então a posição marca `belowMin` e o painel conta o produto no alerta.
RF-23	CA-23-01	Dado gerente, quando consultar faturamento do período, então o líquido é vendas pagas menos estornos do período, lido das views do V6.
RF-23	CA-23-02	Dado item vendido sem custo no período, quando consultar faturamento, então lucro bruto e margem vêm nulos e a cobertura de custo é informada.
RF-23	CA-23-03	Dado painel de gestão aberto, quando a aba estiver visível, então os dados são recarregados a cada 15 s; com a aba oculta, não; erro de carga nunca aparece como zero.
RF-24	CA-24-01	Dado gerente, quando cadastrar usuário, então o e-mail é gravado em minúsculas, a conta exige troca de senha e e-mail repetido em outra caixa retorna 409 `STATE_CONFLICT`; senha fora da política retorna 400 `VALIDATION_FAILED`.
RF-24	CA-24-02	Dado vendedor, quando acessar qualquer rota de `/users`, então a API retorna 403 `FORBIDDEN`.
RF-24	CA-24-03	Dada alteração de papel, situação, senha ou desbloqueio de um usuário, quando ele fizer a próxima requisição com a sessão antiga, então a API retorna 401 `SESSION_REVOKED`.
RF-24	CA-24-04	Dado gerente, quando tentar alterar o próprio papel, a própria situação ou redefinir a própria senha, então a API retorna 409 `SELF_CHANGE_FORBIDDEN`; quando a operação deixaria a loja sem gerente ativo, 409 `LAST_MANAGER`.
RF-24	CA-24-05	Dada senha provisória, quando o usuário entrar, então `areas` traz apenas TROCAR_SENHA e as demais rotas retornam 403 `PASSWORD_CHANGE_REQUIRED` até a troca; senha atual errada na troca retorna 401 e conta para o bloqueio.
RF-24	CA-24-06	Dada qualquer operação de usuário, quando a resposta ou o `audit_event` forem gravados, então nenhum contém senha ou hash.
RF-25	CA-25-01	Dado gerente, quando consultar `GET /customers/{id}/summary`, então total comprado (pagos menos estornos), ticket médio, estornos e formas de pagamento cobrem a loja toda (`scope` STORE), com pendentes e cancelados só como contagem.
RF-25	CA-25-02	Dado vendedor, quando consultar o resumo, então total, ticket médio e formas de pagamento cobrem só os pedidos dele (`scope` OWN) e `refundsTotal` é nulo; contagens, datas e produtos mais comprados cobrem a loja toda.
RF-25	CA-25-03	Dado cliente sem compras, quando consultar o resumo, então contagens e total são zero, ticket médio e datas são nulos e as listas vêm vazias.
RF-26	CA-26-01	Dado vendedor, quando cotar ou vender com desconto manual até 5% e motivo, então a API calcula o valor sobre o subtotal já promocionado (HALF_UP), rateia nos itens e grava o mesmo total no pedido e no pagamento.
RF-26	CA-26-02	Dado desconto acima da alçada do papel da sessão, quando cotar ou vender, então a API retorna 403 `DISCOUNT_ABOVE_LIMIT` com o limite em `details` e não cria pedido nem reserva.
RF-26	CA-26-03	Dado item que ficaria abaixo do custo médio conhecido, quando a política não permitir, então a API retorna 409 `BELOW_COST` sem revelar o custo.
RF-26	CA-26-04	Dado pedido pendente, quando o gerente aprovar um desconto até 20%, então total, itens e pagamento são recalculados, o aprovador fica registrado e a auditoria grava `DISCOUNT_APPROVED`; pedido pago retorna 409.

RF	Critério	Descrição (Dado/Quando/Então)
RF-26	CA-26-05	Dado relatório de vendas do mês, quando houver descontos manuais em pedidos pagos, então `manualDiscounts` mostra por vendedor pedidos, valor concedido e aprovados pelo gerente, e o uso de promoções não inclui a parte manual.

Fonte: `docs/criterios-aceitacao.md`, versão do *commit* `3b0242c` [E-36].

A distribuição dos critérios por requisito é desigual: RF-19 (venda no PDV) e RF-22 (backoffice) têm sete critérios cada e RF-20 e RF-24 têm seis, enquanto nove requisitos têm dois critérios e RF-01 tem apenas um. A concentração acompanha o risco: os requisitos ligados a dinheiro, estoque e autorização receberam mais critérios. RF-03 e RF-04 compartilham a seção de critérios da API original.

APÊNDICE B – MATRIZ DE RASTREABILIDADE

A matriz completa liga cada requisito funcional aos critérios de aceitação, aos casos de teste, à evidência automatizada indicada pela equipe e à verificação realizada nesta auditoria. As quatro primeiras colunas reproduzem `docs/rastreabilidade.md`, cuja situação é de 14/10/2026 [E-36]; a última coluna registra os casos reexecutados em 24/10/2026, detalhados no Apêndice G. O traço indica requisito sem verificação independente nesta auditoria, cuja evidência é apenas a documentada.

Quadro 40 **–** Matriz de rastreabilidade requisito → critério → caso de teste → evidência

RF	Critérios	Casos de teste	Evidência automatizada documentada	Situação documentada	Verificado nesta auditoria
RF-01	CA-01-01	CT-AUTH-001	`AccessMatrixTest` (anônimo → 401 em toda rota protegida), `SessionSecurityTest`	Verde	PUB-01, PUB-02 (401 `UNAUTHENTICATED` sem sessão; CSRF disponível)
RF-02	CA-01-02, CA-01-03	CT-AUTH-002, CT-AUTH-003	`AccessMatrixTest` (VENDEDOR → 403 nas rotas do gerente), `ManagementApiTest`, `GoalsApiTest.managementSummary`	Verde	PUB-02 a PUB-05; RB-01 a RB-08, RB-13; RG-07
RF-03	CA-03-01, CA-03-02	CT-CLIENT-001, CT-CLIENT-002	`CustomersApiTest.invalidCpfIsRejected`, `duplicateDocumentIsRejected`, `CatalogServicesTest` (CustomerService)	Verde	CT-DEMO-06 (CHECK de CPF e de tipo)
RF-04	CA-04-01, CA-04-02	CT-PRODUCT-001, CT-PRODUCT-002	`BackofficeApiTest.productTypeRequiresItsField`, `CatalogServicesTest.decisionTable`	Verde	—
RF-05	CA-05-01 a CA-05-03	CT-ORDER-001 a CT-ORDER-003	`OrderPersistenceTest.demonstrationSalePersistsCustomerAddressPaymentAndTotal`, `OrderServiceRulesTest` (`existingCustomerIsReused`, `newCustomerRules`, `customerIsRequired`)	Verde	—
RF-06	CA-06-01, CA-06-02	CT-ORDER-004, CT-SEC-001	`ProductAndOrderCalculationTest.newOrderRequiresSeller`, `SalesApiTest.unknownPropertiesAreRejected`, `sellerComesOnlyFromSession`	Verde	BD-29, RB-14
RF-07	CA-07-01 a CA-07-03	CT-MONEY-001 a CT-MONEY-003	`ProductAndOrderCalculationTest` (`totalSumsAllItems`, `itemSubtotalMultipliesQuantityOnce`), `OrderPersistenceTest.historicalUnitPriceSurvivesCatalogChangeAndDatabaseReload`, `PricingTest`, `SalesApiTest.inactiveOrSupplyIsNotSoldAndPriceIsFrozen`	Verde	BD-10, BD-11, BD-25, BD-26
RF-08	CA-08-01 a CA-08-03	CT-PAY-001 a CT-PAY-003	`SalesApiTest.paymentConfirmationIsIdempotent`, `PaymentServiceTest`, `ConcurrencyTest`, `RefundAndAnalyticsApiTest.refunds`	Verde	BD-19, BD-22, BD-23
RF-09	CA-09-01, CA-09-02	CT-GOAL-001, CT-GOAL-002	`GoalsApiTest.monthlyGoalsDistribution`	Verde	BD-16

RF	Critérios	Casos de teste	Evidência automatizada documentada	Situação documentada	Verificado nesta auditoria
RF-10	CA-10-01 a CA-10-03	CT-GOAL-003 a CT-GOAL-005	`GoalsApiTest.myPerformance` (85,5%; pendente fora do realizado), `noGoalMeansNoPercentage`	Verde	BD-01 a BD-05, BD-17, BD-20
RF-11	CA-11-01, CA-11-02	CT-PERF-001, CT-SEC-002	`GoalsApiTest.myPerformance`, `noGoalMeansNoPercentage`; `AccessMatrixTest`	Verde	RB-10 a RB-12
RF-12	CA-12-01, CA-12-02	CT-MGMT-001, CT-MGMT-002	`ManagementApiTest`, `GoalsApiTest.managementSummary`; E2E `manager.spec.ts` (CT-WEB-102)	Verde	—
RF-13	CA-13-01 a CA-13-03	CT-STOCK-001 a CT-STOCK-003	`SalesConcurrencyTest.lastUnitIsSoldOnce`, `SalesApiTest.cancelReleasesReservation`, `paymentConfirmationIsIdempotent`; job `bd` (CC-01)	Verde	BD-06, BD-15, BD-19, BD-21, BD-28, CC-01
RF-14	CA-14-01 a CA-14-03	CT-COST-001 a CT-COST-003	`BackofficeApiTest.stockEntries`, `SalesApiTest.catalogListsOnlySellableAvailableProducts` (sem custo), `historicalCostIsKept`	Verde	BD-07, BD-18, RB-09, RG-05
RF-15	CA-15-01, CA-15-02	CT-E2E-001, CT-E2E-002	E2E `sale.spec.ts` (CT-WEB-101, CT-E2E-002)	Verde	—
RF-16	CA-16-01 a CA-16-03	CT-UI-001 a CT-UI-003	E2E `manager.spec.ts` (CT-UI-001), `sale.spec.ts` (CT-WEB-103/CT-UI-002), `session.spec.ts`; `frontend/tests/http.test.ts`	Verde	—
RF-17	CA-17-01 a CA-17-03	CT-SEC-101, CT-SEC-103, CT-SEC-114, CT-WEB-103	`AccessMatrixTest` (260 casos), `MethodSecurityHierarchyTest`, `BackofficeApiTest`; E2E `sale.spec.ts`	Verde	RB-01 a RB-08; RG-01, RG-02
RF-18	CA-18-01 a CA-18-05	CT-AUTH-101, CT-AUTH-102, CT-SEC-120	`LoginLockoutPolicyTest`, `LoginLockoutIntegrationTest`, `LoginAttemptConcurrencyTest` (10 falhas em paralelo, bloqueio uma vez)	Verde	—
RF-19	CA-19-01 a CA-19-07	CT-SALE-101 a 104, CT-STOCK-101 a 107, CT-SEC-110 a 113, CT-SEC-116, CT-WEB-101, CT-WEB-104	`PricingTest`, `RequestHashTest`, `SalesApiTest`, `SalesConcurrencyTest`; E2E `sale.spec.ts`, `flows.spec.ts` (CT-WEB-107, 108)	Verde	BD-28, CC-01
RF-20	CA-20-01 a CA-20-06	CT-CRM-101, 102, 104, 105, 106, CT-CONTRACT-101, CT-WEB-113, CT-WEB-128	`DocumentFormatTest`, `CustomersApiTest`, `AlphanumericCnpjTest`, `FlywayMigrationTest.v10AcceptsAlphanumericCnpj`; E2E `customers.spec.ts`	Verde	CT-DEMO-06

RF	Critérios	Casos de teste	Evidência automatizada documentada	Situação documentada	Verificado nesta auditoria
RF-21	CA-21-01, CA-21-02	CT-CRM-103, CT-SEC-113	`CustomersApiTest.historyHidesOtherSellersTotals`; E2E `sale.spec.ts` (histórico do cliente)	Verde	—
RF-22	CA-22-01 a CA-22-07	CT-STOCK-108, CT-STOCK-109, CT-SALE-104, CT-WEB-105, CT-WEB-106	`BackofficeApiTest`; E2E `flows.spec.ts` (CT-WEB-105, 106)	Verde	BD-09, BD-12, BD-24, RG-03, RG-05, RG-06
RF-23	CA-23-01 a CA-23-03	CT-CONTRACT-102, CT-WEB-102	`scripts/live-contract.mjs`; E2E `manager.spec.ts`	Verde	BD-07, BD-07b, BD-08, BD-31b
RF-24	CA-24-01 a CA-24-06	CT-USR-101 a CT-USR-107, CT-SEC-121, CT-SEC-122, CT-BD-30, CT-CONTRACT-104, CT-WEB-110, CT-WEB-112	`UsersApiTest`, `SessionRevocationTest`, `StrongPasswordValidatorTest`, `FlywayMigrationTest.v8KeepsOneActiveManager`, `06d_testes_evolucao.sql`; E2E `users.spec.ts`	Verde	BD-30a a BD-30c
RF-25	CA-25-01 a CA-25-03	CT-CUS-121 a CT-CUS-123, CT-WEB-111	`CustomersApiTest` (resumo da loja, escopo do vendedor, cliente sem compras), BD-31 no job `bd`; E2E `customers.spec.ts`	Verde	BD-31a a BD-31c
RF-26	CA-26-01 a CA-26-05	CT-DISC-101 a CT-DISC-107, CT-WEB-114, BD-32	`ManualDiscountApiTest`, `ManualDiscountPricingTest`, `06d_testes_evolucao.sql`; E2E `discount.spec.ts`	Verde	BD-32a a BD-32c

Fonte: `docs/rastreabilidade.md` (14/10/2026) e execuções desta auditoria [E-36] [E-11] [E-12] [E-04].

O **Quadro** 41 reproduz a rastreabilidade dos requisitos não funcionais e das propriedades transversais.

Quadro 41 **–** Evidências documentadas para requisitos não funcionais

Item	Evidência documentada pela equipe
Formato único de erro (`ErrorResponse` com `code`)	`ErrorContractTest`, `ErrorResponseTest`, `live-contract.mjs` (erros validados com Ajv)
Dinheiro NUMERIC(12,2) com HALF_UP	`MoneyValidationTest`, `PricingTest`, `FlywayMigrationTest` (V4)
Flyway como única fonte do esquema	`FlywayMigrationTest` (V1 a V11), passo "Verify Flyway schema with Hibernate validation" do job `test (backend)`
Número completo de cartão nunca gravado (D022)	`CardNumberStorageTest`, `FlywayMigrationTest.v7MasksLegacyCardNumbers`
WCAG 2.2 AA e 320 px	E2E `sale.spec.ts` e `manager.spec.ts` (CT-WEB-109, axe nas telas dos dois perfis)
Schemas do frontend iguais ao contrato	`npm run api:check` (CT-CONTRACT-103) no job `frontend`
Sessão revogada na requisição seguinte à mudança de papel, situação ou senha	`SessionRevocationTest`, `UsersApiTest.deactivationRevokesTheSession`; E2E `users.spec.ts` (CT-WEB-112)
Marca SBI v1.0 aplicada em todas as telas	E2E `brand.spec.ts` (CT-WEB-120, CT-WEB-121), `frontend/tests/brand.test.ts` (CT-WEB-122 a CT-WEB-125); capturas em `reports/marca/` com axe sem violação
Massa de demonstração Olist íntegra e idempotente	`09_testes_olist.sql`, `scripts/demo-olist-api.mjs` e `run_olist_bd.sh` (CT-DEMO-01 a CT-DEMO-06) no job `bd`

Fonte: `docs/rastreabilidade.md` [E-36].

A própria matriz da equipe registra quatro lacunas conhecidas: a edição concorrente de cliente é coberta apenas por versão desatualizada, sem duas edições paralelas reais; o faturamento é conferido contra a própria *view*, sem conferência automática contra a massa de demonstração; a duração absoluta da sessão, o *cookie* com prefixo `__Host-` e a limitação por IP ficaram como trabalho futuro; e o logotipo foi reconstruído a partir do ícone do site, porque os arquivos do manual de marca não estavam no repositório. Esta auditoria acrescenta uma divergência documental: o cabeçalho do documento afirma que os PRs da evolução não estavam integrados à `main`, o que não corresponde à versão analisada (achado A-01).

APÊNDICE C – DIAGRAMAS TÉCNICOS

Os treze diagramas solicitados para a documentação foram produzidos a partir do código, das migrações, da configuração das plataformas e do histórico do projeto, em notação Mermaid, cujos arquivos-fonte são texto editável. O **Quadro** 42 indica a localização de cada diagrama no relatório e o arquivo-fonte correspondente no diretório `diagramas/fonte/` entregue com este relatório. Este apêndice apresenta os dois diagramas do modelo físico que não constam do corpo do texto.

Quadro 42 **–** Diagramas produzidos e arquivos-fonte

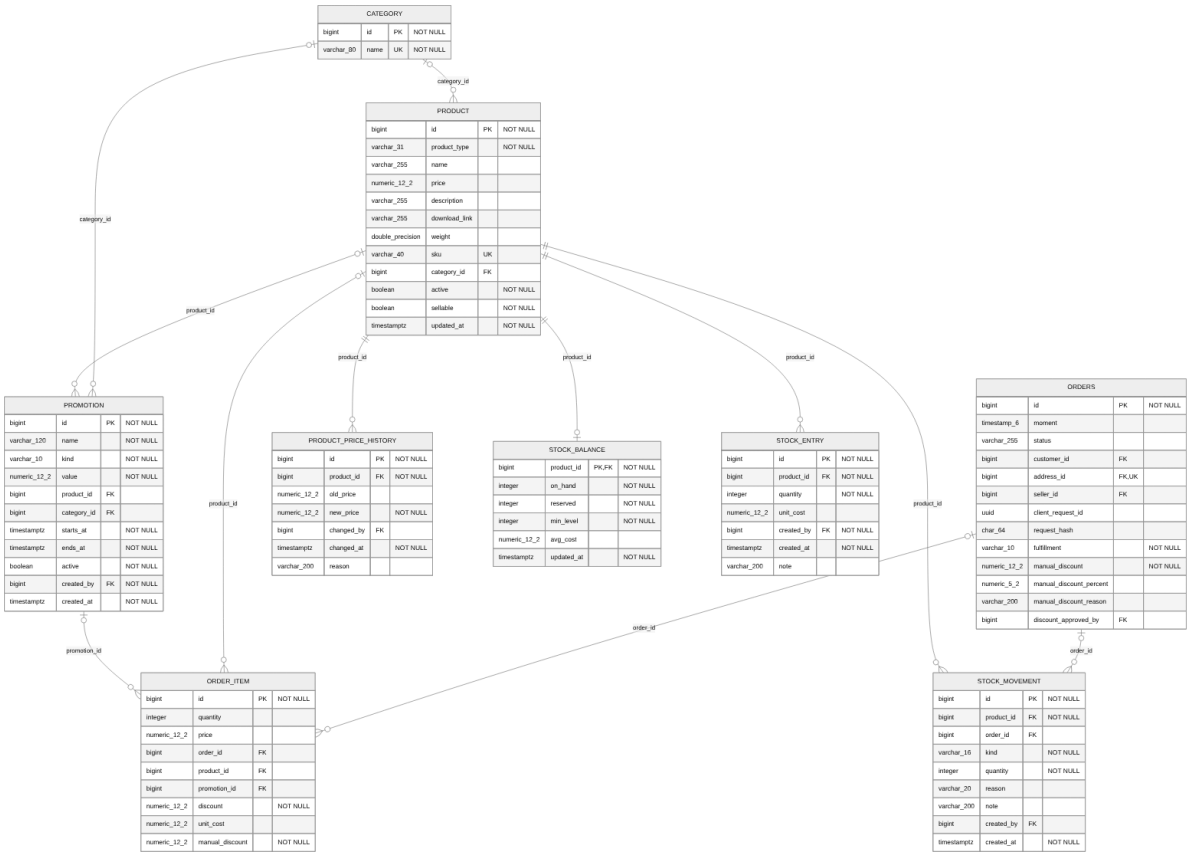
Nº	Diagrama	Localização	Arquivo-fonte	Base de construção
1	Contexto	**Figura** 2	`D01_contexto.mmd`	Código, `next.config.mjs`, `docs/deploy.md`
2	Casos de uso	**Figura** 1	`D02_casos_de_uso.mmd`	Rotas da interface, contrato v5, `SecurityConfig`
3	Arquitetura de implantação	**Figura** 23	`D03_arquitetura_implantacao.mmd`	Vercel, Render, Neon, *workflows*
4	Componentes	**Figura** 3	`D04_componentes.mmd`	Pacotes Java e diretórios do frontend
5	Entidade-relacionamento conceitual	**Figura** 6	`D05_der_conceitual.mmd`	Migrações V1–V11
6	Modelo lógico	**Figura** 7	`D06_modelo_logico.mmd`	Catálogo do PostgreSQL
7a	Modelo físico: vendas	**Figura** 8	`D07a_fisico_vendas.mmd`	Catálogo do PostgreSQL
7b	Modelo físico: catálogo e estoque	**Figura** 24	`D07b_fisico_catalogo_estoque.mmd`	Catálogo do PostgreSQL
7c	Modelo físico: usuários, metas e auditoria	**Figura** 25	`D07c_fisico_usuarios_metas_auditoria.mmd`	Catálogo do PostgreSQL
8	Sequência da venda no PDV	**Figura** 5	`D08_sequencia_venda.mmd`	`SalesService`, controladores, migrações
9	Fluxo de autenticação	**Figura** 4	`D09_fluxo_autenticacao.mmd`	`SecurityConfig`, `security/*`
10	Estados do pedido e do pagamento (fluxo de vendas)	**Figura** 10	`D10_estados_pedido_pagamento.mmd`	`SalesService`, CHECKs de status
11	Fluxo de atualização de estoque	**Figura** 9	`D11_fluxo_estoque.mmd`	`SalesService`, gatilho `stock_entry_apply`
12	Estratégia de testes	**Figura** 11	`D12_estrategia_testes.mmd`	`ci.yml`, plano de testes

Nº	Diagrama	Localização	Arquivo-fonte	Base de construção
13	Linha do tempo	**Figura** 22	`D13_linha_do_tempo.mmd`	*Commits*, Trello, Drive, implantações

Fonte: elaborado pelos autores.

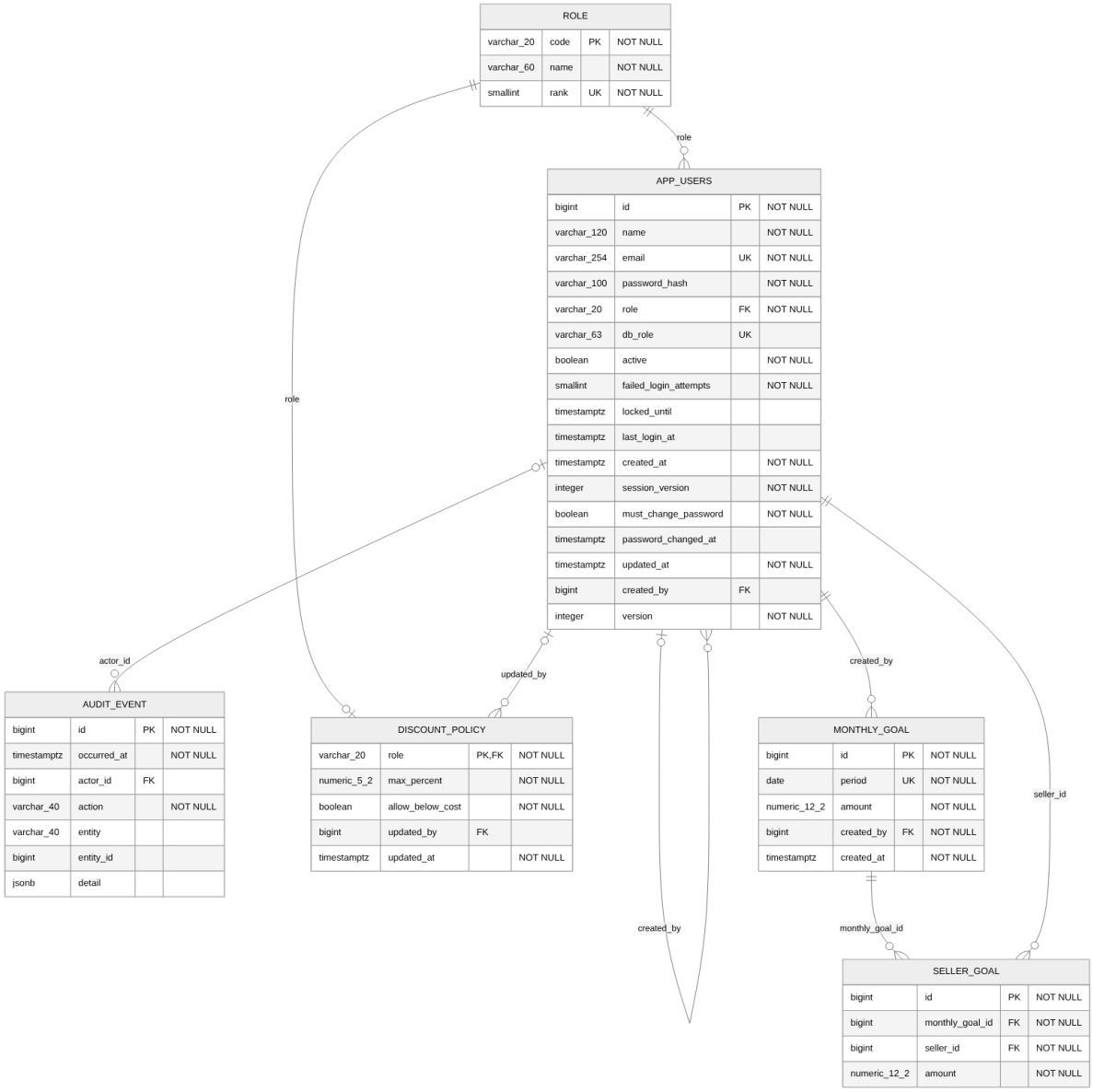
Os diagramas podem ser regenerados com o Mermaid CLI, por exemplo `mmdc -i D07b_fisico_catalogo_estoque.mmd -o D07b.png -s 2 -b white`. Nos modelos físicos, as cardinalidades refletem a nulidade real das chaves estrangeiras: uma chave anulável é representada como participação opcional, mesmo quando o domínio a trata como obrigatória (seção 6.23).

Figura 24 **–** Modelo físico do subdomínio de catálogo e estoque



Fonte: elaborado pelos autores a partir do catálogo do PostgreSQL [E-15].

Figura 25 **–** Modelo físico do subdomínio de usuários, metas e auditoria



Fonte: elaborado pelos autores a partir do catálogo do PostgreSQL [E-15].

APÊNDICE D – DICIONÁRIO DE DADOS

O dicionário foi derivado do catálogo do PostgreSQL (`information_schema` e `pg_catalog`) após a aplicação das migrações V1 a V11 e do pacote de demonstração no *commit* `3b0242c`, e conferido contra o banco publicado, que tem as mesmas contagens de tabelas, *views*, funções, gatilhos, restrições `CHECK`, chaves estrangeiras e índices [E-15] [E-03]. As descrições semânticas foram redigidas a partir das migrações e do código. A versão estruturada, com filtros, está no arquivo `SBI_DICIONARIO_DE_DADOS.xlsx` entregue com este relatório.

Convenções: **Nulo** indica se a coluna aceita valor nulo; **Chaves** usa PK (chave primária), FK (chave estrangeira, com a tabela referenciada), UQ (participa de restrição de unicidade) e CK (número de restrições `CHECK` que mencionam a coluna); `varchar(n)` abrevia `character varying(n)` e `timestamp` indica `timestamp without time zone`.

D.1 TABELA ADDRESS

Endereço. Endereço de cliente ou de entrega de pedido (legado V1). Migrações: V1. Colunas: 8.

Quadro 43 **–** Dicionário de dados da tabela `address`

Coluna	Tipo	Nulo	Padrão	Chaves	Descrição
`id`	bigint	não	identidade	PK	Identificador do endereço.
`street`	varchar(255)	sim	—	—	Logradouro.
`number`	varchar(255)	sim	—	—	Número do imóvel.
`complement`	varchar(255)	sim	—	—	Complemento.
`neighborhood`	varchar(255)	sim	—	—	Bairro.
`city`	varchar(255)	sim	—	—	Município.
`state`	varchar(255)	sim	—	—	Unidade federativa.
`zip_code`	varchar(255)	sim	—	—	CEP.

Fonte: catálogo do PostgreSQL após V1–V11 [E-15].

D.2 TABELA APP_USERS

Usuário da aplicação. Conta de acesso de gerente ou vendedor, com hash BCrypt, situação, bloqueio e versão de sessão. Migrações: V2, V5, V8. Colunas: 17.

Quadro 44 **–** Dicionário de dados da tabela `app_users`

Coluna	Tipo	Nulo	Padrão	Chaves	Descrição
`id`	bigint	não	identidade	PK	Identificador do usuário.
`name`	varchar(120)	não	—	—	Nome exibido.
`email`	varchar(254)	não	—	UQ	E-mail de login (único, também sem distinção de maiúsculas).
`password_hash`	varchar(100)	não	—	—	Hash BCrypt (custo 12) da senha; nunca exposto pela API.
`role`	varchar(20)	não	—	FK → role	Papel de acesso (FK para role.code).
`db_role`	varchar(63)	sim	—	UQ	Login de banco associado, usado na demonstração de RLS por psql.
`active`	boolean	não	true	—	Situação da conta (ativa/inativa).
`failed_login_attempts`	smallint	não	0	CK (1)	Falhas consecutivas de login (incremento atômico).
`locked_until`	timestamptz	sim	—	—	Fim do bloqueio após 5 falhas (15 min).
`last_login_at`	timestamptz	sim	—	—	Último login bem-sucedido.
`created_at`	timestamptz	não	now()	—	Data de criação.
`session_version`	integer	não	0	CK (1)	Versão de sessão; muda para revogar sessões abertas.
`must_change_password`	boolean	não	false	—	Indica senha provisória que exige troca.
`password_changed_at`	timestamptz	sim	—	—	Data da última troca de senha.
`updated_at`	timestamptz	não	now()	—	Data da última alteração.
`created_by`	bigint	sim	app_current_user_id()	FK → app_users	Usuário que cadastrou a conta.
`version`	integer	não	0	CK (1)	Versão para controle otimista de concorrência.

Fonte: catálogo do PostgreSQL após V1–V11 [E-15].

D.3 TABELA AUDIT_EVENT

Evento de auditoria. Trilha de ações relevantes (login, cliente, estoque, usuários, descontos), sem senha nem documento completo. Migrações: V5, V8, V11. Colunas: 7.

Quadro 45 **–** Dicionário de dados da tabela `audit_event`

Coluna	Tipo	Nulo	Padrão	Chaves	Descrição
`id`	bigint	não	identidade	PK	Identificador do evento.
`occurred_at`	timestamptz	não	now()	—	Instante do evento.
`actor_id`	bigint	sim	—	FK → app_users	Usuário que executou a ação (nulo em login de e-mail inexistente).
`action`	varchar(40)	não	—	CK (1)	Tipo da ação (domínio fechado por CHECK).
`entity`	varchar(40)	sim	—	—	Tabela/entidade afetada.
`entity_id`	bigint	sim	—	—	Identificador do registro afetado.
`detail`	jsonb	sim	—	—	Detalhes em JSON, sem senha nem documento completo.

Fonte: catálogo do PostgreSQL após V1–V11 [E-15].

D.4 TABELA CATEGORY

Categoria. Agrupamento de produtos usado no catálogo e no escopo de promoções. Migrações: V5. Colunas: 2.

Quadro 46 **–** Dicionário de dados da tabela `category`

Coluna	Tipo	Nulo	Padrão	Chaves	Descrição
`id`	bigint	não	identidade	PK	Identificador da categoria.
`name`	varchar(80)	não	—	UQ	Nome único da categoria.

Fonte: catálogo do PostgreSQL após V1–V11 [E-15].

D.5 TABELA CUSTOMER

Cliente. Cliente pessoa física (INDIVIDUAL, CPF) ou jurídica (CORPORATE, CNPJ), em herança de tabela única. Migrações: V1, V3, V5, V10. Colunas: 14.

Quadro 47 **–** Dicionário de dados da tabela `customer`

Coluna	Tipo	Nulo	Padrão	Chaves	Descrição
`id`	bigint	não	identidade	PK	Identificador do cliente.
`customer_type`	varchar(31)	não	—	CK (2)	Discriminador: INDIVIDUAL ou CORPORATE.
`name`	varchar(255)	sim	—	—	Nome ou razão social exibida.
`email`	varchar(255)	sim	—	—	E-mail de contato.
`birth_date`	date	sim	—	—	Data de nascimento (pessoa física).
`address_id`	bigint	sim	—	FK → address; UQ	Endereço do cliente (1:1 opcional).

Coluna	Tipo	Nulo	Padrão	Chaves	Descrição
cpf	varchar(255)	sim	—	UQ; CK (2)	CPF com 11 dígitos, único; obrigatório para INDIVIDUAL.
cnpj	varchar(255)	sim	—	UQ; CK (2)	CNPJ de 14 posições (numérico ou alfanumérico), único; obrigatório para CORPORATE.
company_name	varchar(255)	sim	—	—	Razão social (pessoa jurídica).
phone	varchar(20)	sim	—	—	Telefone.
version	integer	não	0	—	Versão para controle otimista (409 em edição desatualizada).
created_by	bigint	sim	app_current_user_id()	FK → app_users	Usuário que cadastrou.
created_at	timestamptz	não	now()	—	Data de cadastro.
updated_at	timestamptz	não	now()	—	Data da última alteração.

Fonte: catálogo do PostgreSQL após V1–V11 [E-15].

D.6 TABELA DISCOUNT_POLICY

Alçada de desconto. Percentual máximo de desconto manual por papel e permissão de venda abaixo do custo. Migrações: V11. Colunas: 5.

Quadro 48 **–** Dicionário de dados da tabela `discount_policy`

Coluna	Tipo	Nulo	Padrão	Chaves	Descrição
role	varchar(20)	não	—	FK → role; PK	Papel ao qual a alçada se aplica (PK e FK para role).
max_percent	numeric(5,2)	não	—	CK (1)	Percentual máximo de desconto manual (0 a 100).
allow_below_cost	boolean	não	false	—	Permite desconto que leve o item abaixo do custo médio.
updated_by	bigint	sim	app_current_user_id()	FK → app_users	Gerente que alterou a alçada.
updated_at	timestamptz	não	now()	—	Data da alteração.

Fonte: catálogo do PostgreSQL após V1–V11 [E-15].

D.7 TABELA MONTHLY_GOAL

Meta mensal da loja. Meta de faturamento da loja para um mês (primeiro dia do mês). Migrações: V5. Colunas: 5.

Quadro 49 **–** Dicionário de dados da tabela `monthly_goal`

Coluna	Tipo	Nulo	Padrão	Chaves	Descrição
`id`	bigint	não	identidade	PK	Identificador da meta.
`period`	date	não	—	UQ; CK (1)	Mês de referência (sempre o dia 1), único.
`amount`	numeric(12,2)	não	—	CK (1)	Valor da meta da loja.
`created_by`	bigint	não	app_current_user_id()	FK → app_users	Gerente que registrou.
`created_at`	timestamptz	não	now()	—	Data de registro.

Fonte: catálogo do PostgreSQL após V1–V11 [E-15].

D.8 TABELA ORDER_ITEM

Item do pedido. Produto vendido no pedido, com preço histórico, descontos e custo atribuído na confirmação. Migrações: V1, V4, V5, V11. Colunas: 9.

Quadro 50 **–** Dicionário de dados da tabela `order_item`

Coluna	Tipo	Nulo	Padrão	Chaves	Descrição
`id`	bigint	não	identidade	PK	Identificador do item.
`quantity`	integer	sim	—	CK (2)	Quantidade vendida (> 0).
`price`	numeric(12,2)	sim	—	CK (2)	Preço unitário histórico no momento da venda.
`order_id`	bigint	sim	—	FK → orders	Pedido ao qual o item pertence.
`product_id`	bigint	sim	—	FK → product	Produto vendido.
`promotion_id`	bigint	sim	—	FK → promotion	Promoção aplicada ao item.
`discount`	numeric(12,2)	não	0	CK (3)	Desconto total do item (promoção + parte manual), ≤ preço × quantidade.
`unit_cost`	numeric(12,2)	sim	—	—	Custo médio atribuído na confirmação (nulo = desconhecido).
`manual_discount`	numeric(12,2)	não	0	CK (2)	Parte manual do desconto rateada no item (≤ discount).

Fonte: catálogo do PostgreSQL após V1–V11 [E-15].

D.9 TABELA ORDERS

Pedido. Venda registrada no balcão ou pelas rotas legadas, com vendedor, cliente, situação, idempotência e desconto manual. Migrações: V1, V2, V3, V5, V11. Colunas: 13.

Quadro 51 **–** Dicionário de dados da tabela `orders`

Coluna	Tipo	Nulo	Padrão	Chaves	Descrição
`id`	bigint	não	identidade	PK	Identificador do pedido.

Coluna	Tipo	Nulo	Padrão	Chaves	Descrição
moment	timestamp(6)	sim	—	—	Data e hora do pedido (horário da loja).
status	varchar(255)	sim	—	CK (1)	Situação: PENDING_PAYMENT, PROCESSING, PAID, SHIPPED, DELIVERED ou CANCELED.
customer_id	bigint	sim	—	FK → customer	Cliente do pedido.
address_id	bigint	sim	—	FK → address; UQ; CK (1)	Endereço de entrega (obrigatório se DELIVERY).
seller_id	bigint	sim	—	FK → app_users; UQ	Vendedor do pedido, sempre o usuário da sessão.
client_request_id	uuid	sim	—	UQ; CK (1)	Chave de idempotência (Idempotency-Key).
request_hash	char(64)	sim	—	CK (2)	SHA-256 do corpo da requisição (64 hex).
fulfillment	varchar(10)	não	PICKUP	CK (2)	Modalidade: PICKUP (retirada) ou DELIVERY (entrega).
manual_discount	numeric(12,2)	não	0	CK (1)	Valor do desconto manual do pedido.
manual_discount_percent	numeric(5,2)	sim	—	CK (2)	Percentual do desconto manual.
manual_discount_reason	varchar(200)	sim	—	CK (1)	Motivo do desconto manual (≥ 5 caracteres).
discount_approved_by	bigint	sim	—	FK → app_users	Gerente que aprovou desconto acima da alçada.

Fonte: catálogo do PostgreSQL após V1–V11 [E-15].

D.10 TABELA PAYMENT

Pagamento. Pagamento único do pedido (CARD, PIX ou BOLETO), em herança de tabela única. Migrações: V1, V4, V5, V7. Colunas: 17.

Quadro 52 **–** Dicionário de dados da tabela payment

Coluna	Tipo	Nulo	Padrão	Chaves	Descrição
id	bigint	não	identidade	PK	Identificador do pagamento.
payment_type	varchar(31)	não	—	CK (2)	Discriminador: CARD, PIX ou BOLETO.
amount	numeric(12,2)	sim	—	CK (1)	Valor do pagamento (= total do pedido).
status	varchar(255)	sim	—	CK (1)	Situação: PENDING, APPROVED, DECLINED ou EXPIRED.
payment_date	timestamp(6)	sim	—	—	Data e hora da aprovação.
order_id	bigint	sim	—	FK → orders; UQ	Pedido pago (1:1).

Coluna	Tipo	Nulo	Padrão	Chaves	Descrição
bar_code	varchar(255)	sim	—	—	Linha digitável do boleto.
due_date	date	sim	—	—	Vencimento do boleto.
card_number	varchar(255)	sim	—	CK (1)	Coluna legada; sempre nula (CHECK ck_payment_card_number_absent).
card_holder	varchar(255)	sim	—	—	Titular do cartão.
installments	integer	sim	—	—	Número de parcelas.
pix_key	varchar(255)	sim	—	—	Chave Pix.
pix_holder_name	varchar(255)	sim	—	—	Titular da chave Pix.
transaction_id	varchar(255)	sim	—	—	Identificador da transação Pix.
pix_key_type	varchar(255)	sim	—	—	Tipo da chave Pix.
card_last4	char(4)	sim	—	CK (2)	Quatro últimos dígitos do cartão.
authorization_code	varchar(20)	sim	—	—	Código de autorização do cartão.

Fonte: catálogo do PostgreSQL após V1–V11 [E-15].

D.11 TABELA PRODUCT

Produto. Produto físico ou digital, vendável ou insumo, com SKU, categoria e preço vigente. Migrações: V1, V4, V5. Colunas: 12.

Quadro 53 **–** Dicionário de dados da tabela `product`

Coluna	Tipo	Nulo	Padrão	Chaves	Descrição
id	bigint	não	identidade	PK	Identificador do produto.
product_type	varchar(31)	não	—	CK (1)	Discriminador: PHYSICAL ou DIGITAL.
name	varchar(255)	sim	—	—	Nome do produto.
price	numeric(12,2)	sim	—	CK (1)	Preço de venda vigente (> 0).
description	varchar(255)	sim	—	—	Descrição.
download_link	varchar(255)	sim	—	—	Link de download (produto digital).
weight	double precision	sim	—	—	Peso em kg (produto físico).
sku	varchar(40)	sim	—	UQ; CK (1)	Código SKU único, em maiúsculas.
category_id	bigint	sim	—	FK → category	Categoria do produto.
active	boolean	não	true	—	Produto ativo para venda e listagem.
sellable	boolean	não	true	—	Vendável (false = insumo).
updated_at	timestamptz	não	now()	—	Data da última alteração.

Fonte: catálogo do PostgreSQL após V1–V11 [E-15].

D.12 TABELA PRODUCT_PRICE_HISTORY

Histórico de preço. Registro de cada alteração de preço, com autor e motivo, gravado por gatilho. Migrações: V5. Colunas: 7.

Quadro 54 **–** Dicionário de dados da tabela `product_price_history`

Coluna	Tipo	Nulo	Padrão	Chaves	Descrição
`id`	bigint	não	identidade	PK	Identificador do registro.
`product_id`	bigint	não	—	FK → product	Produto alterado.
`old_price`	numeric(12,2)	sim	—	—	Preço anterior.
`new_price`	numeric(12,2)	não	—	—	Novo preço.
`changed_by`	bigint	sim	—	FK → app_users	Usuário que alterou (sessão).
`changed_at`	timestamptz	não	now()	—	Instante da alteração.
`reason`	varchar(200)	sim	—	—	Motivo informado.

Fonte: catálogo do PostgreSQL após V1–V11 [E-15].

D.13 TABELA PROMOTION

Promoção. Desconto percentual ou em valor, por produto ou categoria, liberado somente por gerente. Migrações: V5. Colunas: 11.

Quadro 55 **–** Dicionário de dados da tabela `promotion`

Coluna	Tipo	Nulo	Padrão	Chaves	Descrição
`id`	bigint	não	identidade	PK	Identificador da promoção.
`name`	varchar(120)	não	—	—	Nome.
`kind`	varchar(10)	não	—	CK (2)	Tipo: PERCENT ou AMOUNT.
`value`	numeric(12,2)	não	—	CK (2)	Valor ou percentual (> 0; percentual ≤ 100).
`product_id`	bigint	sim	—	FK → product; CK (1)	Escopo por produto (opcional).
`category_id`	bigint	sim	—	FK → category; CK (1)	Escopo por categoria (opcional).
`starts_at`	timestamptz	não	—	CK (1)	Início da vigência.
`ends_at`	timestamptz	não	—	CK (1)	Fim da vigência (> início).
`active`	boolean	não	true	—	Promoção ativa.
`created_by`	bigint	não	app_current_user_id()	FK → app_users	Gerente que liberou (verificado por gatilho).
`created_at`	timestamptz	não	now()	—	Data de criação.

Fonte: catálogo do PostgreSQL após V1–V11 [E-15].

D.14 TABELA REFUND

Estorno. Devolução financeira parcial ou total de um pagamento aprovado. Migrações: V5. Colunas: 6.

Quadro 56 **–** Dicionário de dados da tabela `refund`

Coluna	Tipo	Nulo	Padrão	Chaves	Descrição
`id`	bigint	não	identidade	PK	Identificador do estorno.
`payment_id`	bigint	não	—	FK → payment	Pagamento estornado.
`amount`	numeric(12,2)	não	—	CK (1)	Valor estornado (> 0).
`reason`	varchar(200)	sim	—	—	Motivo.
`created_by`	bigint	não	app_current_user_id()	FK → app_users	Gerente que registrou.
`created_at`	timestamptz	não	now()	—	Data do estorno.

Fonte: catálogo do PostgreSQL após V1–V11 [E-15].

D.15 TABELA ROLE

Papel. Domínio dos papéis de acesso (GERENTE, VENDEDOR) com hierarquia por posto. Migrações: V5. Colunas: 3.

Quadro 57 **–** Dicionário de dados da tabela `role`

Coluna	Tipo	Nulo	Padrão	Chaves	Descrição
`code`	varchar(20)	não	—	PK	Código do papel (GERENTE, VENDEDOR).
`name`	varchar(60)	não	—	—	Nome exibido.
`rank`	smallint	não	—	UQ	Posto hierárquico (GERENTE 20 > VENDEDOR 10).

Fonte: catálogo do PostgreSQL após V1–V11 [E-15].

D.16 TABELA SELLER_GOAL

Meta do vendedor. Parcela da meta mensal atribuída a um vendedor; a soma deve igualar a meta da loja. Migrações: V5. Colunas: 4.

Quadro 58 **–** Dicionário de dados da tabela `seller_goal`

Coluna	Tipo	Nulo	Padrão	Chaves	Descrição
`id`	bigint	não	identidade	PK	Identificador.
`monthly_goal_id`	bigint	não	—	FK → monthly_goal; UQ	Meta mensal distribuída.
`seller_id`	bigint	não	—	FK → app_users; UQ	Vendedor (gatilho exige papel VENDEDOR).

Coluna	Tipo	Nulo	Padrão	Chaves	Descrição
amount	numeric(12,2)	não	—	CK (1)	Valor da meta individual.

Fonte: catálogo do PostgreSQL após V1–V11 [E-15].

D.17 TABELA STOCK_BALANCE

Saldo de estoque. Saldo físico, reservado, mínimo e custo médio ponderado de cada produto. Migrações: V5. Colunas: 6.

Quadro 59 **–** Dicionário de dados da tabela `stock_balance`

Coluna	Tipo	Nulo	Padrão	Chaves	Descrição
product_id	bigint	não	—	FK → product; PK	Produto (PK e FK; uma linha por produto).
on_hand	integer	não	0	CK (2)	Saldo físico (≥ 0).
reserved	integer	não	0	CK (2)	Quantidade reservada por pedidos pendentes (≤ saldo).
min_level	integer	não	0	CK (1)	Estoque mínimo para alerta.
avg_cost	numeric(12,2)	sim	—	—	Custo médio ponderado (nulo = custo não informado).
updated_at	timestamptz	não	now()	—	Data da última atualização.

Fonte: catálogo do PostgreSQL após V1–V11 [E-15].

D.18 TABELA STOCK_ENTRY

Entrada de estoque. Entrada de mercadoria com quantidade e custo unitário; atualiza saldo e custo médio por gatilho. Migrações: V5. Colunas: 7.

Quadro 60 **–** Dicionário de dados da tabela `stock_entry`

Coluna	Tipo	Nulo	Padrão	Chaves	Descrição
id	bigint	não	identidade	PK	Identificador da entrada.
product_id	bigint	não	—	FK → product	Produto recebido.
quantity	integer	não	—	CK (1)	Quantidade (> 0).
unit_cost	numeric(12,2)	sim	—	CK (1)	Custo unitário (nulo = não informado).
created_by	bigint	não	app_current_user_id()	FK → app_users	Usuário que registrou.
created_at	timestamptz	não	now()	—	Data da entrada.
note	varchar(200)	sim	—	—	Observação.

Fonte: catálogo do PostgreSQL após V1–V11 [E-15].

D.19 TABELA STOCK_MOVEMENT

Movimento de estoque. Extrato de entradas, reservas, liberações, saídas e ajustes. Migrações: V5. Colunas: 9.

Quadro 61 **–** Dicionário de dados da tabela `stock_movement`

Coluna	Tipo	Nulo	Padrão	Chaves	Descrição
`id`	bigint	não	identidade	PK	Identificador do movimento.
`product_id`	bigint	não	—	FK → product	Produto movimentado.
`order_id`	bigint	sim	—	FK → orders	Pedido de origem (reserva, liberação, saída).
`kind`	varchar(16)	não	—	CK (3)	Tipo: ENTRADA, RESERVA, LIBERACAO, SAIDA, DEVOLUCAO, AJUSTE_ENTRADA ou AJUSTE_SAIDA.
`quantity`	integer	não	—	CK (1)	Quantidade (> 0).
`reason`	varchar(20)	sim	—	CK (3)	Motivo obrigatório nos ajustes (PERDA, AVARIA, USO_INTERNO, INVENTARIO, DEVOLUCAO_FORNECEDOR).
`note`	varchar(200)	sim	—	—	Observação.
`created_by`	bigint	sim	—	FK → app_users	Usuário responsável.
`created_at`	timestamptz	não	now()	—	Instante do movimento.

Fonte: catálogo do PostgreSQL após V1–V11 [E-15].

D.20 CHAVES ESTRANGEIRAS

O **Quadro** 62 lista as 32 chaves estrangeiras. Apenas uma tem exclusão em cascata (`seller_goal` → `monthly_goal`); as demais usam a ação padrão `NO ACTION`, o que impede excluir registros referenciados.

Quadro 62 **–** Chaves estrangeiras do esquema

Tabela	Coluna(s)	Referencia	Restrição	Ação na exclusão
`app_users`	`created_by`	`app_users(id)`	`app_users_created_by_fkey`	NO ACTION
`app_users`	`role`	`role(code)`	`fk_app_users_role`	NO ACTION
`audit_event`	`actor_id`	`app_users(id)`	`audit_event_actor_id_fkey`	NO ACTION
`customer`	`address_id`	`address(id)`	`client_address_id_fkey`	NO ACTION
`customer`	`created_by`	`app_users(id)`	`customer_created_by_fkey`	NO ACTION
`discount_policy`	`role`	`role(code)`	`discount_policy_role_fkey`	NO ACTION

Tabela	Coluna(s)	Referencia	Restrição	Ação na exclusão
discount_policy	updated_by	app_users(id)	discount_policy_updated_by_fkey	NO ACTION
monthly_goal	created_by	app_users(id)	monthly_goal_created_by_fkey	NO ACTION
order_item	order_id	orders(id)	order_item_order_id_fkey	NO ACTION
order_item	product_id	product(id)	order_item_product_id_fkey	NO ACTION
order_item	promotion_id	promotion(id)	order_item_promotion_id_fkey	NO ACTION
orders	seller_id	app_users(id)	fk_orders_seller	NO ACTION
orders	address_id	address(id)	orders_address_id_fkey	NO ACTION
orders	customer_id	customer(id)	orders_client_id_fkey	NO ACTION
orders	discount_approved_by	app_users(id)	orders_discount_approved_by_fkey	NO ACTION
payment	order_id	orders(id)	payment_order_id_fkey	NO ACTION
product	category_id	category(id)	product_category_id_fkey	NO ACTION
product_price_history	changed_by	app_users(id)	product_price_history_changed_by_fkey	NO ACTION
product_price_history	product_id	product(id)	product_price_history_product_id_fkey	NO ACTION
promotion	category_id	category(id)	promotion_category_id_fkey	NO ACTION
promotion	created_by	app_users(id)	promotion_created_by_fkey	NO ACTION
promotion	product_id	product(id)	promotion_product_id_fkey	NO ACTION
refund	created_by	app_users(id)	refund_created_by_fkey	NO ACTION
refund	payment_id	payment(id)	refund_payment_id_fkey	NO ACTION
seller_goal	monthly_goal_id	monthly_goal(id)	seller_goal_monthly_goal_id_fkey	CASCADE
seller_goal	seller_id	app_users(id)	seller_goal_seller_id_fkey	NO ACTION
stock_balance	product_id	product(id)	stock_balance_product_id_fkey	NO ACTION
stock_entry	created_by	app_users(id)	stock_entry_created_by_fkey	NO ACTION
stock_entry	product_id	product(id)	stock_entry_product_id_fkey	NO ACTION
stock_movement	created_by	app_users(id)	stock_movement_created_by_fkey	NO ACTION
stock_movement	order_id	orders(id)	stock_movement_order_id_fkey	NO ACTION
stock_movement	product_id	product(id)	stock_movement_product_id_fkey	NO ACTION

Fonte: catálogo do PostgreSQL [E-15].

D.21 VIEWS

O esquema tem 18 *views*, criadas pelas migrações V6 (16 definições), V9 e V11. As relações de origem foram extraídas das cláusulas `FROM` e `JOIN` de cada definição.

Quadro 63 **–** Views do esquema

View	Colunas	Relações de origem
v_alerta_estoque	6	v_posicao_estoque
v_catalogo_vendedor	6	category, product, stock_balance
v_cliente_resumo	8	order_item, orders, payment, refund
v_descontos_manuais	6	app_users, orders, payment
v_desempenho_vendedor	8	app_users, monthly_goal, seller_goal, v_lancamento_realizado
v_evolucao_semanal	5	v_lancamento_realizado
v_evolucao_semanal_loja	4	v_evolucao_semanal
v_faturamento_diario	10	order_item, v_lancamento_realizado
v_lancamento_realizado	6	order_item, orders, payment, refund
v_margem_produto	9	order_item, orders, payment, product
v_meu_desempenho	7	v_desempenho_vendedor
v_minha_evolucao_semanal	4	v_evolucao_semanal
v_minhas_vendas	6	orders, payment
v_posicao_estoque	15	category, product, stock_balance
v_promocoes_ativas	8	promotion
v_resumo_loja	7	monthly_goal, v_lancamento_realizado
v_uso_promocoes	5	order_item, promotion
v_vendas_forma_pagamento	4	v_lancamento_realizado

Fonte: catálogo do PostgreSQL [E-15].

D.22 FUNÇÕES E GATILHOS

Quadro 64 **–** Funções do esquema

Função	Argumentos	Retorno	Linguagem	Execução
app_current_role	—	character varying	sql	SECURITY DEFINER, STABLE
app_current_user_id	—	bigint	sql	SECURITY DEFINER, STABLE
fn_cancelar_pedido	p_order_id bigint	void	plpgsql	SECURITY DEFINER, VOLATILE
fn_confirmar_pagamento	p_order_id bigint	boolean	plpgsql	SECURITY DEFINER, VOLATILE
fn_criar_pedido	p_customer_id bigint, p_itens jsonb, p_payment_type character varying	bigint	plpgsql	SECURITY DEFINER, VOLATILE

Função	Argumentos	Retorno	Linguagem	Execução
fn_estornar	p_payment_id bigint, p_amount numeric, p_reason character varying	bigint	plpgsql	SECURITY DEFINER, VOLATILE
trg_goal_sum_check	—	trigger	plpgsql	SECURITY DEFINER, VOLATILE
trg_keep_one_active_manager	—	trigger	plpgsql	invoker, VOLATILE
trg_product_price_history	—	trigger	plpgsql	SECURITY DEFINER, VOLATILE
trg_product_stock_row	—	trigger	plpgsql	SECURITY DEFINER, VOLATILE
trg_promotion_only_manager	—	trigger	plpgsql	SECURITY DEFINER, VOLATILE
trg_seller_goal_is_seller	—	trigger	plpgsql	SECURITY DEFINER, VOLATILE
trg_stock_entry_apply	—	trigger	plpgsql	SECURITY DEFINER, VOLATILE

Fonte: catálogo do PostgreSQL [E-15].

Quadro 65 **–** Gatilhos do esquema

Gatilho	Tabela	Momento e evento	Função	Tipo
keep_one_active_manager	app_users	AFTER DELETE OR UPDATE OF role, active (por linha)	trg_keep_one_active_manager	restrição, adiável até o COMMIT
monthly_goal_sum	monthly_goal	AFTER INSERT OR UPDATE (por linha)	trg_goal_sum_check	restrição, adiável até o COMMIT
product_price_history	product	BEFORE UPDATE OF price (por linha)	trg_product_price_history	comum
product_stock_row	product	AFTER INSERT (por linha)	trg_product_stock_row	comum
promotion_only_manager	promotion	BEFORE INSERT OR UPDATE (por linha)	trg_promotion_only_manager	comum
seller_goal_is_seller	seller_goal	BEFORE INSERT OR UPDATE (por linha)	trg_seller_goal_is_seller	comum
seller_goal_sum	seller_goal	AFTER INSERT OR DELETE OR UPDATE (por linha)	trg_goal_sum_check	restrição, adiável até o COMMIT
stock_entry_apply	stock_entry	AFTER INSERT (por linha)	trg_stock_entry_apply	comum

Fonte: catálogo do PostgreSQL [E-15].

APÊNDICE E – CASOS DE USO

Os vinte casos de uso foram identificados nas rotas da interface, nas operações do contrato v5 e nas regras do código, na versão de referência [E-01] [E-06]. Os casos UC03, UC04, UC12 e UC17 estão descritos em detalhe na seção 4.4; os demais são especificados de forma resumida no **Quadro** 66. Os casos UC14, UC16 e UC20, assim como a exclusão de produto no UC10, são atendidos apenas pela API, sem tela na interface. Em todos os casos aplicam-se as pré-condições gerais de sessão ativa e de troca de senha concluída (exceto UC01 e UC02) e as respostas comuns do contrato: 401 `UNAUTHENTICATED` ou `SESSION_REVOKED` sem sessão válida, 403 `FORBIDDEN` a perfil sem permissão ou sem *token* CSRF em operação de escrita, e 400 `VALIDATION_FAILED` ou `MALFORMED_REQUEST` para entrada inválida.

Quadro 66 **–** Especificação resumida dos casos de uso

UC	Nome	Ator	Fluxo principal	Exceções específicas	Operações da API	RF
UC01	Autenticar-se	Vendedor, Gerente	Obter o *token* CSRF; enviar e-mail e senha; o servidor cria a sessão, renova o identificador e devolve o usuário com as áreas liberadas; a interface abre a primeira área do perfil; sair encerra a sessão	Credencial inválida, conta inativa ou bloqueada: 401 com resposta idêntica; quinta falha seguida bloqueia a conta por 15 minutos; senha provisória libera apenas a troca de senha	`GET /auth/csrf`, `POST /auth/login`, `GET /auth/me`, `POST /auth/logout`	RF-01, RF-18
UC02	Trocar a própria senha	Vendedor, Gerente	Informar a senha atual e a nova senha duas vezes; o servidor valida a política, grava o novo *hash*, renova a sessão e encerra as demais sessões do usuário	Senha atual errada: 401, contada para o bloqueio; senha fora da política (12 a 72 caracteres, até 72 bytes, fora da lista comum, diferente do e-mail): 400	`PUT /auth/password`	RF-24
UC05	Cancelar pedido pendente	Vendedor dono do pedido, Gerente	Abrir o pedido pendente e cancelar após confirmação; o sistema libera a reserva de cada item e marca o pedido como cancelado	Pedido de outro vendedor: 404; pedido pago: 409 `STATE_CONFLICT`; repetição sobre pedido já cancelado: 200 sem efeito	`POST /orders/{id}/cancel`	RF-13, RF-19
UC06	Cadastrar e consultar cliente	Vendedor, Gerente	Buscar por documento exato (só dígitos) ou prefixo do nome; cadastrar pessoa física (CPF) ou jurídica (CNPJ numérico ou alfanumérico); editar dados sem alterar o documento	Dígito verificador inválido: 400; documento já cadastrado: 409 `DUPLICATE_DOCUMENT`; documento no corpo da edição: 400; versão desatualizada: 409 `CONCURRENT_UPDATE`	`GET /customers`, `POST /customers`, `GET /customers/{id}`, `PUT /customers/{id}`	RF-03, RF-20
UC07	Consultar histórico e resumo do cliente	Vendedor, Gerente	Abrir o cliente e consultar o histórico paginado, do mais recente ao mais antigo, e os indicadores de compra	Para o vendedor, totais e formas de pagamento cobrem só os próprios pedidos e o documento vem mascarado; cliente inexistente: 404	`GET /customers/{id}/orders`, `GET /customers/{id}/summary`	RF-21, RF-25
UC08	Consultar Meu desempenho	Vendedor, Gerente	Abrir Meu desempenho e escolher o período; o sistema exibe meta, realizado, atingimento e falta do próprio usuário	Sem meta definida: “Sem meta”, sem percentual (RN-10)	`GET /performance/me`	RF-10, RF-11
UC09	Aplicar desconto manual na alçada	Vendedor, Gerente	No PDV, informar percentual e motivo; o servidor cota sobre o subtotal já promocionado, rateia nos itens e grava o mesmo total no pedido e no pagamento	Acima da alçada do papel: 403 `DISCOUNT_ABOVE_LIMIT`, sem criar pedido; abaixo do custo médio: 409 `BELOW_COST`, sem revelar o custo; motivo com menos de 5 caracteres: 400	`POST /sales/quote`, `POST /sales/orders`	RF-26

UC	Nome	Ator	Fluxo principal	Exceções específicas	Operações da API	RF
UC10	Manter produtos, insumos e categorias	Gerente	Cadastrar produto vendável ou insumo com SKU, categoria e tipo; editar dados sem preço nem situação; ativar ou inativar; criar e renomear categorias	Produto já vendido: exclusão recusada com 409 (deve ser inativado); produto físico sem peso ou digital sem *link*: 400	`GET/POST /products`, `GET/PUT/DELETE /products/{id}`, `PATCH /products/{id}/status`, `GET/POST /categories`, `PUT /categories/{id}`	RF-04, RF-22
UC11	Alterar preço com motivo	Gerente	Informar o novo preço e o motivo; o gatilho grava o histórico com autor e motivo; os itens já vendidos mantêm o preço da venda	Motivo fora de 5 a 200 caracteres ou preço abaixo de 0,01: 400; produto inexistente: 404	`PATCH /products/{id}/price`, `GET /products/{id}/price-history`	RF-22
UC13	Liberar e encerrar promoção	Gerente	Definir desconto percentual ou em valor, escopo por produto ou categoria e janela de vigência; a promoção vigente é aplicada no PDV; encerrar antecipa o fim	Produto ou categoria inexistente: 404; percentual acima de 100 ou janela inválida: 400 `VALIDATION_FAILED`, com as restrições do banco como segunda barreira (RN-11)	`GET /promotions`, `POST /promotions`, `POST /promotions/{id}/close`, `GET /promotions/active`	RF-17, RF-22
UC14	Definir e distribuir metas mensais	Gerente	Informar a meta da loja do mês e a parcela de cada vendedor; o banco confere a soma no `COMMIT`	Soma das parcelas diferente da meta da loja ou meta individual para quem não é vendedor: 400, com os gatilhos do banco como segunda barreira	`GET /goals/monthly`, `PUT /goals/monthly`	RF-09
UC15	Acompanhar faturamento e análises	Gerente	Abrir a visão geral e as análises; escolher o período; o sistema exibe faturamento líquido, lucro bruto, margem, pedidos pagos, metas da equipe, alertas de estoque, margens por produto e uso de promoções	Item sem custo no período: lucro e margem “Não informado”; erro de carga nunca exibido como zero	`GET /management/summary`, `GET /management/revenue`, `GET /analytics/sales`, `GET /analytics/stock`	RF-12, RF-23
UC16	Registrar estorno	Gerente	Informar valor e motivo do estorno de um pagamento aprovado; o realizado é descontado na data do estorno	Pagamento não aprovado: 409 `STATE_CONFLICT`; valor maior que o saldo estornável ou não positivo: 400	`POST /payment/{id}/refund`	RF-08
UC18	Definir alçadas de desconto	Gerente	Consultar e alterar, com motivo, o percentual máximo de desconto manual por papel e a permissão de venda abaixo do custo	Entrada inválida: 400	`GET /discount-policy`, `PUT /discount-policy/{role}`	RF-26
UC19	Aprovar desconto acima da alçada	Gerente	Abrir o pedido pendente e aplicar desconto até a alçada do gerente, com motivo; o sistema recalcula itens, total e pagamento e registra o aprovador	Pedido pago ou cancelado: 409; pedido inexistente: 404	`POST /sales/orders/{id}/discount-approval`	RF-26

UC	Nome	Ator	Fluxo principal	Exceções específicas	Operações da API	RF
UC20	Excluir cliente sem pedidos	Gerente	Solicitar pela API a exclusão de cliente sem pedidos; a interface não oferece essa ação	Cliente com pedido: 409 `RESOURCE_IN_USE`; vendedor: 403	`DELETE /customers/{id}`	RF-20

Fonte: elaborado pelos autores a partir do contrato v5, dos controladores e serviços e das rotas da interface [E-06] [E-01].

APÊNDICE F – PLANO DE TESTES

Este apêndice consolida o plano de testes da equipe, distribuído entre `docs/plano-testes.md` (03/10/2026, com atualizações até 14/10) e `docs/plano-qualidade.md` (versão 0.1 de 09/10/2026), e o plano de verificação aplicado nesta auditoria [E-43] [E-37]. Os dois planos da equipe constam como rascunho, com aprovação nominal pendente.

F.1 OBJETIVOS E ESCOPO

O plano da equipe tem quatro objetivos: demonstrar a conformidade entre requisitos, contrato, API, banco e interface; preservar o comportamento da API original antes das ampliações; exercitar comportamentos positivos e negativos, autorização por perfil, integridade de estoque e regras monetárias; e registrar evidências rastreáveis para a avaliação acadêmica. O escopo abrange a API original, o contrato REST, o banco de dados, a autenticação e a autorização, as metas e o realizado, o estoque e o custo e a interface. Ficam fora do escopo o gateway real de pagamento, os dados reais e a certificação de produção [E-43].

F.2 OBJETIVOS DE QUALIDADE E METAS

Quadro 67 **–** Objetivos de qualidade do plano da equipe

ID	Objetivo	Indicador	Meta declarada
O1	Nenhum defeito grave aberto	Defeitos críticos ou altos não encerrados	0
O2	Contrato respeitado	Casos HTTP de contrato aprovados contra o OpenAPI 3.1	100%
O3	Regras críticas testadas por dentro	Cobertura de *branches* do pacote de serviços	≥ 60% (proposta)
O4	Cobertura global	Cobertura de instruções (JaCoCo)	≥ 70% (proposta)
O5	Manutenibilidade não piora	Achados PMD, SpotBugs e blocos CPD	R2 ≤ R0
O6	Rastreabilidade	RF ligados a CA, CT, execução e defeito	100%
O7	Reprodutibilidade	Resultados reproduzidos ao vivo com um comando	Todos os números do relatório final

Fonte: `docs/plano-qualidade.md`, seção 1 [E-37].

Na versão analisada, O2, O4 e O6 estão atingidos segundo os resultados documentados (seção 8.19); O1 depende do encerramento formal dos defeitos (A-

13); O3 não foi medido separadamente para o pacote de serviços na R1; O5 não foi avaliado, porque a R2 não foi realizada e a R1 registrou aumento absoluto de achados com o crescimento do código; O7 está parcialmente atingido (seção 13.10).

F.3 NÍVEIS, TÉCNICAS E FERRAMENTAS

Quadro 68 **–** Níveis de teste e ferramentas

Nível	O que verifica	Ferramentas	Situação na versão analisada
Unitário	Regras isoladas, cálculos, estados e validações	JUnit 5, Mockito, AssertJ	24 classes, 128 métodos (inventário estático)
Integração	Persistência, restrições, transações, migrações e concorrência com banco real	Spring Boot Test, PostgreSQL 18 descartável (Docker Compose)	20 classes, 119 métodos (inventário estático)
Banco de dados	Regras declarativas, *views*, permissões e concorrência por SQL	`psql`, pacote `docs/bd/pacote-sql`	61 verificações + 11 da massa Olist
Sistema e contrato	API em execução contra o contrato v5 e a sessão	Node 24, Ajv 8 (`live-contract.mjs`, `http-contract.mjs`, `http-session.mjs`)	58 operações declaradas; relatório versionado com 54
Aceitação (ponta a ponta)	Fluxos dos dois perfis pela interface, com acessibilidade	Playwright 1.63, axe-core	8 especificações, 26 casos
Regressão	Defeitos corrigidos não voltam	Testes `[Dxxx]` com marcação de regressão, CI a cada *push* e PR	20 defeitos com teste de regressão

Fonte: `docs/plano-qualidade.md`, `docs/plano-testes.md` e inventário desta auditoria [E-37] [E-43] [E-21].

As técnicas de caixa-preta previstas são particionamento de equivalência, análise de valor-limite, tabela de decisão e transição de estados, aplicadas, respectivamente, a tipos de cliente, produto e pagamento e a perfis; a quantidades 0 e 1, meta zero, saldo 0 e 1 e limites monetários; à autorização por perfil, à soma de metas e à visibilidade de custo; e aos estados de pedido, pagamento, reserva, cancelamento e estorno. A técnica de caixa-branca prevista é a cobertura de instruções e *branches* com JaCoCo e a análise de caminhos independentes dos métodos mais complexos [E-37].

F.4 DADOS E AMBIENTES DE TESTE

Os dados de teste são sintéticos: quatro contas com e-mails `@sbi.tech`, meta da loja de R$ 240.000,00 para setembro de 2026, realizado demonstrativo por vendedor, venda demonstrativa de R$ 5.700,00 e produtos com saldo, sem saldo e

sem custo conhecido, além da massa Olist anonimizada [E-43]. Os ambientes previstos são o banco PostgreSQL 18 descartável por Docker Compose na porta 15432 para integração, a API isolada no perfil `contract` para os testes HTTP, a pilha de demonstração local e a integração contínua no GitHub Actions, em Linux.

F.5 CRITÉRIOS DE ENTRADA, SAÍDA E APROVAÇÃO

Uma rodada formal só começa com *commit* identificado na `main`, integração contínua verde ou falhas classificadas, banco descartável, massa sintética versionada e casos selecionados vinculados a requisito e critério. Ela termina quando todos os casos selecionados foram executados e os relatórios brutos preservados, cada falha foi classificada como de produto, teste, contrato ou ambiente, cada defeito foi registrado com os 13 campos e as métricas da rodada foram geradas e interpretadas. Um caso é aprovado quando o resultado obtido é igual ao esperado com evidência reproduzível; uma correção de defeito é aprovada quando o teste falha antes e passa depois, a regressão completa está verde e outro integrante faz o reteste [E-37].

F.6 PLANO DE VERIFICAÇÃO DESTA AUDITORIA

O plano aplicado nesta auditoria seguiu três restrições: nenhuma operação de escrita no ambiente publicado, nenhum teste ofensivo e nenhuma alteração no repositório. O **Quadro** 69 resume o que foi planejado e o que foi efetivamente executado.

Quadro 69 **–** Plano de verificação desta auditoria

Verificação	Objetivo	Ambiente	Executada	Resultado
Pacote SQL do job `bd`	Confirmar regras, permissões e concorrência no banco	PostgreSQL 16.15 local	Sim	61/61
Massa Olist por SQL	Confirmar carga, idempotência e CRUD	PostgreSQL 16.15 local	Sim (7 de 11)	7/7
Validadores de contrato	Confirmar políticas e esquemas do contrato	Node 22.23.4	Sim	47/47
Auditoria estática do contrato v5	Confirmar operações e esquemas	Node 22.23.4	Sim	0 erros, 236 esquemas
Verificação de tipos e lógica do frontend	Confirmar esquemas gerados e regras de interface	Node 22.23.4, TypeScript 6.0.3	Sim	1/1 e 41/42

Verificação	Objetivo	Ambiente	Executada	Resultado
Construção do frontend	Confirmar o *build* de produção	Node 22.23.4	Tentada	Bloqueada pela rede (fontes externas)
Suítes Java (unitária e integração)	Reproduzir 233 + 649 testes	JDK 25, Maven, Docker	Não	Dependências indisponíveis; aprovadas na integração contínua do *commit* de referência
Contrato ao vivo e ponta a ponta	Reproduzir 58 operações e 24 casos E2E	API e interface locais	Não	Dependências indisponíveis; aprovados na integração contínua do *commit* de referência
Consulta à integração contínua	Confirmar a conclusão das suítes não reproduzidas	API do GitHub Actions	Sim	4/4 *jobs* com sucesso; fumaça publicada sem execução registrada
Consulta ao serviço da API	Confirmar versão implantada e configuração	API do Render	Sim	Implantação ativa do *commit* `3b0242c`
Inspeção do ambiente publicado	Confirmar disponibilidade, 401 padronizado e cabeçalhos	Navegador integrado	Sim	5/5
Comparação do OpenAPI publicado com o v5	Medir conformidade estrutural e de erros	Script de comparação	Sim	58/58; 92/201 erros
Consulta somente leitura ao banco publicado	Confirmar esquema, migrações e volumes	Neon, PostgreSQL 18.6	Sim	Paridade de contagens
Auditoria de dependências	Identificar avisos de segurança	`npm audit`	Sim	6 pacotes no frontend; 1 na raiz
Recálculo das métricas R0 e R1	Confirmar cobertura, testes e análise estática declarados	Relatórios brutos versionados	Sim	Valores idênticos aos declarados
Validação das correções propostas	Confirmar as correções de A-03, A-04, A-05, A-12 e A-16	*Branch* local, Node 22.23.4, Next.js 16.4.3	Sim	Tipos aprovados; 43/43; cabeçalhos presentes; 0 vulnerabilidade de produção

Fonte: elaborado pelos autores [E-11] [E-12] [E-08] [E-09] [E-10] [E-04] [E-05] [E-03] [E-24] [E-22] [E-46] [E-47] [E-48].

APÊNDICE G – CASOS DE TESTE E RESULTADOS

Este apêndice registra, caso a caso, os testes executados nesta auditoria em 24/10/2026 e o catálogo de casos documentado pela equipe. Os resultados executados vêm dos arquivos de evidência indicados; os resultados apenas documentados estão marcados como não executados nesta auditoria. Nenhum resultado foi inferido.

G.1 PACOTE SQL E MASSA OLIST

Ambiente: PostgreSQL 16.15 local, esquema migrado de V1 a V11 com o pacote de demonstração, scripts `run_bd.sh` (61 verificações) e `07_carga_olist.sql`, `08_crud_clientes_olist.sql` e `09_testes_olist.sql` (7 verificações). O resultado esperado é a afirmação verificada pelo próprio script, que emite PASS somente quando a consulta confirma o valor ou a recusa esperada.

Tabela 8 **–** Casos executados na camada de banco

ID	Requisito	Cenário	Resultado esperado	Resultado observado	Status	Evidência
BD-01	RF-10, RF-11	Metas e realizado (massa de setembro)	Allan 68.400 / 120.000 = 57,0%	Conforme o esperado	Aprovado	EV-BD-01
BD-02	RF-10	Metas e realizado (massa de setembro)	Francisco 51,7% e só dois vendedores com meta	Conforme o esperado	Aprovado	EV-BD-01
BD-03	RF-10	Metas e realizado (massa de setembro)	Loja 186.400 / 240.000 = 77,7%, falta 53.600	Conforme o esperado	Aprovado	EV-BD-01
BD-04	RF-10	Metas e realizado (massa de setembro)	Evolução semanal 38.200/41.600/36.800/42.300/27.500	Conforme o esperado	Aprovado	EV-BD-01
BD-05	RF-10 (RN-09)	Metas e realizado (massa de setembro)	Pendentes de 01/10 fora do realizado	Conforme o esperado	Aprovado	EV-BD-01
BD-06	RF-13	Estoque, reserva e confirmação	Saldo, reservado e disponível	Conforme o esperado	Aprovado	EV-BD-01
BD-07	RF-14, RF-23	Custo médio e margem	Margem NULL quando há item sem custo	Conforme o esperado	Aprovado	EV-BD-01
BD-07b	RF-23	Custo médio e margem	iPhone e AirPods de setembro com custo e margem	Conforme o esperado	Aprovado	EV-BD-01
BD-08	RF-23	Custo médio e margem	Margem calculada quando custo conhecido (Apple Watch 32,1%)	Conforme o esperado	Aprovado	EV-BD-01
BD-09	RF-22	Alerta de estoque mínimo	Apple Watch abaixo do mínimo e iPhone fora do alerta	Conforme o esperado	Aprovado	EV-BD-01
BD-10	RF-07 (RN-03)	Limites de domínio (CHECK)	quantidade 0 rejeitada	Conforme o esperado	Aprovado	EV-BD-01
BD-11	RF-07 (RN-03)	Limites de domínio (CHECK)	quantidade 1 aceita	Conforme o esperado	Aprovado	EV-BD-01
BD-12	RF-22	Limites de domínio (CHECK)	preço 0,00 rejeitado (mínimo 0,01)	Conforme o esperado	Aprovado	EV-BD-01
BD-13	RN-11	Limites de domínio (CHECK)	desconto 100,01% rejeitado	Conforme o esperado	Aprovado	EV-BD-01
BD-14	RF-17 (RN-11)	Promoções	promoção exige gerente	Conforme o esperado	Aprovado	EV-BD-01
BD-15	RF-13	Estoque, reserva e confirmação	reservado <= saldo garantido pelo banco	Conforme o esperado	Aprovado	EV-BD-01
BD-16	RF-09	Distribuição de metas	distribuição divergente rejeitada	Conforme o esperado	Aprovado	EV-BD-01
BD-17	RF-10	Metas e realizado (massa de setembro)	Meta zero -> Sem meta e atingimento NULL	Conforme o esperado	Aprovado	EV-BD-01
BD-18	RF-14	Custo médio e margem	Custo médio ponderado = 150,00	Conforme o esperado	Aprovado	EV-BD-01
BD-19	RF-08, RF-13	Estoque, reserva e confirmação	Segunda confirmação sem efeito; iPhone 9/1/8	Conforme o esperado	Aprovado	EV-BD-01

ID	Requisito	Cenário	Resultado esperado	Resultado observado	Status	Evidência
BD-20	RF-10	Metas e realizado (massa de setembro)	Realizado de outubro do Allan = 5.700 após confirmação	Conforme o esperado	Aprovado	EV-BD-01
BD-21	RF-13	Estoque, reserva e confirmação	Cancelamento libera reserva (iPhone reservado 2 -> 1)	Conforme o esperado	Aprovado	EV-BD-01
BD-22	RF-08	Estornos	Estorno de 1.200 entra como -1.200 na data do estorno; setembro preservado	Conforme o esperado	Aprovado	EV-BD-01
BD-23	RF-08 (RN-13)	Estornos	estorno acima do pago rejeitado	Conforme o esperado	Aprovado	EV-BD-01
BD-24	RF-22	Histórico de preço	Histórico de preço com autor gerente	Conforme o esperado	Aprovado	EV-BD-01
BD-25	RF-07	Histórico de preço	Item vendido mantém preço 4.500 após reajuste	Conforme o esperado	Aprovado	EV-BD-01
BD-26	RF-07	Desconto de promoção	Desconto 10% sobre 2 x 300 = 60; total 540	Conforme o esperado	Aprovado	EV-BD-01
BD-27	RN-11	Promoções	promoção fora do escopo rejeitada	Conforme o esperado	Aprovado	EV-BD-01
BD-28	RF-13, RF-19	Estoque, reserva e confirmação	venda acima do disponível rejeitada (LJ409 -> HTTP 409)	Conforme o esperado	Aprovado	EV-BD-01
BD-29	RF-06	Pedido sem vendedor	pedido sem usuário rejeitado	Conforme o esperado	Aprovado	EV-BD-01
RB-01	RF-02, RF-17	Permissões do papel VENDEDOR (GRANT e RLS)	tabela product (custo) negada ao vendedor	Conforme o esperado	Aprovado	EV-BD-01
RB-02	RF-02, RF-17	Permissões do papel VENDEDOR (GRANT e RLS)	margem negada ao vendedor	Conforme o esperado	Aprovado	EV-BD-01
RB-03	RF-02, RF-17	Permissões do papel VENDEDOR (GRANT e RLS)	resultado da loja negado ao vendedor	Conforme o esperado	Aprovado	EV-BD-01
RB-04	RF-02, RF-17	Permissões do papel VENDEDOR (GRANT e RLS)	coluna unit_cost negada ao vendedor	Conforme o esperado	Aprovado	EV-BD-01
RB-05	RF-02 (RN-19)	Permissões do papel VENDEDOR (GRANT e RLS)	coluna card_number negada ao vendedor	Conforme o esperado	Aprovado	EV-BD-01
RB-06	RF-02, RF-17	Permissões do papel VENDEDOR (GRANT e RLS)	alteração de estoque negada	Conforme o esperado	Aprovado	EV-BD-01
RB-07	RF-17 (RN-11)	Permissões do papel VENDEDOR (GRANT e RLS)	criação de promoção negada	Conforme o esperado	Aprovado	EV-BD-01
RB-08	RF-02 (RN-13)	Permissões do papel VENDEDOR (GRANT e RLS)	estorno negado ao vendedor	Conforme o esperado	Aprovado	EV-BD-01
RB-09	RF-14	Permissões do papel VENDEDOR (GRANT e RLS)	Catálogo do vendedor sem colunas de custo	Conforme o esperado	Aprovado	EV-BD-01

ID	Requisito	Cenário	Resultado esperado	Resultado observado	Status	Evidência
RB-10	RF-02, RF-11	Permissões do papel VENDEDOR (GRANT e RLS)	Vendedor ve somente os próprios pedidos (BOLA)	Conforme o esperado	Aprovado	EV-BD-01
RB-11	RF-11	Permissões do papel VENDEDOR (GRANT e RLS)	Meu desempenho retorna somente o Allan	Conforme o esperado	Aprovado	EV-BD-01
RB-12	RF-11	Permissões do papel VENDEDOR (GRANT e RLS)	Minha evolução semanal = 12.000/15.000/13.400/18.000/10.000	Conforme o esperado	Aprovado	EV-BD-01
RB-13	RF-02	Permissões do papel VENDEDOR (GRANT e RLS)	cancelamento de pedido alheio negado	Conforme o esperado	Aprovado	EV-BD-01
RB-14	RF-06	Permissões do papel VENDEDOR (GRANT e RLS)	Pedido criado pela vendedora tem seller = Allan	Conforme o esperado	Aprovado	EV-BD-01
RG-01	RF-17	Permissões do papel GERENTE	Gerente ve todos os pedidos	Conforme o esperado	Aprovado	EV-BD-01
RG-02	RF-17	Permissões do papel GERENTE	Gerente le posição de estoque com custo	Conforme o esperado	Aprovado	EV-BD-01
RG-03	RF-22 (RN-21)	Permissões do papel GERENTE	Atualização de preço registra gerente e motivo	Conforme o esperado	Aprovado	EV-BD-01
RG-04	RN-11	Permissões do papel GERENTE	Gerente libera promoção, visível como ativa	Conforme o esperado	Aprovado	EV-BD-01
RG-05	RF-14, RF-22	Permissões do papel GERENTE	Entrada de estoque soma saldo e recalcula custo (2x1900 + 5x1950)	Conforme o esperado	Aprovado	EV-BD-01
RG-06	RF-22	Permissões do papel GERENTE	saldo só muda por entrada/movimento	Conforme o esperado	Aprovado	EV-BD-01
RG-07	RF-02	Permissões do papel GERENTE	exclusão física de pedidos negada	Conforme o esperado	Aprovado	EV-BD-01
BD-30a	RF-24 (RN-18)	Último gerente ativo	último gerente não pode ser rebaixado	Conforme o esperado	Aprovado	EV-BD-01
BD-30b	RF-24 (RN-18)	Último gerente ativo	último gerente não pode ser desativado	Conforme o esperado	Aprovado	EV-BD-01
BD-30c	RF-24 (RN-18)	Último gerente ativo	rebaixamento aceito com outro gerente ativo	Conforme o esperado	Aprovado	EV-BD-01
BD-31a	RF-25	Resumo do cliente	Resumo dos clientes soma 186.400 em 65 pedidos pagos (seed)	Conforme o esperado	Aprovado	EV-BD-01
BD-31b	RF-23, RF-25	Resumo do cliente	Resumo dos clientes igual ao faturamento líquido de v_faturamento_diário	Conforme o esperado	Aprovado	EV-BD-01
BD-31c	RF-25	Resumo do cliente	Marina Costa: primeira e última compra e pedidos pagos coerentes com os pedidos	Conforme o esperado	Aprovado	EV-BD-01

ID	Requisito	Cenário	Resultado esperado	Resultado observado	Status	Evidência
BD-32a	RF-26	Alçadas de desconto manual	Alçadas iniciais: VENDEDOR 5% e GERENTE 20%, sem venda abaixo do custo	Conforme o esperado	Aprovado	EV-BD-01
BD-32b	RF-26	Alçadas de desconto manual	desconto manual exige motivo	Conforme o esperado	Aprovado	EV-BD-01
BD-32c	RF-26	Alçadas de desconto manual	parte manual nunca passa do desconto do item	Conforme o esperado	Aprovado	EV-BD-01
CC-01	RF-13, RF-19 (RN-04)	Estoque, reserva e confirmação	duas reservas simultâneas da última unidade: exatamente uma aceita	Conforme o esperado	Aprovado	EV-BD-01
CT-DEMO-06	RF-03, RF-20	Massa Olist (carga e consistência)	roteiro 08: CRUD com as três recusas esperadas (CHECK de CPF, tipo, FK de pedidos)	Conforme o esperado	Aprovado	EV-BD-02
CT-DEMO-01a	RNF-04	Massa Olist (carga e consistência)	60 clientes (8 PJ), 42 produtos em 7 categorias, 219 pedidos e 237 itens	Conforme o esperado	Aprovado	EV-BD-02
CT-DEMO-01b	RNF-04	Massa Olist (carga e consistência)	faturamento bruto 449.690,00 e pagamentos aprovados com o mesmo total	Conforme o esperado	Aprovado	EV-BD-02
CT-DEMO-02	RF-25, RNF-04	Massa Olist (carga e consistência)	CLI-001 com 14 pedidos pagos, itens e resumo iguais ao CSV	Conforme o esperado	Aprovado	EV-BD-02
CT-DEMO-03	RNF-04	Massa Olist (carga e consistência)	carga repetida sem duplicar SKU, cliente, pedido ou entrada de estoque	Conforme o esperado	Aprovado	EV-BD-02
CT-DEMO-04	RF-13, RNF-04	Massa Olist (carga e consistência)	saldo = entrada - vendido, nunca negativo, reservado 0	Conforme o esperado	Aprovado	EV-BD-02
CT-DEMO-07	RNF-04	Massa Olist (carga e consistência)	produtos da massa nas 7 categorias Apple, com nome do CSV; nenhuma categoria do catálogo antigo	Conforme o esperado	Aprovado	EV-BD-02

Fonte: execução desta auditoria, 24/10/2026 [E-11] [E-12].

O caso CT-DEMO-05, que exercita a massa Olist pela API, não foi executado nesta auditoria, porque depende da API em execução local.

G.2 VALIDADORES DE CONTRATO

Ambiente: Node 22.23.4, `npm test` na raiz do repositório. Os nomes dos testes são os do código-fonte. O resultado esperado de cada caso é a asserção do próprio teste; o observado é o registro TAP.

Tabela 9 **–** Casos executados nos validadores de contrato

Nº	Arquivo	Caso (nome no código)	Status
1	`audit-contract`	planned target has references and profile policies	Aprovado
2	`audit-contract`	broken reference is detected	Aprovado
3	`audit-contract`	protected endpoint cannot declare public security	Aprovado
4	`audit-contract`	anonymous alternative cannot bypass session policy	Aprovado
5	`audit-contract`	login and logout require CSRF and document rejection	Aprovado
6	`audit-contract`	session login rejects bearer response and malformed cookie scheme	Aprovado
7	`audit-contract`	archived v2 still passes its original bearer policies	Aprovado
8	`audit-contract`	manager policy cannot admit a seller	Aprovado
9	`audit-contract`	seller projection cannot allow unknown root or nested fields	Aprovado
10	`audit-contract`	explicit sensitive field in seller projection is detected	Aprovado
11	`audit-contract`	required path parameters cannot disappear	Aprovado
12	`audit-contract`	v4 draft passes with the executable access matrix	Aprovado
13	`audit-contract`	v4 accepts implemented operations; v3 stays planning-only	Aprovado
14	`audit-contract`	v4 x-roles must match the access matrix	Aprovado
15	`audit-contract`	v4 seller responses stay closed and free of cost data	Aprovado
16	`audit-contract`	v4 request schemas reject unknown properties	Aprovado
17	`audit-contract`	v4 sale creation requires Idempotency-Key and protected operations keep session security	Aprovado
18	`audit-contract`	v5 keeps every v4 operation and passes with the executable access matrix	Aprovado
19	`audit-contract`	v5 user administration stays manager-only and closed	Aprovado
20	`audit-contract`	planned operations wait for their matrix row; implemented ones require it	Aprovado
21	`check-commit-messages`	mensagem limpa em Conventional Commits passa	Aprovado
22	`check-commit-messages`	corpo vazio passa	Aprovado
23	`check-commit-messages`	cada termo proibido falha, em qualquer caixa	Aprovado
24	`check-commit-messages`	linha de sessão falha; a palavra session em texto comum passa	Aprovado
25	`check-commit-messages`	termo dentro de outra palavra não conta; comentario do Git fica de fora	Aprovado
26	`check-commit-messages`	endereço antigo do site falha na mensagem e no arquivo; o endereço oficial passa	Aprovado
27	`check-commit-messages`	--tracked: nenhum arquivo versionado tem endereço antigo do site	Aprovado

Nº	Arquivo	Caso (nome no código)	Status
28	validate-contract	2020-12 tuple keywords are executed	Aprovado
29	validate-contract	all target schema locations compile and validate against 2020-12	Aprovado
30	validate-contract	session login response rejects token fields and CSRF response requires the exact header	Aprovado
31	validate-contract	seller projection accepts valid sample and rejects sensitive fields at both levels	Aprovado
32	validate-contract	formats reject invalid email, wrong currency and non numeric money	Aprovado
33	validate-contract	sales order rejects zero quantity and empty items	Aprovado
34	validate-contract	invalid JSON Schema and broken references fail	Aprovado
35	validate-contract	official OpenAPI document validates refs and schemas without mutating downloaded schema	Aprovado
36	validate-contract	official document checks reject missing info, malformed parameter and extra operation field	Aprovado
37	validate-contract	media-range format accepts JSON and wildcards, rejects malformed content keys	Aprovado
38	validate-contract	error revision validates and preserves original baseline and successful responses	Aprovado
39	validate-contract	400 and 404 error payloads validate with nullable details and local timestamp	Aprovado
40	validate-contract	SignedMoney accepts negative results while Money stays non-negative	Aprovado
41	validate-contract	checkResponse resolves v4 shared responses such as NotFound	Aprovado
42	validate-contract	v4 document validates against the official OpenAPI 3.1 schema	Aprovado
43	validate-contract	v4 sale request rejects seller, price and total fields	Aprovado
44	validate-contract	v4 sale request enforces quantity bounds and payment variants	Aprovado
45	validate-contract	v4 error response requires code and keeps details as strings or null	Aprovado
46	validate-contract	v4 customer projections mask documents and keep the document immutable	Aprovado
47	validate-contract	v5 accepts numeric and alphanumeric CNPJ, masked the same way, and keeps letters uppercase	Aprovado

Fonte: `EV-CT-01_npm-test-raiz_2026-10-24.tap` [E-08].

Complementarmente, a auditoria estática do contrato v5 (`audit-contract.mjs`) encontrou 46 caminhos, 58 operações e nenhum erro, e a validação dos esquemas (`validate-contract.mjs`) percorreu 236 esquemas sem achados [E-07].

G.3 LÓGICA DO FRONTEND

Ambiente: Node 22.23.4 e TypeScript 6.0.3, `npm run typecheck` (aprovado) e `npm test` em `frontend/`.

Tabela 10 **–** Casos executados na lógica do frontend

Nº	Caso (nome no código)	Status
1	[CT-WEB-122] nome acessível único da marca	Aprovado
2	[CT-WEB-122] símbolo do componente tem a mesma geometria do favicon app/icon.svg	Aprovado
3	variantes do logo: negativa sobre Tinta e principal sobre fundo claro	Aprovado
4	[CT-WEB-123] ícones: os seis do manual e os derivados, decorativos por padrão	Aprovado
5	menu em seções GESTAO, ATENDIMENTO e MEU ESPACO; Trocar senha sempre presente	Aprovado
6	[CT-WEB-124] formatos do manual: hora 14h30, período, ausente e valor curto	**Reprovado**
7	[CT-WEB-125] nenhum nome proibido da marca na interface nem na documentacao	Aprovado
8	adiciona, soma e limita ao disponível; promoção vigente vai pre-preenchida (C-08)	Aprovado
9	produto sem disponível não entra e o carrinho para em 50 itens (C-14)	Aprovado
10	mesma chave enquanto o corpo não muda; corpo novo gera chave nova (CT-WEB-104)	Aprovado
11	409 STOCK_INSUFFICIENT: detalhe vira disponível e o carrinho é preservado com a quantidade ajustada	Aprovado
12	carrinho e chave ficam no sessionStorage por usuário e sobrevivem a JSON inválido	Aprovado
13	desconto manual digitado volta com o carrinho depois de recarregar a página	Aprovado
14	"há N dias" conta dias de calendário no horário da loja (America/Fortaleza)	Aprovado
15	documento repetido (409) traz o id do cliente já cadastrado; outros erros não	Aprovado
16	desconto manual só vai no corpo com percentual de 0,01 a 100 (2 casas) e motivo de 5 a 200 caracteres	Aprovado
17	mutação busca CSRF uma vez, envia o cabeçalho e usa cookies da mesma origem	Aprovado
18	POST /sales/orders envia Idempotency-Key; GET não envia CSRF	Aprovado
19	403 em mutação renova o CSRF uma vez (token trocado no login)	Aprovado
20	ErrorResponse.code vira mensagem em pt-BR sem código HTTP; details preservados	Aprovado
21	401 fora do login chama onUnauthorized; corpo do servidor não vaza na mensagem	Aprovado
22	resposta fora do contrato falha em teste (safeParse)	Aprovado
23	falha de rede vira ApiError com mensagem de conexão	Aprovado
24	login: mesma mensagem genérica no 401 e token renovado antes e depois	Aprovado
25	403 de regra de negócio (DISCOUNT_ABOVE_LIMIT) não repete a mutação e devolve os details	Aprovado
26	502, 503 e 504 sem ErrorResponse dizem que o sistema esta iniciando, sem código HTTP	Aprovado
27	miniatura pelo SKU, pelo nome (resumo do cliente) ou icone da linha	Aprovado
28	toda foto do manifesto existe em public/produtos e tem autor e licença	Aprovado
29	menu vem das áreas do servidor; vendedor não ve Gestão (CT-WEB-103)	Aprovado
30	destino depois do login respeita ?next= interno e ignora destino externo	Aprovado
31	CPF e CNPJ conferem dígito verificador	Aprovado
32	dinheiro e percentual em pt-BR; ausente vira travessão	Aprovado

Nº	Caso (nome no código)	Status
33	mensagens por código, com fallback por status e sem código HTTP	Aprovado
34	formulário de cliente validado pelo schema gerado do contrato	Aprovado
35	valor digitado aceita vírgula ou ponto decimal	Aprovado
36	problemas do schema saem com o rótulo do campo, nunca com o nome técnico do corpo	Aprovado
37	senha provisória leva sempre para /conta/senha, mesmo com ?next=	Aprovado
38	Usuarios aparece no menu só com a área USUARIOS	Aprovado
39	situação: inativo, bloqueado até HH:MM no horário da loja, ativo	Aprovado
40	linha do próprio usuário não oferece papel, situação nem redefinição de senha (CT-USR-104)	Aprovado
41	troca de senha: confere tamanho, bytes, e-mail, igualdade e confirmação antes de enviar	Aprovado
42	todo ErrorCode do contrato tem mensagem em pt-BR	Aprovado

Fonte: `EV-WEB-01_frontend-typecheck-unit_2026-10-24.tap` [E-09].

O caso 6 (CT-WEB-124) esperava a forma compacta “R$ 47 mil” e obteve “R$ 47,0 mil”. A diferença decorre da versão do Node e dos dados de internacionalização do ambiente de execução; a integração contínua usa Node 24, em que o caso consta como aprovado [E-22]. A falha foi registrada como achado A-03, de origem ambiental; a correção proposta nesta auditoria remove a dependência da versão do Node, e o caso passou a ser aprovado no mesmo ambiente (seção 15.4) [E-48].

G.4 VERIFICAÇÕES NO AMBIENTE PUBLICADO

Ambiente: navegador integrado, requisições GET sem credenciais a `https://sbi-app-alpha.vercel.app`, em 24/10/2026, das 18h58 às 19h00 (horário de Brasília). Nenhuma requisição de escrita foi feita.

Tabela 11 **–** Casos executados no ambiente publicado

ID	Requisito	Cenário	Resultado esperado	Resultado observado	Status	Evidência
PUB-01	RF-01	`GET /api/backend/auth/csrf` sem sessão	200 com nome do cabeçalho e *token* CSRF	200, `headerName` igual a `X-CSRF-TOKEN` (*token* omitido na evidência)	Aprovado	EV-PUB-03
PUB-02	RF-01, RF-02	`GET /api/backend/auth/me` sem sessão	401 com `ErrorResponse` e `code UNAUTHENTICATED`	401, `UNAUTHENTICATED`, `details` nulo	Aprovado	EV-PUB-03
PUB-03	RF-02, RF-19	`GET /api/backend/sales/catalog` sem sessão	401 sem dados do catálogo	401, `UNAUTHENTICATED`	Aprovado	EV-PUB-03

ID	Requisito	Cenário	Resultado esperado	Resultado observado	Status	Evidência
PUB-04	RF-02, RF-12	`GET /api/backend/management/summary` sem sessão	401 sem dados gerenciais	401, `UNAUTHENTICATED`	Aprovado	EV-PUB-03
PUB-05	RF-02	`GET` em rota inexistente da API, sem sessão	401, sem revelar a inexistência da rota	401, `UNAUTHENTICATED`	Aprovado	EV-PUB-03

Fonte: inspeção desta auditoria [E-04].

Também foram observados, sem compor a contagem de casos: a tela `/entrar` com resposta 200, campos rotulados e atributos `autocomplete` adequados; o redirecionamento de `/gestao` sem sessão para `/entrar?next=%2Fgestao`; os cabeçalhos de segurança da API (`no-store`, `nosniff`, `DENY` e HSTS); e a ausência desses cabeçalhos nas páginas (achado A-05) [E-04].

G.5 CATÁLOGO DE CASOS DOCUMENTADO PELA EQUIPE

O catálogo formal da equipe tem 43 casos, com os 10 campos exigidos pelo enunciado, técnica, nível e risco [E-44]. O status documentado refere-se às execuções de referência da equipe de 09/10/2026 e à integração contínua; nenhum desses casos foi reexecutado nesta auditoria, porque todos dependem do JDK 25, do Maven Central ou da API em execução. As etapas da integração contínua que os executam terminaram com sucesso no *commit* de referência [E-46].

Quadro 70 **–** Casos de teste do catálogo da equipe

ID	Título	Requisito	Técnica	Nível	Status documentado	Nesta auditoria
CT-MONEY-001	Total da venda demonstrativa	RF-07 / CA-07-01	Baseado em requisitos	Unitário, Integração	Passou	Não executado
CT-MONEY-002	Quantidade multiplicada uma única vez	RF-07 / CA-07-02	Particionamento de equivalência	Unitário, Regressão	Passou	Não executado
CT-MONEY-003	Preço histórico preservado após mudança do catálogo	RF-07 / CA-07-03	Transição de estados	Integração	Passou	Não executado
CT-MONEY-004	Pedido sem itens	RF-07	Análise de valor-limite	Unitário	Passou	Não executado
CT-MONEY-005	Normalização para 2 casas com HALF_UP	RNF-03	Análise de valor-limite	Unitário	Passou	Não executado
CT-MONEY-006	Soma de centavos sem erro de ponto flutuante	RNF-03	Particionamento de equivalência	Unitário	Passou	Não executado
CT-MONEY-007	Limites de preço e pagamento na entrada	RNF-03	Análise de valor-limite	Unitário	Passou	Não executado
CT-REG-D017	Menor valor monetário aceito	RNF-03 / D017	Análise de valor-limite	Unitário, Regressão	Passou	Não executado
CT-ORDER-001	Pedido válido é criado	RF-05 / CA-05-01	Baseado em requisitos	Integração	Passou (parcial)	Não executado
CT-ORDER-002	Cliente novo persistido com o pedido	RF-05 / CA-05-02	Particionamento de equivalência	Integração	Passou	Não executado
CT-ORDER-003	Pedido sem cliente é rejeitado	RF-05 / CA-05-03	Testes de casos inválidos	Unitário	Passou	Não executado
CT-ORDER-004	Vendedor do pedido vem da sessão	RF-06 / CA-06-01	Tabela de decisão	Unitário, Integração	Passou	Não executado
CT-ORDER-005	clientId inexistente	RF-05	Testes de casos inválidos	Unitário	Passou	Não executado
CT-ORDER-006	Cliente novo sem documento ou de tipo inválido	RF-03, RF-05	Particionamento de equivalência	Unitário	Passou	Não executado
CT-ORDER-007	Produto inexistente no pedido	RF-05	Testes de casos inválidos	Unitário	Passou	Não executado
CT-ORDER-009	Pagamento do pedido sem dado obrigatório	RF-05, RF-08	Tabela de decisão	Unitário	Passou	Não executado
CT-PAY-001	Pagamento pendente aprovado confirma o pedido	RF-08 / CA-08-01	Transição de estados	Unitário	Passou	Não executado
CT-REG-D012	Valor pago diferente do total	RF-08 / D012	Análise de valor-limite	Unitário, Regressão	Passou	Não executado

ID	Título	Requisito	Técnica	Nível	Status documentado	Nesta auditoria
CT-PAY-011	Comparação com o total depois do HALF_UP	RF-08, RNF-03	Análise de valor-limite	Unitário, Regressão	Passou	Não executado
CT-BB-BOL-02	Vencimento do boleto	RF-08 / D003	Análise de valor-limite	Unitário, Regressão	Passou	Não executado
CT-REG-D004	Pagamento já processado não é reprocessado	RF-08 / D004	Transição de estados	Unitário, Regressão	Passou	Não executado
CT-REG-D006	Boleto sem vencimento	RF-08 / D006	Testes de casos inválidos	Unitário, Regressão	Passou	Não executado
CT-REG-D016	Tipo de pagamento independente do locale	RF-08 / D016	Particionamento de equivalência	Unitário, Regressão	Passou	Não executado
CT-REG-D018	Pagamento tardio não regride o pedido	RF-08 / D018	Transição de estados	Unitário, Regressão	Passou	Não executado
CT-REG-D019-PAG	Processamento simultâneo do mesmo pagamento	RNF-02 / D019	Baseado em risco	Integração, Regressão	Passou	Não executado
CT-PAY-005	Cada tipo cria a subclasse correta	RF-08	Particionamento de equivalência	Unitário	Passou	Não executado
CT-PAY-009	Troca de tipo substitui o pagamento	RF-08	Transição de estados	Unitário	Passou	Não executado
CT-WB-PIX-02	Classificação da chave PIX (caminhos)	RF-08	Caixa-branca (caminhos independentes)	Unitário	Passou	Não executado
CT-BB-FRETE-01	Frete na fronteira de 2 kg	RF-04, RF-07	Análise de valor-limite	Unitário	Passou	Não executado
CT-ST-DT	Tabela completa de transições de status	RF-08 / D010	Tabela de decisão / transição de estados	Unitário, Regressão	Passou	Não executado
CT-REG-D019-ST	Trocas de status simultâneas	RNF-02 / D019	Baseado em risco	Integração, Regressão	Passou	Não executado
CT-ADDR-DT	Alteração de endereço por status	RF-05	Tabela de decisão	Unitário	Passou	Não executado
CT-PROD-DT	Cadastro de produto por tipo	RF-04 / CA-04-01, CA-04-02	Tabela de decisão	Unitário	Passou	Não executado
CT-CLIENT-001	Cliente sem documento obrigatório	RF-03 / CA-03-01, CA-03-02 / D008	Particionamento de equivalência	Unitário, Regressão	Passou	Não executado
CT-REG-D011	Excluir recurso inexistente e existente	D011	Particionamento de equivalência	Unitário, Regressão	Passou	Não executado
CT-BD-001	Migração V3 de dinheiro legado	RNF-02, RNF-03	Análise de valor-limite	Integração	Passou	Não executado
CT-REG-D020	Valor legado logo acima do limite	RNF-02 / D020	Análise de valor-limite	Integração, Regressão	Passou	Não executado

ID	Título	Requisito	Técnica	Nível	Status documentado	Nesta auditoria
CT-CONTRACT-001	API ao vivo conforme o contrato OpenAPI	RF-04, RNF-07	Baseado em requisitos (contrato)	Sistema	Passou	Não executado
CT-AUTH-001	Acesso sem sessão e login inválido	RF-01 / CA-01-01	Tabela de decisão	Sistema	Passou	Não executado
CT-AUTH-002	Vendedor não lê dados restritos	RF-02 / CA-01-02	Tabela de decisão	Sistema	Passou	Não executado
CT-UI-LOGIN-001	Login e logout pela interface	RF-01, RF-16	Baseado em requisitos	Aceitação (E2E)	Passou	Não executado
CT-UI-LOGIN-002	Erro de senha acessível e tela móvel	RNF-05	Testes de casos inválidos	Aceitação (E2E)	Passou	Não executado
CT-REG-D021	Senha do banco ausente	RNF-04 / D021	Testes de casos inválidos	Unitário, Regressão	Passou	Não executado

Fonte: `docs/catalogo-casos.md`, versão 0.1 de 09/10/2026 [E-44].

Além do catálogo formal, o inventário estático do repositório registra 44 classes de teste Java (24 unitárias, com 128 métodos de teste, e 20 de integração, com 119 métodos), 8 arquivos de testes unitários do frontend (42 casos) e 8 especificações de ponta a ponta (26 casos), além da fumaça do ambiente publicado (5 casos) [E-21]. Os totais documentados pela equipe para a suíte Java (233 unitários e 649 de integração) incluem testes parametrizados, como os 260 casos da matriz de acesso, e por isso superam a contagem de métodos.

APÊNDICE H – REGISTRO DE DEFEITOS

O registro reúne os 20 defeitos documentados pela equipe e os 17 achados desta auditoria. Os campos seguem o modelo de 13 campos do enunciado; por limitação de espaço, a tabela apresenta os campos essenciais, e o registro completo da equipe, com descrição, passos de reprodução, evidência, responsável e reteste, está em `docs/defeitos.md` [E-23]. Valores de cartão de teste citados no registro original foram omitidos.

H.1 DEFEITOS REGISTRADOS PELA EQUIPE

Quadro 71 **–** Defeitos registrados pela equipe

ID	Título	Severidade	Prioridade	Resultado esperado	Resultado obtido	Correção	Status registrado
H-01	Quantidade multiplicada duas vezes no total do pedido	Crítica	Alta	Preço do item R$ 100,00; total R$ 200,00.	Preço do item R$ 200,00; total R$ 400,00 (`expected: 100.0 but was: 200.0`).	`main` (03/10)	Corrigido e retestado
H-02	Erro genérico devolvia HTTP 500 com `status: 400` no corpo	Média	Média	HTTP 500 e `status: 500`, sem detalhes internos.	HTTP 500 e `status: 400`.	`main` (04/10)	Corrigido e retestado
D003	Boleto vencido marca o pedido como PAID	Alta	Alta	Pagamento EXPIRED; pedido continua PENDING_PAYMENT.	Pedido PAID.	`192f068`	Corrigido na main
D004	Pagamento não PENDING é processado de novo	Alta	Alta	Erro de negócio (`PaymentException`, 400) sem alterar o pagamento.	Reprocessamento ou NullPointerException (500).	`192f068`	Corrigido na main
D006	Boleto sem vencimento aceito; processamento lança NullPointerException	Alta	Alta	400 na criação, com mensagem `dueDate is required`.	Criação aceita; NullPointerException no processamento.	`192f068`	Corrigido na main
D008	Cliente sem CPF/CNPJ lança `ProductException`	Baixa	Baixa	400 com erro de cliente mencionando `cpf`.	`ProductException`.	`a5e3b51`	Corrigido na main
D010	Status do pedido regride (ex.: DELIVERED → PENDING_PAYMENT)	Alta	Alta	400 (`OrderException`); status mantido.	Status alterado para PENDING_PAYMENT.	`a5e3b51`	Corrigido na main
D011	DELETE de recurso inexistente não retorna 404	Média	Média	404 com `ApiErrorResponse`.	204.	`a5e3b51`	Corrigido na main
D012	Pagamento com valor diferente do total do pedido é aceito	Crítica	Alta	400; nada gravado.	Pagamento gravado e aprovável.	`192f068`	Corrigido na main
D016	Tipo de pagamento depende do locale da JVM	Média	Média	Pagamento PIX criado.	`Invalid payment type: pix`.	`192f068`	Corrigido na main
D017	Valor abaixo de meio centavo é aceito e gravado como 0,00	Alta	Alta	400 (o menor valor aceito é 0,005, que arredonda para 0,01).	Produto criado com preço 0,00.	`ac064e4`	Corrigido na main
D018	Pagamento tardio volta pedido enviado/entregue para PAID	Alta	Alta	400 (`PaymentException`); pedido continua SHIPPED; pagamento continua PENDING.	Pedido PAID.	`ac064e4`	Corrigido na main

ID	Título	Severidade	Prioridade	Resultado esperado	Resultado obtido	Correção	Status registrado
D019	Requisições simultâneas passam pela validação de status	Média	Alta	Estado final SHIPPED; no pagamento, uma requisição aprovada e a outra recusada.	Estado final PAID (`expected: SHIPPED but was: PAID`); pagamento aprovado duas vezes (`Expected size: 1 but was: 0`).	`ac064e4`	Corrigido na main
D020	Migração V3 arredonda antes de conferir o limite da coluna	Baixa	Média	Migração falha com mensagem clara; nada é convertido.	Migração concluída com 9999999999,99.	`ac064e4`	Corrigido na main
D021	Senha do banco ausente gera erro de autenticação confuso	Baixa	Média	Inicialização interrompida com mensagem que cita as variáveis.	Erro de autenticação do PostgreSQL.	`5bc0736`	Corrigido na main
D022	Rota legada de pagamento grava o número completo do cartão	Alta	Média	Banco sem o número completo; no máximo os 4 últimos dígitos.	`card_number = [número omitido]`, conforme o código atual de `CardPayment` e `PaymentService` (não executado como teste).	ETAPA 2C	Corrigido
D023	`GET /orders` legado responde 500 quando existe pedido de retirada (sem endereço)	Média	Alta	Não detalhado no registro (somente resumo).	Não detalhado no registro (somente resumo).	ETAPA 2C (`5995323`)	Corrigido
D024	Inteiro do corpo aceita decimal: quantidade `1.5` vira `1`	Média	Alta	Não detalhado no registro (somente resumo).	Não detalhado no registro (somente resumo).	ETAPA 2C (`cbcd7a1`)	Corrigido
D025	Contrato v4 rejeita `StockMovement.reason` nulo (tipo aceita `null`, `enum` não)	Baixa	Média	Não detalhado no registro (somente resumo).	Não detalhado no registro (somente resumo).	ETAPA 2C (`0c48a1a`, D-EXEC-06)	Corrigido
D026	OpenAPI ao vivo diverge do v4: `GET /auth/csrf` com parâmetro `csrf` e códigos 201/204 não documentados	Média	Média	Não detalhado no registro (somente resumo).	Não detalhado no registro (somente resumo).	ETAPA 2C (`b478ebf`)	Corrigido

Fonte: `docs/defeitos.md`, versão 0.1 de 09/10/2026, com acréscimos até 14/10/2026 [E-23].

Os identificadores D001, D002, D005, D007, D009 e D013 a D015 não aparecem no repositório nem no quadro, e o registro pede que se confirme se foram descartados. Os defeitos D023 a D026 constam apenas do resumo, sem registro detalhado. Nenhum defeito está no estado Encerrado, porque o reteste por integrante que não implementou a correção não foi registrado (achado A-13).

H.2 ACHADOS DESTA AUDITORIA

Todos os achados estão abertos na data da análise. A coluna Proposta remete ao roteiro do capítulo 15. Para A-03, A-04, A-05, A-12 e A-16, esta auditoria preparou correções validadas localmente e ainda não aplicadas ao repositório (seção 15.4) [E-48].

Quadro 72 **–** Achados desta auditoria

ID	Descrição	Evidência	Resultado esperado	Resultado observado	Severidade	Prioridade	Proposta
A-01	Documentação desatualizada ou contraditória: README e relatório final citam “um gerente e três vendedores” no ambiente publicado; `rastreabilidade.md` afirma que os PRs da evolução não estavam integrados; a matriz inicial de `criterios-aceitacao.md` marca requisitos como pendentes; `docs/bd/dicionario-dados.md` descreve o modelo anterior à V5; `revisao-equipe.md` cita o código `CSRF_INVALID`, inexistente	Leitura dos documentos no *commit* `3b0242c`; consulta a `app_users` no Neon (2 gerentes e 2 vendedores) [E-01] [E-03] [E-32]	Documentos coerentes com o código e com o ambiente publicado	Cinco documentos com estado anterior ou incorreto	Baixa	Alta	EV-07
A-02	O OpenAPI gerado pela API omite respostas de erro declaradas no contrato v5	Comparação de `/v3/api-docs` publicado com o contrato v5 [E-05]	As 201 respostas de erro do contrato documentadas no OpenAPI publicado	109 de 201 ausentes (400: 39; 404: 31; 409: 22; 403: 17); 58/58 operações e códigos de sucesso presentes	Média	Alta	EV-04
A-03	O caso CT-WEB-124 depende da versão do Node e dos dados de internacionalização	`npm test` do frontend em Node 22.23.4 [E-09]	“R$ 47 mil”	“R$ 47,0 mil”; aprovado na CI com Node 24, segundo a equipe	Baixa	Média	EV-11

ID	Descrição	Evidência	Resultado esperado	Resultado observado	Severidade	Prioridade	Proposta
A-04	Dependência `next` 16.3.8 com avisos de segurança críticos publicados após o último *commit*	`npm audit` do frontend [E-24]; aviso GHSA-2464-fhr4-w9xw (GITHUB, 2026)	Nenhuma dependência de produção com aviso crítico ou alto	`next` (crítico, faixa até 16.4.0), `undici` (alto) e outros 4 pacotes de desenvolvimento ou moderados; exposição não confirmada, pois o SBI não usa `next/image`	Alta	Alta	EV-01
A-05	Páginas HTML sem cabeçalhos de segurança	Inspeção de `/entrar` no ambiente publicado [E-04]	CSP com `frame-ancestors`, `X-Content-Type-Options`, `Referrer-Policy` e `Permissions-Policy`	Ausentes nas páginas; presentes nas respostas da API	Média	Média	EV-09
A-06	Senha de banco de desenvolvimento em texto no histórico do Git	Busca no histórico de *commits*, sem reprodução do valor [E-25]	Nenhum segredo no histórico	Introduzida em `application.properties` (`182a5a2`, 31/08/2026) e em `docker-compose.yml` (`153093a`, 03/10/2026); removida em `3753711` (14/10/2026)	Média	Média	EV-08
A-07	Pasta oficial do Drive editável por qualquer pessoa com o *link*	Metadados de permissão da pasta [E-27]	Acesso restrito à equipe e aos docentes	Edição pública por *link*	Média	Alta	EV-03
A-08	Contas de demonstração dos docentes, previstas para inspeções autorizadas, inexistentes no banco publicado	Consulta somente leitura a `app_users` no Neon [E-32]	Três contas de avaliação ativas	Somente as quatro contas da equipe	Média	Alta	EV-02
A-09	Relatório versionado do contrato ao vivo desatualizado	`reports/contrato/live-v5/` [E-33]	58 operações verificadas, como declarado	54 operações, execução de 14/10/2026 09h54	Baixa	Média	EV-05
A-10	Métricas R1 anteriores à evolução de 14/10/2026 e medição R2 não realizada	`reports/metricas/r1/metrics.json` (gerado em 14/10/2026) e inventário de testes [E-22] [E-21]	Métricas medidas sobre a versão entregue	R1 com 544 testes, sem as classes da evolução (por exemplo, `UsersApiTest`); R2 não realizada	Baixa	Média	EV-05

ID	Descrição	Evidência	Resultado esperado	Resultado observado	Severidade	Prioridade	Proposta
A-11	Índice `idx_orders_seller_id` redundante com `uq_orders_seller_request`; 15 índices do esquema sem uso na amostra de produção	Catálogo e `pg_stat_user_indexes` no Neon [E-13] [E-14]	Índices justificados por consultas reais	Índice redundante e índices com `idx_scan` igual a zero	Baixa	Baixa	EV-17
A-12	`spring.jpa.show-sql=true` no perfil padrão	`application.properties` e `docs/deploy.md` [E-01] [E-41]	Registro de SQL desativado fora do desenvolvimento	Ativado em `application.properties`; em produção, desativado pela variável `SPRING_JPA_SHOW_SQL=false`, segundo `docs/deploy.md`	Baixa	Baixa	EV-13
A-13	Defeitos sem encerramento formal e estados internos divergentes	`docs/defeitos.md` [E-23]	Defeitos encerrados após reteste independente	Nenhum encerrado; registro em versão 0.1	Baixa	Média	EV-06
A-14	Quadro do Trello sem atualização desde 12/10/2026	Ações do quadro [E-26]	Quadro coerente com o estado do projeto	Última ação em 12/10; 25 cartões vencidos; 12% dos itens de *checklist* concluídos	Baixa	Média	EV-07
A-15	Capturas versionadas com miniaturas de produto quebradas	`reports/marca/` de 20/10/2026 [E-30]	Capturas que representem a interface atual	Miniaturas quebradas, causa corrigida no PR #45, capturas não refeitas	Baixa	Baixa	EV-10
A-16	Barra de meta quase cheia em “Meu desempenho” sem meta definida	Captura `vendedor-meu-desempenho.png` e componente `GoalBar` [E-30] [E-01]	Barra vazia ou ausente quando não há meta	Barra em cerca de 87% da largura (escala `max(meta, realizado, 1) × 1,15`)	Baixa	Média	EV-10
A-17	Bloqueio por conta sem limitação por origem permite bloquear contas conhecidas	Leitura de `LockoutAuthenticationProvider` e do README [E-17]	Tentativas limitadas também por origem	Cinco falhas de qualquer origem bloqueiam a conta por 15 minutos	Média	Média	EV-12

Fonte: elaborado pelos autores.

APÊNDICE I – MANUAL TÉCNICO DE INSTALAÇÃO

Este manual consolida os procedimentos do README e de `docs/deploy.md` na versão de referência e os procedimentos executados nesta auditoria [E-01] [E-41]. Cada procedimento indica se foi executado nesta auditoria. Nenhuma senha, *token* ou URL de banco consta deste manual: os valores devem ser definidos pelo responsável em variáveis de ambiente locais ou nos painéis das plataformas.

I.1 REQUISITOS

Quadro 73 **–** Requisitos de instalação por procedimento

Procedimento	Requisitos
Pilha de demonstração local	Docker Desktop (ou Docker Engine com Compose); acesso à internet para baixar imagens e dependências
Desenvolvimento local	Docker Desktop, JDK 25, Node.js 20.9 ou superior (a CI usa 24), VS Code opcional
Testes Java	JDK 25, Maven Wrapper (Maven 3.9.11), Docker para o PostgreSQL 18 descartável na porta 15432
Reprodução do pacote SQL sem Docker	PostgreSQL 16 ou superior com `psql`, Python 3 com `bcrypt` (somente para gerar o *hash* das contas de demonstração)
Publicação	Contas na Vercel, no Render, no Neon e no GitHub, com acesso ao repositório

Fonte: README, `docs/deploy.md` e `reproducao/LEIA-ME.md` [E-01] [E-41].

I.2 OBTENÇÃO DO CÓDIGO

O repositório é privado e exige autorização do responsável. Após obter acesso: `git clone <URL do repositório> sbi`, `cd sbi` e, para reproduzir exatamente a versão analisada, `git checkout 3b0242c`.

I.3 PILHA DE DEMONSTRAÇÃO EM UM COMANDO

Procedimento oficial da equipe, não executado nesta auditoria. Define-se uma senha para as contas de demonstração (`SBI_DEMO_PASSWORD`, de 8 a 72 caracteres, ou uma por conta em `SBI_DEMO_PASSWORD_MATHEUS`, `_GABRIEL`, `_ALLAN` e `_FRANCISCO`) e uma senha aleatória para o banco (`SBI_DB_PASSWORD`), e executa-se `docker compose -f docker-compose.demo.yml -p sbi-demo up --build`. A interface fica em `http://localhost:3000` e o banco de demonstração em `127.0.0.1:5433` (banco `sbi_demo`, usuário `sbi`). A massa Olist é carregada em seguida por `scripts/demo-`

`olist.ps1 -Check`. Para encerrar e apagar os dados: `docker compose -f docker-compose.demo.yml -p sbi-demo down` [E-01].

I.4 AMBIENTE DE DESENVOLVIMENTO

Procedimento oficial da equipe, não executado nesta auditoria. No VS Code, a tarefa “SBI: iniciar tudo” (`Ctrl+Shift+B`) executa `scripts/dev.ps1`, que sobe um PostgreSQL descartável em `127.0.0.1:5434`, a API no perfil `demo` em `http://localhost:8080` e o frontend em `http://localhost:3000`. As senhas ficam em `.env.local`, gerado na primeira execução e ignorado pelo Git [E-01].

I.5 PERFIS E VARIÁVEIS DE AMBIENTE

Quadro 74 **–** Perfis da API e variáveis obrigatórias

Perfil	Uso	Variáveis principais
padrão	Desenvolvimento com o banco `postgres-order`	`SPRING_DATASOURCE_PASSWORD` (obrigatória)
`demo`	Contas da equipe e massa sintética (`db/demo`); deve estar ativo desde a primeira migração	`SBI_DEMO_PASSWORD` ou senhas por conta; datasource
`integration`	Testes de integração com PostgreSQL descartável na porta 15432	definidas pelos scripts de teste
`contract`	API isolada para testes HTTP de contrato, com contas sintéticas	senha gerada por execução

Fonte: README [E-01].

No frontend, `BACKEND_ORIGIN` define a origem da API usada pelo *rewrite* (padrão `http://127.0.0.1:8080`); como o valor é gravado no *build*, sua alteração exige novo *build*. `SESSION_COOKIE_SECURE=false` só deve ser usado em HTTP local [E-01] [E-41].

I.6 EXECUÇÃO DOS TESTES

Quadro 75 **–** Comandos de teste

Suíte	Comando	Executado nesta auditoria
Unitários Java	`./mvnw -q verify`	Não
Integração Java, cobertura e análise estática	`scripts/test.ps1 -Mode Integration`	Não
Contrato ao vivo, E2E e axe	`scripts/test-contract.ps1 -HttpOnly -Frontend`	Não
Validadores de contrato	`npm ci && npm test` (raiz)	Sim (47/47)

Suíte	Comando	Executado nesta auditoria
Auditoria estática do contrato	`node scripts/audit-contract.mjs <contrato v5>` e `node scripts/validate-contract.mjs <contrato v5>`	Sim
Frontend	`cd frontend && npm ci && npm run typecheck && npm test`	Sim (41/42)
Esquemas do frontend iguais ao contrato	`npm run api:check` (em `frontend/`)	Não
Build do frontend	`npm run build` (em `frontend/`)	Tentado; bloqueado pela rede
Métricas	`node scripts/quality-metrics.mjs reports/metricas/r1 "R1"`	Não

Fonte: README e execuções desta auditoria [E-01] [E-08] [E-09] [E-10].

I.7 REPRODUÇÃO DO PACOTE SQL SEM DOCKER

Procedimento executado com sucesso nesta auditoria, em Linux, com os scripts do diretório `reproducao/` entregue com este relatório:

a) iniciar um cluster PostgreSQL local com autenticação `trust` restrita ao *socket* local e um superusuário de teste;

b) definir `PGHOST`, `PGPORT` e `PGUSER` e executar `reproducao/apply_migrations.sh <repositório> order_management_test`, que cria o banco, aplica V1 a V11 e `db/demo` na ordem do Flyway, substitui `${flyway:defaultSchema}` por `public`, executa cada arquivo em transação própria e cria as contas de demonstração com o *hash* BCrypt de uma senha aleatória, sem gravá-la em disco;

c) copiar `reproducao/run_bd_local.sh` para `docs/bd/pacote-sql/` da cópia local do repositório e, nessa pasta, executar `DB=order_management_test SUPERUSER=<superusuário> bash run_bd_local.sh`; o resultado esperado é `PASS: 61 FAIL: 0`;

d) para a massa Olist, executar com `psql 07_carga_olist.sql` (duas vezes, para verificar a idempotência), `08_crud_clientes_olist.sql` e `09_testes_olist.sql`.

O procedimento difere do oficial em três pontos, discutidos na seção 14.6: versão do PostgreSQL, aplicação das migrações por `psql` e migração Java V6_1 substituída por SQL equivalente.

I.8 PUBLICAÇÃO

Procedimento documentado pela equipe, na ordem Neon, Render, carga Olist e Vercel [E-41]:

a) **Neon:** criar projeto com PostgreSQL 18, usar o banco `neondb` e copiar o *host* de conexão direta, sem *pooler*, porque o Flyway e o contexto de sessão usado pelos gatilhos exigem conexão direta;
b) **Render:** criar serviço web Docker a partir do repositório, com verificação de saúde em `/auth/csrf` e as variáveis `SPRING_PROFILES_ACTIVE=demo`, datasource, `SBI_DEMO_PASSWORD`, `SESSION_COOKIE_SECURE=true` e opções de memória da JVM; o Flyway cria o esquema e as contas na primeira inicialização;
c) **Carga Olist:** executar o *workflow* manual `neon-olist.yml`, que usa o segredo `NEON_API_KEY` e a variável `NEON_PROJECT_ID` do GitHub;
d) **Vercel:** importar o repositório com diretório raiz `frontend` e definir `BACKEND_ORIGIN` com a URL da API;
e) **Opcional:** habilitar `neon-preview.yml` para criar uma *branch* do banco por *pull request*.

I.9 SOLUÇÃO DE PROBLEMAS

Quadro 76 **–** Problemas conhecidos e soluções

Sintoma	Causa	Solução
Primeira requisição demora de 1 a 3 minutos ou a tela de login avisa que o servidor está acordando	API suspensa pelo plano gratuito do Render	Aquecer acessando `/auth/csrf` da API alguns minutos antes do uso [E-41]
API não inicia	`SPRING_DATASOURCE_PASSWORD` ausente	Definir a variável; a API interrompe a inicialização com mensagem explícita (defeito D021 corrigido)
Frontend chama a API errada após mudança de URL	`BACKEND_ORIGIN` gravado no *build*	Refazer o *build* na Vercel
Build do frontend falha ao baixar fontes	`next/font/google` exige acesso a `fonts.googleapis.com`	Executar em rede com acesso a esse domínio
Caso CT-WEB-124 falha	Versão do Node diferente da CI	Usar Node 24 (A-03)
Migração ou gatilho falha no Neon	Conexão pelo *pooler* em modo transação	Usar o *host* de conexão direta

Fonte: `docs/deploy.md`, README e execuções desta auditoria [E-41] [E-01] [E-10].

APÊNDICE J – MANUAL DO USUÁRIO

Este manual descreve o uso do SBI pela interface, por perfil, na versão de referência. Ele foi elaborado a partir das rotas e dos textos da interface, do roteiro de demonstração da equipe e das capturas versionadas [E-01] [E-30]. As senhas das contas não constam deste manual: são entregues pela equipe por canal seguro.

J.1 ACESSO AO SISTEMA

a) Acesse `https://sbi-app-alpha.vercel.app`. Se a API estiver suspensa pelo plano gratuito, a tela de login avisa que o servidor está acordando; aguarde de 1 a 3 minutos.
b) Informe e-mail e senha e selecione Entrar. A opção “Mostrar senha” exibe o texto digitado.
c) Na primeira entrada com senha provisória, o sistema abre apenas a tela Trocar senha. A nova senha deve ter de 12 a 72 caracteres, não pode ser comum nem igual ao e-mail.
d) Após cinco tentativas erradas seguidas, a conta fica bloqueada por 15 minutos; o gerente pode desbloqueá-la antes.
e) Para sair, use o botão Sair no rodapé do menu. Se o gerente alterar o seu papel, situação ou senha, a sessão é encerrada na ação seguinte e o sistema pede novo login.

O menu lateral mostra apenas as áreas liberadas para o perfil: GESTÃO (somente gerente), ATENDIMENTO e MEU ESPAÇO. Acessar diretamente o endereço de uma área não liberada exibe “Sem acesso a esta área”.

J.2 PERFIL VENDEDOR

Quadro 77 **–** Tarefas do vendedor

Tarefa	Onde	Como fazer
Registrar uma venda	Vender	Busque o produto por nome ou SKU (atalho “/”), adicione ao carrinho e ajuste as quantidades; a promoção vigente é aplicada automaticamente. Selecione “Continuar para cliente e pagamento”, busque o cliente por CPF, CNPJ ou nome (ou cadastre-o), escolha a forma de pagamento (Pix, cartão ou boleto) e a modalidade (retirada na loja ou entrega) e crie o pedido. O total é sempre calculado pelo servidor

Tarefa	Onde	Como fazer
Aplicar desconto manual	Vender, campo “Desconto manual (opcional)”	Informe o percentual e o motivo (5 a 200 caracteres). O limite do vendedor é 5%; acima dele, o sistema informa “Desconto acima do seu limite” e o gerente pode aprovar o desconto no pedido pendente
Confirmar o pagamento	Vender (resumo da venda) ou Vendas	Após receber o pagamento, confirme-o; o sistema exibe “Pagamento confirmado. Estoque baixado.” Repetir a confirmação não gera efeito
Cancelar um pedido pendente	Vendas, detalhe da venda	Cancele após a confirmação do diálogo; a reserva de estoque é liberada. Pedido pago não pode ser cancelado
Consultar vendas	Vendas	O vendedor vê apenas as próprias vendas, com filtro por situação e paginação
Atender cliente	Clientes	Busque por documento ou nome, cadastre pessoa física ou jurídica, edite contato e endereço (o documento não pode ser alterado) e consulte histórico e indicadores. Para o vendedor, os documentos aparecem mascarados e os valores consideram apenas as próprias vendas
Acompanhar a meta	Meu desempenho	Escolha o mês para ver meta, realizado (vendas pagas menos estornos), atingimento e falta. Sem meta definida, os campos exibem “Sem meta”
Trocar a senha	Trocar senha	Informe a senha atual e a nova senha duas vezes. As outras sessões abertas da conta são encerradas

Fonte: elaborado pelos autores a partir da interface e de `docs/roteiro-demonstracao.md` [E-01].

J.3 PERFIL GERENTE

O gerente executa todas as tarefas do vendedor e, adicionalmente, as do **Quadro** 78.

Quadro 78 **–** Tarefas exclusivas do gerente

Tarefa	Onde	Como fazer
Acompanhar a loja	Visão geral	Escolha o período para ver faturamento líquido, lucro bruto, margem, pedidos pagos, gráfico diário, formas de pagamento, ranking de vendedores, produtos mais vendidos, meta do mês e alerta de estoque. O painel se atualiza a cada 15 segundos com a aba visível. “Não informado” indica item vendido sem custo conhecido
Manter produtos e categorias	Produtos e preços	Use “Novo produto” para cadastrar produto físico (com peso) ou digital (com *link* de download), vendável ou insumo; edite dados, ative ou inative e mantenha as categorias. Para retirar um produto do PDV, inative-o; a interface não oferece exclusão
Alterar preço	Produtos e preços, ação Preço	Informe o novo preço e o motivo; consulte o Histórico de preço, com autor e motivo. Vendas anteriores mantêm o preço da época
Movimentar estoque	Estoque	Na posição de estoque, registre Entrada (quantidade e custo unitário opcional, que recalcula o custo médio), Ajuste com motivo (perda, avaria, uso interno, inventário ou devolução ao fornecedor) e estoque Mínimo. O Extrato de movimentos filtra por produto, tipo e período
Liberar promoção e definir alçadas	Promoções	Use “Liberar promoção” para definir nome, tipo (percentual ou valor por unidade), produto ou categoria e vigência; encerre quando necessário. Em “Alçadas por papel”, altere com motivo o desconto máximo de cada papel e a permissão de venda abaixo do custo

Tarefa	Onde	Como fazer
Aprovar desconto acima da alçada	Vendas, detalhe de pedido pendente	Use “Aprovar desconto (gerente)”, com percentual até a alçada do gerente e motivo; o total e o pagamento são recalculados
Consultar análises	Análises	Escolha o mês para ver realizado por semana, formas de pagamento, margem por produto, uso de promoções, descontos manuais por vendedor e valor do estoque a custo e a preço de venda
Administrar usuários	Usuários	Cadastre usuário com nome, e-mail, papel e senha provisória; edite, altere papel ou situação com motivo, desbloqueie e redefina a senha. A própria conta não oferece essas ações, e o sistema impede deixar a loja sem gerente ativo

Fonte: elaborado pelos autores a partir da interface e de `docs/roteiro-demonstracao.md` [E-01].

As funções de definição de metas mensais, estorno e exclusão de cliente ou de produto não têm tela na interface; são executadas pela API documentada no Swagger (`/swagger-ui/index.html`), com sessão de gerente, conforme os casos UC10, UC14, UC16 e UC20 do Apêndice E.

J.4 MENSAGENS FREQUENTES

Quadro 79 **–** Mensagens frequentes e ação recomendada

Situação	Significado	O que fazer
Servidor acordando na tela de login	A API estava suspensa por inatividade	Aguardar de 1 a 3 minutos
Sessão encerrada porque o acesso mudou	O gerente alterou papel, situação ou senha da conta	Entrar novamente
Estoque insuficiente ao criar o pedido	Outra venda reservou as unidades disponíveis	Ajustar a quantidade ao disponível informado; o carrinho é preservado
Desconto acima do seu limite	Percentual maior que a alçada do papel	Reduzir o percentual ou pedir aprovação ao gerente
Documento já cadastrado	CPF ou CNPJ pertence a outro cliente	Usar o cliente existente, oferecido pela própria tela
Sem acesso a esta área	Área exclusiva de outro perfil	Usar as áreas do menu
Pedido não encontrado	Pedido inexistente ou de outro vendedor	Verificar o número do pedido

Fonte: elaborado pelos autores a partir das mensagens da interface e do contrato v5 [E-01] [E-06].

APÊNDICE K – MATRIZ DE EVIDÊNCIAS

A matriz associa cada identificador citado no texto, no formato [E-nn], à afirmação que sustenta, à fonte, à evidência e ao local onde ela pode ser conferida. O status segue a classificação do **Quadro** 3. As consultas foram feitas em 24/10/2026 e, para o *pull request* nº 47, no início de 25/10/2026 (horário de Brasília), sobre o *commit* `3b0242c` quando se trata do repositório. Os arquivos citados na coluna Localização com prefixo `evidencias/`, `reproducao/` ou `correcoes-propostas/` acompanham este relatório; os demais caminhos referem-se ao repositório na versão de referência.

Quadro 80 **–** Matriz de evidências

ID	Afirmação sustentada	Origem	Evidência	Localização	Status
E-01	Código, estrutura, histórico e documentação da versão de referência	Repositório oficial (privado)	*Commit* `3b0242c`, de 21/10/2026; 113 *commits* na `main`; 1.245 arquivos versionados	`github.com/matheussiqueira-dev/SBI--Super-Business-Intelligence`	Verificada
E-02	A produção na Vercel executa o *commit* `3b0242c`	Vercel, projeto `sbi-app`	Implantação `dpl_ApRPtFqstvaRHnEgkRRG2h2NKe7o`, estado READY, *branch* `main`	Painel da Vercel; `evidencias/EV-PUB-03_inspecao-publicada_2026-10-24.json`	Verificada
E-03	Versão, migrações, estrutura e volumes do banco publicado	Neon, projeto `wispy-glade-88962628`	Consultas somente leitura: PostgreSQL 18.6, 15 migrações, contagens de objetos e linhas, configuração do projeto	`evidencias/EV-BD-04_neon-producao_2026-10-24.json`	Verificada
E-04	Disponibilidade, respostas 401 padronizadas, cabeçalhos e telas públicas	Aplicação publicada	Requisições GET sem credenciais; capturas em largura de computador e em 375 px	`evidencias/EV-PUB-01`, `EV-PUB-02` e `EV-PUB-03`	Verificada
E-05	Conformidade do OpenAPI publicado com o contrato v5	`/api/backend/v3/api-docs`	76 operações; 58/58 do contrato; 0 divergência de sucesso; 109/201 erros ausentes	`evidencias/EV-PUB-04` e `EV-PUB-05`	Verificada
E-06	Operações, esquemas, papéis e respostas do contrato	Contrato v5	OpenAPI 3.1.0, 46 caminhos, 58 operações, 113 esquemas	`config/openapi/target-loja-gestao-openapi-2026-10-14-v5.json`	Verificada
E-07	Contrato v5 estruturalmente válido	`audit-contract.mjs`, `validate-contract.mjs`	0 erros; 236 esquemas sem achados	`evidencias/EV-CT-02` e `EV-CT-03`	Verificada
E-08	Validadores de contrato aprovados	`npm test` na raiz	47/47 em Node 22.23.4	`evidencias/EV-CT-01_npm-test-raiz_2026-10-24.tap`	Verificada
E-09	Verificação de tipos e lógica do frontend	`npm run typecheck` e `npm test` em `frontend/`	Tipos aprovados; 41/42	`evidencias/EV-WEB-01_frontend-typecheck-unit_2026-10-24.tap`	Verificada
E-10	Limites do ambiente de análise e *build* bloqueado	Execuções desta auditoria	Falha ao baixar fontes externas; Maven Central e Docker indisponíveis	`evidencias/EV-WEB-02_frontend-build-bloqueado-rede_2026-10-24.log`	Verificada
E-11	Regras, permissões e concorrência no banco	Pacote SQL do job `bd`	61/61 em PostgreSQL 16.15	`evidencias/EV-BD-01_resultado_bd_pg16_2026-10-24.txt`	Verificada

ID	Afirmação sustentada	Origem	Evidência	Localização	Status
E-12	Carga Olist, idempotência e CRUD por SQL	`07_carga_olist.sql`, `08_crud_clientes_olist.sql`, `09_testes_olist.sql`	7/7; CRUD com recusas esperadas	`evidencias/EV-BD-02`, `EV-BD-05` e `EV-BD-06`	Verificada
E-13	Planos de execução e uso de índices	`EXPLAIN ANALYZE` local	Varredura sequencial no volume atual; índices usados quando forçados	`evidencias/EV-BD-03_explain-indices_pg16_2026-10-24.txt`	Verificada
E-14	Índices sem uso em produção	`pg_stat_user_indexes` no Neon	16 índices com `idx_scan` igual a zero (15 do esquema e 1 do Flyway)	`evidencias/EV-BD-04_neon-producao_2026-10-24.json`	Verificada
E-15	Catálogo do esquema: 19 tabelas, 162 colunas, 18 *views*, 13 funções, 8 gatilhos, 52 CHECK, 32 FK, 48 índices	Catálogo do PostgreSQL após V1–V11	Extração de `information_schema` e `pg_catalog`	`SBI_DICIONARIO_DE_DADOS.xlsx`; Apêndice D	Verificada
E-16	Conteúdo das migrações e da massa de demonstração	`src/main/resources/db`	Leitura de V1–V11 e `db/demo`	`src/main/resources/db/migration`, `db/demo`	Verificada
E-17	Autenticação, autorização, sessão e bloqueio	Código de segurança	`SecurityConfig`, `security/*`, `LockoutAuthenticationProvider`	`src/main/java/.../config`, `.../security`	Verificada
E-18	Formato padronizado de erro e códigos	Código de tratamento de erros	`GlobalExceptionHandler`, `ErrorResponse`, `ErrorCode`	`src/main/java/.../exceptions`	Verificada
E-19	Regras de venda, reserva e confirmação	Código do PDV	`SalesService`, `Pricing`, `RequestHash`	`src/main/java/.../api/sales`	Verificada
E-20	Integração contínua e *workflows*	GitHub Actions	`ci.yml` (quatro *jobs*), `neon-preview.yml`, `neon-olist.yml`, `published-smoke.yml`	`.github/workflows/`	Verificada (leitura); conclusão da execução em E-46
E-21	Inventário de testes	Repositório	44 classes Java (128 + 119 métodos), 8 arquivos unitários do frontend (42 casos), 8 especificações E2E (26 casos)	`src/test`, `frontend/tests`, `frontend/e2e`	Verificada
E-22	Resultados declarados pela equipe e métricas R0/R1	Relatório final e relatórios de métricas	233 unitários, 649 de integração, 24 E2E, contrato 58/58; R0 e R1 recalculadas dos relatórios brutos	`docs/relatorio-final.md`, `reports/metricas/`; `evidencias/EV-QA-01_metricas-r0-r1-recalculadas_2026-10-24.txt`	Parcialmente verificada
E-23	Registro de defeitos	Equipe	20 defeitos com 13 campos, nenhum encerrado	`docs/defeitos.md`	Documentada
E-24	Avisos de segurança em dependências	`npm audit`	6 pacotes no frontend (1 crítico, 1 alto); 1 na raiz (moderado)	`evidencias/EV-SEC-01` e `EV-SEC-02`	Verificada

ID	Afirmação sustentada	Origem	Evidência	Localização	Status
E-25	Senha de desenvolvimento no histórico do Git	Histórico do repositório	Introduzida em `182a5a2` (31/08) e `153093a` (03/10), removida em `3753711` (14/10); valor não reproduzido	Histórico do Git (consulta local)	Verificada
E-26	Planejamento e acompanhamento no Trello	Quadro `ombHh0vo`	6 listas, 94 cartões, 448 ações, última ação em 12/10/2026	`trello.com/b/ombHh0vo`	Verificada
E-27	Conteúdo e permissões da pasta oficial	Google Drive	4 arquivos, sem subpastas; edição pública por *link*	Pasta `13MA9KBPV3kl6OfPjHm4Dvj73PagR3Dj-`	Verificada
E-28	Exigências da disciplina de Garantia da Qualidade	Enunciado do projeto	Grupo de até 3 alunos; ≥ 30 casos; ≥ 3 técnicas de caixa-preta; reprodutibilidade	Documento do projeto (PDF do enunciado)	Verificada
E-29	Planejamento inicial e escopo de 26/09/2026	Plano de Execução e Manual do projeto	Sistema sob teste, cronograma, esforço e exclusões de escopo	Pasta oficial do Drive; `docs/referencias/Manual-Projeto-A3.md`	Verificada
E-30	Telas autenticadas e acessibilidade	Capturas versionadas	20 capturas de 20/10/2026; 17 verificações axe sem violação	`reports/marca/`	Documentada
E-31	Telas da demonstração local de 14/10/2026	Pasta de trabalho do responsável	Capturas consultadas para conferência	Pasta local do responsável	Documentada (não citada no texto)
E-32	Contas existentes no banco publicado	Neon, tabela `app_users`	4 contas ativas (2 gerentes, 2 vendedores), sem bloqueio nem troca de senha pendente; nenhuma conta de docente	Consulta somente leitura; dados pessoais não reproduzidos	Verificada
E-33	Relatório versionado do contrato ao vivo	`reports/contrato/live-v5/live-contract.json`	54 operações, 0 divergências, execução de 14/10/2026	Repositório	Verificada
E-34	Documentos locais de planejamento	Pasta de trabalho do responsável	Planos e cópias de segurança do histórico	Pasta local do responsável	Documentada
E-35	Exigência da disciplina de Banco de Dados	Documento do projeto e cartão BD-04 do Trello	Banco funcional com SQL e operações CRUD	Projeto da disciplina; Trello	Verificada
E-36	Requisitos, critérios de aceitação, validação e rastreabilidade	Documentação da equipe	26 RF, 85 critérios, matriz RF → CA → CT	`docs/requisitos.md`, `docs/criterios-aceitacao.md`, `docs/validacao-requisitos.md`, `docs/rastreabilidade.md`	Documentada
E-37	Plano de qualidade	Documentação da equipe	Objetivos O1–O7, atributos ISO/IEC 25010, critérios de entrada e saída	`docs/plano-qualidade.md`	Documentada

ID	Afirmação sustentada	Origem	Evidência	Localização	Status
E-38	Avaliação heurística e correções de usabilidade	Documentação da equipe	24 problemas corrigidos no PR #45	`docs/relatorio-usabilidade.md`	Documentada
E-39	Pacote SQL e modelo ERP/PDV	Documentação da equipe	Modelo, roteiros e resultado de 12/10/2026	`docs/bd/pacote-sql/`, `docs/bd/modelo-erp-pdv.md`	Verificada (reexecutado)
E-40	Matriz de risco	Documentação da equipe	Probabilidade × impacto	`docs/analise-risco.md`	Documentada
E-41	Implantação, variáveis e operação	Documentação da equipe	Ordem Neon → Render → Olist → Vercel; aquecimento; variáveis sem valores	`docs/deploy.md`	Documentada
E-42	Decisões arquiteturais	Documentação da equipe	Registros D-EVO, ERP/PDV, pilha e autenticação	`docs/decisoes/`	Documentada
E-43	Plano de testes	Documentação da equipe	Níveis, técnicas, dados, critérios	`docs/plano-testes.md`	Documentada
E-44	Catálogo de casos de teste	Documentação da equipe	43 casos com 10 campos, técnica, nível e risco	`docs/catalogo-casos.md`	Documentada
E-45	Metas de qualidade propostas	Documentação da equipe	MQ-01 a MQ-07, aguardando aprovação	`docs/decisoes/metas-qualidade.md`	Documentada
E-46	Conclusão da integração contínua do *commit* de referência e execuções dos *workflows* manuais	GitHub Actions (API REST, somente leitura)	Execução 39668791724, de 21/10/2026: *jobs* `test`, `bd`, `contract` e `frontend` e todas as etapas com sucesso; relatórios e *logs* não baixados; `published-smoke.yml` sem execuções; carga Olist no Neon em 15/10 e 19/10	`evidencias/EV-CI-01_execucao-ci-3b0242c_2026-10-24.json`	Verificada (conclusão); totais documentados
E-47	Serviço da API no Render e implantação ativa	Render (API, somente leitura)	Serviço `sbi-api`: Docker, plano gratuito, Virgínia, uma instância, saúde em `/auth/csrf`, implantação automática por *commit* na `main`; implantação ativa do *commit* `3b0242c`, de 21/10/2026; variáveis de ambiente não lidas	`evidencias/EV-REN-01_render-servico-e-implantacoes_2026-10-24.json`	Verificada
E-48	Correções propostas para A-03, A-04, A-05, A-12 e A-16 e sua validação	Esta auditoria (*branch* local sobre `3b0242c`, não enviada ao repositório)	5 *commits*; tipos aprovados; 43/43 testes do frontend; 0 vulnerabilidade nas dependências de produção; cabeçalhos presentes nas páginas	`correcoes-propostas/`; `evidencias/EV-WEB-03`, `EV-WEB-04` e `EV-SEC-03`	Verificada

ID	Afirmação sustentada	Origem	Evidência	Localização	Status
E-49	Avatares dos usuários e reações animadas, em integração	*Pull request* nº 47 (`feat/user-profile-avatars`, *commit* `a54635d`); GitHub Actions e Vercel (somente leitura)	Aberto e não mesclado; CI aprovada na segunda tentativa; pré-visualizações e tentativa de produção na Vercel com erro de provisionamento; avatar ausente na produção (404); 11/11 testes de avatar e 52/53 da suíte local; 54/54 com o *patch* de correções; 14 imagens com *hash* SHA-256	`evidencias/EV-AV-01_avatares-pr47_2026-10-25.json` e `EV-AV-02_frontend-unit-pr47_2026-10-25.tap`	Verificada (código e CI); não publicada

Fonte: elaborado pelos autores.

APÊNDICE L – GLOSSÁRIO TÉCNICO

Quadro 81 **–** Glossário técnico

Termo	Definição no contexto do SBI
Alçada de desconto	Percentual máximo de desconto manual que um papel pode conceder sem aprovação; 5% para o vendedor e 20% para o gerente na configuração inicial.
Ameaça à validade	Fator que pode tornar incorreta a conclusão de um estudo, como diferença de ambiente ou de amostra.
Auditoria (trilha de)	Registro, na tabela `audit_event`, de ações relevantes com autor, entidade e detalhe, sem senha, *hash* ou documento completo.
axe-core	Biblioteca de verificação automática de acessibilidade usada nos testes de ponta a ponta.
BCrypt	Função de *hash* de senhas com custo ajustável, usada para armazenar as senhas das contas.
BOLA	*Broken Object Level Authorization*: falha que permite acessar objeto de outro usuário pela troca do identificador; no SBI, o pedido de outro vendedor responde 404.
Branch (cobertura de)	Proporção de decisões do código exercitadas pelos testes.
Caixa-preta (teste)	Teste projetado a partir da especificação, sem considerar a estrutura interna do código.
Caixa-branca (teste)	Teste projetado a partir da estrutura interna do código, como caminhos e decisões.
Commit	Registro atômico de alterações no histórico do Git.
Contract-first	Estratégia em que o contrato da API é definido antes da implementação e serve de oráculo para os testes.
CRUD	Operações de criação, leitura, atualização e exclusão de dados.
CSRF	*Cross-Site Request Forgery*: ataque que induz o navegador a enviar requisição autenticada; mitigado por *token* exigido em operações de escrita.
Custo médio ponderado	Custo unitário recalculado a cada entrada de estoque, ponderado pelas quantidades.
DTO	*Data Transfer Object*: objeto que define o formato de dados trocados entre cliente e API.
E2E	Teste de ponta a ponta, que exercita a interface, a API e o banco em conjunto.
`ErrorResponse`	Formato padronizado de erro da API, com `timestamp`, `status`, `error`, `code`, `message` e `details`.
Estorno	Devolução financeira parcial ou total de um pagamento aprovado, descontada do realizado na data em que ocorre.
Flyway	Ferramenta de versionamento do esquema do banco por migrações numeradas.
Gatilho de restrição adiado	Gatilho executado no `COMMIT` da transação, usado para validar regras que envolvem várias linhas.
Idempotência	Propriedade de uma operação que, repetida com os mesmos dados, produz o mesmo efeito de uma única execução.
Insumo	Produto cadastrado com `sellable = false`, que não aparece no PDV.
JaCoCo	Ferramenta de medição de cobertura de código Java.
Massa Olist	Recorte do conjunto público de pedidos da Olist, com nomes e documentos sintéticos, usado como dados de demonstração.
Meta	Valor de faturamento esperado para a loja ou para um vendedor em um mês.
Migração	Script versionado que altera o esquema ou os dados do banco de forma reproduzível.
OpenAPI	Especificação para descrever APIs HTTP: operações, parâmetros, corpos e respostas.

Termo	Definição no contexto do SBI
PDV	Ponto de venda: tela de atendimento no balcão, com busca de produtos, carrinho, cotação e finalização.
Realizado	Soma dos pagamentos aprovados menos os estornos, no período e no fuso da loja.
Reserva de estoque	Quantidade comprometida por pedidos pendentes; o disponível é o saldo físico menos o reservado.
Rewrite	Regra do Next.js que encaminha requisições do domínio do frontend para a API, preservando a mesma origem para o *cookie* de sessão.
RLS	*Row-Level Security*: política do PostgreSQL que filtra linhas conforme o usuário do banco; demonstrativa no SBI.
R0, R1, R2	Rodadas de medição de métricas de qualidade antes, durante e depois do ciclo de melhoria.
Sessão (versão de)	Contador `session_version` incrementado para revogar as sessões abertas de um usuário.
SKU	*Stock Keeping Unit*: código único de identificação do produto.
View	Consulta armazenada no banco, usada nos painéis e nas restrições de visibilidade de dados.
Zod	Biblioteca de validação de esquemas em TypeScript, cujos esquemas do SBI são gerados a partir do contrato.

Fonte: elaborado pelos autores.

APÊNDICE M – SITUAÇÃO EM 25 DE OUTUBRO DE 2026

Este apêndice registra o que mudou depois da análise. O corpo do relatório continua a descrever a versão de referência (*commit* `3b0242c`, de 21/10/2026), e os resultados de testes, as métricas e as capturas dos capítulos anteriores continuam válidos como retrato dessa versão. Entre 04h29 e 09h37 de 25/10/2026 (horário de Brasília), oito *pull requests* foram integrados à *branch* `main` e publicados na Vercel e, quando mudavam a API, no Render, com todas as verificações da integração contínua aprovadas; a situação resultante está no **Quadro** 82.

Quadro 82 **–** Situação dos *pull requests* e dos achados em 25/10/2026

Item	Na análise (24/10/2026)	Em 25/10/2026
Pull request nº 50 – identidade visual v2	Não existia.	Integrado (`a03f6bd`) e publicado. A interface passou a seguir o manual de marca v2.0; as capturas e a descrição da interface neste relatório correspondem à versão de referência.
Pull request nº 47 – avatares (seção 10.13)	Aberto e não mesclado; no ambiente publicado, a imagem do avatar respondia 404.	Integrado (`687cfc2`), adaptado à identidade v2 e publicado; as imagens dos avatares respondem 200.
Pull request nº 51 – créditos dos avatares	Não existia.	Integrado (`daafee0`): ajusta o texto sobre as expressões dos avatares na página de créditos.
Pull request nº 48 – perfil CEO	Alterações locais fora do escopo (seção 1.6).	Integrado (`e7e5f2c`) e publicado: papel CEO acima do gerente, painel executivo em `/executivo`, planejamento de metas e créditos do Projeto A3 em todas as telas.
Pull request nº 52 – desempenho	Não existia.	Integrado (`af1f10f`) e publicado: senhas verificadas com BCrypt de custo 10 (antes 12), painéis que deixam de atualizar após 10 minutos sem uso e cache de inicialização da API corrigido no Render.
Pull request nº 53 – endereços do repositório	Não existia.	Integrado (`2123f8d`): links e metadados com o endereço novo do repositório e fatos atualizados no README e na arquitetura.
Pull request nº 54 – organização das pastas	Não existia.	Integrado (`56a1b69`): relatórios brutos de qualidade deixam de ser versionados, as saídas dos testes de contrato vão para `target/contrato/` e os registros de etapas encerradas, para `docs/historico/`. Os caminhos citados neste relatório referem-se à versão de referência.
Achados A-03, A-04, A-05, A-12 e A-16 (seção 15.4)	Correções preparadas em *patch*, não aplicadas.	Aplicadas pelo *pull request* nº 49 (`b1e22f0`): `next` 16.4.3; cabeçalhos de segurança nas páginas, conferidos no ambiente publicado; `compactMoney` sem a notação compacta do `Intl`; barra de meta oculta sem meta; registro de SQL desligado por padrão.
A-08 – contas dos docentes	Pendente.	Contas de Paulo, Christian e Bruno criadas pela migração `V12_1` com o papel CEO e associadas aos avatares do catálogo; acesso conferido pela equipe no ambiente publicado. As senhas não constam deste relatório.

Item	Na análise (24/10/2026)	Em 25/10/2026
A-07 – pasta do Drive	Editável por qualquer pessoa com o *link*.	Mantida como “Qualquer pessoa com o link: Editor” por decisão do responsável; o risco do achado continua aceito.
A-02 e A-17	Abertos.	Abertos; não fazem parte do *patch*.
Implantação na Vercel (seção 10.13)	Pré-visualizações e uma tentativa de produção falhavam com “Resource provisioning failed”.	Causa: banco Supabase pausado, conectado ao projeto sem uso pelo SBI, com ação obrigatória em cada implantação. A conexão foi removida, e a produção está no *commit* `56a1b69`.
Integração contínua	Execuções de 24/10, entre 22h13 e 22h24, não iniciadas por pendência de faturamento da conta no GitHub.	Execuções normalizadas; todas as verificações dos *pull requests* nº 47 a 54 aprovadas antes da integração.

Fonte: GitHub, Vercel, Render e Neon, consultados pelos autores em 25/10/2026.